

一本小小的

MyBatis 源码分析书

田忠波 编著



MyBatis

田忠波 版权所有

版权声明

本作品采用[知识共享署名-非商业性使用-禁止演绎 4.0 国际许可协议](#)进行许可。

版权所有，侵权必究。

您可以自由地：

共享—在任何媒介以任何形式复制、发行本作品

惟须遵守下列条件：

署名—您必须给出适当的署名，提供指向本许可协议的链接，同时标明是否（对原始作品）作了修改。您可以用任何合理的方式来署名，但是不得以任何方式暗示许可人为您或您的使用背书。

非商业性使用—您不得将本作品用于商业目的。

禁止演绎—如果您再混合、转换、或者基于该作品创作，您不可以分发修改作品。

版本记录

版本号	修订人	更新时间	备注
1.0.0	田忠波	2018.09.09	第一次发布

注：本书目前由我个人维护，若后续有更新，请到 tianxiaobo.com 索取最新版本。

前言

我从七月份开始阅读 MyBatis 源码，并在随后的 40 天内陆续更新了 7 篇文章。起初，我只是打算通过博客的形式进行分享。但在写作的过程中，发现要分析的代码太多，以至于文章篇幅特别大。在这 7 篇文章中，有 4 篇文章字数超过了 1 万，最长的一篇文章约有 2.7 万字（含代码）。考虑到超长文章对读者不太友好，以及拆分文章工作量也不小等问题。遂决定将博文整理成电子书，方便大家阅读。

本书是按照 MyBatis 的工作流程进行编排，这样做的好处在于全书的内容会比较连贯。大家可以跟着本书的内容，从头到尾学习 MyBatis 源码。本书共包含七章内容，第一章介绍了 MyBatis 的用法，并对比了 MyBatis 与其他框架的优缺点。从第二章开始，本书进入源码分析阶段。第二章分析了配置文件的解析过程，包括但不限于<properties>、<setting>、<typeAlias>等节点解析过程的分析。第三章分析了映射文件的解析过程，包括<cache>、<sql>、<resultMap>等节点。第四章从 SQL 执行的源头开始，一路进行追踪，较为完整的分析了 MyBatis 执行 SQL 的过程。第五章分析了 MyBatis 提供的两种数据源。第六章和第七章分别分析了缓存机制与插件机制的实现。本书中出现的一些测试代码已经上传到 GitHub 上，有需要的同学自取。地址：[mybatis-test](https://github.com/tianxiaobo/mybatis-test)。

考虑到自己工作刚满两年，不管是技术能力，还是工作经验，均处于入门水平。同时这 也是我写的第一本电子书，经验不足。因此对于书中写的不好的地方，还请大家见谅，同时也希望大家多多指导。最后感谢大家阅读我的书。祝大家工作顺利，生活愉快。

个人网站：<https://www.tianxiaobo.com>

<https://www.coolblog.xyz>

博客园：<http://www.cnblogs.com/nullllun/>

邮 箱：coolblog.xyz@outlook.com

田忠波

2018 年 8 月 27 日于杭州

目录

第 1 章 MyBatis 入门.....	1
1.1 MyBatis 是什么	1
1.2 为什么要使用 MyBatis	2
1.2.1 使用 MyBatis 访问数据库	3
1.2.2 使用 JDBC 访问数据库	6
1.2.3 使用 SpringJDBC 访问数据库	9
1.2.4 使用 Hibernate 访问数据库	12
1.3 如何使用 MyBatis.....	18
1.3.1 单独使用 MyBatis.....	18
1.3.2 在 Spring 中使用 MyBatis.....	28
1.4 本章小结	34
第 2 章 配置文件解析过程	35
2.1 配置文件解析过程分析	35
2.1.1 解析<properties>节点	39
2.1.2 解析<settings>节点	42
2.1.3 设置<settings>内容到 Configuration 中	61
2.1.4 解析<typeAliases>节点	62
2.1.5 解析<plugins>节点	69
2.1.6 解析<environments>节点	70
2.1.7 解析<typeHandlers>节点	72
2.2 本章小结	79
第 3 章 映射文件解析过程	80
3.1 映射文件解析解析入口	80
3.2 解析映射文件	84
3.2.1 解析<cache>节点	86
3.2.2 解析<cache-ref>节点	93
3.2.3 解析<resultMap>节点	96
3.2.4 解析<sql>节点	115
3.2.5 解析 SQL 语句节点	119
3.3 Mapper 接口绑定过程分析	137

3.4 处理未完成解析的节点	139
3.5 本章小结	142
第4章 SQL 执行流程.....	143
4.1 SQL 执行入口	144
4.1.1 为 Mapper 接口创建代理对象	144
4.1.2 执行代理逻辑	146
4.2 查询语句的执行过程	158
4.2.1 selectOne 方法分析	159
4.2.2 获取 BoundSql	166
4.2.3 创建 StatementHandler	187
4.2.4 设置运行时参数到 SQL 中	189
4.2.5 #{} 占位符的解析与参数的设置过程梳理	194
4.2.6 处理查询结果	196
4.3 更新语句的执行过程	231
4.3.1 更新语句执行过程全貌	231
4.3.2 KeyGenerator	236
4.3.3 处理更新结果	242
4.4 SQL 执行过程总结	243
4.5 本章小结	244
第5章 内置数据源	245
5.1 内置数据源初始化过程	245
5.2 UnpooledDataSource	248
5.2.1 初始化数据库驱动	249
5.2.2 获取数据库连接	250
5.3 PooledDataSource	252
5.3.1 辅助类介绍	252
5.3.2 获取连接	255
5.3.3 回收连接	260
5.4 本章小结	263
第6章 缓存机制	264
6.1 缓存类介绍	264
6.1.1 PerpetualCache	265

6.1.2 LruCache.....	267
6.1.3 BlockingCache.....	270
6.2 CacheKey.....	273
6.3 一级缓存.....	276
6.4 二级缓存.....	280
6.5 本章小结.....	294
第7章 插件机制.....	295
7.1 插件机制原理.....	295
7.1.1 植入插件逻辑.....	296
7.1.2 执行插件逻辑.....	300
7.2 实现一个分页插件.....	302
7.3 本章小结.....	306
附录 MyBatis 源码分析系列文章列表	307

第 1 章

MyBatis 入门

MyBatis 是一个容易上手的持久层框架，使用者通过简单的学习即可掌握其常用特性的用法，这也是 MyBatis 被广泛使用的原因之一。在深入分析 MyBatis 源码前，我想先向大家介绍一下 MyBatis 的一些基础知识。本章将从三个角度向大家介绍 MyBatis，分别是 MyBatis 是什么，为什么要使用，以及如何使用。如果大家对 MyBatis 比较了解，可以跳过本章。

1.1 MyBatis 是什么

MyBatis 的前身是 iBatis，iBatis 是 Apache 软件基金会下的一个开源项目。2010 年该项目从 Apache 基金会迁出，并改名为 MyBatis。同期，iBatis 停止维护。

MyBatis 是一种半自动化的 Java 持久层框架(persistenceframework)，其通过注解或 XML 的方式将对象和 SQL 关联起来。之所以说它是半自动的，是因为和 Hibernate 等一些可自动生成 SQL 的 ORM(ObjectRelationalMapping)框架相比，使用 MyBatis 需要用户自行维护 SQL。维护 SQL 的工作比较繁琐，但也有好处。比如我们可控制 SQL 逻辑，可对其进行优化，以提高效率。

1.2 为什么要使用 MyBatis

我们在使用 Java 程序访问数据库时，有多种可选方案。比如我们可通过编写最原始的 JDBC 代码访问数据库，或是通过 Spring 提供的 JdbcTemplate 访问数据库。除此之外，我们还可以选择 Hibernate，亦或是本书的主角 MyBatis 等。在有多个可选项的情况下，我们为什么选择 MyBatis 呢？要回答这个问题，我们需要将 MyBatis 与这几种框架对比一下，高下立判。当然，技术之间一般没有高下之分。从应用场景的角度来说，符合应用场景需求的技术才是合适的选择。接下来我们通过写代码的方式，来对比这几种数据库访问技术的优缺点，并在最后说明 MyBatis 的适用场景。

这里先把本节所用到的一些公共类和配置贴出来，后面但凡用到这些类和配置的地方，大家可以到这里进行查看。如下：

```
public class Article {  
    private Integer id;  
    private String title;  
    private String author;  
    private String content;  
    private Date createTime;  
  
    // 省略 getter/setter 和 toString  
}
```

数据库相关配置放在了 jdbc.properties 文件中，详细内容如下：

```
jdbc.driver=com.mysql.cj.jdbc.Driver  
jdbc.url=jdbc:mysql://localhost:3306/myblog?useUnicode=true .....  
jdbc.username=root  
jdbc.password=****
```

表记录如下：

id	title	author	content	create_time
3	Spring IOC 容器源码分析系列文章导读	coolblog.xyz	Spring IOC 容器源码分析系列文章导读	2018-05-30 00:00:00
4	Spring IOC 容器源码分析 - 获取单例...	coolblog.xyz	Spring IOC 容器源码分析 - 获取单例...	2018-06-01 00:00:00
5	Spring IOC 容器源码分析 - 循环依赖...	coolblog.xyz	Spring IOC 容器源码分析 - 循环依赖...	2018-06-08 00:00:00
6	Spring AOP 源码分析系列文章导读	coolblog.xyz	Spring AOP 源码分析系列文章导读	2018-06-17 00:00:00
7	Spring AOP 源码分析 - 创建代理对象	coolblog.xyz	Spring AOP 源码分析 - 创建代理对象	2018-06-20 00:00:00
8	Spring MVC 原理探秘 - 一个请求的...	coolblog.xyz	Spring MVC 原理探秘 - 一个请求的...	2018-06-29 00:00:00
9	Spring MVC 原理探秘 - 容器的创建...	coolblog.xyz	Spring MVC 原理探秘 - 容器的创建...	2018-06-30 00:00:00

1.2.1 使用 MyBatis 访问数据库

前面说过，MyBatis 是一种半自动化的 Java 持久层框架，需要用户自行维护 SQL。这里，我们把 SQL 放在 XML 中，文件名称为 ArticleMapper.xml。如下：

```
<mapper namespace="xyz.coolblog.chapter1.dao.ArticleDao">
  <select id="findByAuthorAndCreateTime" resultType="Article">
    SELECT
      `id`, `title`, `author`, `content`, `create_time`
    FROM
      `article`
    WHERE
      `author` = #{author} AND `create_time` > #{createTime}
  </select>
</mapper>
```

上面的 SQL 用于从 article 表中查询出某个作者从某个时候到现在所写的文章记录。在 MyBatis 中，SQL 映射文件需要与数据访问接口对应起来，比如上面的配置对应 xyz.coolblog.dao.ArticleDao 接口，这个接口的定义如下：

```
public interface ArticleDao {
    List<Article> findByAuthorAndCreateTime(@Param("author") String author,
        @Param("createTime") String createTime);
}
```

要想让 MyBatis 跑起来，还需要进行一些配置。比如配置数据源、配置 SQL 映射文件的位置信息等。本节所使用到的配置如下：

```
<configuration>
  <properties resource="jdbc.properties"/>
  <typeAliases>
    <typeAlias alias="Article"
      type="xyz.coolblog.chapter1.model.ArticleDO"/>
    <typeAlias alias="Author" type="xyz.coolblog.model.AuthorDO"/>
  </typeAliases>
  <environments default="development">
    <environment id="development">
      <transactionManager type="JDBC"/>
      <dataSource type="POOLED">
        <property name="driver" value="${jdbc.driver}"/>
        <property name="url" value="${jdbc.url}"/>
        <property name="username" value="${jdbc.username}"/>
        <property name="password" value="${jdbc.password}"/>
      </dataSource>
    </environment>
  </environments>
  <mappers>
    <mapper resource="chapter1/mapper/ArticleMapper.xml"/>
  </mappers>
</configuration>
```

到此，MyBatis 所需的环境就配置好了，下面写点测试代码把 MyBatis 跑起来。

```
public class MyBatisTest {
  private SqlSessionFactory sqlSessionFactory;

  @Before
  public void prepare() throws IOException {
    String resource = "chapter1/mybatis-config.xml";
```

```

        InputStream inputStream = Resources.getResourceAsStream(resource);
        sqlSessionFactory = new
            SqlSessionFactoryBuilder().build(inputStream);
        inputStream.close();
    }

    @Test
    public void testMyBatis() throws IOException {
        SqlSession session = sqlSessionFactory.openSession();
        try {
            ArticleDao articleDao = session.getMapper(ArticleDao.class);
            List<Article> articles = articleDao.
                findByAuthorAndCreateTime("coolblog.xyz", "2018-06-10");
        } finally {
            session.commit();
            session.close();
        }
    }
}

```

在上面的测试代码中，prepare 方法用于创建 SqlSessionFactory 工厂，该工厂的用途是创建 SqlSession。通过 SqlSession 可为我们的数据库访问接口 ArticleDao 接口生成一个代理对象。MyBatis 会将接口方法 findByAuthorAndCreateTime 和 SQL 映射文件中配置的 SQL 关联起来，这样调用该方法等同于执行相关的 SQL。

上面测试代码的运行结果如下：



The screenshot shows a test execution window with the following content:

```

1 test passed - 1s 419ms © 2018 tianxiaobo.com
DEBUG [main] - ==> Preparing: SELECT `id`, `title`, `author`, `content`, `create_time` FROM `blog_article` WHERE `au
DEBUG [main] - ==> Parameters: coolblog.xyz(String), 2018-06-10(String)
TRACE [main] - <== Columns: id, title, author, content, create_time
TRACE [main] - <== Row: 6, Spring AOP 源码分析系列文章导读, coolblog.xyz, <<BLOB>>, 2018-06-17 00:00:00
TRACE [main] - <== Row: 7, Spring AOP 源码分析 - 创建代理对象, coolblog.xyz, <<BLOB>>, 2018-06-20 00:00:00
TRACE [main] - <== Row: 8, Spring MVC 原理探秘 - 一个请求的旅行过程, coolblog.xyz, <<BLOB>>, 2018-06-29 00:00:00
TRACE [main] - <== Row: 9, Spring MVC 原理探秘 - 容器的创建过程, coolblog.xyz, <<BLOB>>, 2018-06-30 00:00:00
DEBUG [main] - <== Total: 4

```


如上，大家在学习 MyBatis 框架时，可以配置一下 MyBatis 的日志，这样可把 MyBatis 的调试信息打印出来，方便观察 SQL 的执行过程。在上面的结果中，`==>` 符号所在的行表示向数据库中输入的 SQL 及相关参数。`<==` 符号所在的行则表示 SQL 的执行结果。上面输入输出不难看懂，这里就不多说了。

1.2.2 使用 JDBC 访问数据库

在初学 Java 编程阶段，多数朋友应该都是直接通过写 JDBC 代码访问数据库。这种方式的代码执行流程一般为加载数据库驱动，创建数据库连接对象、创建 Statement 对象、执行 SQL，以及处理结果集等，过程比较固定。下面我们再手写一遍 JDBC 代码，回忆一下初学 Java 的场景。

```
public class JdbcTest {

    @Test
    public void testJdbc() {
        String url = "jdbc:mysql://localhost:3306/myblog?user=root&.....";

        Connection conn = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(url);

            String author = "coolblog.xyz";
            String date = "2018.06.10";
            String sql = "SELECT id, title, author, content, create_time"
                + " FROM article"
                + " WHERE author = '" + author
                + "' AND create_time > '" + date + "'";

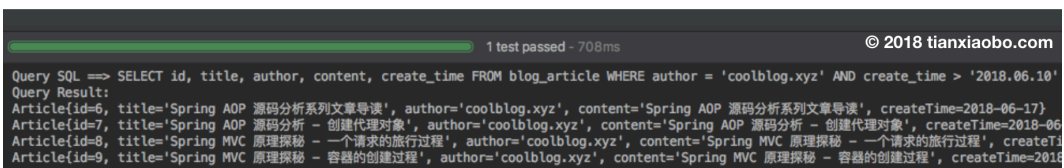
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql);
            List<Article> articles = new ArrayList<>(rs.getRow());
```

```

        while (rs.next()) {
            Article article = new Article();
            article.setId(rs.getInt("id"));
            article.setTitle(rs.getString("title"));
            article.setAuthor(rs.getString("author"));
            article.setContent(rs.getString("content"));
            article.setCreateTime(rs.getDate("create_time"));
            articles.add(article);
        }
        System.out.println("Query SQL ==> " + sql);
        System.out.println("Query Result: ");
        articles.forEach(System.out::println);
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}
}

```

代码比较简单，就不解释了。下面来看一下测试结果：



```

1 test passed - 708ms © 2018 tianxiaobo.com
Query SQL ==> SELECT id, title, author, content, create_time FROM blog_article WHERE author = 'coolblog.xyz' AND create_time > '2018.06.10'
Query Result:
Article{id=6, title='Spring AOP 源码分析系列文章导读', author='coolblog.xyz', content='Spring AOP 源码分析系列文章导读', createTime=2018-06-17}
Article{id=7, title='Spring AOP 源码分析 - 创建代理对象', author='coolblog.xyz', content='Spring AOP 源码分析 - 创建代理对象', createTime=2018-06-17}
Article{id=8, title='Spring MVC 原理探秘 - 一个请求的旅行过程', author='coolblog.xyz', content='Spring MVC 原理探秘 - 一个请求的旅行过程', createTime=2018-06-17}
Article{id=9, title='Spring MVC 原理探秘 - 容器的创建过程', author='coolblog.xyz', content='Spring MVC 原理探秘 - 容器的创建过程', createTime=2018-06-17}

```

上面代码的步骤比较多，但核心步骤只有两部，分别是执行 SQL 和处理查询结果。从开发人员的角度来说，我们也只关心这两个步骤。如果每次为了执行某个 SQL 都要写很多额外的代码。比如打开驱动，创建数据库连接，就显得很繁琐了。当然我们可以将这些额外的步骤封装起来，这样每次调用封装好的方法即可。这样确实可以解决代码繁琐，冗余的问题。不过，使用 JDBC 并非仅会导致代码繁琐，冗余的问题。在上面的代码中，我们通过字符串对 SQL 进行拼接。这样做会导致两个问题，第一是拼接 SQL 可能会导致 SQL 出错，比如少了个逗号或者多了个单引号等。第二是将 SQL 写在代码中，如果要改动 SQL，就需要到代码中进行更改。这样做是不合适的，因为改动 Java 代码就需要重新编译 Java 文件，然后再打包发布。同时，将 SQL 和 Java 代码混在一起，会降低代码的可读性，不利于维护。关于拼接 SQL 问题，是有相应的处理方法。比如可以使用 PreparedStatement，在解决拼接 SQL 问题的同时，还可解决 SQL 注入的问题。

除了上面所说的一些问题，直接使用 JDBC 访问数据库还会导致什么问题呢？这次我们将目光转移到执行结果的处理逻辑上。从上面的代码中可以看出，我们需要手动从 ResultSet 中取出数据，然后再设置到 Article 对象中。好在我们的 Article 属性不多，所以这样做看起来也没什么。假如 Article 对象有几十个属性，再用上面的方式接收查询结果，会非常的麻烦。而且可能还会因为属性太多，导致忘记设置某些属性。以上的代码还有一个问题，用户需要自行处理受检异常，这也是导致代码繁琐的一个原因。哦，还有一个问题，差点忘了。用户还需要手动管理数据库连接，开始要手动获取数据库连接。使用好后，又要手动关闭数据库连接。不得不说，真麻烦。

没想到直接使用 JDBC 访问数据库会有这么多的问题。但这并不意味着完全不可在项目直接 JDBC，应视情况而定。如果项目非常小，且对数据库依赖比较低。直接使用 JDBC 也很方便，不需要像 MyBatis 那样搞一堆配置了。

MyBatis VS JDBC

与 JDBC 相比，MyBatis 缺点比较明显，它的配置比较多，特别是 SQL 映射文件。如果一个大型项目中有几十上百个 Dao 接口，就需要有同等数量的 SQL 映射文件，这些映射文件需要用户自行维护。不过与 JDBC 相比，维护映射文件不是什么大问题。不然如果把同等数量的 SQL 像 JDBC 那样写在代码中，那维护的代价才叫大，出错的可能性也会增大。除了配置文件的问题，大家会发现使用 MyBatis 访问数据库好像过程也很繁琐啊。它的步骤大致如下：

1. 读取配置文件
2. 创建 SqlSessionFactoryBuilder 对象
3. 通过 SqlSessionFactoryBuilder 对象创建 SqlSessionFactory

4. 通过 SqlSessionFactory 创建 SqlSession
5. 为 Dao 接口生成代理类
6. 调用接口方法访问数据库

如果每次执行一个 SQL 要简历如上几步步骤, 那和 JDBC 比较起来, 也就没什么优势了。好在, 事实并非如此。SqlSessionFactoryBuilder 和 SqlSessionFactory 以及 SqlSession 等对象的作用域和生命周期是不一样的, 这一点在 MyBatis 官方文档中有所说明。我这里照搬一下, SqlSessionFactoryBuilder 对象用于构建 SqlSessionFactory, 只要构建好, SqlSessionFactoryBuilder 对象就可以丢弃了。SqlSessionFactory 是一个工厂类, 一旦被创建就应该在应用运行期间一直存在, 不应该丢弃或重建。SqlSession 不是线程安全的, 所以不应被多线程共享。官方推荐的使用方式是有按需创建, 用完即销毁。因此, 以上步骤中, 第 1、2 和第 3 步只需执行一次。第 4 和第 5 步需要进行多次创建。至于第 6 步, 这一步是必须的。相比之下, MyBatis 访问数据库的步骤要比 JDBC 简练不少。同时, 使用 MyBatis 无需处理受检异常, 比如 SQLException。另外, 把 SQL 写在配置文件中, 进行集中管理, 利于维护。同时将 SQL 从代码中剥离, 在提高代码的可读性的同时, 也避免了拼接 SQL 可能会导致的错误。除了上面所说这些, MyBatis 会将查询结果映射为相应的对象, 无需用户自行处理 ResultSet。

总的来说, MyBatis 在易用性上要比 JDBC 好太多。不过这里拿 MyBatis 和 JDBC 进行对比并不太合适。JDBC 作为 Java 平台的数据库访问规范, 它仅提供一种访问数据库的能力。至于使用者觉得 JDBC 流程繁琐, 还要自行处理异常等问题, 这些还真不怪 JDBC。比如 SQLException 这个异常, JDBC 没法处理, 也不应该处理, 抛给调用者处理也是理所应当。至于繁杂的步骤, 这仅是从使用者的角度考虑的, 从 JDBC 的角度来说, 这里的每个步骤对于完成一个数据访问请求来说都是必须的。至于 MyBatis, 它是构建在 JDBC 技术之上的, 对访问数据库的操作进行了简化, 方便用户使用。综上所述, JDBC 可看做是一种基础服务, MyBatis 则是构建在基础服务之上的框架, 它们的目标是不同的。

1.2.3 使用 SpringJDBC 访问数据库

上一节演示了 JDBC 访问数据的过程, 通过演示及分析, 大家应该感受到了直接使用 JDBC 的一些痛点。为了解决其中的一些痛点, SpringJDBC 应运而生。SpringJDBC 在 JDBC 基础上, 进行了一层比较薄的包装, 易用性得到了不少提升。那下面我们来看看如何使用 SpringJDBC。我们在使用 SpringJDBC 之前, 需要进行一些配置。这里我把配置信息放在了 application.xml 文件中, 后面写测试代码时, 让容器去加载这个配置。配置内容如下:

```
<beans>
    <context:property-placeholder location="jdbc.properties"/>

    <bean id="dataSource"
        class="org.springframework.jdbc.datasource.DriverManagerDataSource">
        <property name="driverClassName" value="${jdbc.driver}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>
    <bean id="jdbcTemplate"
        class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource" />
    </bean>
</beans>
```

如上, JdbcTemplate 封装了一些访问数据库的方法, 下面我们会通过此对象访问数据库。演示代码如下:

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("classpath:chapter1/application.xml")
public class SpringJdbcTest {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Test
    public void testSpringJdbc() {
        String author = "coolblog.xyz";
        String date = "2018.06.10";
        String sql = "SELECT id, title, author, content, create_time"
            + " FROM blog_article"
            + " WHERE author = '" + author
```

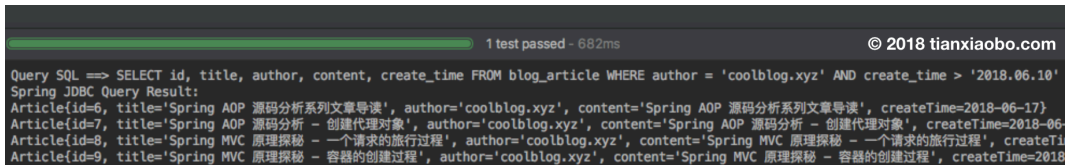
```

        + "' AND create_time > '" + date + "'";
    List<Article> articles = jdbcTemplate.query(sql, (rs, rowNum) -> {
        Article article = new Article();
        article.setId(rs.getInt("id"));
        article.setTitle(rs.getString("title"));
        article.setAuthor(rs.getString("author"));
        article.setContent(rs.getString("content"));
        article.setCreateTime(rs.getDate("create_time"));
        return article;
    });

    System.out.println("Query SQL ==> " + sql);
    System.out.println("Spring JDBC Query Result: ");
    articles.forEach(System.out::println);
}
}

```

测试结果如下：



```

1 test passed - 682ms
© 2018 tianxiaobo.com
Query SQL ==> SELECT id, title, author, content, create_time FROM blog_article WHERE author = 'coolblog.xyz' AND create_time > '2018.06.10'
Spring JDBC Query Result:
Article{id=6, title='Spring AOP 源码分析系列文章导读', author='coolblog.xyz', content='Spring AOP 源码分析系列文章导读', createTime=2018-06-17}
Article{id=7, title='Spring AOP 源码分析 - 创建代理对象', author='coolblog.xyz', content='Spring AOP 源码分析 - 创建代理对象', createTime=2018-06-17}
Article{id=8, title='Spring MVC 原理探秘 - 一个请求的旅行过程', author='coolblog.xyz', content='Spring MVC 原理探秘 - 一个请求的旅行过程', createTime=2018-06-17}
Article{id=9, title='Spring MVC 原理探秘 - 容器的创建过程', author='coolblog.xyz', content='Spring MVC 原理探秘 - 容器的创建过程', createTime=2018-06-17}

```

从上面的代码中可以看出，SpringJDBC 还是比较容易使用的。不过它也是存在一定缺陷的，比如 SQL 仍是写在代码中。又比如，对于较为复杂的结果（数据库返回的记录包含多列数据），需要用户自行处理 ResultSet 等。不过与 JDBC 相比，使用 SpringJDBC 无需手动加载数据库驱动，获取数据库连接，以及创建 Statement 对象等操作。总的来说，易用性上得到了不少的提升。

这里就不对比 SpringJDBC 和 MyBatis 的优缺点了。SpringJDBC 仅对 JDBC 进行了一层比较薄的封装，相关对比可以参考上一节的部分分析，这里不再赘述。

1.2.4 使用 Hibernate 访问数据库

本节会像之前的章节一样, 我会先写代码进行演示, 然后再对比 Hibernate 和 MyBatis 的区别。需要特别说明的是, 我在工作中没有用过 Hibernate, 对 Hibernate 也仅停留在了解的程度上。本节所用的测试代码都是现学现卖的, 可能有些地方写的会有问题, 或者不是最佳实践。因此, 关于测试代码, 大家看看就好。若有不妥之处, 也欢迎指出。

使用 Hibernate, 需要先进行环境配置, 主要是关于数据库方面的配置。这里为了演示, 我们简单配置一下。如下:

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/myblog?...
    </property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">****</property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQL5Dialect
    </property>
    <property name="hibernate.show_sql">true</property>

    <mapping resource="chapter1/mapping/Article.hbm.xml" />
  </session-factory>
</hibernate-configuration>
```

下面再配置一下实体类和表之间的映射关系, 也就是上面配置中出现的 Article.hbm.xml。不过这个配置不是必须的, 可用注解进行替换。

```
<hibernate-mapping package="xyz.coolblog.chapter1.model">
  <class table="blog_article" name="Article">
    <id name="id" column="id">
      <generator class="native" />
    </id>
    <property name="title" column="title" />
    <property name="author" column="author" />
    <property name="content" column="content" />
    <property name="createTime" column="create_time" />
  </class>
</hibernate-mapping>
```

测试代码如下：

```
public class HibernateTest {

    private SessionFactory buildSessionFactory;

    @Before
    public void init() {
        Configuration configuration = new Configuration();
        configuration.configure("chapter1/hibernate.cfg.xml");
        buildSessionFactory = configuration.buildSessionFactory();
    }

    @After
    public void destroy() {
        buildSessionFactory.close();
    }

    @Test
    public void testORM() {
        System.out.println("-----✧ ORM Query ✧-----");
        Session session = null;
    }
}
```



```
try {
    session = buildSessionFactory.openSession();
    int id = 6;
    Article article = session.get(Article.class, id);
    System.out.println("ORM Query Result: ");
    System.out.println(article);
    System.out.println();
} finally {
    if (Objects.nonNull(session)) {
        session.close();
    }
}
}

@Test
public void testHQL() {
    System.out.println("-----✧ HQL Query ✧+-----");
    Session session = null;
    try {
        session = buildSessionFactory.openSession();
        String hql = "from Article where author = :author"
            + " and create_time > :createTime";
        Query query = session.createQuery(hql);
        query.setParameter("author", "coolblog.xyz");
        query.setParameter("createTime", "2018.06.10");

        List<Article> articles = query.list();
        System.out.println("HQL Query Result: ");
        articles.forEach(System.out::println);
        System.out.println();
    } finally {
        if (Objects.nonNull(session)) {
```

```

        session.close();
    }
}

@Test
public void testJpaCriteria() throws ParseException {
    System.out.println("-----✧ JPA Criteria ✧-----");
    Session session = null;
    try {
        session = buildSessionFactory.openSession();
        CriteriaBuilder criteriaBuilder = session.getCriteriaBuilder();
        CriteriaQuery<Article> criteriaQuery =
            criteriaBuilder.createQuery(Article.class);

        Root<Article> article = criteriaQuery.from(Article.class);

        SimpleDateFormat sdf = new SimpleDateFormat("yyyy.MM.dd");
        Predicate greaterThan = criteriaBuilder.greaterThan(
            article.get("createTime"), sdf.parse("2018.06.10"));
        Predicate equal = criteriaBuilder
            .equal(article.get("author"), "coolblog.xyz");
        criteriaQuery.select(article).where(equal, greaterThan);

        Query<Article> query = session.createQuery(criteriaQuery);
        List<Article> articles = query.getResultList();
        System.out.println("JPA Criteria Query Result: ");
        articles.forEach(System.out::println);
    } finally {
        if (Objects.nonNull(session)) { session.close(); }
    }
}
}

```

这里我写了三种不同的查询方法，对于比较简单的查询，可以通过 OID 的方式进行，也就是 testORM 方法中对应的代码。这种方式不需要写 SQL，完全由 Hibernate 去生成。生成的 SQL 如下：

```
select
    article0_.id as id1_0_0_,
    article0_.title as title2_0_0_,
    article0_.author as author3_0_0_,
    article0_.content as content4_0_0_,
    article0_.create_time as create_t5_0_0_
from
    article article0_
where
    article0_.id=?
```

第二种方式是通过 HQL 进行查询，查询过程对应测试类中的 testHQL 方法。这种方式需要写一点 HQL，并为其设置相应的参数。最终生成的 SQL 如下：

```
select
    article0_.id as id1_0_,
    article0_.title as title2_0_,
    article0_.author as author3_0_,
    article0_.content as content4_0_,
    article0_.create_time as create_t5_0_
from
    article article0_
where
    article0_.author=? and create_time>?
```

第三种方式是通过 JPACriteria 进行查询，JPA Criteria 具有类型安全、面向对象和语义化的特点。使用 JPA Criteria，我们可以用写 Java 代码的方式进行数据库操作，无需手写 SQL。第二种方式和第三种方式进行的是同样的查询，所以生成的 SQL 区别不大，这里就不贴出来了。下面看一下测试代码的运行结果：

```

All 3 tests passed - 6s 820ms
© 2018 tianxiaobo.com

+ HQL Query +
Hibernate: select article0_.id as id1_0_, article0_.title as title2_0_, article0_.author as author3_0_, article0_.content as content4_0_, article0_.create_time as create_time5_0_ from article0_ where article0_.id=6
HQL Query Result:
Article{id=6, title='Spring AOP 源码分析系列文章导读', author='coolblog.xyz', content='Spring AOP 源码分析系列文章导读', createTime=2018-06-17 13:00:00.0}
Article{id=7, title='Spring AOP 源码分析 - 创建代理对象', author='coolblog.xyz', content='Spring AOP 源码分析 - 创建代理对象', createTime=2018-06-20 13:00:00.0}
Article{id=8, title='Spring MVC 原理揭秘 - 一个请求的旅行过程', author='coolblog.xyz', content='Spring MVC 原理揭秘 - 一个请求的旅行过程', createTime=2018-06-29 13:00:00.0}
Article{id=9, title='Spring MVC 原理揭秘 - 容器的创建过程', author='coolblog.xyz', content='Spring MVC 原理揭秘 - 容器的创建过程', createTime=2018-06-30 13:00:00.0}

+ ORM Query +
Hibernate: select article0_.id as id1_0_, article0_.title as title2_0_, article0_.author as author3_0_, article0_.content as content4_0_, article0_.create_time as create_time5_0_ from article0_ where article0_.id=6
ORM Query Result:
Article{id=6, title='Spring AOP 源码分析系列文章导读', author='coolblog.xyz', content='Spring AOP 源码分析系列文章导读', createTime=2018-06-17 13:00:00.0}

+ JPA Criteria +
Hibernate: select article0_.id as id1_0_, article0_.title as title2_0_, article0_.author as author3_0_, article0_.content as content4_0_, article0_.create_time as create_time5_0_ from article0_ where article0_.id=6
JPA Criteria Query Result:
Article{id=6, title='Spring AOP 源码分析系列文章导读', author='coolblog.xyz', content='Spring AOP 源码分析系列文章导读', createTime=2018-06-17 13:00:00.0}
Article{id=7, title='Spring AOP 源码分析 - 创建代理对象', author='coolblog.xyz', content='Spring AOP 源码分析 - 创建代理对象', createTime=2018-06-20 13:00:00.0}
Article{id=8, title='Spring MVC 原理揭秘 - 一个请求的旅行过程', author='coolblog.xyz', content='Spring MVC 原理揭秘 - 一个请求的旅行过程', createTime=2018-06-29 13:00:00.0}
Article{id=9, title='Spring MVC 原理揭秘 - 容器的创建过程', author='coolblog.xyz', content='Spring MVC 原理揭秘 - 容器的创建过程', createTime=2018-06-30 13:00:00.0}

```

MyBatis VS Hibernate

在 Java 中，就持久层框架来说，MyBatis 和 Hibernate 都是很热门的框架。关于这两个框架孰好孰坏，在网上也有很广泛的讨论。这两个框架之间的区别比较大，下面我们来聊聊。

从映射关系上来说，Hibernate 是把实体类（POJO）和表进行了关联，是一种完整的 ORM(O/Rmapping)框架。而 MyBatis 则是将数据访问接口（Dao）与 SQL 进行了关联，本质上算是一种 SQL 映射。从使用的角度来说，使用 Hibernate 通常不需要写 SQL，让框架自动生成即可。但 MyBatis 则不行，再简单的数据库访问操作都需要有与之对应的 SQL。另一方面，由于 Hibernate 可自动生成 SQL，所以进行数据库移植时，代价要小一点。而由于使用 MyBatis 需要手写 SQL，加之 SQL 方言的存在。这就导致在进行数据库移植时，出现 SQL 无法正常使用的情况，此时需要根据 SQL 方言修改 SQL。不过好在移植数据库的情况很少见，这个差异可以忽略。

上面从两个维度对 Hibernate 和 MyBatis 进行了对比，但目前也只是说了他们的一些不同点。下面我们来分析一下这两个框架的适用场景。

Hibernate 可自动生成 SQL，降低使用成本。但同时也要意识到，这样做也是有代价的，会损失灵活性。比如，如果当我们需要手动优化 SQL 时，会发现很难改变 Hibernate 自动生成的 SQL。因此对于 Hibernate 来说，它适用于一些需求比较稳定，变化比较小的项目，譬如 OA、CRM 等。

与 Hibernate 相反，MyBatis 需要手动维护 SQL，这会增加使用成本。但同时，使用者可灵活控制 SQL 的行为，这为改动和优化 SQL 提供了便利性。所以 MyBatis 适合应用在一些需要快速迭代，需求变化大的项目中，这也就是为什么 MyBatis 在互联网公司中使用的比较广泛的原因。除此之外，MyBatis 还提供了插件机制，使用者可以按需定制插件。这也是 MyBatis 灵活性的一个体现。

1.3 如何使用 MyBatis

本节,我们一起来了解一下如何使用 MyBatis 框架。在上一节中,我简单演示了 MyBatis 的使用方法。不过那个示例太简单,本节演示一个略为复杂的例子。这个例子复杂度和真实的项目还是有一些差距的,仅做演示使用。本节除了演示单独使用 MyBatis 的过程,还会将 MyBatis 与 Spring 进行整合,演示在 Spring 中使用 MyBatis 的过程。

1.3.1 单独使用 MyBatis

本节演示的场景是个人网站的作者和文章之间的关联场景。在一个网站中,一篇文章对应一名作者,一个作者对应多篇文章。下面我们来看一下作者和文章的定义,如下:

```
public class Author {
    private Integer id;
    private String name;
    private Integer age;
    private SexEnum sex;
    private String email;
    private List<ArticleDO> articles;
}

public class Article {
    private Integer id;
    private String title;
    private ArticleTypeEnum type;
    private AuthorDO author;
    private String content;
    private Date createTime;
}
```

如上, Author 中包含了一组对 Article 的引用,这是一对多的关系。Article 中则包含了一个对 Author 的引用,这是一对一的关系。除此之外,这里使用了两个常量,一个用于表示性别,另一个用于表示文章类型,它们的定义如下:

```
public enum SexEnum {  
    MAN,  
    FEMALE,  
    UNKNOWN;  
}  
  
public enum ArticleTypeEnum {  
    JAVA(1),  
    DUBBO(2),  
    SPRING(4),  
    MYBATIS(8);  
  
    private int code;  
  
    ArticleTypeEnum(int code) {  
        this.code = code;  
    }  
  
    public int code() {  
        return code;  
    }  
  
    public static ArticleTypeEnum find(int code) {  
        for (ArticleTypeEnum at : ArticleTypeEnum.values()) {  
            if (at.code == code) {  
                return at;  
            }  
        }  
  
        return null;  
    }  
}
```

本节使用了两张表，分别用于存储文章和作者信息。这两种表的内容如下：

id	author_id	title	type	content	create_time
1	1	MyBatis 源码分析系列文章导读	8	MyBatis 源码分析系列文章导读	2018-07-15 15:30:09
2	2	HashMap 源码详细分析(JDK1.8)	1	HashMap 源码详细分析(JDK1.8)	2018-01-18 15:29:13
3	1	Java CAS 原理分析	1	Java CAS 原理分析	2018-05-15 15:28:33
4	1	Spring IOC 容器源码分析 - 获取单例 bean	4	Spring IOC 容器源码分析 - 获取单例 bean	2018-06-01 00:00:00
5	1	Spring IOC 容器源码分析 - 循环依赖的解决办法	4	Spring IOC 容器源码分析 - 循环依赖的解决办法	2018-06-08 00:00:00
6	2	Spring AOP 源码分析系列文章导读	4	Spring AOP 源码分析系列文章导读	2018-06-17 00:00:00
7	2	Spring AOP 源码分析 - 创建代理对象	4	Spring AOP 源码分析 - 创建代理对象	2018-06-20 00:00:00
8	1	Spring MVC 原理探秘 - 一个请求的旅行过程	4	Spring MVC 原理探秘 - 一个请求的旅行过程	2018-06-29 00:00:00
9	2	Spring MVC 原理探秘 - 容器的创建过程	4	Spring MVC 原理探秘 - 容器的创建过程	2018-06-30 00:00:00
10	2	Spring IOC 容器源码分析系列文章导读	4	Spring IOC 容器源码分析系列文章导读	2018-05-30 00:00:00

table: article

id	name	age	sex	email
1	coolblog.xyz	28	0	coolblog.xyz@outlook.com
2	nullllun	29	1	coolblog.xyz@outlook.com

table: author

下面来看一下数据库访问层的接口定义，如下：

```
public interface ArticleDao {
    Article findOne(@Param("id") int id);
}

public interface AuthorDao {
    Author findOne(@Param("id") int id);
}
```

与这两个接口对应的 SQL 被配置在了下面的两个映射文件中。我们先来看第一个映射文件 AuthorMapper.xml 的内容。

```
<!-- AuthorMapper.xml -->
<mapper namespace="xyz.coolblog.chapter1.dao2.AuthorDao">
    <resultMap id="articleResult" type="Article">
        <id property="id" column="article_id" />
        <result property="title" column="title"/>
        <result property="type" column="type"/>
    </resultMap>

```

```

    <result property="content" column="content"/>
    <result property="createTime" column="create_time"/>
</resultMap>

<resultMap id="authorResult" type="Author">
    <id property="id" column="id"/>
    <result property="name" column="name"/>
    <result property="age" column="age"/>
    <result property="sex" column="sex"
        typeHandler="org.apache.ibatis.type.EnumOrdinalTypeHandler"/>
    <result property="email" column="email"/>
    <collection property="articles" resultMap="articleResult"/>
</resultMap>

<select id="findOne" resultMap="authorResult">
    SELECT
        au.id, au.name, au.age, au.sex, au.email, ar.id as article_id,
        ar.title, ar.type, ar.content, ar.create_time
    FROM
        author au, article ar
    WHERE
        au.id = ar.author_id AND au.id = #{id}
</select>
</mapper>

```

注意看上面的 `<resultMap>` 配置, 这个标签中包含了一个一对多的配置 `<collection>`, 这个配置引用了一个 id 为 `articleResult` 的。除了要注意一对多的配置, 这里还要下面这行配置:

```

<result property="sex" column="sex"
    typeHandler="org.apache.ibatis.type.EnumOrdinalTypeHandler"/>

```

前面说过 `Author` 的 `sex` 属性是一个枚举, 但这个属性在数据表中是以整型值进行存储的。所以向数据表写入或者查询数据时, 要进行类型转换。写入时, 需要将 `SexEnum` 转成

int。查询时，则需要把 int 转成 SexEnum。由于这两个是完全不同的类型，不能通过强转进行转换，所以需要使用一个中间类进行转换，这个中间类就是 EnumOrdinalTypeHandler。这个类会按照枚举顺序进行转换，比如在 SexEnum 中，MAN 的顺序是 0。存储时，EnumOrdinalTypeHandler 会将 MAN 替换为 0。查询时，又会将 0 转换为 MAN。除了 EnumOrdinalTypeHandler，MyBatis 还提供了另一个枚举类型处理器 EnumTypeHandler。这个则是按照枚举的字面值进行转换，比如该处理器可将枚举 MAN 与字符串"MAN"进行相互转换。

下面来看一下 ArticleMapper.xml 的配置内容：

```
<!-- ArticleMapper.xml -->
<mapper namespace="xyz.coolblog.chapter1.dao2.ArticleDao">
    <resultMap id="authorResult" type="Author">
        <id property="id" column="author_id"/>
        <result property="name" column="name"/>
        <result property="age" column="age"/>
        <result property="sex" column="sex"
            typeHandler="org.apache.ibatis.type.EnumOrdinalTypeHandler"/>
        <result property="email" column="email"/>
    </resultMap>

    <resultMap id="articleResult" type="Article">
        <id property="id" column="id" />
        <result property="title" column="title"/>
        <result property="type" column="type"
            typeHandler="xyz.coolblog.mybatis.ArticleTypeHandler"/>
        <result property="content" column="content"/>
        <result property="createTime" column="create_time"/>
        <association property="author" javaType="Author"
            resultMap="authorResult"/>
    </resultMap>

    <select id="findOne" resultMap="articleResult">
        SELECT
            ar.id, ar.author_id, ar.title, ar.type, ar.content,
            ar.create_time, au.name, au.age, au.sex, au.email
    </select>
</mapper>
```

```
FROM
    article ar, author au
WHERE
    ar.author_id = au.id AND ar.id = #{id}
</select>
</mapper>
```

如上，ArticleMapper.xml 中包含了一个一对一的配置 <association>，这个配置引用了另一个 id 为 authorResult 的。除了一对一的配置外，这里还有一个自定义类型处理器 ArticleTypeHandler 需要大家注意。这个自定义类型处理器用于处理 ArticleTypeEnum 枚举类型。大家如果注意看前面贴的 ArticleTypeEnum 的源码，会发现每个枚举值有自己的编号定义。比如 JAVA 的编号为 1，DUBBO 的编号为 2，SPRING 的编号为 8。因此这里我们不能再使用 EnumOrdinalTypeHandler 对 ArticleTypeHandler 进行类型转换，需要自定义一个类型转换器。下面我们来看一下这个类型转换器的定义。

```
public class ArticleTypeHandler extends BaseTypeHandler<ArticleTypeEnum> {

    @Override
    public void setNonNullParameter(PreparedStatement ps, int i,
        ArticleTypeEnum parameter, JdbcType jdbcType) throws SQLException {
        // 获取枚举的 code 值，并设置到 PreparedStatement 中
        ps.setInt(i, parameter.code());
    }

    @Override
    public ArticleTypeEnum getNullableResult(
        ResultSet rs, String columnName) throws SQLException {
        // 从 ResultSet 中获取 code
        int code = rs.getInt(columnName);
        // 解析 code 对应的枚举，并返回
        return ArticleTypeEnum.find(code);
    }
}
```

```
@Override
public ArticleTypeEnum getNullableResult(ResultSet rs, int columnIndex)
    throws SQLException {
    int code = rs.getInt(columnIndex);
    return ArticleTypeEnum.find(code);
}

@Override
public ArticleTypeEnum getNullableResult(CallableStatement cs,
    int columnIndex) throws SQLException {
    int code = cs.getInt(columnIndex);
    return ArticleTypeEnum.find(code);
}
}
```

若需实现自定义类型处理器，可继承 `BaseTypeHandler`，并实现相关的抽象方法。上面的代码比较简单，我也进行了一些注释，就不过多分析了。前面贴了实体类，数据访问类，以及 SQL 映射文件。最后还差一个 MyBatis 的配置文件，这里也贴出来。如下：

```
<!-- mybatis-config2.xml -->
<configuration>
    <properties resource="jdbc.properties"/>

    <typeAliases>
        <typeAlias alias="Article"
            type="xyz.coolblog.chapter1.model2.ArticleDO"/>
        <typeAlias alias="Author"
            type="xyz.coolblog.chapter1.model2.AuthorDO"/>
    </typeAliases>

    <typeHandlers>
        <typeHandler handler="xyz.coolblog.mybatis.ArticleTypeHandler"
            javaType="xyz.coolblog.constant.ArticleTypeEnum"/>
    </typeHandlers>
</configuration>
```

```
</typeHandlers>
<environments default="development">
  <environment id="development">
    <transactionManager type="JDBC"/>
    <dataSource type="POOLED">
      <property name="driver" value="${jdbc.driver}"/>
      <property name="url" value="${jdbc.url}"/>
      <property name="username" value="${jdbc.username}"/>
      <property name="password" value="${jdbc.password}"/>
    </dataSource>
  </environment>
</environments>

<mappers>
  <mapper resource="chapter1/mapper2/AuthorMapper.xml"/>
  <mapper resource="chapter1/mapper2/ArticleMapper.xml"/>
</mappers>
</configuration>
```

下面通过一个表格简单解释配置中出现的一些标签。

标签名称	用途
properties	用于配置全局属性，这样在配置文件中，可以通过占位符 \${} 进行属性值配置
typeAliases	用于定义别名。如上所示，这里把 xyz.coolblog.model.ArticleDO 的别名定义为 Article，这样在 SQL 映射文件中，就可以直接使用别名，而不用每次都输入长长的全限定类名
typeHandlers	用于定义全局的类型处理器，如果这里配置了，SQL 映射文件中就不需要再次进行配置。前面为了讲解需要，我在 SQL 映射文件中也配置了 ArticleTypeHandler，其实是多余的
environments	用于配置事务，以及数据源
mappers	用于配置 SQL 映射文件的位置信息

以上仅介绍了一些比较常用的配置，更多的配置信息，建议大家去阅读 [MyBatis 官方文档](#)。到这里，我们把所有的准备工作都做完了。那么接下来，写点测试代码测试一下。

```
public class MyBatisTest2 {

    private SqlSessionFactory sqlSessionFactory;

    @Before
    public void prepare() throws IOException {
        String resource = "chapter1/mybatis-config2.xml";
        InputStream inputStream = Resources.getResourceAsStream(resource);
        sqlSessionFactory =
            new SqlSessionFactoryBuilder().build(inputStream);
        inputStream.close();
    }

    @Test
    public void testOne2One() {
        SqlSession session = sqlSessionFactory.openSession();
        try {
            ArticleDao articleDao = session.getMapper(ArticleDao.class);
            ArticleDO article = articleDao.findOne(1);

            AuthorDO author = article.getAuthor();
            article.setAuthor(null);
            System.out.println("\nauthor info:");
            System.out.println(author);
            System.out.println("\narticles info:");
            System.out.println(article);
        } finally {
            session.close();
        }
    }
}
```

```

@Test
public void testOne2Many() {
    SqlSession session = sqlSessionFactory.openSession();
    try {
        AuthorDao authorDao = session.getMapper(AuthorDao.class);
        AuthorDO author = authorDao.findOne(1);

        List<ArticleDO> arts = author.getArticles();
        List<ArticleDO> articles =
            Arrays.asList(arts.toArray(new ArticleDO[arts.size()]));
        arts.clear();

        System.out.println("\nauthor info:");
        System.out.println(author);
        System.out.println("\narticles info:");
        articles.forEach(System.out::println);
    } finally {
        session.close();
    }
}
}

```

第一个测试方法用于从数据库中查询某篇文章和相应的作者信息。它的运行结果如下：

```

1 test passed - 1s 662ms
© 2018 tianxiaobo.com
DEBUG [main] - ==> Preparing: SELECT ar.id, ar.author_id AS artilece_author_id, ar.title, ar.type, ar.content, ar.create_time, au.id AS author_id, au.name
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, artilece_author_id, title, type, content, create_time, author_id, name, age, sex, email
TRACE [main] - <== Row: 1, 1, MyBatis 源码分析系列文章导读, 8, <<BLOB>>, 2018-07-15 15:38:09, 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com
DEBUG [main] - <== Total: 1

author info:
Author{id=1, name='tianxiaobo', age=28, sex=MAN, email='coolblog.xyz@outlook.com', articles=null}

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', author=null, content='MyBatis 源码分析系列文章导读', type=MYBATIS, createTime=Mon Jul 16 04:38:09 CST 2018}

```

第二个测试方法用于查询某位作者，及其所写的文章的信息。它的运行结果如下：

```

1 test passed - 1s 600ms
© 2018 tianxiaobo.com

DEBUG [main] - ==> Preparing: SELECT au.id, au.name, au.age, au.sex, au.email, ar.id as article_id, ar.title, ar.type, ar.content, ar.create_time FROM author au,
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, name, age, sex, email, article_id, title, type, content, create_time
TRACE [main] - <== Row: 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com, 1, MyBatis 源码分析系列文章导读, 0, <<BLOB>>, 2018-07-15 15:30:09
TRACE [main] - <== Row: 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com, 3, Java CAS 原理分析, 1, <<BLOB>>, 2018-05-15 15:28:33
TRACE [main] - <== Row: 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com, 4, Spring IOC 容器源码分析 - 获取单例 bean, 4, <<BLOB>>, 2018-06-01 00:00:00
TRACE [main] - <== Row: 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com, 5, Spring IOC 容器源码分析 - 循环依赖的解决办法, 4, <<BLOB>>, 2018-06-08 00:00:00
TRACE [main] - <== Row: 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com, 8, Spring MVC 原理揭秘 - 一个请求的旅行过程, 4, <<BLOB>>, 2018-06-29 00:00:00
DEBUG [main] - <== Total: 5

author info:
Author{id=1, name='tianxiaobo', age=28, sex=MAN, email='coolblog.xyz@outlook.com', articles=[]}

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', author=null, content='MyBatis 源码分析系列文章导读', type=MYBATIS, createTime=Mon Jul 16 04:30:09 CST 2018}
Article{id=3, title='Java CAS 原理分析', author=null, content='Java CAS 原理分析', type=JAVA, createTime=Wed May 16 04:28:33 CST 2018}
Article{id=4, title='Spring IOC 容器源码分析 - 获取单例 bean', author=null, content='Spring IOC 容器源码分析 - 获取单例 bean', type=SPRING, createTime=Fri Jun 01 13:00:00 CST 2018}
Article{id=5, title='Spring IOC 容器源码分析 - 循环依赖的解决办法', author=null, content='Spring IOC 容器源码分析 - 循环依赖的解决办法', type=SPRING, createTime=Fri Jun 08 00:00:00 CST 2018}
Article{id=8, title='Spring MVC 原理揭秘 - 一个请求的旅行过程', author=null, content='Spring MVC 原理揭秘 - 一个请求的旅行过程', type=SPRING, createTime=Fri Jun 29 13:00:00 CST 2018}

```

到此，关于 MyBatis 的使用方法就演示完了。以上实例仅演示了 MyBatis 框架的一些常规特性，对于一些较为高级的特性，比如 MyBatis 的插件机制，分页，以及缓存等均未涉及到。对于一些比较生僻的特性，比如对象工厂，鉴别器等，本节内容也无法覆盖到。因此，本节所演示的实例仅是一个常规实例，并非完整示例，望周知。

1.3.2 在 Spring 中使用 MyBatis

在实际开发中，我们一般会将 MyBatis 和 Spring 整合在一起使用。这样，我们可以通过 bean 注入的方式使用各种 Dao 接口。MyBatis 和 Spring 原本是两个完全不相关的框架，要想把两者整合起来，需要一个中间框架。这个框架一方面负责加载和解析 MyBatis 相关配置。另一方面，该框架还会通过 Spring 提供的拓展点，把各种 Dao 接口及其对应的对象放入 bean 工厂中。这样，我们才可以通过 bean 注入的方式获取到这些 Dao 接口对应的 bean。那么问题来了，具有如此能力的框架是谁呢？答案是 mybatis-spring。

接下来，通过一个实例，演示两个框架的整合过程。我的测试项目是基于 Maven 构建的，所以这里先来看一下 pom 文件的内容。

```

<project>
  <properties>
    <spring.version>4.3.17.RELEASE</spring.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.mybatis</groupId>
      <artifactId>mybatis</artifactId>
      <version>3.4.6</version>
    </dependency>
  </dependencies>
</project>

```

```
</dependency>
<dependency>
    <groupId>org.mybatis</groupId>
    <artifactId>mybatis-spring</artifactId>
    <version>1.3.2</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-core</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-beans</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>${spring.version}</version>
</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-test</artifactId>
    <version>${spring.version}</version>
    <scope>test</scope>
</dependency>
</dependencies>
</project>
```


为了减少配置文件所占的文章篇幅,上面的配置经过了一定的简化。仅列出了与 MyBatis 和 Spring 相关依赖的坐标,其他依赖忽略。下面将 MyBatis 中的一些类配置到 Spring 的配置文件中。

如上,上面就是将 MyBatis 整合到 Spring 中所需的一些配置。这里,我们将数据源配置到 Spring 配置文件中。配置完数据源,接下来配置 SqlSessionFactory, SqlSessionFactory 的用途大家都知道,不用过多解释了。再接下来是配置 MapperScannerConfigurer,这个类用于扫描某个包下的数据访问接口,并将这些接口注册到 Spring 容器中。这样,我们就可以在其他的 bean 中注入 Dao 接口的实现类,无需再从 SqlSession 中获取接口实现类。

```
<!-- application-mybatis.xml -->
<beans>
    <context:property-placeholder location="jdbc.properties"/>

    <!-- 配置数据源 -->
    <bean id="dataSource"
        class="org.apache.ibatis.datasource.pooled.PooledDataSource">
        <property name="driver" value="${jdbc.driver}" />
        <property name="url" value="${jdbc.url}" />
        <property name="username" value="${jdbc.username}" />
        <property name="password" value="${jdbc.password}" />
    </bean>

    <!-- 配置 SqlSessionFactory -->
    <bean id="sqlSessionFactory"
        class="org.mybatis.spring.SqlSessionFactoryBean">
        <!-- 配置 mybatis-config.xml 路径 -->
        <property name="configLocation"
            value="classpath:chapter1/mybatis-config3.xml"/>
        <!-- 给 SqlSessionFactory 配置数据源, 这里引用上面的数据源配置 -->
        <property name="dataSource" ref="dataSource"/>
        <!-- 配置 SQL 映射文件 -->
        <property name="mapperLocations" value="chapter1/mapper2/*.xml"/>
    </bean>
```

```
<!-- 配置 MapperScannerConfigurer -->
<bean class="org.mybatis.spring.mapper.MapperScannerConfigurer">
    <!-- 配置 Dao 接口所在的包 -->
    <property name="basePackage" value="xyz.coolblog.chapter1.dao2"/>
</bean>
</beans>
```

将 MyBatis 配置到 Spring 中后，为了让我们的程序正常运行，这里还需要为 MyBatis 提供一份配置。相关配置如下：

```
<!-- mybatis-config3.xml -->
<configuration>
    <settings>
        <setting name="cacheEnabled" value="true"/>
    </settings>

    <typeAliases>
        <typeAlias alias="Article"
            type="xyz.coolblog.chapter1.model2.Article"/>
        <typeAlias alias="Author"
            type="xyz.coolblog.chapter1.model2.Author"/>
    </typeAliases>

    <typeHandlers>
        <typeHandler handler="xyz.coolblog.mybatis.ArticleTypeHandler"
            javaType="xyz.coolblog.constant.ArticleTypeEnum"/>
    </typeHandlers>
</configuration>
```

这里的 mybatis-config.xml 和上一节的配置不太一样，移除了数据源和 SQL 映射文件路径的配置。需要注意的是，<settings> 必须配置在 mybatis-config.xml 中。其他的配置都不是必须项，可放在 Spring 的配置文件中，这里偷了个懒。

到此，Spring 整合 MyBatis 的配置工作就完成了，接下来写点测试代码跑跑看。

```
@RunWith(SpringJUnit4ClassRunner.class)
@ContextConfiguration("classpath:chapter1/application-mybatis.xml")
public class SpringWithMyBatisTest implements ApplicationContextAware {

    private ApplicationContext applicationContext;
    /** 自动注入 AuthorDao, 无需再通过 SqlSession 获取 */
    @Autowired
    private AuthorDao authorDao;
    @Autowired
    private ArticleDao articleDao;

    @Before
    public void printBeanInfo() {
        ListableBeanFactory lbf = applicationContext;
        String[] beanNames = lbf.getBeanDefinitionNames();
        Arrays.sort(beanNames);
        System.out.println("\n-----☆ bean name ☆-----");
        Arrays.asList(beanNames).subList(0, 5).forEach(System.out::println);
        AuthorDao authorDao =
            (AuthorDao) applicationContext.getBean("authorDao");
        ArticleDao articleDao =
            (ArticleDao) applicationContext.getBean("articleDao");
        System.out.println("\n-----☆ bean class info ☆-----");
        System.out.println("AuthorDao Class: " + authorDao.getClass());
        System.out.println("ArticleDao Class: " + articleDao.getClass());
        System.out.println("\n-----xxxx-----xxx-----\n");
    }

    @Test
    public void testOne2One() {
        Article article = articleDao.findOne(1);
        Author author = article.getAuthor();
        article.setAuthor(null);
    }
}
```

```

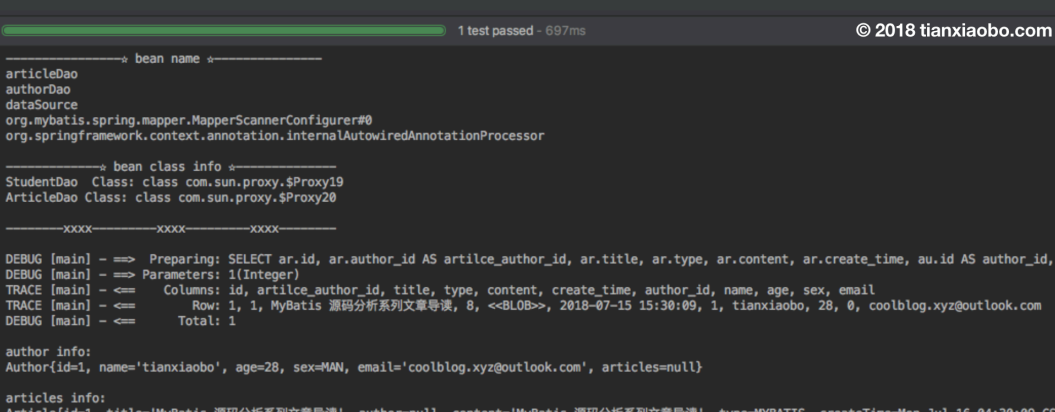
        System.out.println("\nauthor info:");
        System.out.println(author);
        System.out.println("\narticles info:");
        System.out.println(article);
    }

    @Test
    public void testOne2Many() {
        Author author = authorDao.findOne(1);
        System.out.println("\nauthor info:");
        System.out.println(author);
        System.out.println("\narticles info:");
        author.getArticles().forEach(System.out::println);
    }

    @Override
    public void setApplicationContext(ApplicationContext applicationContext)
        throws BeansException {
        this.applicationContext = applicationContext;
    }
}

```

如上代码，为了证明我们的整合配置生效了，上面专门写了一个方法，用于输出 ApplicationContext 中 bean 的信息。下面来看一下 testOne2One 测试方法的输出结果。



```

1 test passed - 697ms
© 2018 tianxiaobo.com

----- bean name -----
articleDao
authorDao
dataSource
org.mybatis.spring.mapper.MapperScannerConfigurer#0
org.springframework.context.annotation.internalAutowiredAnnotationProcessor

----- bean class info -----
StudentDao Class: class com.sun.proxy.$Proxy19
ArticleDao Class: class com.sun.proxy.$Proxy20

-----xxxx-----xxxx-----xxxx-----
DEBUG [main] - ==> Preparing: SELECT ar.id, ar.author_id AS artilece_author_id, ar.title, ar.type, ar.content, ar.create_time, au.id AS author_id,
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, artilece_author_id, title, type, content, create_time, author_id, name, age, sex, email
TRACE [main] - <== Row: 1, 1, MyBatis 源码分析系列文章导读, 8, <<BLOB>>, 2018-07-15 15:30:09, 1, tianxiaobo, 28, 0, coolblog.xyz@outlook.com
DEBUG [main] - <== Total: 1

author info:
Author{id=1, name='tianxiaobo', age=28, sex=MAN, email='coolblog.xyz@outlook.com', articles=null}

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', author=null, content='MyBatis 源码分析系列文章导读', type=MYBATIS, createTime=Mon Jul 16 04:30:09 CS

```

如上所示, beannname 的前两行就是我们的 Dao 接口的名称, 它们的实现类则是 JDK 的动态代理生成的。同时 testOne2One 方法也正常运行了, 由此可知, 我们的整合配置生效了。

1.4 本章小结

本章介绍了 MyBatis 的一些背景知识, 并比较了 JDBC、SpringJDBC、Hibernate 和 MyBatis 等 4 种持久层技术的优缺点, 以及各自的使用场景。最后演示了单独使用 MyBatis, 和在 Spring 中使用 MyBatis 等两种不同的方式。总的来说, 本章内容应该较为充分的解答了 MyBatis 是什么, 为何要使用, 以及如何使用这三个问题。

第 2 章

配置文件解析过程

从本章开始，正式进入源码分析阶段。按照 MyBatis 启动流程，本章将会对 MyBatis 解析配置文件的过程进行分析。我们在使用 MyBatis 框架时，通常会进行一定的设置，使其能更好的满足我们的需求。对于一个框架来说，提供较为丰富的配置文件，也是其灵活性的体现。本章将会介绍 MyBatis 配置文件中的大部分节点解析过程，包括但不限于 properties、settings、typeAliases、typeHandlers 等，对于一些生僻的配置，本章将不会进行分析，比如 objectFactory、objectWrapperFactory 等。对于这部分内容，大家若是感兴趣，可自行分析。

2.1 配置文件解析过程分析

我们在使用 MyBatis 时，第一步要做的事情一般是根据配置文件构建 SqlSessionFactory 对象。相关代码大致如下：

```
String resource = "mybatis-config.xml";
InputStream inputStream = Resources.getResourceAsStream(resource);
SqlSessionFactory sqlSessionFactory =
    new SqlSessionFactoryBuilder().build(inputStream);
```

在上面代码中，我们首先会使用 MyBatis 提供的工具类 Resources 加载配置文件，得到一个输入流。然后再通过 SqlSessionFactoryBuilder 对象的 build 方法构建 SqlSessionFactory 对象。这里的 build 方法是我们分析配置文件解析过程的入口方法。下面我们来看一下这个方法的代码：

```
// -☆- SqlSessionFactoryBuilder
public SqlSessionFactory build(InputStream inputStream) {
    // 调用重载方法
    return build(inputStream, null, null);
}

public SqlSessionFactory build(InputStream inputStream, String environment,
Properties properties) {
    try {
        // 创建配置文件解析器
        XMLConfigBuilder parser =
            new XMLConfigBuilder(inputStream, environment, properties);
        // 调用 parse 方法解析配置文件，生成 Configuration 对象
        return build(parser.parse());
    } catch (Exception e) {
        throw ExceptionFactory.wrapException(".....", e);
    } finally {
        ErrorContext.instance().reset();
        try {
            inputStream.close();
        } catch (IOException e) {.....}
    }
}

public SqlSessionFactory build(Configuration config) {
    // 创建 DefaultSqlSessionFactory
    return new DefaultSqlSessionFactory(config);
}
```

从上面的代码中,我们大致可以猜出 MyBatis 配置文件是通过 XMLConfigBuilder 进行解析的。不过目前这里还没有非常明确的解析逻辑,所以我们继续往下看。这次来看一下 XMLConfigBuilder 的 parse 方法,如下:

```
// -☆- XMLConfigBuilder
public Configuration parse() {
    if (parsed) {
        throw new BuilderException(".....");
    }
    parsed = true;
    // 解析配置
    parseConfiguration(parser.evalNode("/configuration"));
    return configuration;
}
```

到这里大家可以看到一些端倪了,注意一个 xpath 表达式—— /configuration。这个表达式代表的是 MyBatis 配置文件的 <configuration> 节点,这里通过 xpath 选中这个节点,并传递给 parseConfiguration 方法。我们继续跟下去。

```
private void parseConfiguration(XNode root) {
    try {
        // 解析 properties 配置
        propertiesElement(root.evalNode("properties"));

        // 解析 settings 配置,并将其转换为 Properties 对象
        Properties settings =
            settingsAsProperties(root.evalNode("settings"));

        // 加载 vfs
        loadCustomVfs(settings);

        // 解析 typeAliases 配置
        typeAliasesElement(root.evalNode("typeAliases"));
    }
```



```
// 解析 plugins 配置
pluginElement(root.evalNode("plugins"));

// 解析 objectFactory 配置
objectFactoryElement(root.evalNode("objectFactory"));

// 解析 objectWrapperFactory 配置
objectWrapperFactoryElement(root.evalNode("objectWrapperFactory"));

// 解析 reflectorFactory 配置
reflectorFactoryElement(root.evalNode("reflectorFactory"));

// settings 中的信息设置到 Configuration 对象中
settingsElement(settings);

// 解析 environments 配置
environmentsElement(root.evalNode("environments"));

// 解析 databaseIdProvider, 获取并设置 databaseId 到 Configuration 对象
databaseIdProviderElement(root.evalNode("databaseIdProvider"));

// 解析 typeHandlers 配置
typeHandlerElement(root.evalNode("typeHandlers"));
// 解析 mappers 配置
mapperElement(root.evalNode("mappers"));
} catch (Exception e) {
    throw new BuilderException(".....");
}
}
```

到此，一个完整的配置解析过程就呈现出来了，每个节点的解析逻辑均封装在了相应的方法中。我在分析这些方法时，并不会按照上面代码中所呈现的解析顺序进行分析，而是做了一定的调整。

2.1.1 解析<properties>节点

<properties>节点的解析工作由 propertiesElement 这个方法完成的, 在分析方法的源码前, 我们先来看一下<properties>节点是如何配置的。如下:

```
<properties resource="jdbc.properties">
    <property name="jdbc.username" value="coolblog"/>
    <property name="hello" value="world"/>
</properties>
```

上面配置包含了一个 resource 属性, 和两个子节点。接下面, 我们参照上面的配置, 来分析 propertiesElement 方法的逻辑。如下。

```
// -☆- XMLConfigBuilder
private void propertiesElement(XNode context) throws Exception {
    if (context != null) {
        // 解析 propertis 的子节点, 并将这些节点内容转换为属性对象 Properties
        Properties defaults = context.getChildrenAsProperties();
        // 获取 propertis 节点中的 resource 和 url 属性值
        String resource = context.getStringAttribute("resource");
        String url = context.getStringAttribute("url");

        // 两者都不用空, 则抛出异常
        if (resource != null && url != null) {
            throw new BuilderException("...");
        }
        if (resource != null) {
            // 从文件系统中加载并解析属性文件
            defaults.putAll(Resources.getResourceAsProperties(resource));
        } else if (url != null) {
            // 通过 url 加载并解析属性文件
            defaults.putAll(Resources.getUrlAsProperties(url));
        }
    }
}
```

```
        Properties vars = configuration.getVariables();
        if (vars != null) {
            defaults.putAll(vars);
        }
        parser.setVariables(defaults);
        // 将属性值设置到 configuration 中
        configuration.setVariables(defaults);
    }
}

public Properties getChildrenAsProperties() {
    Properties properties = new Properties();
    // 获取并遍历子节点
    for (XNode child : getChildren()) {
        // 获取 property 节点的 name 和 value 属性
        String name = child.getStringAttribute("name");
        String value = child.getStringAttribute("value");
        if (name != null && value != null) {
            // 设置属性到属性对象中
            properties.setProperty(name, value);
        }
    }
    return properties;
}

// -☆- XNode
public List<XNode> getChildren() {
    List<XNode> children = new ArrayList<XNode>();
    // 获取子节点列表
    NodeList nodeList = node.getChildNodes();
    if (nodeList != null) {
        for (int i = 0, n = nodeList.getLength(); i < n; i++) {
```

```

Node node = nodeList.item(i);
if (node.getNodeType() == Node.ELEMENT_NODE) {
    // 将节点对象封装到 XNode 中, 并将 XNode 对象放入 children 列表中
    children.add(new XNode(xpathParser, node, variables));
}
}
}
return children;
}

```

上面就是<properties>节点解析过程, 不是很复杂。主要包含三个步骤, 一是解析<properties>节点的子节点, 并将解析结果设置到 Properties 对象中。二是从文件系统或通过网络读取属性配置, 这取决于<properties>节点的 resource 和 url 是否为空。第二步对应的代码比较简单, 这里就不分析了。最后一步则是将包含属性信息的 Properties 对象设置到 XPathParser 和 Configuration 中。

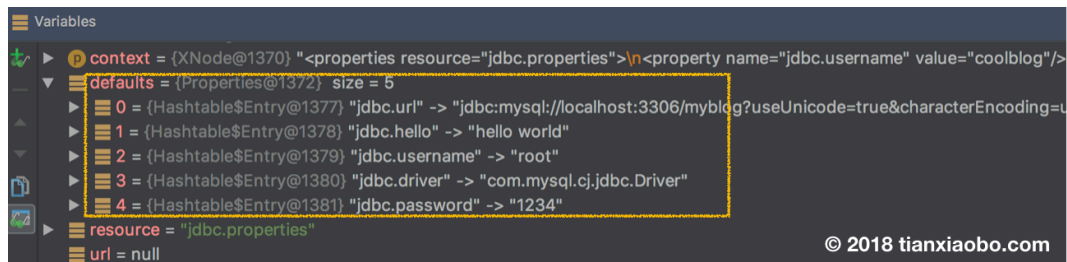
需要注意的是, propertiesElement 方法是先解析<properties>节点的子节点内容, 然后再从文件系统或者网络读取属性配置, 并将所有的属性及属性值都放入到 defaults 属性对象中。这会导致同名属性覆盖的问题, 也就是从文件系统, 或者网络上读取到的属性和属性值会覆盖掉<properties>子节点中同名的属性和及值。假如上面配置中的 jdbc.properties 内容如下:

```

jdbc.driver=com.mysql.cj.jdbc.Driver
jdbc.url=jdbc:mysql://localhost:3306/myblog?...
jdbc.username=root
jdbc.password=1234

```

与 properties 子节点内容合并后, 结果如下:



如上，原 jdbc.username 值为 coolblog，现在被覆盖为了 root。同名属性覆盖是一个比较小的点，大家开发过程中注意一下即可，其他的就没什么了。

2.1.2 解析<settings>节点

<settings>相关配置是 MyBatis 中非常重要的配置，这些配置用于调整 MyBatis 运行时的行为。settings 配置繁多，在对这些配置不熟悉的情况下，保持默认配置即可。关于<settings>相关配置，MyBatis 官网上进行了比较详细的描述，大家可以去了解一下。在本节中，暂时还用不到这些配置，所以即使不了解这些配置也没什么关系。下面先来看一个比较简单的配置，如下：

```
<settings>
  <setting name="cacheEnabled" value="true"/>
  <setting name="lazyLoadingEnabled" value="true"/>
  <setting name="autoMappingBehavior" value="PARTIAL"/>
</settings>
```

接下来，对照上面的配置，来分析相关源码

```
// -☆- XMLConfigBuilder
private Properties settingsAsProperties(XNode context) {
    if (context == null) {
        return new Properties();
    }
    // 获取 settings 子节点中的内容
    Properties props = context.getChildrenAsProperties();

    // 创建 Configuration 类的“元信息”对象
    MetaClass metaConfig = MetaClass.forClass(
        Configuration.class, localReflectorFactory);
    for (Object key : props.keySet()) {
        // 检测 Configuration 中是否存在相关属性，不存在则抛出异常
        if (!metaConfig.hasSetter(String.valueOf(key))) {
            throw new BuilderException(".....");
        }
    }
}
```

```

    }
}
return props;
}

```

如上，`settingsAsProperties` 方法看起来并不复杂，不过这是一个假象。在上面的代码中出现了个陌生的类 `MetaClass`，这个类是用来做什么的呢？答案是用来解析目标类的一些元信息，比如类的成员变量，`getter/setter` 方法等。关于这个类的逻辑，待会我会详细解析。接下来，简单总结一下上面代码的逻辑。

1. 解析 `settings` 子节点的内容，并将解析结果转成 `Properties` 对象
2. 为 `Configuration` 创建元信息对象
3. 通过 `MetaClass` 检测 `Configuration` 中是否存在某个属性的 `setter` 方法，不存在则抛异常
4. 若通过 `MetaClass` 的检测，则返回 `Properties` 对象，方法逻辑结束

下面，我们重点关注一下第 2 步和第 3 步的流程。这两步流程对应的代码较为复杂，需要一点耐心阅读。

1. 元信息对象创建过程

元信息类 `MetaClass` 的构造方法为私有类型，所以不能直接创建，必须使用其提供的 `forClass` 方法进行创建。它的创建逻辑如下：

```

public class MetaClass {
    private final ReflectorFactory reflectorFactory;
    private final Reflector reflector;

    private MetaClass(Class<?> type, ReflectorFactory reflectorFactory) {
        this.reflectorFactory = reflectorFactory;
        // 根据类型创建 Reflector
        this.reflector = reflectorFactory.findForClass(type);
    }
}

```

```
public static MetaClass forClass(  
    Class<?> type, ReflectorFactory reflectorFactory) {  
    // 调用构造方法  
    return new MetaClass(type, reflectorFactory);  
}  
  
// 省略其他方法  
}
```

上面的代码看起来很简单，不过这只是冰山一角。上面代码出现了两个新的类 `ReflectorFactory` 和 `Reflector`，`MetaClass` 通过引入这些新类帮助它完成功能。下面我们看一下 `hasSetter` 方法的源码就知道是怎么回事了。

```
// -☆- MetaClass  
public boolean hasSetter(String name) {  
    // 属性分词器，用于解析属性名  
    PropertyTokenizer prop = new PropertyTokenizer(name);  
    // hasNext 返回 true，则表明 name 是一个复合属性，后面会进行分析  
    if (prop.hasNext()) {  
        // 调用 reflector 的 hasSetter 方法  
        if (reflector.hasSetter(prop.getName())) {  
            // 为属性创建创建 MetaClass  
            MetaClass metaProp = metaClassForProperty(prop.getName());  
            // 再次调用 hasSetter  
            return metaProp.hasSetter(prop.getChildren());  
        } else {  
            return false;  
        }  
    } else {  
        // 调用 reflector 的 hasSetter 方法  
        return reflector.hasSetter(prop.getName());  
    }  
}
```

从上面的代码中，我们可以看出 MetaClass 中的 hasSetter 方法最终调用了 Reflector 的 hasSetter 方法。关于 Reflector 的 hasSetter 方法，这里先不进行分析，Reflector 这个类的逻辑较为复杂，本节会在随后进行详细说明。下面来简单介绍一下上面代码中出现的几个类：

1. ReflectorFactory：顾名思义，Reflector 的工厂类，兼有缓存 Reflector 对象的功能
2. Reflector：反射器，用于解析和存储目标类中的元信息
3. PropertyTokenizer：属性名分词器，用于处理较为复杂的属性名

上面的描述比较简单，仅通过上面的描述还不能让大家对这几个类有更深入的理解。所以下面单独分析一下这几个类的逻辑，首先是 ReflectorFactory。ReflectorFactory 是一个接口，MyBatis 中目前只有一个实现类 DefaultReflectorFactory，它的分析如下：

1.1 DefaultReflectorFactory

DefaultReflectorFactory 用于创建 Reflector，同时兼有缓存的功能，它的源码如下：

```
public class DefaultReflectorFactory implements ReflectorFactory {

    private boolean classCacheEnabled = true;
    /** 目标类和反射器映射缓存 */
    private final ConcurrentMap<Class<?>, Reflector> reflectorMap =
        new ConcurrentHashMap<Class<?>, Reflector>();

    // 省略部分代码

    @Override
    public Reflector findForClass(Class<?> type) {
        // classCacheEnabled 默认为 true
        if (classCacheEnabled) {
            // 从缓存中获取 Reflector 对象
            Reflector cached = reflectorMap.get(type);
            // 缓存为空，则创建一个新的 Reflector 实例，并放入缓存中
            if (cached == null) {
                cached = new Reflector(type);
                // 将 <type, cached> 映射缓存到 map 中，方便下次取用
                reflectorMap.put(type, cached);
            }
        }
    }
}
```



```
    }  
    return cached;  
  } else {  
    // 创建一个新的 Reflector 实例  
    return new Reflector(type);  
  }  
}  
}
```

如上，DefaultReflectorFactory 的 findForClass 方法逻辑包含两个访存操作，和一个对象创建操作。逻辑不是很复杂，代码注释的比较详细了，就不多说了。

1.2 Reflector

Reflector 这个类的用途主要是是通过反射获取目标类的 getter 方法及其返回值类型，setter 方法及其参数值类型等元信息。并将获取到的元信息缓存到相应的集合中，供后续使用。Reflector 本身代码比较多，这里不能一一分析。我将会分析该类的部分逻辑，分别如下：

1. Reflector 构造方法及成员变量分析
2. getter 方法解析过程
3. setter 方法解析过程

下面我们按照上面的顺序进行分析，先来分析 Reflector 的构造方法。

● Reflector 构造方法及成员变量分析

Reflector 构造方法中包含了很多初始化逻辑，目标类的元信息解析过程也是在构造方法中完成的，这些元信息最终会被保存到 Reflector 的成员变量中。下面我们先来看看 Reflector 的构造方法和相关的成员变量定义，如下：

```
public class Reflector {  
  
    private final Class<?> type;  
    private final String[] readablePropertyNames;  
    private final String[] writeablePropertyNames;
```

```
private final Map<String, Invoker> setMethods =
    new HashMap<String, Invoker>();
private final Map<String, Invoker> getMethods =
    new HashMap<String, Invoker>();
private final Map<String, Class<?>> setTypes =
    new HashMap<String, Class<?>>();
private final Map<String, Class<?>> getTypes =
    new HashMap<String, Class<?>>();
private Constructor<?> defaultConstructor;

private Map<String, String> caseInsensitivePropertyMap =
    new HashMap<String, String>();

public Reflector(Class<?> clazz) {
    type = clazz;
    // 解析目标类的默认构造方法，并赋值给 defaultConstructor 变量
    addDefaultConstructor(clazz);

    // 解析 getter 方法，并将解析结果放入 getMethods 中
    addGetMethods(clazz);

    // 解析 setter 方法，并将解析结果放入 setMethods 中
    addSetMethods(clazz);

    // 解析属性字段，并将解析结果添加到 setMethods 或 getMethods 中
    addFields(clazz);

    // 从 getMethods 映射中获取可读属性名数组
    readablePropertyNames = getMethods.keySet()
        .toArray(new String[getMethods.keySet().size()]);
    // 从 setMethods 映射中获取可写属性名数组
    writablePropertyNames = setMethods.keySet()
        .toArray(new String[setMethods.keySet().size()]);
```

```
// 将所有属性名的大写形式作为键，属性名作为值，
// 存入到 caseInsensitivePropertyMap 中
for (String propName : readablePropertyNames) {
    caseInsensitivePropertyMap
        .put(propName.toUpperCase(Locale.ENGLISH), propName);
}
for (String propName : writeablePropertyNames) {
    caseInsensitivePropertyMap.put(propName
        .toUpperCase(Locale.ENGLISH), propName);
}
}

// 省略其他方法
}
```

如上，Reflector 的构造方法逻辑比较多，看起来比较复杂。不过好在一些较为复杂的逻辑都封装在了相应的方法中，这样整体的逻辑就比较清晰了。Reflector 构造方法所做的事情均已进行了注释，大家对照着注释先看一下。构造方法中所调用的其他方法的细节待会会进行分析。下面使用表格列举一下 Reflector 部分成员变量的用途。如下：

变量名	用途
readablePropertyNames	可读属性名称数组，用于保存 <code>getter</code> 方法对应的属性名称
writeablePropertyNames	可写属性名称数组，用于保存 <code>setter</code> 方法对应的属性名称
setMethods	用于保存属性名称到 <code>Invoke</code> 的映射。 <code>setter</code> 方法会被封装到 <code>MethodInvoker</code> 对象中， <code>Invoke</code> 实现类比较简单，大家自行分析
getMethods	用于保存属性名称到 <code>Invoke</code> 的映射。同上， <code>getter</code> 方法也会被封装到 <code>MethodInvoker</code> 对象中
setTypes	用于保存 <code>setter</code> 对应的属性名与参数类型的映射

变量名	用途
getTypes	用于保存 getter 对应的属性名与返回值类型的映射
caseInsensitivePropertyMap	用于保存大写属性名与属性名之间的映射，比如 <NAME, name>

上面列举了一些集合变量，这些变量用于缓存各种原信息。关于这些变量，这里描述的不太好懂，主要是这些变量用途不太好解释。要想了解这些变量更多的细节，还是要深入到源码才行。所以我们趁热打铁，继续往下分析。

• getter 方法解析过程

getter 方法解析的逻辑被封装在了 addGetMethods 方法中，这个方法除了会解析形如 getXXX 的方法，同时也会解析 isXXX 方法。该方法的源码分析如下：

```
private void addGetMethods(Class<?> cls) {
    Map<String, List<Method>> conflictingGetters =
        new HashMap<String, List<Method>>();

    // 获取当前类，接口，以及父类中的方法。该方法逻辑不是很复杂，这里就不展开了
    Method[] methods = getClassMethods(cls);
    for (Method method : methods) {
        // getter 方法不应该有参数，若存在参数，则忽略当前方法
        if (method.getParameterTypes().length > 0) {
            continue;
        }

        String name = method.getName();
        // 过滤出以 get 或 is 开头的方法
        if ((name.startsWith("get") && name.length() > 3)
            || (name.startsWith("is") && name.length() > 2)) {
            // 将 getXXX 或 isXXX 等方法名转成相应的属性，比如 getName -> name
            name = PropertyNamer.methodToProperty(name);
        }
    }
}
```

```
    /*
     * 将冲突的方法添加到 conflictingGetters 中。考虑这样一种情况：
     *
     * getTitle 和 isTitle 两个方法经过 methodToProperty 处理，
     * 均得到 name = title，这会导致冲突。
     *
     * 对于冲突的方法，这里先统一起存起来，后续再解决冲突
     */
    addMethodConflict(conflictingGetters, name, method);
}
}

// 解决 getter 冲突
resolveGetterConflicts(conflictingGetters);
}
```

addGetMethods 方法的代码不是很多，但是逻辑有点多。这里总结一下：

1. 获取当前类，接口，以及父类中的方法
2. 遍历上一步获取的方法数组，并过滤出以 get 和 is 开头的方法
3. 将方法名转换成相应的属性名
4. 将属性名和方法对象添加到冲突集合中
5. 解决冲突

在上面的执行流程中，前三步比较简单，大家自行分析吧。第 4 步也不复杂，下面我会把源码贴出来，大家看一下就能懂。在这几步中，第 5 步逻辑比较复杂，这一步逻辑我们重点关注一下。

```
/** 添加属性名和方法对象到冲突集合中 */
private void addMethodConflict(Map<String,
    List<Method>> conflictingMethods, String name, Method method) {
    List<Method> list = conflictingMethods.get(name);
    if (list == null) {
        list = new ArrayList<Method>();
    }
}
```

```

        conflictingMethods.put(name, list);
    }
    list.add(method);
}

/** 解决冲突 */
private void resolveGetterConflicts(
    Map<String, List<Method>> conflictingGetters) {
    for (Entry<String, List<Method>> entry : conflictingGetters.entrySet()) {
        Method winner = null;
        String propName = entry.getKey();
        for (Method candidate : entry.getValue()) {
            if (winner == null) {
                winner = candidate;
                continue;
            }
            // 获取返回值类型
            Class<?> winnerType = winner.getReturnType();
            Class<?> candidateType = candidate.getReturnType();

            /*
             * 两个方法的返回值类型一致，若两个方法返回值类型均为 boolean，
             * 则选取 isXXX 方法为 winner。否则无法决定哪个方法更为合适，
             * 只能抛出异常
             */
            if (candidateType.equals(winnerType)) {
                if (!boolean.class.equals(candidateType)) {
                    throw new ReflectionException(".....");
                }

                /*
                 * 如果方法返回值类型为 boolean，且方法名以 "is" 开头，
                 * 则认为候选方法 candidate 更为合适
                 */
            } else if (candidate.getName().startsWith("is")) {

```

```
        winner = candidate;
    }

    /*
     * winnerType 是 candidateType 的子类，类型上更为具体，
     * 则认为当前的 winner 仍是合适的，无需做什么事情
     */
} else if (candidateType.isAssignableFrom(winnerType)) {
    /*
     * candidateType 是 winnerType 的子类，此时认为 candidate 方法
     * 更为合适，故将 winner 更新为 candidate
     */
} else if (winnerType.isAssignableFrom(candidateType)) {
    winner = candidate;
} else {
    throw new ReflectionException(".....");
}
}

// 将筛选出的方法添加到 getMethods 中，并将方法返回值添加到 getTypes 中
addGetMethod(propName, winner);
}
}

private void addGetMethod(String name, Method method) {
    if (isValidPropertyName(name)) {
        getMethods.put(name, new MethodInvoker(method));
        // 解析返回值类型
        Type returnType = TypeParameterResolver
            .resolveReturnType(method, type);
        // 将返回值类型由 Type 转为 Class，并将转换后的结果缓存到 setTypes 中
        getTypes.put(name, typeToClass(returnType));
    }
}
```

以上就是解除冲突的过程，代码有点长，不太容易看懂。这里大家只要记住解决冲突的规则即可理解上面代码的逻辑。相关规则如下：

1. 冲突方法返回值类型具有继承关系，子类返回值对应方法被认为是更合适的选择
2. 冲突方法的返回值类型相同，如果返回值类型为 boolean，那么以 is 开头的方法则是更合适的选择
3. 冲突方法的返回值类型相同，但类型非 boolean，此时出现歧义，抛出异常
4. 冲突方法的返回值类型不相关，无法确定哪个是更好的选择，此时直接抛异常

分析完 getter 方法的解析过程，下面继续分析 setter 方法的解析过程。

● setter 方法解析过程

与 getter 方法解析过程相比，setter 方法的解析过程与此有一定的区别。主要体现在冲突出现的原因，以及冲突的解决方法上。下面，我们深入源码来找出两者之间的区别。

```
private void addSetMethods(Class<?> cls) {
    Map<String, List<Method>> conflictingSetters =
        new HashMap<String, List<Method>>();
    // 获取当前类，接口，以及父类中的方法。该方法逻辑不是很复杂，这里就不展开了
    Method[] methods = getClassMethods(cls);
    for (Method method : methods) {
        String name = method.getName();
        // 过滤出 setter 方法，且方法仅有一个参数
        if (name.startsWith("set") && name.length() > 3) {
            if (method.getParameterTypes().length == 1) {
                name = PropertyNamer.methodToProperty(name);
                // setter 方法发生冲突原因是：可能存在重载情况，比如：
                //     void setSex(int sex);
                //     void setSex(SexEnum sex);
                addMethodConflict(conflictingSetters, name, method);
            }
        }
    }
}
// 解决 setter 冲突
```



```
        resolveSetterConflicts(conflictingSetters);
    }
```

从上面的代码和注释中,我们可知道 setter 方法之间出现冲突的原因。即方法存在重载,方法重载导致 methodToProperty 方法解析出的属性名完全一致。而 getter 方法之间出现冲突的原因是 getXXX 和 isXXX 对应的属性名一致。既然冲突发生了,要进行调停,那接下来继续来看看调停冲突的逻辑。

```
private void resolveSetterConflicts(
    Map<String, List<Method>> conflictingSetters) {
    for (String propName : conflictingSetters.keySet()) {
        List<Method> setters = conflictingSetters.get(propName);
        /*
         * 获取 getter 方法的返回值类型, 由于 getter 方法不存在重载的情况,
         * 所以可以用它的返回值类型反推哪个 setter 的更为合适
         */
        Class<?> getterType = getTypes.get(propName);
        Method match = null;
        ReflectionException exception = null;
        for (Method setter : setters) {
            // 获取参数类型
            Class<?> paramType = setter.getParameterTypes()[0];
            if (paramType.equals(getterType)) {
                // 参数类型和返回类型一致, 则认为是最好的选择, 并结束循环
                match = setter;
                break;
            }
            if (exception == null) {
                try {
                    // 选择一个更为合适的方法
                    match = pickBetterSetter(match, setter, propName);
                } catch (ReflectionException e) {
                    match = null;
                }
            }
        }
    }
}
```

```

        exception = e;
    }
}
}
// 若 match 为空，表示没找到更为合适的方法，此时抛出异常
if (match == null) {
    throw exception;
} else {
    // 将筛选出的方法放入 setMethods 中，并将方法参数值添加到 setTypes 中
    addSetMethod(propName, match);
}
}
}

/** 从两个 setter 方法中选择一个更为合适方法 */
private Method pickBetterSetter(Method setter1,
    Method setter2, String property) {
    if (setter1 == null) {
        return setter2;
    }
    Class<?> paramType1 = setter1.getParameterTypes()[0];
    Class<?> paramType2 = setter2.getParameterTypes()[0];

    // 如果参数 2 可赋值给参数 1，即参数 2 是参数 1 的子类，
    // 则认为参数 2 对应的 setter 方法更为合适
    if (paramType1.isAssignableFrom(paramType2)) {
        return setter2;

        // 这里和上面情况相反
    } else if (paramType2.isAssignableFrom(paramType1)) {
        return setter1;
    }
}

```

```
// 两种参数类型不相关，这里抛出异常
throw new ReflectionException(".....");
}

private void addSetMethod(String name, Method method) {
    if (isValidPropertyName(name)) {
        setMethods.put(name, new MethodInvoker(method));
        // 解析参数类型列表
        Type[] paramTypes = TypeParameterResolver
            .resolveParamTypes(method, type);
        // 将参数类型由 Type 转为 Class，并将转换后的结果缓存到 setTypes
        setTypes.put(name, typeToClass(paramTypes[0]));
    }
}
```

关于 setter 方法冲突的解析规则，这里也总结一下吧。如下：

1. 冲突方法的参数类型与 getter 的返回类型一致，则认为是最好的选择
2. 冲突方法的参数类型具有继承关系，子类参数对应的方法被认为是更合适的选择
3. 冲突方法的参数类型不相关，无法确定哪个是更好的选择，此时直接抛异常

到此关于 setter 方法的解析过程就说完了。我在前面说过 MetaClass 的 hasSetter 最终调用了 Refactor 的 hasSetter 方法，那么现在是时候分析 Refactor 的 hasSetter 方法了。代码如下：

```
public boolean hasSetter(String propertyName) {
    return setMethods.keySet().contains(propertyName);
}
```

代码如上，仅两行，很简单，就不多说了。

3. PropertyTokenizer

对于较为复杂的属性，需要进行进一步解析才能使用。那什么样的属性是复杂属性呢？来看个测试代码就知道了。

```
public class MetaClassTest {

    private class Author {
        private Integer id;
        private String name;
        private Integer age;
        /** 一个作者对应多篇文章 */
        private Article[] articles;

        // 省略 getter/setter
    }

    private class Article {
        private Integer id;
        private String title;
        private String content;
        /** 一篇文章对应一个作者 */
        private Author author;

        // 省略 getter/setter
    }

    @Test
    public void testHasSetter() {
        // 为 Author 创建元信息对象
        MetaClass authorMeta = MetaClass.forClass(
            Author.class, new DefaultReflectorFactory());
        System.out.println("-----☆ Author ☆-----");
        System.out.println("id -> " + authorMeta.hasSetter("id"));
        System.out.println("name -> " + authorMeta.hasSetter("name"));
        System.out.println("age -> " + authorMeta.hasSetter("age"));
        // 检测 Author 中是否包含 Article[] 的 setter
        System.out.println("articles->" + authorMeta.hasSetter("articles"));
    }
}
```

```

System.out.println("articles[] -> " +
    authorMeta.hasSetter("articles[]"));
System.out.println("title -> " + authorMeta.hasSetter("title"));

// 为 Article 创建元信息对象
MetaClass articleMeta = MetaClass.forClass(
    Article.class, new DefaultReflectorFactory());
System.out.println("\n-----☆ Article ☆-----");
System.out.println("id -> " + articleMeta.hasSetter("id"));
System.out.println("title -> " + articleMeta.hasSetter("title"));
System.out.println("content -> " +
    articleMeta.hasSetter("content"));
// 下面两个均为复杂属性，分别检测 Article 类中的 Author 类
// 是否包含 id 和 name 的 setter 方法
System.out.println("author.id->" +
    articleMeta.hasSetter("author.id"));
System.out.println("author.name->" +
    articleMeta.hasSetter("author.name"));
}
}

```

如上，Article 类中包含了一个 Author 引用。然后我们调用 articleMeta 的 hasSetter 检测 author.id 和 author.name 属性是否存在，我们的期望结果为 true。测试结果如下：

```

1 test passed - 65ms

-----☆ Author ☆-----
id -> true
name -> true
age -> true
articles -> true
articles[] -> true
title -> false
} ①
} ②
} ③

-----☆ Article ☆-----
id -> true
title -> true
content -> true
author.id -> true
author.name -> true
} ④
} ⑤

```

© 2018 tianxiaobo.com

如上，标记⑤处的输出均为 true，我们的预期达到了。标记②处检测 Article 数组的是否存在 setter 方法，结果也均为 true。这说明 PropertyTokenizer 对数组和复合属性均进行了处理。那它是如何处理的呢？答案如下：

```
public class PropertyTokenizer implements Iterator<PropertyTokenizer> {

    private String name;
    private final String indexedName;
    private String index;
    private final String children;

    public PropertyTokenizer(String fullname) {
        // 检测传入的参数中是否包含字符 '.'
        int delim = fullname.indexOf('.');
        if (delim > -1) {
            /*
             * 以点位为界，进行分割。比如：
             *     fullname = www.coolblog.xyz
             *
             * 以第一个点为分界符：
             *     name = www
             *     children = coolblog.xyz
             */
            name = fullname.substring(0, delim);
            children = fullname.substring(delim + 1);
        } else {
            // fullname 中不存在字符 '.'
            name = fullname;
            children = null;
        }
        indexedName = name;
        // 检测传入的参数中是否包含字符 '['
        delim = name.indexOf('[');
    }
}
```

```
        if (delim > -1) {
            /*
             * 获取中括号里的内容，比如：
             *    1. 对于数组或 List 集合：[] 中的内容为数组下标，
             *       比如 fullname = articles[1], index = 1
             *    2. 对于 Map: [] 中的内容为键，
             *       比如 fullname = xxxMap[keyName], index = keyName
             */
            index = name.substring(delim + 1, name.length() - 1);

            // 获取分解符前面的内容，比如
            // fullname = articles[1], name = articles
            name = name.substring(0, delim);
        }
    }

    // 省略 getter

    @Override
    public boolean hasNext() {
        return children != null;
    }

    @Override
    public PropertyTokenizer next() {
        // 对 children 进行再次切分，用于解析多重复合属性
        return new PropertyTokenizer(children);
    }
}
```

以上是 PropertyTokenizer 的源码分析，注释的比较多，应该分析清楚了。大家如果看懂了上面的分析，可自行举例进行测试，以加深理解。

本节为了分析 MetaClass 的 hasSetter 方法，并把与这个方法相关的源码均分析了一遍。

其实，如果想简单点分析，可直接把 MetaClass 当成一个黑盒，然后用一句话告诉大家 hasSetter 方法有什么用即可。但是这样做我觉的内容太虚，没什么深度。关于 MetaClass 及相关源码大家第一次看可能会有点吃力，看不懂可以先放一放。后面多看几遍，动手写点测试代码调试一下，可以帮助理解。

好了，关于<settings>节点的解析过程就先分析到这里，继续往下分析。

2.1.3 设置<settings>内容到 Configuration 中

上一节讲了<settings>节点的解析过程，节点内容解析出来后，需要有一个存放的地方，以使其他代码可以找到这些配置。这个存放地方就是 Configuration 对象，本节来看一下将<settings>节点内容设置到 Configuration 对象中的过程。如下：

```
private void settingsElement(Properties props) throws Exception {
    // 设置 autoMappingBehavior 属性，默认值为 PARTIAL
    configuration.setAutoMappingBehavior(AutoMappingBehavior
        .valueOf(props.getProperty("autoMappingBehavior", "PARTIAL")));
    configuration.setAutoMappingUnknownColumnBehavior(
        AutoMappingUnknownColumnBehavior.valueOf(
            props.getProperty("autoMappingUnknownColumnBehavior", "NONE")));
    // 设置 cacheEnabled 属性，默认值为 true
    configuration.setCacheEnabled(
        booleanValueOf(props.getProperty("cacheEnabled"), true));

    // 解析默认的枚举处理器
    Class<? extends TypeHandler> typeHandler=(Class<? extends TypeHandler>)
        resolveClass(props.getProperty("defaultEnumTypeHandler"));
    // 设置默认枚举处理器
    configuration.setDefaultEnumTypeHandler(typeHandler);
    configuration.setCallSettersOnNulls(
        booleanValueOf(props.getProperty("callSettersOnNulls"), false));
    configuration.setUseActualParamName(
        booleanValueOf(props.getProperty("useActualParamName"), true));
}
```


上面代码处理调用 Configuration 的 setter 方法，没太多其他逻辑。这里来看一下上面出现的一个调用 resolveClass，它的源码如下：

```
// -☆- BaseBuilder
protected Class<?> resolveClass(String alias) {
    if (alias == null) {
        return null;
    }
    try {
        // 通过别名解析
        return resolveAlias(alias);
    } catch (Exception e) {
        throw new BuilderException("Error resolving class. Cause: " + e, e);
    }
}

protected final TypeAliasRegistry typeAliasRegistry;

protected Class<?> resolveAlias(String alias) {
    // 通过别名注册器解析别名对于的类型 Class
    return typeAliasRegistry.resolveAlias(alias);
}
```

这里出现了一个新的类 TypeAliasRegistry，大家对于它可能会觉得陌生，但是对于 typeAlias 应该不会陌生。TypeAliasRegistry 的用途就是将别名和类型进行映射，这样就可以用别名表示某个类了，方便使用。既然聊到了别名，那下面我们不妨看看别名的配置的解析过程。

2.1.4 解析<typeAliases>节点

在 MyBatis 中，我们可以为自己写的一些类定义一个别名。这样在使用的时候，只需要输入别名即可，无需再把全限定的类名写出来。在 MyBatis 中，我们有两种方式进行别名配

置。第一种是仅配置包名，让 MyBatis 去扫描包中的类型，并根据类型得到相应的别名。这种方式可配合 Alias 注解使用，即通过注解为某个类配置别名，而不是让 MyBatis 按照默认规则生成别名。这种方式的配置如下：

```
<typeAliases>
  <package name="xyz.coolblog.chapter2.model1"/>
  <package name="xyz.coolblog.chapter2.model2"/>
</typeAliases>
```

第二种方式是通过手动的方式，明确为某个类型配置别名。这种方式的配置如下：

```
<typeAliases>
  <typeAlias alias="article" type="xyz.coolblog.chapter2.model.Article" />
  <typeAlias alias="author" type="xyz.coolblog.chapter2.model.Author" />
</typeAliases>
```

对比这两种方式，第一种自动扫描的方式配置起来比较简单，缺点也不明显。唯一能想到缺点可能就是 MyBatis 会将某个包下所有符合要求的类的别名都解析出来，并形成映射关系。如果你不想让某些类被扫描，这个好像做不到，没发现 MyBatis 提供了相关的排除机制。不过我觉得这并不是什么大问题，最多是多解析并缓存了一些别名到类型的映射，在时间和空间上产生了一些的消耗而已。当然，如果无法忍受这些消耗，可以使用第二种配置方式，通过手工的方式精确配置某些类型的别名。不过这种方式比较繁琐，特别是配置项比较多时。至于两种方式怎么选择，这个看具体的情况了。配置项非常少时，两种皆可。比较多的话，还是让 MyBatis 自行扫描吧。

以上介绍了两种不同的别名配置方式，下面我们来看一下两种不同的别名配置是怎样解析的。代码如下：

```
// ☆ XMLConfigBuilder
private void typeAliasesElement(XNode parent) {
    if (parent != null) {
        for (XNode child : parent.getChildren()) {
            // ☆ 从指定的包中解析别名和类型的映射
            if ("package".equals(child.getName())) {
```

```
String typeAliasPackage = child.getStringAttribute("name");
configuration.getTypeAliasRegistry()
    .registerAliases(typeAliasPackage);

// ☆ 从 typeAlias 节点中解析别名和类型的映射
} else {
    // 获取 alias 和 type 属性值, alias 不是必填项, 可为空
    String alias = child.getStringAttribute("alias");
    String type = child.getStringAttribute("type");
    try {
        // 加载 type 对应的类型
        Class<?> clazz = Resources.classForName(type);

        // 注册别名到类型的映射
        if (alias == null) {
            typeAliasRegistry.registerAlias(clazz);
        } else {
            typeAliasRegistry.registerAlias(alias, clazz);
        }
    } catch (ClassNotFoundException e) {
        throw new BuilderException(".....");
    }
}
}
}
}
```

如上, 上面的代码通过一个 if-else 条件分支来处理两种不同的配置, 这里我用☆标注了出来。下面我们来分别看一下这两种配置方式的解析过程。

1. 从<typeAlias>节点中解析并注册别名

在别名的配置中, type 属性是必须要配置的, 而 alias 属性则不是必须的。这个在配置

文件的 DTD 中有所规定。如果使用者未配置 alias 属性，则需要 MyBatis 自行为目标类型生成别名。对于别名为空的情况，注册别名的任务交由 registerAlias(Class<?>) 方法处理。若不为空，则由 registerAlias(String,Class<?>) 进行别名注册。这两个方法的分析如下：

```
public void registerAlias(Class<?> type) {
    // 获取全路径类名的简称
    String alias = type.getSimpleName();
    Alias aliasAnnotation = type.getAnnotation(Alias.class);
    if (aliasAnnotation != null) {
        // 从注解中取出别名
        alias = aliasAnnotation.value();
    }
    // 调用重载方法注册别名和类型映射
    registerAlias(alias, type);
}

public void registerAlias(String alias, Class<?> value) {
    if (alias == null) { throw new TypeException("..."); }
    // 将别名转成小写
    String key = alias.toLowerCase(Locale.ENGLISH);
    // 如果 TYPE_ALIASES 中存在着某个类型映射，这里判断当前类型与映射中的类型
    // 是否一致，不一致则抛出异常，不允许一个别名对应两种类型
    if (TYPE_ALIASES.containsKey(key) && TYPE_ALIASES.get(key) != null
        && !TYPE_ALIASES.get(key).equals(value)) {
        throw new TypeException(".....");
    }
    // 缓存别名到类型映射
    TYPE_ALIASES.put(key, value);
}
```

如上，若用户未明确配置 alias 属性，MyBatis 会使用类名的小写形式作为别名。比如，全限定类名 xyz.coolblog.model.Author 的别名为 author。若类中有 @Alias 注解，则从注解中取值作为别名。上面代码的逻辑不是很复杂，注释的也比较清楚了，就不多说了。继续分析注册别名的第二种方式。

2. 从指定的包中解析并注册别名

从指定的包中解析并注册别名过程主要由别名的解析和注册两步组成。下面来看一下相关代码：

```
public void registerAliases(String packageName) {
    // 调用重载方法注册别名
    registerAliases(packageName, Object.class);
}

public void registerAliases(String packageName, Class<?> superType) {
    ResolverUtil<Class<?>> resolverUtil = new ResolverUtil<Class<?>>();
    // 查找某个包下的父类为 superType 的类。从调用栈来看，这里的
    // superType = Object.class，所以 ResolverUtil 将查找所有的类。
    // 查找完成后，查找结果将会被缓存到内部集合中。
    resolverUtil.find(new ResolverUtil.IsA(superType), packageName);
    // 获取查找结果
    Set<Class<? extends Class<?>>> typeSet = resolverUtil.getClasses();
    for (Class<?> type : typeSet) {
        // 忽略匿名类，接口，内部类
        if (!type.isAnonymousClass() && !type.isInterface() &&
            !type.isMemberClass()) {
            // 为类型注册别名
            registerAlias(type);
        }
    }
}
```

上面的代码不多，相关流程也不复杂，可简单总结为下面两个步骤：一是查找指定包下的所有类；二是遍历查找到的类型集合，为每个类型注册别名

在这两步流程中，第2步流程对应的代码上一节已经分析过了，这里不再赘述。第1步的功能理解起来不难，但是背后对应的代码有点多。限于篇幅原因，这里我不打算详细分析这一部分的代码，只做简单的流程总结。如下：

1. 通过 VFS（虚拟文件系统）获取指定包下的所有文件的路径名，
2. 比如 xyz/coolblog/model/Article.class
3. 筛选以.class 结尾的文件名
4. 将路径名转成全限定的类名，通过类加载器加载类名
5. 对类型进行匹配，若符合匹配规则，则将其放入内部集合中

3. 注册 MyBatis 内部类及常见类型的别名

最后，我们来看一下一些 MyBatis 内部类及一些常见类型的别名注册过程。如下：

```
// -☆- Configuration
public Configuration() {
    // 注册事务工厂的别名
    typeAliasRegistry.registerAlias("JDBC", JdbcTransactionFactory.class);
    // 省略部分代码，下同

    // 注册数据源的别名
    typeAliasRegistry.registerAlias(
        "POOLED", PooledDataSourceFactory.class);

    // 注册缓存策略的别名
    typeAliasRegistry.registerAlias("FIFO", FifoCache.class);
    typeAliasRegistry.registerAlias("LRU", LruCache.class);

    // 注册日志类的别名
    typeAliasRegistry.registerAlias("SLF4J", Slf4jImpl.class);
    typeAliasRegistry.registerAlias("LOG4J", Log4jImpl.class);

    // 注册动态代理工厂的别名
    typeAliasRegistry.registerAlias("CGLIB", CglibProxyFactory.class);
    typeAliasRegistry.registerAlias(
        "JAVASSIST", JavassistProxyFactory.class);
}
```

```
// ☆- TypeAliasRegistry
public TypeAliasRegistry() {
    // 注册 String 的别名
    registerAlias("string", String.class);
    // 注册基本类型包装类的别名
    registerAlias("byte", Byte.class);
    // 省略部分代码，下同

    // 注册基本类型包装类数组的别名
    registerAlias("byte[]", Byte[].class);
    // 注册基本类型的别名
    registerAlias("_byte", byte.class);
    // 注册基本类型包装类的别名
    registerAlias("_byte[]", byte[].class);

    // 注册 Date, BigDecimal, Object 等类型的别名
    registerAlias("date", Date.class);
    registerAlias("decimal", BigDecimal.class);
    registerAlias("object", Object.class);
    // 注册 Date, BigDecimal, Object 等数组类型的别名
    registerAlias("date[]", Date[].class);
    registerAlias("decimal[]", BigDecimal[].class);
    registerAlias("object[]", Object[].class);

    // 注册集合类型的别名
    registerAlias("map", Map.class);
    registerAlias("list", List.class);
    registerAlias("collection", Collection.class);
    registerAlias("iterator", Iterator.class);

    // 注册 ResultSet 的别名
    registerAlias("ResultSet", ResultSet.class);
}
```

我记得以前配置<select>标签的 resultType 属性，由于不知道有别名这回事，傻傻的使用全限定类名进行配置。当时还觉得这样配置一定不会出错吧，很放心。现在想想有点搞笑。

好了，以上就是别名解析的全部流程，大家看懂了吗？如果觉得没啥障碍的话，那继续往下看呗。

2.1.5 解析<plugins>节点

插件是 MyBatis 提供的一个拓展机制，通过插件机制我们可在 SQL 执行过程中的某些点上做一些自定义操作。实现一个插件需要比简单，首先需要让插件类实现 `Interceptor` 接口。然后在插件类上添加 `@Intercepts` 和 `@Signature` 注解，用于指定想要拦截的目标方法。MyBatis 允许拦截下面接口中的一些方法：

- Executor: update, query, flushStatements, commit, rollback, getTransaction, close, isClosed
- ParameterHandler: getParameterObject, setParameters
- ResultSetHandler: handleResultSets, handleOutputParameters
- StatementHandler: prepare, parameterize, batch, update, query

比较常见的插件有分页插件、分表插件等，有兴趣的朋可以去了解下。在分析插件的配置的解析过程之前，我们先来了解一下插件的配置。如下：

```
<plugins>
  <plugin interceptor="xyz.coolblog.mybatis.ExamplePlugin">
    <property name="key" value="value"/>
  </plugin>
</plugins>
```

解析过程如下：

```
private void pluginElement(XNode parent) throws Exception {
    if (parent != null) {
        for (XNode child : parent.getChildren()) {
            String interceptor = child.getStringAttribute("interceptor");
```



```
// 获取配置信息
Properties properties = child.getChildrenAsProperties();
// 解析拦截器的类型，并创建拦截器
Interceptor interceptorInstance =
    (Interceptor) resolveClass(interceptor).newInstance();
// 设置属性
interceptorInstance.setProperties(properties);
// 添加拦截器到 Configuration 中
configuration.addInterceptor(interceptorInstance);
    }
}
}
```

如上，插件解析的过程还是比较简单的。首先是获取配置，然后再解析拦截器类型，并实例化拦截器。最后向拦截器中设置属性，并将拦截器添加到 Configuration 中。好了，关于插件配置的分析就先到这。

2.1.6 解析<environments>节点

在 MyBatis 中，事务管理器和数据源是配置在<environments>节点中的。它们的配置大致如下：

```
<environments default="development">
  <environment id="development">
    <transactionManager type="JDBC"/>
    <dataSource type="POOLED">
      <property name="driver" value="${jdbc.driver}"/>
      <property name="url" value="${jdbc.url}"/>
      <property name="username" value="${jdbc.username}"/>
      <property name="password" value="${jdbc.password}"/>
    </dataSource>
  </environment>
</environments>
```

接下来我们对照上面的配置进行分析。

```
private void environmentsElement(XNode context) throws Exception {
    if (context != null) {
        if (environment == null) {
            // 获取 default 属性
            environment = context.getStringAttribute("default");
        }
        for (XNode child : context.getChildren()) {
            // 获取 id 属性
            String id = child.getStringAttribute("id");
            // 检测当前 environment 节点的 id 与其父节点 environments 的
            // 属性 default 内容是否一致，一致则返回 true，否则返回 false
            if (isSpecifiedEnvironment(id)) {
                // 解析 transactionManager 节点
                TransactionFactory txFactory = transactionManagerElement(
                    child.evalNode("transactionManager"));
                // 解析 dataSource 节点，逻辑和插件的解析逻辑很相似，不在赘述
                DataSourceFactory dsFactory =
                    dataSourceElement(child.evalNode("dataSource"));
                // 创建 DataSource 对象
                DataSource dataSource = dsFactory.getDataSource();
                Environment.Builder environmentBuilder =
                    new Environment.Builder(id)
                        .transactionFactory(txFactory)
                        .dataSource(dataSource);
                // 构建 Environment 对象，并设置到 configuration 中
                configuration.setEnvironment(environmentBuilder.build());
            }
        }
    }
}
```

<environments>节点的解析过程没什么特别之处，按部就班解析就行了，不多说了。

2.1.7 解析<typeHandlers>节点

在向数据库存储或读取数据时,我们需要将数据库字段类型和 Java 类型进行一个转换。比如数据库中有 CHAR 和 VARCHAR 等类型,但 Java 中没有这些类型,不过 Java 有 String 类型。所以我们在从数据库中读取 CHAR 和 VARCHAR 类型的数据时,就可以把它们转成 String。在 MyBatis 中,数据库类型和 Java 类型之间的转换任务是委托给类型处理器 TypeHandler 去处理的。MyBatis 提供了一些常见类型的类型处理器,除此之外,我们还可以自定义类型处理器以非常见类型转换的需求。这里我就不演示自定义类型处理器的编写方法了,没用过或者不熟悉的同学可以 MyBatis 官方文档,或者我在第一章中所写的示例。下面,我们来看一下类型处理器的配置方法:

```
<!-- 自动扫描 -->
<typeHandlers>
    <package name="xyz.coolblog.handlers"/>
</typeHandlers>

<!-- 手动配置 -->
<typeHandlers>
    <typeHandler jdbcType="TINYINT"
        javaType="xyz.coolblog.constant.ArticleTypeEnum"
        handler="xyz.coolblog.mybatis.ArticleTypeHandler"/>
</typeHandlers>
```

使用自动扫描的方式注册类型处理器时,应使用 @MappedTypes 和 @MappedJdbcTypes 注解配置 javaType 和 jdbcType。关于注解,这里就不演示了,比较简单,大家自行尝试。下面开始分析代码。

```
private void typeHandlerElement(XNode parent) throws Exception {
    if (parent != null) {
        for (XNode child : parent.getChildren()) {
            // 从指定的包中注册 TypeHandler
            if ("package".equals(child.getName())) {
```

```

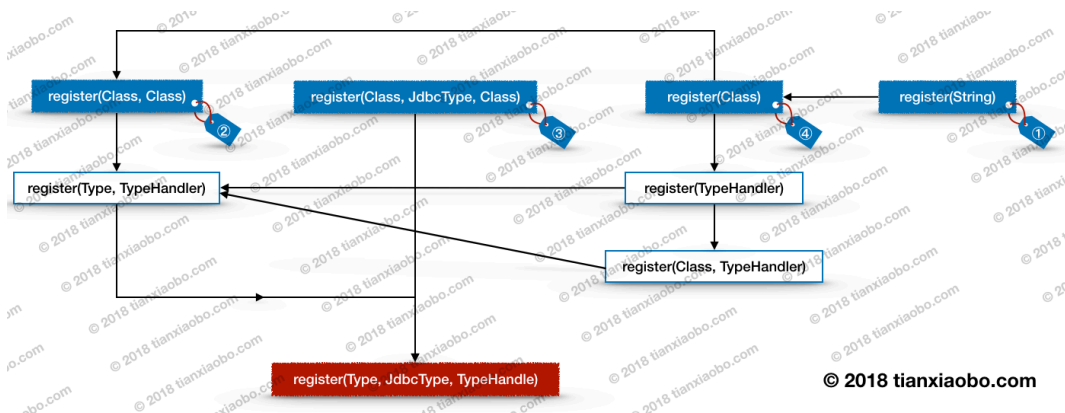
        String typeHandlerPackage = child.getStringAttribute("name");
        // 注册方法 ①
        typeHandlerRegistry.register(typeHandlerPackage);
    } else { // 从 typeHandler 节点中解析别名到类型的映射
        String javaTypeName = child.getStringAttribute("javaType");
        String jdbcTypeName = child.getStringAttribute("jdbcType");
        String handlerTypeName = child.getStringAttribute("handler");

        // 解析上面获取到的属性值
        Class<?> javaTypeClass = resolveClass(javaTypeName);
        JdbcType jdbcType = resolveJdbcType(jdbcTypeName);
        Class<?> typeHandlerClass = resolveClass(handlerTypeName);

        // 根据 javaTypeClass 和 jdbcType 值的情况进行不同的注册策略
        if (javaTypeClass != null) {
            if (jdbcType == null) {
                // 注册方法 ②
                typeHandlerRegistry.register(
                    javaTypeClass, typeHandlerClass);
            } else {
                // 注册方法 ③
                typeHandlerRegistry.register(
                    javaTypeClass, jdbcType, typeHandlerClass);
            }
        } else {
            // 注册方法 ④
            typeHandlerRegistry.register(typeHandlerClass);
        }
    }
}
}
}
}

```

上面代码中用于解析 XML 部分的代码比较简单，没什么需要特别说明的。除此之外，上面的代码中调用了 4 个不同的类型处理器注册方法。这些注册方法的逻辑不难理解，但是重载方法很多，上面调用的注册方法只是重载方法的一部分。由于重载太多且重载方法之间互相调用，导致这一块的代码有点凌乱。我一开始在整理这部分代码时，也很抓狂。后来把重载方法的调用图画了出来，才理清了代码。如下图。



在上面的调用图中，每个蓝色背景框下都有一个标签。每个标签上面都已一个编号，这些编号与上面代码中的标签是一致的。这里我把蓝色背景框内的方法称为开始方法，红色背景框内的方法称为终点方法，白色背景框内的方法称为中间方法。下面我会分析从每个开始方法向下分析，为了避免冗余分析，我会按照③→②→④→①的顺序进行分析。大家在阅读代码分析时，可以参照上面的图片，辅助理解。

1.register(Class,JdbcType,Class)方法分析

当代码执行到此方法时，表示 `javaTypeClass != null && jdbcType != null` 条件成立，即使用户明确配置了 `javaType` 和 `jdbcType` 属性的值。下面我们来看一下该方法的分析。

```
public void register(Class<?> javaTypeClass,
    JdbcType jdbcType, Class<?> typeHandlerClass) {
    // 调用终点方法
    register(javaTypeClass, jdbcType,
        getInstance(javaTypeClass, typeHandlerClass));
}
```

```

/** 类型处理器注册过程的终点 */
private void register(Type javaType,
    JdbcType jdbcType, TypeHandler<?> handler) {
    if (javaType != null) {
        // JdbcType 到 TypeHandler 的映射
        Map<JdbcType, TypeHandler<?>> map = TYPE_HANDLER_MAP.get(javaType);
        if (map == null || map == NULL_TYPE_HANDLER_MAP) {
            map = new HashMap<JdbcType, TypeHandler<?>>();
            // 存储 javaType 到 Map<JdbcType, TypeHandler> 的映射
            TYPE_HANDLER_MAP.put(javaType, map);
        }
        map.put(jdbcType, handler);
    }

    // 存储所有的 TypeHandler
    ALL_TYPE_HANDLERS_MAP.put(handler.getClass(), handler);
}

```

上面的代码只有两层调用，比较简单。同时，所谓的注册过程也就是把类型和处理器进行映射而已，没什么特别之处。关于这个方法就先分析到这里，继续往下分析。下面的方法对应注册方法②。

2. register(Class,Class)方法分析

当代码执行到此方法时，表示 `javaTypeClass != null && jdbcType == null` 条件成立，即使用者仅设置了 `javaType` 属性的值。下面我们来看一下该方法的分析。

```

public void register(Class<?> javaTypeClass, Class<?> typeHandlerClass) {
    // 调用中间方法 register(Type, TypeHandler)
    register(javaTypeClass, getInstance(javaTypeClass, typeHandlerClass));
}

```

```

private <T> void register(Type javaType,
    TypeHandler<? extends T> typeHandler) {
    // 获取 @MappedJdbcTypes 注解
    MappedJdbcTypes mappedJdbcTypes =
        typeHandler.getClass().getAnnotation(MappedJdbcTypes.class);
    if (mappedJdbcTypes != null) {
        // 遍历 @MappedJdbcTypes 注解中配置的值
        for (JdbcType handledJdbcType : mappedJdbcTypes.value()) {
            // 调用终点方法，参考上一小节分析
            register(javaType, handledJdbcType, typeHandler);
        }
        if (mappedJdbcTypes.includeNullJdbcType()) {
            // 调用终点方法，jdbcType = null
            register(javaType, null, typeHandler);
        }
    } else {
        // 调用终点方法，jdbcType = null
        register(javaType, null, typeHandler);
    }
}

```

上面的代码包含三层调用，其中终点方法的逻辑上一节已经分析过，这里不再赘述。上面的逻辑也比较简单，主要做的事情是尝试从注解中获取 `JdbcType` 的值。这个方法就分析这么多，下面分析注册方法④。

3. register(Class)方法分析

当代码执行到此方法时，表示 `javaTypeClass==null&&jdbcType!=null` 条件成立，即使用者未配置 `javaType` 和 `jdbcType` 属性的值。该方法的分析如下。

```

public void register(Class<?> typeHandlerClass) {
    boolean mappedTypeFound = false;
    // 获取 @MappedTypes 注解

```

```
MappedTypes mappedTypes = typeHandlerClass
    .getAnnotation(MappedTypes.class);
if (mappedTypes != null) {
    // 遍历 @MappedTypes 注解中配置的值
    for (Class<?> javaTypeClass : mappedTypes.value()) {
        // 调用注册方法 @
        register(javaTypeClass, typeHandlerClass);
        mappedTypeFound = true;
    }
}
if (!mappedTypeFound) {
    // 调用中间方法 register(TypeHandler)
    register(getInstance(null, typeHandlerClass));
}
}

public <T> void register(TypeHandler<T> typeHandler) {
    boolean mappedTypeFound = false;
    // 获取 @MappedTypes 注解
    MappedTypes mappedTypes =
        typeHandler.getClass().getAnnotation(MappedTypes.class);
    if (mappedTypes != null) {
        for (Class<?> handledType : mappedTypes.value()) {
            // 调用中间方法 register(Type, TypeHandler)
            register(handledType, typeHandler);
            mappedTypeFound = true;
        }
    }
    // 自动发现映射类型
    if (!mappedTypeFound && typeHandler instanceof TypeReference) {
        try {
            TypeReference<T> typeReference = (TypeReference<T>) typeHandler;
            // 获取参数模板中的参数类型，并调用中间方法 register(Type, TypeHandler)

```



```

        register(typeReference.getRawType(), typeHandler);
        mappedTypeFound = true;
    } catch (Throwable t) {
    }
}

if (!mappedTypeFound) {
    // 调用中间方法 register(Class, TypeHandler)
    register((Class<T>) null, typeHandler);
}
}

public <T> void register(
    Class<T> javaType, TypeHandler<? extends T> typeHandler) {
    // 调用中间方法 register(Type, TypeHandler)
    register((Type) javaType, typeHandler);
}

```

上面的代码比较多，不过不用太担心。不管是通过注解的方式，还是通过反射的方式，它们最终目的是为了解析出 `javaType` 的值。解析完成后，这些方法会调用中间方法 `register(Type, TypeHandler)`，这个方法负责解析 `jdbcType`，该方法上一节已经分析过。一个负责解析 `javaType`，另一个负责解析 `jdbcType`，逻辑比较清晰了。那我们趁热打铁，继续分析下一个注册方法，编号为①。

4. register(String)方法分析

本节代码的主要是用于自动扫描类型处理器，并调用其他方法注册扫描结果。该方法的分析如下：

```

public void register(String packageName) {
    ResolverUtil<Class<?>> resolverUtil = new ResolverUtil<Class<?>>();
    // 从指定包中查找 TypeHandler
    resolverUtil.find(new ResolverUtil.IsA(TypeHandler.class), packageName);
    Set<Class<? extends Class<?>>> handlerSet = resolverUtil.getClasses();
}

```

```
for (Class<?> type : handlerSet) {  
    // 忽略内部类，接口，抽象类等  
    if (!type.isAnonymousClass() && !type.isInterface()  
        && !Modifier.isAbstract(type.getModifiers())) {  
        // 调用注册方法 ④  
        register(type);  
    }  
}  
}
```

上面代码的逻辑比较简单，其中注册方法④已经在上一节分析过了，这里就不多说了。由以上分析可知，类型处理器的解析过程不复杂，但是注册过程由于重载方法间相互调用，导致调用路线比较复杂。这个时候需要想办法理清方法的调用路线，理清后，整个逻辑就清晰明了了。

2.2 本章小结

本章对 MyBatis 配置文件中<properties>、<settings>、<typeAliase>、<typeHandlers>等节点的解析过程进行了详细的分析。本章所关注的点不局限于这些节点的解析过程，在此过程中，本章额外分析了其他模块的一些源码。这些模块在随后的章节中还会出现，大家应尽量搞懂这些模块源码

第 3 章

映射文件解析过程

本章，我们来分析一下映射文件解析的过程。与配置文件不同，映射文件用于配置 SQL 语句，字段映射关系等。映射文件中包含<cache>、<cache-ref>、<resultMap>、<sql>、<select|insert|update|delete>等二级节点，这些节点将在接下来内容中进行分析。本章除了分析常规的 XML 解析过程外，还会向大家介绍 Mapper 接口的绑定过程，以及其他一些知识。内容较多，需要有一定的耐心阅读。

3.1 映射文件解析解析入口

映射文件的解析过程是配置文件解析过程的一部分，MyBatis 会在解析配置文件的过程中对映射文件进行解析。解析逻辑封装在 mapperElement 方法中，下面来看一下。

```
// -☆- XMLConfigBuilder
private void mapperElement(XNode parent) throws Exception {
    if (parent != null) {
```

```
for (XNode child : parent.getChildren()) {
    if ("package".equals(child.getName())) {
        // 获取 <package> 节点中的 name 属性
        String mapperPackage = child.getStringAttribute("name");
        // 从指定包中查找 mapper 接口，并根据 mapper 接口解析映射配置
        configuration.addMappers(mapperPackage);
    } else {
        // 获取 resource/url/class 等属性
        String resource = child.getStringAttribute("resource");
        String url = child.getStringAttribute("url");
        String mapperClass = child.getStringAttribute("class");

        // resource 不为空，且其他两者为空，则从指定路径中加载配置
        if (resource != null && url == null && mapperClass == null){
            ErrorContext.instance().resource(resource);
            InputStream inputStream =
                Resources.getResourceAsStream(resource);
            XMLMapperBuilder mapperParser = new XMLMapperBuilder(
                inputStream, configuration, resource,
                configuration.getSqlFragments());
            // 解析映射文件
            mapperParser.parse();

            // url 不为空，且其他两者为空，则通过 url 加载配置
        } else if (resource == null &&
            url != null && mapperClass == null) {
            ErrorContext.instance().resource(url);
            InputStream inputStream = Resources.getUrlAsStream(url);
            XMLMapperBuilder mapperParser = new XMLMapperBuilder(
                inputStream, configuration, url,
                configuration.getSqlFragments());
            // 解析映射文件
            mapperParser.parse();
        }
    }
}
```

```
        // mapperClass 不为空，且其他两者为空，
        // 则通过 mapperClass 解析映射配置
    } else if (resource == null &&
        url == null && mapperClass != null) {
        Class<?> mapperInterface =
            Resources.classForName(mapperClass);
        configuration.addMapper(mapperInterface);

        // 以上条件不满足，则抛出异常
    } else {
        throw new BuilderException(".....");
    }
}
}
}
}
```

上面代码的主要逻辑是遍历 mappers 的子节点，并根据节点属性值判断通过何种方式加载映射文件或映射信息。这里把配置在注解中的内容称为映射信息，以 XML 为载体的配置称为映射文件。在 MyBatis 中，共有四种加载映射文件或映射信息的方式。第一种是从文件系统中加载映射文件；第二种是通过 URL 的方式加载映射文件；第三种是通过 mapper 接口加载映射信息，映射信息可以配置在注解中，也可以配置在映射文件中。最后一种是通过包扫描的方式获取到某个包下的所有类，并使用第三种方式为每个类解析映射信息。

以上简单介绍了 MyBatis 加载映射文件或信息的几种方式。需要注意的是，在 MyBatis 中，通过注解配置映射信息的方式是有一定局限性的，这一点 MyBatis 官方文档中描述的比较清楚。这里引用一下：

因为最初设计时，MyBatis 是一个 XML 驱动的框架。配置信息是基于 XML 的，而且映射语句也是定义在 XML 中的。而到了 MyBatis3，就有新选择了。MyBatis3 构建在全面且强大的基于 Java 语言的配置 API 之上。这个配置 API 是基于 XML 的 MyBatis 配置的基础，也是新的基于注解配置的基础。注解提供了一种简单的方式来实现简单映射语句，而不会引入大量的开销。

注意：不幸的是，Java 注解的表达力和灵活性十分有限。尽管很多时间都花在调查、设计和试验上，**最强大的 MyBatis 映射并不能用注解来构建**——并不是在开玩笑，的确是这样。

如上，请注意用黑体标注了内容。限于 Java 注解的表达力和灵活性，通过注解的方式并不能完全发挥 MyBatis 的能力。因此，对于一些较为复杂的配置信息，我们还是应该通过 XML 的方式进行配置。在接下来的章节中，我会重点分析以 XML 为载体的映射文件的解析过程。如果能看懂此种配置方式的解析过程，那么基于注解的解析过程也不在话下。

下面开始分析映射文件的解析过程，在分析之前，先来看一下映射文件解析入口。

```
// -☆- XMLMapperBuilder
public void parse() {
    // 检测映射文件是否已经被解析过
    if (!configuration.isResourceLoaded(resource)) {
        // 解析 mapper 节点
        configurationElement(parser.evalNode("/mapper"));
        // 添加资源路径到“已解析资源集合”中
        configuration.addLoadedResource(resource);
        // 通过命名空间绑定 Mapper 接口
        bindMapperForNamespace();
    }

    // 处理未完成解析的节点
    parsePendingResultMaps();
    parsePendingCacheRefs();
    parsePendingStatements();
}
```

映射文件解析入口逻辑包含三个核心操作，如下：

1. 解析 mapper 节点
2. 通过命名空间绑定 Mapper 接口
3. 处理未完成解析的节点

这三个操作对应的逻辑将会在随后的章节中依次进行分析，下面先来分析第一个操作对应的逻辑。

3.2 解析映射文件

映射文件包含多种二级节点，比如 `<cache>`，`<resultMap>`，`<sql>` 以及 `<select|insert|update|delete>` 等。除此之外，还包含了一些三级节点，比如 `<include>`，`<if>`，`<where>` 等。这些节点的解析过程将会在接下来的内容中陆续进行分析。在分析之前，我们先来看一个映射文件配置示例。

```
<mapper namespace="xyz.coolblog.dao.AuthorDao">

    <cache/>

    <resultMap id="authorResult" type="Author">
        <id property="id" column="id"/>
        <result property="name" column="name"/>
        <!-- ... -->
    </resultMap>

    <sql id="table">
        author
    </sql>

    <select id="findOne" resultMap="authorResult">
        SELECT
            id, name, age, sex, email
        FROM
            <include refid="table"/>
        WHERE
            id = #{id}
    </select>

    <!-- <insert|update|delete/> -->
</mapper>
```

上面是一个比较简单的映射文件，还有一些的节点未出现在上面。以上配置中每种节点的解析逻辑都封装在了相应的方法中，这些方法由 XMLMapperBuilder 类的 configurationElement 方法统一调用。该方法的逻辑如下：

```
private void configurationElement(XNode context) {
    try {
        // 获取 mapper 命名空间
        String namespace = context.getStringAttribute("namespace");
        if (namespace == null || namespace.equals("")) {
            throw new BuilderException(".....");
        }

        // 设置命名空间到 builderAssistant 中
        builderAssistant.setCurrentNamespace(namespace);
        // 解析 <cache-ref> 节点
        cacheRefElement(context.evalNode("cache-ref"));
        // 解析 <cache> 节点
        cacheElement(context.evalNode("cache"));
        // 已废弃配置，这里不做分析
        parameterMapElement(context.evalNodes("/mapper/parameterMap"));

        // 解析 <resultMap> 节点
        resultMapElements(context.evalNodes("/mapper/resultMap"));
        // 解析 <sql> 节点
        sqlElement(context.evalNodes("/mapper/sql"));
        // 解析 <select>、...、<delete> 等节点
        buildStatementFromContext(
            context.evalNodes("select|insert|update|delete"));
    } catch (Exception e) {
        throw new BuilderException(".....");
    }
}
```


上面代码的执行流程清晰明了。在阅读源码时,我们按部就班的分析每个方法调用即可。不过本章在叙述的过程中会对分析顺序进行一些调整,本章将会先分析<cache>节点的解析过程,然后再分析<cache-ref>节点,之后会按照顺序分析其他节点的解析过程。

3.2.1 解析<cache>节点

MyBatis 提供了一、二级缓存,其中一级缓存是 SqlSession 级别的,默认为开启状态。二级缓存配置在映射文件中,使用者需要显示配置才能开启。如果无特殊要求,二级缓存的配置很简单。如下:

```
<cache/>
```

如果我们想修改缓存的一些属性,可以像下面这样配置。

```
<cache
  eviction="FIFO"
  flushInterval="60000"
  size="512"
  readOnly="true"/>
```

根据上面的配置创建出的缓存有以下特点:

1. 按先进先出的策略淘汰缓存项
2. 缓存的容量为 512 个对象引用
3. 缓存每隔 60 秒刷新一次
4. 缓存返回的对象是写安全的,即在外部分修改对象不会影响到缓存内部存储对象

除了上面两种配置方式,我们还可以给 MyBatis 配置第三方缓存或者自己实现的缓存等。比如,我们将 Ehcache 缓存整合到 MyBatis 中,可以这样配置。

```
<cache type="org.mybatis.caches.ehcache.EhcacheCache"/>
  <property name="timeToIdleSeconds" value="3600"/>
  <property name="timeToLiveSeconds" value="3600"/>
  <property name="maxEntriesLocalHeap" value="1000"/>
  <property name="maxEntriesLocalDisk" value="10000000"/>
```

```
<property name="memoryStoreEvictionPolicy" value="LRU"/>
</cache>
```

以上简单介绍了几种缓存配置方式。关于 MyBatis 缓存更多的知识，后面会单独进行介绍，本章就不深入说明了。下面我们来分析一下缓存配置的解析逻辑，如下：

```
private void cacheElement(XNode context) throws Exception {
    if (context != null) {
        // 获取各种属性
        String type = context.getStringAttribute("type", "PERPETUAL");
        Class<? extends Cache> typeClass =
            typeAliasRegistry.resolveAlias(type);
        String eviction = context.getStringAttribute("eviction", "LRU");
        Class<? extends Cache> evictionClass =
            typeAliasRegistry.resolveAlias(eviction);
        Long flushInterval = context.getLongAttribute("flushInterval");
        Integer size = context.getIntAttribute("size");
        boolean readWrite = !context.getBooleanAttribute("readOnly", false);
        boolean blocking = context.getBooleanAttribute("blocking", false);

        // 获取子节点配置
        Properties props = context.getChildrenAsProperties();

        // 构建缓存对象
        builderAssistant.useNewCache(typeClass, evictionClass,
            flushInterval, size, readWrite, blocking, props);
    }
}
```

上面代码中，大段代码用来解析<cache>节点的属性和子节点，这些代码没什么好说的。缓存对象的构建逻辑封装在 BuilderAssistant 类的 useNewCache 方法中，下面我们来看一下该方法的逻辑。

```
// -☆- MapperBuilderAssistant
public Cache useNewCache(Class<? extends Cache> typeClass,
    Class<? extends Cache> evictionClass, Long flushInterval,
    Integer size, boolean readWrite, boolean blocking, Properties props) {
    // 使用建造模式构建缓存实例
    Cache cache = new CacheBuilder(currentNamespace)
        .implementation(valueOrDefault(typeClass, PerpetualCache.class))
        .addDecorator(valueOrDefault(evictionClass, LruCache.class))
        .clearInterval(flushInterval).size(size)
        .readWrite(readWrite).blocking(blocking)
        .properties(props).build();

    // 添加缓存到 Configuration 对象中
    configuration.addCache(cache);
    // 设置 currentCache 遍历，即当前使用的缓存
    currentCache = cache;
    return cache;
}
```

上面使用了建造模式构建 Cache 实例,Cache 实例构建过程略为复杂,我们跟下去看看。

```
// -☆- CacheBuilder
public Cache build() {
    // 设置默认的缓存类型（PerpetualCache）和缓存装饰器（LruCache）
    setDefaultImplementations();
    // 通过反射创建缓存
    Cache cache = newBaseCacheInstance(implementation, id);
    setCacheProperties(cache);
    // 仅对内置缓存 PerpetualCache 应用装饰器
    if (PerpetualCache.class.equals(cache.getClass())) {
        // 遍历装饰器集合，应用装饰器
        for (Class<? extends Cache> decorator : decorators) {
```

```

        // 通过反射创建装饰器实例
        cache = newCacheDecoratorInstance(decorator, cache);
        // 设置属性值到缓存实例中
        setCacheProperties(cache);
    }
    // 应用标准的装饰器, 比如 LoggingCache、SynchronizedCache
    cache = setStandardDecorators(cache);
} else if (!LoggingCache.class.isAssignableFrom(cache.getClass())) {
    // 应用具有日志功能的缓存装饰器
    cache = new LoggingCache(cache);
}
return cache;
}

```

上面的构建过程流程较为复杂, 这里总结一下。如下:

1. 设置默认的缓存类型及装饰器
2. 应用装饰器到 PerpetualCache 对象上
3. 应用标准装饰器
4. 对非 LoggingCache 类型的缓存应用 LoggingCache 装饰器

在以上 4 个步骤中, 最后一步的逻辑很简单, 无需多说。下面按顺序分析前 3 个步骤对应的逻辑, 如下:

```

private void setDefaultImplementations() {
    if (implementation == null) {
        // 设置默认的缓存实现类
        implementation = PerpetualCache.class;
        if (decorators.isEmpty()) {
            // 添加 LruCache 装饰器
            decorators.add(LruCache.class);
        }
    }
}
}

```

以上代码主要做的事情是在 `implementation` 为空的情况下，为它设置一个默认值。如果大家仔细看前面的方法，会发现 MyBatis 做了不少判空的操作。比如：

```
// 判空操作 1，若用户未设置 cache 节点的 type 和 eviction 属性，
// 这里设置默认值 PERPETUAL
String type = context.getStringAttribute("type", "PERPETUAL");
String eviction = context.getStringAttribute("eviction", "LRU");

// 判空操作 2，若 typeClass 或 evictionClass 为空，
// valueOrDefault 方法会为它们设置默认值
Cache cache = new CacheBuilder(currentNamespace)
    .implementation(valueOrDefault(typeClass, PerpetualCache.class))
    .addDecorator(valueOrDefault(evictionClass, LruCache.class))
    // 省略部分代码
    .build();
```

既然前面已经做了两次判空操作，`implementation` 不可能为空，那么 `setDefaultImplementations` 方法似乎没有存在的必要了。其实不然，如果有人不按套路写代码。比如：

```
Cache cache = new CacheBuilder(currentNamespace)
    // 忘记设置 implementation
    .build();
```

这里忘记设置 `implementation`，或人为的将 `implementation` 设为空。如果不对 `implementation` 进行判空，会导致 `build` 方法在构建实例时触发空指针异常，对于框架来说，出现空指针异常是很尴尬的，这是一个低级错误。这里以及之前做了这么多判空，就是为了避免出现空指针的情况，以提高框架的健壮性。

我们在使用 MyBatis 内置缓存时，一般不用为它们配置自定义属性。但使用第三方缓存时，则应按需进行配置。比如前面演示 MyBatis 整合 Ehcache 时，就为 Ehcache 配置了一些必要的属性。下面我们来看一下这部分配置是如何设置到缓存实例中的。

```
private void setCacheProperties(Cache cache) {
    if (properties != null) {
        // 为缓存实例生成一个“元信息”实例，forObject 方法调用层次比较深，
        // 但最终调用了 MetaClass 的 forClass 方法。
        MetaObject metaCache = SystemMetaObject.forObject(cache);
        for (Map.Entry<Object, Object> entry : properties.entrySet()) {
            String name = (String) entry.getKey();
            String value = (String) entry.getValue();
            if (metaCache.hasSetter(name)) {
                // 获取 setter 方法的参数类型
                Class<?> type = metaCache.getSetterType(name);

                // 根据参数类型对属性值进行转换，并将转换后的值
                // 通过 setter 方法设置到 Cache 实例中
                if (String.class == type) {
                    metaCache.setValue(name, value);
                } else if (int.class == type || Integer.class == type) {
                    /*
                     * 此处及以下分支包含两个步骤：
                     * 1.类型转换 → Integer.valueOf(value)
                     * 2.将转换后的值设置到缓存实例中 →
                     *     metaCache.setValue(name, value)
                     */
                    metaCache.setValue(name, Integer.valueOf(value));
                } else if (long.class == type || Long.class == type) {
                    metaCache.setValue(name, Long.valueOf(value));
                }
                else if (short.class == type || Short.class == type) {...}
                else if (byte.class == type || Byte.class == type) {...}
                else if (float.class == type || Float.class == type) {...}
                else if (boolean.class == type || Boolean.class == type) {...}
                else if (double.class == type || Double.class == type) {...}
                else { throw new CacheException("....."); }
            }
        }
    }
}
```

```

        }
    }
}

// 如果缓存类实现了 InitializingObject 接口，
// 则调用 initialize 方法执行初始化逻辑
if (InitializingObject.class.isAssignableFrom(cache.getClass())) {
    try {
        ((InitializingObject) cache).initialize();
    } catch (Exception e) {
        throw new CacheException(".....");
    }
}
}
}

```

上面的大段代码用于对属性值进行类型转换，和设置转换后的值到 Cache 实例中。关于上面代码中出现的 MetaObject，大家可以自己尝试分析一下。最后，我们来看一下设置标准装饰器的过程。如下：

```

private Cache setStandardDecorators(Cache cache) {
    try {
        // 创建“元信息”对象
        MetaObject metaCache = SystemMetaObject.forObject(cache);
        if (size != null && metaCache.hasSetter("size")) {
            // 设置 size 属性，
            metaCache.setValue("size", size);
        }
        if (clearInterval != null) {
            // clearInterval 不为空，应用 ScheduledCache 装饰器
            cache = new ScheduledCache(cache);
            ((ScheduledCache) cache).setClearInterval(clearInterval);
        }
    }
}

```

```

    if (readWrite) {
        // readWrite 为 true, 应用 SerializedCache 装饰器
        cache = new SerializedCache(cache);
    }
    /*
     * 应用 LoggingCache, SynchronizedCache 装饰器,
     * 使原缓存具备打印日志和线程同步的能力
     */
    cache = new LoggingCache(cache);
    cache = new SynchronizedCache(cache);
    if (blocking) {
        // blocking 为 true, 应用 BlockingCache 装饰器
        cache = new BlockingCache(cache);
    }
    return cache;
} catch (Exception e) {
    throw new CacheException(".....");
}
}

```

以上代码用于为缓存应用一些基本的装饰器，除了 LoggingCache 和 SynchronizedCache 这两个是必要的装饰器，其他的装饰器应用与否，取决于用户的配置。。

3.2.2 解析<cache-ref>节点

在 MyBatis 中，二级缓存是可以共用的。这需要通过<cache-ref>节点为命名空间配置参照缓存，比如像下面这样。

```

<!-- Mapper1.xml -->
<mapper namespace="xyz.coolblog.dao.Mapper1">
    <!-- Mapper1 与 Mapper2 共用一个二级缓存 -->
    <cache-ref namespace="xyz.coolblog.dao.Mapper2"/>

```



```
</mapper>

<!-- Mapper2.xml -->
<mapper namespace="xyz.coolblog.dao.Mapper2">
    <cache/>
</mapper>
```

接下来，我们对照上面的配置分析 cache-ref 的解析过程。

```
private void cacheRefElement(XNode context) {
    if (context != null) {
        configuration.addCacheRef(builderAssistant.getCurrentNamespace(),
            context.getStringAttribute("namespace"));
        // 创建 CacheRefResolver 实例
        CacheRefResolver cacheRefResolver = new CacheRefResolver(
            builderAssistant, context.getStringAttribute("namespace"));
        try {
            // 解析参照缓存
            cacheRefResolver.resolveCacheRef();
        } catch (IncompleteElementException e) {
            // 捕捉 IncompleteElementException 异常，并将 cacheRefResolver
            // 存入到 Configuration 的 incompleteCacheRefs 集合中
            configuration.addIncompleteCacheRef(cacheRefResolver);
        }
    }
}
```

上所示，<cache-ref>节点的解析逻辑封装在了 CacheRefResolver 的 resolveCacheRef 方法中，我们一起看一下这个方法的逻辑。

```
// ☆- CacheRefResolver
public Cache resolveCacheRef() {
```

```
// 调用 builderAssistant 的 useNewCache(namespace) 方法
return assistant.useCacheRef(cacheRefNamespace);
}

// -☆- MapperBuilderAssistant
public Cache useCacheRef(String namespace) {
    if (namespace == null) {
        throw new BuilderException(".....");
    }
    try {
        unresolvedCacheRef = true;
        // 根据命名空间从全局配置对象（Configuration）中查找相应的缓存实例
        Cache cache = configuration.getCache(namespace);

        /*
         * 若未查找到缓存实例，此处抛出异常。这里存在两种情况导致未查找到
         * cache 实例，分别如下：
         *     1.使用者在 <cache-ref> 中配置了一个不存在的命名空间，
         *     导致无法找到 cache 实例
         *     2.使用者所引用的缓存实例还未创建
         */
        if (cache == null) {
            throw new IncompleteElementException(".....");
        }

        // 设置 cache 为当前使用缓存
        currentCache = cache;
        unresolvedCacheRef = false;
        return cache;
    } catch (IllegalArgumentException e) {
        throw new IncompleteElementException(".....");
    }
}
```

以上是 cache-ref 的解析过程，逻辑并不复杂。不过这里要注意 cache 为空的情况，我在代码中已经注释了可能导致 cache 为空的两种情况。第一种情况比较好理解，第二种情况稍微复杂点，但是也不难理解。我会在 3.4 节进行解释说明，这里先不分析。

3.2.3 解析<resultMap>节点

resultMap 是 MyBatis 框架中常用的特性，主要用于映射结果。resultMap 是 MyBatis 提供的一个强力武器，这一点官方文档中有所描述，这里引用一下。

resultMap 元素是 MyBatis 中最重要最强大的元素。它可以让你从 90% 的 JDBC ResultSets 数据提取代码中解放出来，并在一些情形下允许你做一些 JDBC 不支持的事情。实际上，在对复杂语句进行联合映射的时候，它很可能可以代替数千行的同等功能的代码。ResultMap 的设计思想是，简单的语句不需要明确的结果映射，而复杂一点的语句只需要描述它们的关系就行了。

resultMap 元素是 MyBatis 中最重要最强大的元素，它可以把大家从 JDBC ResultSets 数据提取的工作中解放出来。通过 resultMap 和自动映射，可以让 MyBatis 帮助我们完成 ResultSet → Object 的映射，这将会大大提高了开发效率。关于 resultMap 的用法，我相信大家都比较熟悉了，所以这里我就不介绍了。当然，如果大家不熟悉也没关系，MyBatis 的[官方文档](#)上对此进行了详细的介绍，大家不妨去看看。

好了，其他的就不多说了，下面开始分析 resultMap 配置的解析过程。

```
// -☆- XMLMapperBuilder
private void resultMapElements(List<XNode> list) throws Exception {
    // 遍历 <resultMap> 节点列表
    for (XNode resultMapNode : list) {
        try {
            // 解析 resultMap 节点
            resultMapElement(resultMapNode);
        } catch (IncompleteElementException e) {
        }
    }
}
```

```
private ResultMap resultMapElement(XNode resultMapNode) throws Exception {
    // 调用重载方法
    return resultMapElement(resultMapNode,
        Collections.<ResultMapping>emptyList());
}

private ResultMap resultMapElement(XNode resultMapNode,
    List<ResultMapping> additionalResultMappings) throws Exception {
    ErrorContext.instance().activity("processing " +
        resultMapNode.getValueBasedIdentifier());

    // 获取 id 和 type 属性
    String id = resultMapNode.getStringAttribute("id",
        resultMapNode.getValueBasedIdentifier());
    String type = resultMapNode.getStringAttribute("type",
        resultMapNode.getStringAttribute("ofType",
            resultMapNode.getStringAttribute("resultType",
                resultMapNode.getStringAttribute("javaType"))));
    // 获取 extends 和 autoMapping
    String extend = resultMapNode.getStringAttribute("extends");
    Boolean autoMapping = resultMapNode.getBooleanAttribute("autoMapping");

    // 解析 type 属性对应的类型
    Class<?> typeClass = resolveClass(type);
    Discriminator discriminator = null;
    List<ResultMapping> resultMappings = new ArrayList<ResultMapping>();
    resultMappings.addAll(additionalResultMappings);

    // 获取并遍历 <resultMap> 的子节点列表
    List<XNode> resultChildren = resultMapNode.getChildren();
    for (XNode resultChild : resultChildren) {
        if ("constructor".equals(resultChild.getName())) {
            // 解析 constructor 节点, 并生成相应的 ResultMapping

```

```
        processConstructorElement(resultChild, typeClass, resultMappings);
    } else if ("discriminator".equals(resultChild.getName())) {
        // 解析 discriminator 节点
        discriminator = processDiscriminatorElement(
            resultChild, typeClass, resultMappings);
    } else {
        List<ResultFlag> flags = new ArrayList<ResultFlag>();
        if ("id".equals(resultChild.getName())) {
            // 添加 ID 到 flags 集合中
            flags.add(ResultFlag.ID);
        }
        // 解析 id 和 property 节点, 并生成相应的 ResultMapping
        resultMappings.add(buildResultMappingFromContext(
            resultChild, typeClass, flags));
    }
}

ResultMapResolver resultMapResolver = new ResultMapResolver(
    builderAssistant, id, typeClass, extend, discriminator,
    resultMappings, autoMapping);

try {
    // 根据前面获取到的信息构建 ResultMap 对象
    return resultMapResolver.resolve();
} catch (IncompleteElementException e) {
    /*
     * 如果发生 IncompleteElementException 异常,
     * 这里将 resultMapResolver 添加到 incompleteResultMaps 集合中
     */
    configuration.addIncompleteResultMap(resultMapResolver);
    throw e;
}
}
```

上面的代码比较多，看起来有点复杂，这里总结一下：

1. 获取<resultMap>节点的各种属性
2. 遍历<resultMap>的子节点，并根据子节点名称执行相应的解析逻辑
3. 构建 ResultMap 对象
4. 若构建过程中发生异常，则将 resultMapResolver 添加到 incompleteResultMaps 集合中

在上面流程，第1步和最后一步都是一些常规操作，无需过多解释。第2步和第3步分别是<resultMap>节点的子节点解析过程，以及 ResultMap 对象的构建过程。这两个过程是接下来要重点分析，比较重要。上面代码中还出现了鉴别器 discriminator 相关逻辑，鉴别器不是很常用的特性，大家知道它有什么用就行了，本章就不分析了。

1. 解析<id>和<result>节点

在<resultMap>节点中，子节点<id>和<result>都是常规配置，比较常见，相信大家对此也比较熟悉了。那下面我们直接分析这两个节点的解析过程。

```
private ResultMapping buildResultMappingFromContext(XNode context, Class<?>
resultType, List<ResultFlag> flags) throws Exception {
    String property;
    // 根据节点类型获取 name 或 property 属性
    if (flags.contains(ResultFlag.CONSTRUCTOR)) {
        property = context.getStringAttribute("name");
    } else {
        property = context.getStringAttribute("property");
    }

    // 获取其他各种属性
    String column = context.getStringAttribute("column");
    String javaType = context.getStringAttribute("javaType");
    String jdbcType = context.getStringAttribute("jdbcType");
    String nestedSelect = context.getStringAttribute("select");

    // 解析 resultMap 属性，该属性出现在 <association> 和 <collection> 节点中。
```

```
// 若这两个节点不包含 resultMap 属性，则调用 processNestedResultMappings 方法
// 解析嵌套 resultMap。
String nestedResultMap = context.getStringAttribute("resultMap",
    processNestedResultMappings(context,
        Collections.<ResultMapping>emptyList()));

String notNullColumn = context.getStringAttribute("notNullColumn");
String columnPrefix = context.getStringAttribute("columnPrefix");
String typeHandler = context.getStringAttribute("typeHandler");
String resultSet = context.getStringAttribute("resultSet");
String foreignColumn = context.getStringAttribute("foreignColumn");
boolean lazy = "lazy".equals(context.getStringAttribute("fetchType",
    configuration.isLazyLoadingEnabled() ? "lazy" : "eager"));

// 解析 javaType、typeHandler 的类型以及枚举类型 JdbcType
Class<?> javaTypeClass = resolveClass(javaType);
Class<? extends TypeHandler<?>> typeHandlerClass =
    (Class<? extends TypeHandler<?>>) resolveClass(typeHandler);
JdbcType jdbcTypeEnum = resolveJdbcType(jdbcType);

// 构建 ResultMapping 对象
return builderAssistant.buildResultMapping(
    resultType, property, column, javaTypeClass, jdbcTypeEnum,
    nestedSelect, nestedResultMap, notNullColumn, columnPrefix,
    typeHandlerClass, flags, resultSet, foreignColumn, lazy);
}
```

上面的方法主要用于获取<id>和<result>节点的属性，其中，resultMap 属性的解析过程要相对复杂一些。该属性存在于<association>和<collection>节点中。下面以<association>节点为例，演示该节点的两配置方式，分别如下：

第一种配置方式是通过 resultMap 属性引用其他的<resultMap>节点，配置如下：

```

<resultMap id="articleResult" type="Article">
  <id property="id" column="id"/>
  <result property="title" column="article_title"/>
  <association property="article_author" column="article_author_id"
    resultMap="authorResult"/>
</resultMap>

<resultMap id="authorResult" type="Author">
  <id property="id" column="author_id"/>
  <result property="name" column="author_name"/>
</resultMap>

```

第二种配置方式是采取 resultMap 嵌套的方式进行配置，如下：

```

<resultMap id="articleResult" type="Article">
  <id property="id" column="id"/>
  <result property="title" column="article_title"/>
  <association property="article_author" javaType="Author">
    <id property="id" column="author_id"/>
    <result property="name" column="author_name"/>
  </association>
</resultMap>

```

如上配置，<association>的子节点是一些结果映射配置，这些结果配置最终也会被解析成 ResultMap。我们可以看看解析过程是怎样的，如下：

```

private String processNestedResultMappings(XNode context,
List<ResultMapping> resultMappings) throws Exception {
    // 判断节点名称
    if ("association".equals(context.getName())
        || "collection".equals(context.getName())
        || "case".equals(context.getName())) {

```



```

        if (context.getStringAttribute("select") == null) {
            // resultMapElement 是解析 resultMap 入口方法
            ResultMap resultMap = resultMapElement(context, resultMappings);
            // 返回 resultMap id
            return resultMap.getId();
        }
    }
    return null;
}

```

如上，<association>的子节点由 resultMapElement 方法解析成 ResultMap，并在最后返回 resultMap.id。对于<resultMap>节点，id 的值配置在该节点的 id 属性中。但<association>节点无法配置 id 属性，那么该 id 如何产生的呢？答案在 XNode 类的 getValueBasedIdentifier 方法中，这个方法具体逻辑这里就不分析了。下面直接看一下以上配置中的<association>节点解析成 ResultMap 后的 id 值，如下：

```
id = mapper_resultMap[articleResult]_association[article_author]
```

关于嵌套 resultMap 的解析逻辑就先分析到这，下面分析 ResultMapping 的构建过程。

```

public ResultMapping buildResultMapping(Class<?> resultType,
    String property, String column, Class<?> javaType,
    JdbcType jdbcType, String nestedSelect, String nestedResultMap,
    String notNullColumn, String columnPrefix,
    Class<? extends TypeHandler<?>> typeHandler, List<ResultFlag> flags,
    String resultSet, String foreignColumn, boolean lazy) {

    // 若 javaType 为空，这里根据 property 的属性进行解析。关于下面方法中的参数，
    // 这里说明一下：
    //   - resultType: 即 <resultMap type="xxx"/> 中的 type 属性
    //   - property: 即 <result property="xxx"/> 中的 property 属性
    Class<?> javaTypeClass =
        resolveResultJavaType(resultType, property, javaType);
}

```

```

// 解析 TypeHandler
TypeHandler<?> typeHandlerInstance =
    resolveTypeHandler(javaTypeClass, typeHandler);

// 解析 column = {property1=column1, property2=column2} 的情况,
// 这里会将 column 拆分成多个 ResultMapping
List<ResultMapping> composites = parseCompositeColumnName(column);

// 通过建造模式构建 ResultMapping
return new ResultMapping
    .Builder(configuration, property, column, javaTypeClass)
    .jdbcType(jdbcType)
    .nestedQueryId(applyCurrentNamespace(nestedSelect, true))
    .nestedResultMapId(applyCurrentNamespace(nestedResultMap, true))
    .resultSet(resultSet).typeHandler(typeHandlerInstance)
    .flags(flags == null ? new ArrayList<ResultFlag>() : flags)
    .composites(composites)
    .notNullColumns(parseMultipleColumnNames(notNullColumn))
    .columnPrefix(columnPrefix).foreignColumn(foreignColumn)
    .lazy(lazy).build();
}

// -☆- ResultMapping.Builder
public ResultMapping build() {
    // 将 flags 和 composites 两个集合变为不可修改集合
    resultMapping.flags = Collections.unmodifiableList(resultMapping.flags);
    resultMapping.composites =
        Collections.unmodifiableList(resultMapping.composites);
    // 从 TypeHandlerRegistry 中获取相应 TypeHandler
    resolveTypeHandler();
    validate();
    return resultMapping;
}

```

ResultMapping 的构建过程不是很复杂，首先是解析 javaType 类型，并创建 typeHandler 实例。然后处理复合 column。最后通过建造器构建 ResultMapping 实例。关于上面方法中出现的一些方法调用，这里接不跟下去分析了，大家可以自己看看。

2. 解析<constructor>节点

一般情况下，我们所定义的实体类都是简单的 Java 对象，即 POJO。这种对象包含一些私有属性和相应的 getter/setter 方法，通常这种 POJO 可以满足大部分需求。但如果你想使用不可变类存储查询结果，则就需要做一些改动。比如把 POJO 的 setter 方法移除，增加构造方法用于初始化成员变量。对于这种不可变的 Java 类，需要通过带有参数的构造方法进行初始化（反射也可以达到同样目的）。下面举个例子说明一下：

```
public class Article {  
  
    // ...  
  
    public Article(Integer id, String title, String content) {  
        this.id = id;  
        this.title = title;  
        this.content = content;  
    }  
}
```

如上，Article 的构造方法对应的配置如下：

```
<constructor>  
    <idArg column="id" name="id"/>  
    <arg column="title" name="title"/>  
    <arg column="content" name="content"/>  
</constructor>
```

下面，分析 constructor 节点的解析过程。如下：

```

private void processConstructorElement(XNode resultChild,
    Class<?> resultType, List<ResultMapping> resultMappings)
    throws Exception {
    // 获取子节点列表
    List<XNode> argChildren = resultChild.getChildren();
    for (XNode argChild : argChildren) {
        List<ResultFlag> flags = new ArrayList<ResultFlag>();
        // 向 flags 中添加 CONSTRUCTOR 标志
        flags.add(ResultFlag.CONSTRUCTOR);
        if ("idArg".equals(argChild.getName())) {
            // 向 flags 中添加 ID 标志
            flags.add(ResultFlag.ID);
        }
        // 构建 ResultMapping, 上一节已经分析过
        resultMappings.add(
            buildResultMappingFromContext(argChild, resultType, flags));
    }
}

```

上面方法的逻辑并不复杂。首先是获取并遍历子节点列表，然后为每个子节点创建 flags 集合，并添加 CONSTRUCTOR 标志。对于 idArg 节点，额外添加 ID 标志。最后一步则是构建 ResultMapping，该步逻辑前面已经分析过，这里就不多说了。

3. ResultMap 对象构建过程分析

前面用了不少的篇幅来分析<resultMap>子节点的解析过程。通过前面的分析，我们可知<id>，<result>等节点最终都被解析成了 ResultMapping。在得到这些 ResultMapping 后，紧接着要做的事情是构建 ResultMap。如果说 ResultMapping 与单条结果映射相对应，那 ResultMap 与什么对应呢？答案是…。答案暂时还不能说，我们到源码中去找寻吧。下面，让我们带着这个疑问开始本节的源码分析。

前面分析了很多源码，大家可能都忘了 ResultMap 构建的入口了。这里再贴一下，如下：

```
private ResultMap resultMapElement(XNode resultMapNode,
    List<ResultMapping> additionalResultMappings) throws Exception {

    // 获取 resultMap 节点中的属性
    // ...

    // 解析 resultMap 对应的类型
    // ...

    // 遍历 resultMap 节点的子节点, 构建 ResultMapping 对象
    // ...

    // 创建 ResultMap 解析器
    ResultMapResolver resultMapResolver = new ResultMapResolver(
        builderAssistant, id, typeClass, extend, discriminator,
        resultMappings, autoMapping);
    try {
        // 根据前面获取到的信息构建 ResultMap 对象
        return resultMapResolver.resolve();
    } catch (IncompleteElementException e) {
        configuration.addIncompleteResultMap(resultMapResolver);
        throw e;
    }
}
```

如上, ResultMap 的构建逻辑分装在 ResultMapResolver 的 resolve 方法中, 下面我从该方法进行分析。

```
// -☆- ResultMapResolver
public ResultMap resolve() {
    return assistant.addResultMap(this.id, this.type, this.extend,
        this.discriminator, this.resultMappings, this.autoMapping);
}
```

上面的方法将构建 `ResultMap` 实例的任务委托给了 `MapperBuilderAssistant` 的 `addResultMap`，我们跟进到这个方法中看看。

```
// -☆- MapperBuilderAssistant
public ResultMap addResultMap(
    String id, Class<?> type, String extend, Discriminator discriminator,
    List<ResultMapping> resultMappings, Boolean autoMapping) {

    // 为 ResultMap 的 id 和 extend 属性值拼接命名空间
    id = applyCurrentNamespace(id, false);
    extend = applyCurrentNamespace(extend, true);

    if (extend != null) {
        if (!configuration.hasResultMap(extend)) {
            throw new IncompleteElementException(".....");
        }

        ResultMap resultMap = configuration.getResultMap(extend);
        List<ResultMapping> extendedResultMappings = new
            ArrayList<ResultMapping>(resultMap.getResultMappings());
        // 为拓展 ResultMappings 取出重复项
        extendedResultMappings.removeAll(resultMappings);

        boolean declaresConstructor = false;
        // 检测当前 resultMappings 集合中是否包含 CONSTRUCTOR 标志的元素
        for (ResultMapping resultMapping : resultMappings) {
            if (resultMapping.getFlags().contains(ResultFlag.CONSTRUCTOR)) {
                declaresConstructor = true;
                break;
            }
        }

        // 如果当前 <resultMap> 节点中包含 <constructor> 子节点，
```

```
// 则将拓展 ResultMapping 集合中的包含 CONSTRUCTOR 标志的元素移除
if (declaresConstructor) {
    Iterator<ResultMapping> extendedResultMappingsIter =
        extendedResultMappings.iterator();
    while (extendedResultMappingsIter.hasNext()) {
        if (extendedResultMappingsIter.next().getFlags()
            .contains(ResultFlag.CONSTRUCTOR)) {
            extendedResultMappingsIter.remove();
        }
    }
}

// 将扩展 resultMappings 集合合并到当前 resultMappings 集合中
resultMappings.addAll(extendedResultMappings);
}

// 构建 ResultMap
ResultMap resultMap = new ResultMap
    .Builder(configuration, id, type, resultMappings, autoMapping)
    .discriminator(discriminator)
    .build();
configuration.addResultMap(resultMap);
return resultMap;
}
```

上面的方法主要用于处理 resultMap 节点的 extend 属性，extend 不为空的话，这里将当前 resultMappings 集合和扩展 resultMappings 集合合并二为一。随后，通过建造模式构建 ResultMap 实例。过程如下：

```
// -☆- ResultMap
public ResultMap build() {
    if (resultMap.id == null) {
        throw new IllegalArgumentException("ResultMaps must have an id");
    }
}
```

```
}
resultMap.mappedColumns = new HashSet<String>();
resultMap.mappedProperties = new HashSet<String>();
resultMap.idResultMappings = new ArrayList<ResultMapping>();
resultMap.constructorResultMappings = new ArrayList<ResultMapping>();
resultMap.propertyResultMappings = new ArrayList<ResultMapping>();
final List<String> constructorArgNames = new ArrayList<String>();

for (ResultMapping resultMapping : resultMap.resultMappings) {
    // 检测 <association> 或 <collection> 节点
    // 是否包含 select 和 resultMap 属性
    resultMap.hasNestedQueries = resultMap.hasNestedQueries ||
        resultMapping.getNestedQueryId() != null;
    resultMap.hasNestedResultMaps = resultMap.hasNestedResultMaps ||
        (resultMapping.getNestedResultMapId() != null &&
            resultMapping.getResultSet() == null);

    final String column = resultMapping.getColumn();
    if (column != null) {
        // 将 column 转换成大写, 并添加到 mappedColumns 集合中
        resultMap.mappedColumns.add(column.toUpperCase(Locale.ENGLISH));
    } else if (resultMapping.isCompositeResult()) {
        for (ResultMapping compositeResultMapping :
            resultMapping.getComposites()) {
            final String compositeColumn =
                compositeResultMapping.getColumn();
            if (compositeColumn != null) {
                resultMap.mappedColumns.add(
                    compositeColumn.toUpperCase(Locale.ENGLISH));
            }
        }
    }
}
```



```
// 添加属性 property 到 mappedProperties 集合中
final String property = resultMapping.getProperty();
if (property != null) {
    resultMap.mappedProperties.add(property);
}

// 检测当前 resultMapping 是否包含 CONSTRUCTOR 标志
if (resultMapping.getFlags().contains(ResultFlag.CONSTRUCTOR)) {
    // 添加 resultMapping 到 constructorResultMappings 中
    resultMap.constructorResultMappings.add(resultMapping);
    // 添加属性 (constructor 节点的 name 属性) 到 constructorArgNames 中
    if (resultMapping.getProperty() != null) {
        constructorArgNames.add(resultMapping.getProperty());
    }
} else {
    // 添加 resultMapping 到 propertyResultMappings 中
    resultMap.propertyResultMappings.add(resultMapping);
}

if (resultMapping.getFlags().contains(ResultFlag.ID)) {
    // 添加 resultMapping 到 idResultMappings 中
    resultMap.idResultMappings.add(resultMapping);
}

}

if (resultMap.idResultMappings.isEmpty()) {
    resultMap.idResultMappings.addAll(resultMap.resultMappings);
}

if (!constructorArgNames.isEmpty()) {
    // 获取构造方法参数列表，篇幅原因，这个方法不分析了
    final List<String> actualArgNames =
        argNamesOfMatchingConstructor(constructorArgNames);
}
```

```

        if (actualArgNames == null) {
            throw new BuilderException(".....");
        }

        // 对 constructorResultMappings 按照构造方法参数列表的顺序进行排序
        Collections.sort(resultMap.constructorResultMappings,
            new Comparator<ResultMapping>() {

                @Override
                public int compare(ResultMapping o1, ResultMapping o2) {
                    int paramIdx1 = actualArgNames.indexOf(o1.getProperty());
                    int paramIdx2 = actualArgNames.indexOf(o2.getProperty());
                    return paramIdx1 - paramIdx2;
                }
            });
    }

    // 将以下这些集合变为不可修改集合
    resultMap.resultMappings =
        Collections.unmodifiableList(resultMap.resultMappings);
    resultMap.idResultMappings =
        Collections.unmodifiableList(resultMap.idResultMappings);
    resultMap.constructorResultMappings =
        Collections.unmodifiableList(resultMap.constructorResultMappings);
    resultMap.propertyResultMappings =
        Collections.unmodifiableList(resultMap.propertyResultMappings);
    resultMap.mappedColumns =
        Collections.unmodifiableSet(resultMap.mappedColumns);

    return resultMap;
}

```

以上代码看起来很复杂, 实际上这是假象。以上代码主要做的事情就是将 ResultMapping 实例及属性分别存储到不同的集合中, 仅此而已。ResultMap 中定义了五种不同的集合, 下面分别介绍一下这几种集合。

集合名称	用途
mappedColumns	用于存储 <id>、<result>、<idArg>、<arg> 节点 column 属性
mappedProperties	用于存储 <id> 和 <result> 节点的 property 属性, 或 <idArgs> 和 <arg> 节点的 name 属性
idResultMappings	用于存储 <id> 和 <idArg> 节点对应的 ResultMapping 对象
propertyResultMappings	用于存储 <id> 和 <result> 节点对应的 ResultMapping 对象
constructorResultMappings	用于存储 <idArgs> 和 <arg> 节点对应的 ResultMapping 对象

上面干巴巴的描述不够直观。下面我们写点代码测试一下, 并把这些集合的内容打印到控制台上, 大家直观感受一下。先定义一个映射文件, 如下:

```
<mapper namespace="xyz.coolblog.chapter3.dao.ArticleDao">
  <resultMap id="articleResult" type="xyz.coolblog.chapter3.model.Article">
    <constructor>
      <idArg column="id" name="id"/>
      <arg column="title" name="title"/>
      <arg column="content" name="content"/>
    </constructor>
    <id property="id" column="id"/>
    <result property="author" column="author"/>
    <result property="createTime" column="create_time"/>
  </resultMap>
</mapper>
```

测试代码如下:

```
public class ResultMapTest {

    @Test
    public void printResultMapInfo() throws Exception {
        Configuration configuration = new Configuration();
        String resource = "chapter3/mapper/ArticleMapper.xml";
        InputStream inputStream = Resources.getResourceAsStream(resource);
        XMLMapperBuilder builder = new XMLMapperBuilder(inputStream,
            configuration, resource, configuration.getSqlFragments());
        builder.parse();

        ResultMap resultMap = configuration.getResultMap("articleResult");

        System.out.println("\n-----+💡 mappedColumns 💡+-----");
        System.out.println(resultMap.getMappedColumns());

        System.out.println("\n-----+💡 mappedProperties 💡+-----");
        System.out.println(resultMap.getMappedProperties());

        System.out.println("\n-----+💡 idResultMappings 💡+-----");
        resultMap.getIdResultMappings().forEach(
            rm -> System.out.println(simplify(rm)));

        System.out.println("\n-----+💡 propertyResultMappings 💡+-----");
        resultMap.getPropertyResultMappings().forEach(
            rm -> System.out.println(simplify(rm)));

        System.out.println("\n-----+💡 constructorResultMappings 💡+-----");
        resultMap.getConstructorResultMappings().forEach(
            rm -> System.out.println(simplify(rm)));

        System.out.println("\n-----+💡 resultMappings 💡+-----");
```

```

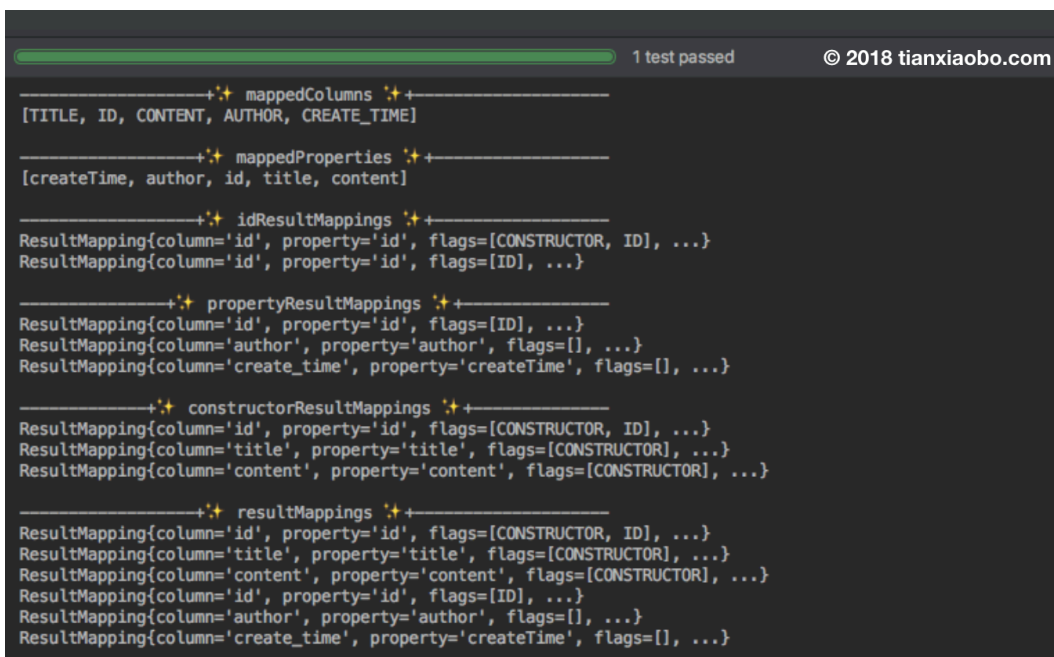
        resultMap.getResultMappings().forEach(
            rm -> System.out.println(simplify(rm));

        inputStream.close();
    }

    /** 简化 ResultMapping 输出结果 */
    private String simplify(ResultMapping resultMap) {
        return String.format(
            "ResultMapping{column='%s', property='%s', flags=%s, ...}",
            resultMap.getColumn(), resultMap.getProperty(),
            resultMap.getFlags());
    }
}

```

我们把 5 个集合转给你的内容都打印出来，结果如下：



```

1 test passed © 2018 tianxiaobo.com

-----+* mappedColumns *+-----
[TITLE, ID, CONTENT, AUTHOR, CREATE_TIME]

-----+* mappedProperties *+-----
[createTime, author, id, title, content]

-----+* idResultMappings *+-----
ResultMapping{column='id', property='id', flags=[CONSTRUCTOR, ID], ...}
ResultMapping{column='id', property='id', flags=[ID], ...}

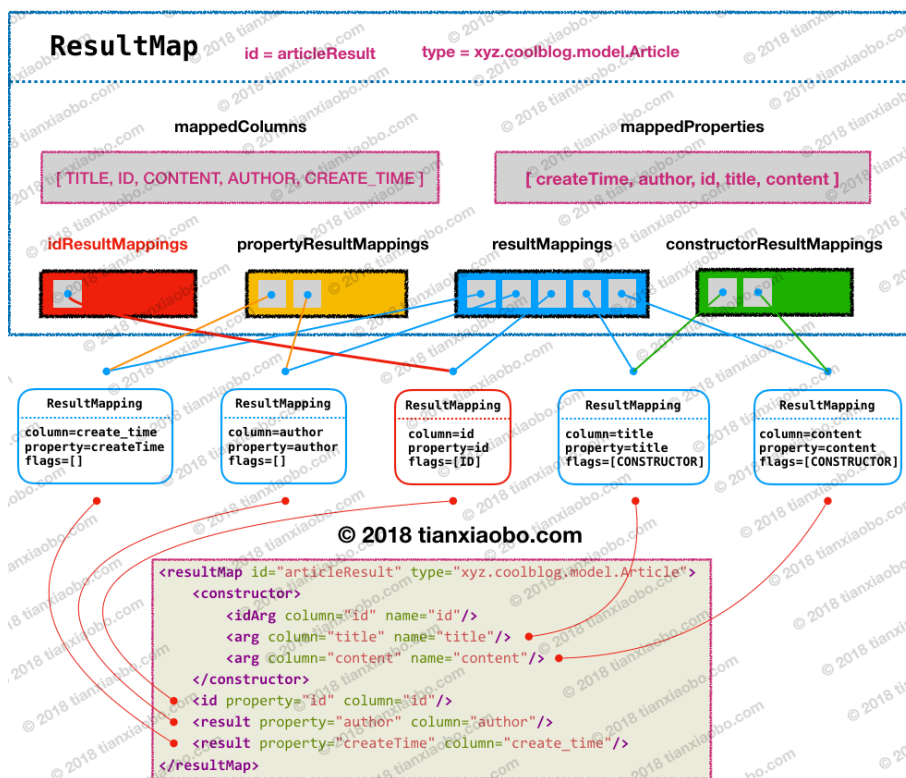
-----+* propertyResultMappings *+-----
ResultMapping{column='id', property='id', flags=[ID], ...}
ResultMapping{column='author', property='author', flags=[], ...}
ResultMapping{column='create_time', property='createTime', flags=[], ...}

-----+* constructorResultMappings *+-----
ResultMapping{column='id', property='id', flags=[CONSTRUCTOR, ID], ...}
ResultMapping{column='title', property='title', flags=[CONSTRUCTOR], ...}
ResultMapping{column='content', property='content', flags=[CONSTRUCTOR], ...}

-----+* resultMappings *+-----
ResultMapping{column='id', property='id', flags=[CONSTRUCTOR, ID], ...}
ResultMapping{column='title', property='title', flags=[CONSTRUCTOR], ...}
ResultMapping{column='content', property='content', flags=[CONSTRUCTOR], ...}
ResultMapping{column='id', property='id', flags=[ID], ...}
ResultMapping{column='author', property='author', flags=[], ...}
ResultMapping{column='create_time', property='createTime', flags=[], ...}

```

如上，结果比较清晰明了，不需要过多解释了。我们参照上面配置文件及输出的结果，把 resultMap 的大致轮廓画出来。如下：



到这里，<resultMap>节点的解析过程就分析完了。总的来说，该节点的解析过程还是比较复杂的。好了，其他的就不多说了，继续后面的分析。

3.2.4 解析<sql>节点

<sql>节点用来定义一些可重用的 SQL 语句片段，比如表名，或表的列名等。在映射文件中，我们可以通过<include>节点引用<sql>节点定义的内容。下面我来演示一下<sql>节点的使用方式，如下：

```

<sql id="table">
  article
</sql>

```

```
<select id="findOne" resultType="Article">
    SELECT id, title FROM <include refid="table"/> WHERE id = #{id}
</select>

<update id="update" parameterType="Article">
    UPDATE <include refid="table"/> SET title = #{title} WHERE id = #{id}
</update>
```

如上，上面配置中，`<select>`和`<update>`节点通过`<include>`引入定义在`<sql>`节点中的表名。上面的配置比较常规，除了静态文本，`<sql>`节点还支持属性占位符`${}`。比如：

```
<sql id="table">
    ${table_prefix}_article
</sql>
```

如果属性 `table_prefix = blog`，那么`<sql>`节点中的内容最终为 `blog_article`。了解了`<sql>`节点的用法，下面分析一下 `sql` 节点的解析过程，如下：

```
private void sqlElement(List<XNode> list) throws Exception {
    if (configuration.getDatabaseId() != null) {
        // 调用 sqlElement 解析 <sql> 节点
        sqlElement(list, configuration.getDatabaseId());
    }

    // 再次调用 sqlElement，不同的是，这次调用，该方法的第二个参数为 null
    sqlElement(list, null);
}
```

这个方法需要大家注意一下，如果 `Configuration` 的 `databaseId` 不为空，`sqlElement` 方法会被调用了两次。第一次传入具体的 `databaseId`，用于解析带有 `databaseId` 属性，且属性值与此相等的`<sql>`节点。第二次传入的 `databaseId` 为空，用于解析未配置 `databaseId` 属性的`<sql>`节点。这里是个小细节，大家注意一下就好。我们继续往下分析。

```
private void sqlElement(List<XNode> list, String requiredDatabaseId)
    throws Exception {
    for (XNode context : list) {
        // 获取 id 和 databaseId 属性
        String databaseId = context.getStringAttribute("databaseId");
        String id = context.getStringAttribute("id");

        // id = currentNamespace + "." + id
        id = builderAssistant.applyCurrentNamespace(id, false);

        // 检测当前 databaseId 和 requiredDatabaseId 是否一致
        if (databaseIdMatchesCurrent(id, databaseId, requiredDatabaseId)) {
            // 将 <id, XNode> 键值对缓存到 sqlFragments 中
            sqlFragments.put(id, context);
        }
    }
}
```

这个方法逻辑比较简单，首先是获取<sql>节点的 id 和 databaseId 属性，然后为 id 属性值拼接命名空间。最后，通过检测当前 databaseId 和 requiredDatabaseId 是否一致，来决定保存还是忽略当前的<sql>节点。下面，我们来看一下 databaseId 的匹配逻辑是怎样的。

```
private boolean databaseIdMatchesCurrent(
    String id, String databaseId, String requiredDatabaseId) {
    if (requiredDatabaseId != null) {
        // 当前 databaseId 和目标 databaseId 不一致时，返回 false
        if (!requiredDatabaseId.equals(databaseId)) {
            return false;
        }
    } else {
        // 如果目标 databaseId 为空，但当前 databaseId 不为空。两者不一致，返回 false
    }
}
```



```
        if (databaseId != null) {
            return false;
        }
        // 如果当前 <sql> 节点的 id 与之前的 <sql> 节点重复, 且先前节点
        // databaseId 不为空。则忽略当前节点, 并返回 false
        if (this.sqlFragments.containsKey(id)) {
            XNode context = this.sqlFragments.get(id);
            if (context.getStringAttribute("databaseId") != null) {
                return false;
            }
        }
    }
    return true;
}
```

这里总结一下 databaseId 的匹配规则。

1. databaseId 与 requiredDatabaseId 不一致, 即失配, 返回 false
2. 当前节点与之前的节点出现 id 重复的情况, 若之前的<sql>节点 databaseId 属性
3. 不为空, 返回 false
4. 若以上两条规则均匹配失败, 此时返回 true

在上面三条匹配规则中, 第二条规则稍微难理解一点。这里简单分析一下, 考虑下面这种配置。

```
<!-- databaseId 不为空 -->
<sql id="table" databaseId="mysql">
    article
</sql>

<!-- databaseId 为空 -->
<sql id="table">
    article
</sql>
```

在上面配置中，两个`<sql>`节点的 `id` 属性值相同，`databaseId` 属性不一致。假设 `configuration.databaseId = mysql`，第一次调用 `sqlElement` 方法，第一个`<sql>`节点对应的 `XNode` 会被放入到 `sqlFragments` 中。第二次调用 `sqlElement` 方法时，`requiredDatabaseId` 参数为空。由于 `sqlFragments` 中已包含了一个 `id` 节点，且该节点的 `databaseId` 不为空，此时匹配逻辑返回 `false`，第二个节点不会被保存到 `sqlFragments`。

上面的分析内容涉及到了 `databaseId`，关于 `databaseId` 的用途，简单介绍一下。`databaseId` 用于标明数据库厂商的身份，不同厂商有自己的 SQL 方言，MyBatis 可以根据 `databaseId` 执行不同 SQL 语句。`databaseId` 在`<sql>`节点中有什么用呢？这个问题也不难回答。`<sql>`节点用于保存 SQL 语句片段，如果 SQL 语句片段中包含方言的话，那么该`<sql>`节点只能被同一 `databaseId` 的查询语句或更新语句引用。

3.2.5 解析 SQL 语句节点

前面分析了`<cache>`、`<cache-ref>`、`<resultMap>`以及`<sql>`节点，从这一节开始，我们来分析映射文件中剩余的几个节点，分别是`<select>`、`<insert>`、`<update>`以及`<delete>`等。这几个节点中存储的是相同的内容，都是 SQL 语句，所以这几个节点的解析过程也是相同的。在进行代码分析之前，这里需要特别说明一下：为了避免和`<sql>`节点混淆，同时也为了描述方便，这里把`<select>`、`<insert>`、`<update>`以及`<delete>`等节点统称为 SQL 语句节点。好了，下面开始本节的分析。

```
private void buildStatementFromContext(List<XNode> list) {
    if (configuration.getDatabaseId() != null) {
        // 调用重载方法构建 Statement
        buildStatementFromContext(list, configuration.getDatabaseId());
    }
    // 调用重载方法构建 Statement, requiredDatabaseId 参数为空
    buildStatementFromContext(list, null);
}

private void buildStatementFromContext(List<XNode> list, String
requiredDatabaseId) {
    for (XNode context : list) {
```

```
// 创建 Statement 建造类
final XMLStatementBuilder statementParser = new XMLStatementBuilder(
    configuration, builderAssistant, context, requiredDatabaseId);
try {
    // 解析 Statement 节点，并将解析结果存储到
    // configuration 的 mappedStatements 集合中
    statementParser.parseStatementNode();
} catch (IncompleteElementException e) {
    // 解析失败，将解析器放入 Configuration 的 incompleteStatements 集合中
    configuration.addIncompleteStatement(statementParser);
}
}
```

上面的解析方法没有什么实质性的解析逻辑，我们继续往下分析。

```
public void parseStatementNode() {
    // 获取 id 和 databaseId 属性
    String id = context.getStringAttribute("id");
    String databaseId = context.getStringAttribute("databaseId");

    // 根据 databaseId 进行检测，检测逻辑和上一节基本一致，这里不再赘述
    if(!databaseIdMatchesCurrent(id, databaseId, this.requiredDatabaseId)) {
        return;
    }

    // 获取各种属性
    Integer fetchSize = context.getIntAttribute("fetchSize");
    Integer timeout = context.getIntAttribute("timeout");
    String parameterMap = context.getStringAttribute("parameterMap");
    String parameterType = context.getStringAttribute("parameterType");
    Class<?> parameterTypeClass = resolveClass(parameterType);
    String resultMap = context.getStringAttribute("resultMap");
}
```

```
String resultType = context.getStringAttribute("resultType");
String lang = context.getStringAttribute("lang");
LanguageDriver langDriver = getLanguageDriver(lang);

// 通过别名解析 resultType 对应的类型
Class<?> resultTypeClass = resolveClass(resultType);
String resultSetType = context.getStringAttribute("resultSetType");

// 解析 Statement 类型, 默认为 PREPARED
StatementType statementType = StatementType.valueOf(
    context.getStringAttribute(
        "statementType", StatementType.PREPARED.toString()));

// 解析 ResultSetType
ResultSetType resultSetTypeEnum = resolveResultSetType(resultSetType);

// 获取节点的名称, 比如 <select> 节点名称为 select
String nodeName = context.getNode().getNodeName();
// 根据节点名称解析 SqlCommandType
SqlCommandType sqlCommandType = SqlCommandType.valueOf(
    nodeName.toUpperCase(Locale.ENGLISH));
boolean isSelect = sqlCommandType == SqlCommandType.SELECT;
boolean flushCache=context.getBooleanAttribute("flushCache", !isSelect);
boolean useCache = context.getBooleanAttribute("useCache", isSelect);
boolean resultOrdered =
    context.getBooleanAttribute("resultOrdered", false);

// 解析 <include> 节点
XMLIncludeTransformer includeParser =
    new XMLIncludeTransformer(configuration, builderAssistant);
includeParser.applyIncludes(context.getNode());

// 解析 <selectKey> 节点
```

```
processSelectKeyNodes(id, parameterTypeClass, langDriver);

// 解析 SQL 语句
SqlSource sqlSource = langDriver.createSqlSource(
    configuration, context, parameterTypeClass);
String resultSets = context.getStringAttribute("resultSets");
String keyProperty = context.getStringAttribute("keyProperty");
String keyColumn = context.getStringAttribute("keyColumn");

KeyGenerator keyGenerator;
String keyStatementId = id + SelectKeyGenerator.SELECT_KEY_SUFFIX;
keyStatementId = builderAssistant
    .applyCurrentNamespace(keyStatementId, true);
if (configuration.hasKeyGenerator(keyStatementId)) {
    // 获取 KeyGenerator 实例
    keyGenerator = configuration.getKeyGenerator(keyStatementId);
} else {
    // 创建 KeyGenerator 实例
    keyGenerator = context.getBooleanAttribute("useGeneratedKeys",
        configuration.isUseGeneratedKeys() &&
        SqlCommandType.INSERT.equals(sqlCommandType)) ?
        Jdbc3KeyGenerator.INSTANCE : NoKeyGenerator.INSTANCE;
}

// 构建 MappedStatement 对象，并将该对象存储到
// Configuration 的 mappedStatements 集合中
builderAssistant.addMappedStatement(id, sqlSource, statementType,
    sqlCommandType, fetchSize, timeout, parameterMap,
    parameterTypeClass, resultMap, resultTypeClass, resultSetTypeEnum,
    flushCache, useCache, resultOrdered, keyGenerator, keyProperty,
    keyColumn, databaseId, langDriver, resultSets);
}
```

上面的代码比较长，看起来有点复杂。不过如果大家耐心看一下源码，会发现，上面的代码中起码有一半的代码是用来获取节点属性，以及解析部分属性等。抛去这部分代码，以上代码做的事情如下。

1. 解析<include>节点
2. 解析<selectKey>节点
3. 解析 SQL，获取 SqlSource
4. 构建 MappedStatement 实例

以上流程对应的代码比较复杂，每个步骤都能分析出一些东西来，接下来我们按照顺序进行分析。

1. 解析<include>节点

<include>节点的解析逻辑封装在 `applyIncludes` 中，该方法的代码如下：

```
public void applyIncludes(Node source) {
    Properties variablesContext = new Properties();
    Properties configurationVariables = configuration.getVariables();
    if (configurationVariables != null) {
        // 将 configurationVariables 中的数据添加到 variablesContext 中
        variablesContext.putAll(configurationVariables);
    }

    // 调用重载方法处理 <include> 节点
    applyIncludes(source, variablesContext, false);
}
```

上面代码创建了一个新的 `Properties` 对象，并将全局 `Properties` 添加到其中。这样做的原因是 `applyIncludes` 的重载方法会向 `Properties` 中添加新的元素，如果直接将全局 `Properties` 传给重载方法，会造成全局 `Properties` 被污染。这是个细节，其他就没什么了，我们继续往下看。

```
private void applyIncludes(Node source,
    final Properties variablesContext, boolean included) {
```

```
// ☆ 第一个条件分支
if (source.getNodeName().equals("include")) {

    // 获取 <sql> 节点。若 refid 中包含属性占位符 ${},
    // 则需先将属性占位符替换为对应的属性值
    Node toInclude = findSqlFragment(
        getStringAttribute(source, "refid"), variablesContext);

    // 解析<include>的子节点<property>, 并将解析结果与 variablesContext 融合,
    // 然后返回融合后的 Properties。若 <property> 节点的 value 属性中存在
    // 占位符 ${}, 则将占位符替换为对应的属性值
    Properties toIncludeContext =
        getVariablesContext(source, variablesContext);

    /*
     * 这里是一个递归调用, 用于将 <sql> 节点内容中出现的属性占位符 ${}
     * 替换为对应的属性值。这里要注意一下递归调用的参数:
     *
     * - toInclude: <sql> 节点对象
     * - toIncludeContext: <include> 子节点 <property> 的解析结果与
     *                    全局变量融合后的结果
     */
    applyIncludes(toInclude, toIncludeContext, true);

    // 如果 <sql> 和 <include> 节点不在一个文档中,
    // 则从其他文档中将 <sql> 节点引入到 <include> 所在文档中
    if (toInclude.getOwnerDocument() != source.getOwnerDocument()) {
        toInclude=source.getOwnerDocument().importNode(toInclude,true);
    }
    // 将 <include> 节点替换为 <sql> 节点
    source.getParentNode().replaceChild(toInclude, source);
    while (toInclude.hasChildNodes()) {
        // 将 <sql> 中的内容插入到 <sql> 节点之前
    }
}
```

```

        toInclude.getParentNode().insertBefore(
            toInclude.getFirstChild(), toInclude);
    }

    // 前面已经将 <sql> 节点的内容插入到 dom 中了,
    // 现在不需要 <sql> 节点了, 这里将该节点从 dom 中移除
    toInclude.getParentNode().removeChild(toInclude);

    // ☆ 第二个条件分支
} else if (source.getNodeType() == Node.ELEMENT_NODE) {
    if (included && !variablesContext.isEmpty()) {
        NamedNodeMap attributes = source.getAttributes();
        for (int i = 0; i < attributes.getLength(); i++) {
            Node attr = attributes.item(i);
            // 将 source 节点属性中的占位符 ${} 替换成具体的属性值
            attr.setNodeValue(PropertyParser.parse(
                attr.getNodeValue(), variablesContext));
        }
    }

    NodeList children = source.getChildNodes();
    for (int i = 0; i < children.getLength(); i++) {
        // 递归调用
        applyIncludes(children.item(i), variablesContext, included);
    }

    // ☆ 第三个条件分支
} else if (included && source.getNodeType() == Node.TEXT_NODE &&
    !variablesContext.isEmpty()) {
    // 将文本 (text) 节点中的属性占位符 ${} 替换成具体的属性值
    source.setNodeValue(PropertyParser.parse(
        source.getNodeValue(), variablesContext));
}
}

```


上面的代码如果从上往下读，不太容易看懂。因为上面的方法由三个条件分支，外加两个递归调用组成，代码的执行顺序并不是由上而下。要理解上面的代码，我们需要定义一些配置，并将配置带入到具体代码中，逐行进行演绎。不过，更推荐的方式是使用 IDE 进行单步调试。为了便于讲解，我把上面代码中的三个分支都用★标记了出来，大家注意一下。好了，必要的准备工作做好了，下面开始演绎代码的执行过程。演绎所用的测试配置如下：

```
<mapper namespace="xyz.coolblog.dao.ArticleDao">
  <sql id="table">
    ${table_name}
  </sql>
  <select id="findOne" resultType="xyz.coolblog.dao.Article">
    SELECT
      id, title
    FROM
      <include refid="table">
        <property name="table_name" value="article"/>
      </include>
    WHERE id = #{id}
  </select>
</mapper>
```

我们先来看一下 applyIncludes 方法第一次被调用时的状态，如下：

参数值：

source = <select> 节点

节点类型：ELEMENT_NODE

variablesContext = [] // 无内容

included = false

执行流程：

1. 进入条件分支 2
2. 获取 <select> 子节点列表
3. 遍历子节点列表，将子节点作为参数，进行递归调用

第一次调用 `applyIncludes` 方法, `source=<select>`, 代码进入条件分支 2。在该分支中, 首先要获取`<select>`节点的子节点列表。可获取到的子节点如下:

编号	子节点	类型	描述
1	SELECT id, title FROM	TEXT_NODE	文本节点
2	<include refid="table"/>	ELEMENT_NODE	普通节点
3	WHERE id = #{id}	TEXT_NODE	文本节点

在获取到子节点类列表后, 接下来要做的事情是遍历列表, 然后将子节点作为参数进行递归调用。在上面三个子节点中, 子节点 1 和子节点 3 都是文本节点, 调用过程一致。因此, 本节只会演示子节点 1 和子节点 2 的递归调用过程。先来演示子节点 1 的调用过程, 如下:

第一层		第二层	第三层
第一次调用: 参数值: <code>source = <select></code> 节点 节点类型: <code>ELEMENT_NODE</code> <code>variablesContext = []</code> // 无内容 <code>included = false</code> 执行流程: 1. 进入条件分支2 2. 获取 <code><select></code> 子节点列表 3. 遍历子节点列表, 将子节点作为参数, 进行递归调用	调用 返回	第二次调用: 参数值: <code>source = [#text: SELECT id, title FROM]</code> 节点类型: <code>TEXT_NODE</code> <code>variablesContext = []</code> // 无内容 <code>included = false</code> 执行流程: 1. 进入条件分支3 2. 不满足分支条件, 本次调用返回	© 2018 tianxiaobo.com

节点 1 的调用过程比较简单, 只有两层调用。接下来我们再看一下子节点 2 的调用过程, 如下:

第一层	第二层	第三层	第四层
第一次调用: 参数值: <code>source = <select></code> 节点 节点类型: <code>ELEMENT_NODE</code> <code>variablesContext = []</code> // 无内容 <code>included = false</code> 执行流程: 1. 进入条件分支2 2. 获取 <code><select></code> 子节点列表 3. 遍历子节点列表, 将子节点作为参数, 进行递归调用	第二次调用: 参数值: <code>source = <include></code> 节点 节点类型: <code>ELEMENT_NODE</code> <code>variablesContext = []</code> // 无内容 <code>included = false</code> 执行流程: 1. 进入条件分支1 2. 查找 <code><sql></code> 节点 3. 获取 <code><include></code> 子节点 <code><property></code> 内容。此时 <code>toIncludeContext = [{"table_name": "article"}]</code> 4. 递归调用	第三次调用: 参数值: <code>source = <sql></code> 节点 节点类型: <code>ELEMENT_NODE</code> <code>variablesContext = [{"table_name": "article"}]</code> <code>included = true</code> 执行流程: 1. 进入条件分支2 2. 对 <code><sql></code> 节点属性中的占位符解析替换 3. 获取 <code><sql></code> 子节点列表 4. 将子节点作为参数, 并进行递归调用	第四次调用: 参数值: <code>source = [#text: \${table_name}]</code> 节点类型: <code>TEXT_NODE</code> <code>variablesContext = [{"table_name": "article"}]</code> <code>included = true</code> 执行流程: 1. 进入条件分支3 2. 将文本中的占位符 <code>\${table_name}</code> 替换为 <code>article</code> 3. 调用结束, 返回上一层

上面是子节点 2 的调用过程, 共有四层调用, 略为复杂。大家自己也对着配置, 把源码走一遍, 然后记录每一次调用的一些状态, 这样才能更好的理解 `applyIncludes` 方法的逻辑。

2. 解析<selectKey>节点

对于一些不支持自增主键的数据库来说，我们在插入数据时，需要明确指定主键数据。以 Oracle 数据库为例，Oracle 数据库不支持自增主键，但它提供了自增序列工具。我们每次向数据库中插入数据时，可以先通过自增序列获取主键数据，然后再进行插入。这里涉及到两次数据库查询操作，但我们并不能在一个<select>节点中同时配置两个 select 语句，这会导致 SQL 语句出错。对于这个问题，MyBatis 提供的<selectKey>可以很好的解决。下面我们看一段配置：

```
<insert id="saveAuthor">
    <selectKey keyProperty="id" resultType="int" order="BEFORE">
        select author_seq.nextval from dual
    </selectKey>
    insert into Author
        (id, name, password)
    values
        ({id}, #{username}, #{password})
</insert>
```

在上面的配置中，查询语句会先于插入语句执行，这样我们就可以在插入时获取到主键的值。关于<selectKey>的用法，这里不过多介绍了。下面我们来看一下<selectKey>节点的解析过程。

```
private void processSelectKeyNodes(String id, Class<?> parameterTypeClass,
LanguageDriver langDriver) {
    List<XNode> selectKeyNodes = context.evalNodes("selectKey");
    if (configuration.getDatabaseId() != null) {
        // 解析 <selectKey> 节点, databaseId 不为空
        parseSelectKeyNodes(id, selectKeyNodes, parameterTypeClass,
            langDriver, configuration.getDatabaseId());
    }
    // 解析 <selectKey> 节点, databaseId 为空
```

```

    parseSelectKeyNodes(id, selectKeyNodes,
        parameterTypeClass, langDriver, null);
    // 将 <selectKey> 节点从 dom 树中移除
    removeSelectKeyNodes(selectKeyNodes);
}

```

从上面的代码中可以看出，<selectKey>节点在解析完成后，会被从 dom 树中移除。这样后续可以更专注的解析<insert>或<update>节点中的 SQL，无需再额外处理<selectKey>节点。继续往下看。

```

private void parseSelectKeyNodes(String parentId, List<XNode> list,
    Class<?> parameterTypeClass, LanguageDriver langDriver,
    String skRequiredDatabaseId) {
    for (XNode nodeToHandle : list) {
        // id = parentId + !selectKey, 比如 saveUser!selectKey
        String id = parentId + SelectKeyGenerator.SELECT_KEY_SUFFIX;
        // 获取 <selectKey> 节点的 databaseId 属性
        String databaseId = nodeToHandle.getStringAttribute("databaseId");
        // 匹配 databaseId
        if (databaseIdMatchesCurrent(id, databaseId, skRequiredDatabaseId)) {
            // 解析 <selectKey> 节点
            parseSelectKeyNode(id, nodeToHandle,
                parameterTypeClass, langDriver, databaseId);
        }
    }
}

private void parseSelectKeyNode(String id, XNode nodeToHandle,
    Class<?> parameterTypeClass, LanguageDriver langDriver,
    String databaseId) {

    // 获取各种属性
    String resultType = nodeToHandle.getStringAttribute("resultType");
}

```

```
Class<?> resultTypeClass = resolveClass(resultType);
StatementType statementType = StatementType.valueOf(
    nodeToHandle.getStringAttribute(
        "statementType", StatementType.PREPARED.toString()));
String keyProperty = nodeToHandle.getStringAttribute("keyProperty");
String keyColumn = nodeToHandle.getStringAttribute("keyColumn");
boolean executeBefore = "BEFORE".equals(
    nodeToHandle.getStringAttribute("order", "AFTER"));

// 设置默认值
boolean useCache = false;
boolean resultOrdered = false;
KeyGenerator keyGenerator = NoKeyGenerator.INSTANCE;
Integer fetchSize = null;
Integer timeout = null;
boolean flushCache = false;
String parameterMap = null;
String resultMap = null;
ResultSetType resultSetTypeEnum = null;

// 创建 SqlSource
SqlSource sqlSource = langDriver.createSqlSource(
    configuration, nodeToHandle, parameterTypeClass);
// <selectKey> 节点中只能配置 SELECT 查询语句,
// 因此 sqlCommandType 为 SqlCommandType.SELECT
SqlCommandType sqlCommandType = SqlCommandType.SELECT;

// 构建 MappedStatement, 并将 MappedStatement
// 添加到 Configuration 的 mappedStatements map 中
builderAssistant.addMappedStatement(id, sqlSource, statementType,
    sqlCommandType, fetchSize, timeout, parameterMap, parameterTypeClass,
    resultMap, resultTypeClass, resultSetTypeEnum, flushCache, useCache,
    resultOrdered, keyGenerator, keyProperty, keyColumn,
    databaseId, langDriver, null);
```

```

// id = namespace + "." + id
id = builderAssistant.applyCurrentNamespace(id, false);
MappedStatement keyStatement =
    configuration.getMappedStatement(id, false);
// 创建 SelectKeyGenerator, 并添加到 keyGenerators map 中
configuration.addKeyGenerator(id,
    new SelectKeyGenerator(keyStatement, executeBefore));
}

```

上面的源码比较长,但大部分代码都是一些基础代码,不是很难理解。以上代码比较重要的一些步骤如下:

1. 创建 SqlSource 实例
2. 构建并缓存 MappedStatement 实例
3. 构建并缓存 SelectKeyGenerator 实例

在这三步中,第1步和第2步调用的是公共逻辑,其他地方也会调用,这两步对应的源码后续会进行讲解。第3步则是创建一个 SelectKeyGenerator 实例,SelectKeyGenerator 创建的过程比较简单,所以就不多说了。下面分析一下 SqlSource 和 MappedStatement 实例的创建过程。

3.解析 SQL 语句

前面分析了<include>和<selectKey>节点的解析过程,这两个节点解析完成后,都会以不同的方式从 dom 树中消失。所以目前的 SQL 语句节点由一些文本节点和普通节点组成,比如<if>、<where>等。下面我们来看一下移除掉<include>和<selectKey>节点后的 SQL 语句节点是如何解析的。

```

// -☆- XMLLanguageDriver
public SqlSource createSqlSource(Configuration configuration,
    XNode script, Class<?> parameterType) {
    XMLScriptBuilder builder = new XMLScriptBuilder(
        configuration, script, parameterType);
    return builder.parseScriptNode();
}

```

```
// -☆- XMLScriptBuilder
public SqlSource parseScriptNode() {
    // 解析 SQL 语句节点
    MixedSqlNode rootSqlNode = parseDynamicTags(context);
    SqlSource sqlSource = null;
    // 根据 isDynamic 状态创建不同的 SqlSource
    if (isDynamic) {
        sqlSource = new DynamicSqlSource(configuration, rootSqlNode);
    } else {
        sqlSource = new RawSqlSource(
            configuration, rootSqlNode, parameterType);
    }
    return sqlSource;
}
```

如上,SQL 语句的解析逻辑被封装在了 XMLScriptBuilder 类的 parseScriptNode 方法中。该方法首先会调用 parseDynamicTags 解析 SQL 语句节点,在解析过程中,会判断节点是否包含一些动态标记,比如\${} 占位符以及动态 SQL 节点等。若包含动态标记,则会将 isDynamic 设为 true。后续可根据 isDynamic 创建不同的 SqlSource。下面,我们来看一下 parseDynamicTags 方法的逻辑。

```
/** 该方法用于初始化 nodeHandlerMap 集合,该集合后面会用到 */
private void initNodeHandlerMap() {
    nodeHandlerMap.put("trim", new TrimHandler());
    nodeHandlerMap.put("where", new WhereHandler());
    nodeHandlerMap.put("set", new SetHandler());
    nodeHandlerMap.put("foreach", new ForEachHandler());
    nodeHandlerMap.put("if", new IfHandler());
    nodeHandlerMap.put("choose", new ChooseHandler());
    nodeHandlerMap.put("when", new IfHandler());
    nodeHandlerMap.put("otherwise", new OtherwiseHandler());
    nodeHandlerMap.put("bind", new BindHandler());
}
```

```

protected MixedSqlNode parseDynamicTags(XNode node) {
    List<SqlNode> contents = new ArrayList<SqlNode>();
    NodeList children = node.getNode().getChildNodes();
    // 遍历子节点
    for (int i = 0; i < children.getLength(); i++) {
        XNode child = node.newXNode(children.item(i));
        if (child.getNode().getNodeType() == Node.CDATA_SECTION_NODE ||
            child.getNode().getNodeType() == Node.TEXT_NODE) {
            // 获取文本内容
            String data = child.getStringBody("");
            TextSqlNode textSqlNode = new TextSqlNode(data);
            // 若文本中包含 ${} 占位符，也被认为是动态节点
            if (textSqlNode.isDynamic()) {
                contents.add(textSqlNode);
                // 设置 isDynamic 为 true
                isDynamic = true;
            } else {
                // 创建 StaticTextSqlNode
                contents.add(new StaticTextSqlNode(data));
            }
            // child 节点是 ELEMENT_NODE 类型，比如 <if>、<where> 等
        } else if (child.getNode().getNodeType() == Node.ELEMENT_NODE) {
            // 获取节点名称，比如 if、where、trim 等
            String nodeName = child.getNode().getNodeName();
            // 根据节点名称获取 NodeHandler
            NodeHandler handler = nodeHandlerMap.get(nodeName);
            // 如果 handler 为空，表明当前节点对与 MyBatis 来说，是未知节点。
            // MyBatis 无法处理这种节点，故抛出异常
            if (handler == null) {
                throw new BuilderException(".....");
            }
            // 处理 child 节点，生成相应的 SqlNode
            handler.handleNode(child, contents);
        }
    }
}

```

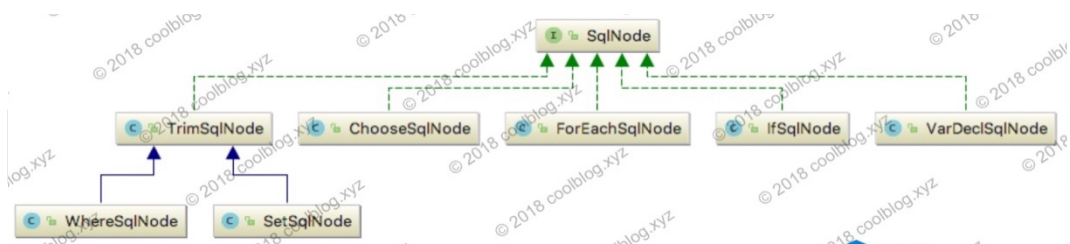


```

        // 设置 isDynamic 为 true
        isDynamic = true;
    }
}
return new MixedSqlNode(contents);
}

```

上面方法的逻辑我前面已经说过,主要是用来判断节点是否包含一些动态标记,比如\${}占位符以及动态 SQL 节点等。这里,不管是动态 SQL 节点还是静态 SQL 节点,我们都可以把它们看成是 SQL 片段,一个 SQL 语句由多个 SQL 片段组成。在解析过程中,这些 SQL 片段被存储在 contents 集合中。最后,该集合会被传给 MixedSqlNode 构造方法,用于创建 MixedSqlNode 实例。从 MixedSqlNode 类名上可知,它会存储多种类型的 SqlNode。除了上面代码中已出现的几种 SqlNode 实现类,还有一些 SqlNode 实现类未出现在上面的代码中。但它们也参与了 SQL 语句节点的解析过程,这里我们来看一下这些幕后的 SqlNode 类。



上面的 SqlNode 实现类用于处理不同的动态 SQL 逻辑,这些 SqlNode 是如何生成的呢?答案是由各种 NodeHandler 生成。我们再回到上面的代码中,可以看到这样一句代码:

```
handler.handleNode(child, contents);
```

该代码用于处理动态 SQL 节点,并生成相应的 SqlNode。下面来简单分析一下 WhereHandler 的代码。

```

/** 定义在 XMLScriptBuilder 中 */
private class WhereHandler implements NodeHandler {

    public WhereHandler() {
    }
}

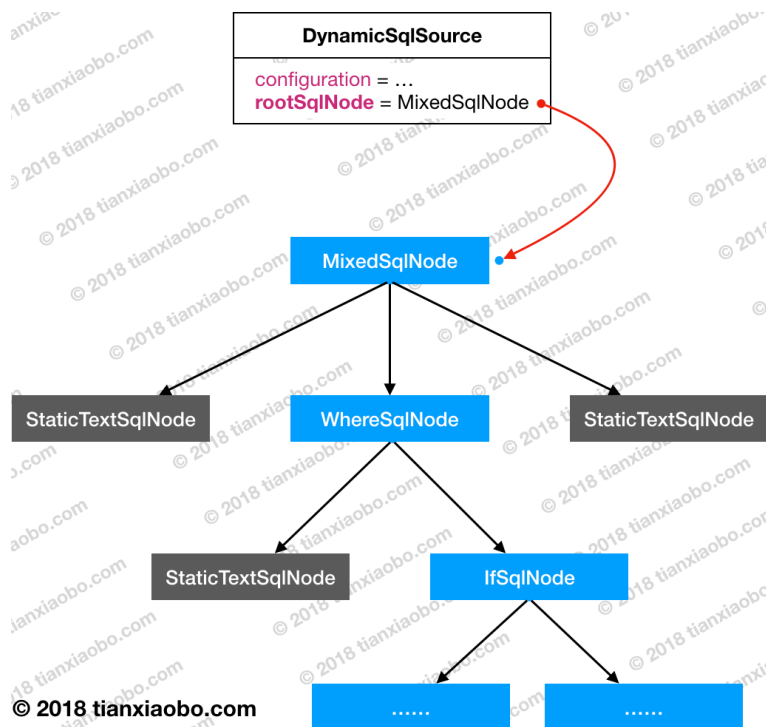
```

```

@Override
public void handleNode(XNode nodeToHandle, List<SqlNode> targetContents){
    // 调用 parseDynamicTags 解析 <where> 节点
    MixedSqlNode mixedSqlNode = parseDynamicTags(nodeToHandle);
    // 创建 WhereSqlNode
    WhereSqlNode where = new WhereSqlNode(configuration, mixedSqlNode);
    // 添加到 targetContents
    targetContents.add(where);
}
}

```

如上，handleNode 方法内部会再次调用 parseDynamicTags 解析<where>节点中的内容，这样又会生成一个 MixedSqlNode 对象。最终，整个 SQL 语句节点会生成一个具有树状结构的 MixedSqlNode。如下图：



到此,SQL 语句的解析过程就分析完了。现在,我们已经将 XML 配置解析了 SqlSource,但这还没有结束。SqlSource 中只能记录 SQL 语句信息,除此之外,这里还有一些额外的信息需要记录。因此,我们需要一个类能够同时存储 SqlSource 和其他的信息。这个类就是 MappedStatement。下面我们来看一下它的构建过程。

4.构建 MappedStatement

SQL 语句节点可以定义很多属性,这些属性和属性值最终存储在 MappedStatement 中。下面我们看一下 MappedStatement 的构建过程是怎样的。

```
public MappedStatement addMappedStatement(  
    String id, SqlSource sqlSource, StatementType statementType,  
    SqlCommandType sqlCommandType, Integer fetchSize, Integer timeout,  
    String parameterMap, Class<?> parameterType, String resultMap,  
    Class<?> resultType, ResultSetType resultSetType, boolean flushCache,  
    boolean useCache, boolean resultOrdered, KeyGenerator keyGenerator,  
    String keyProperty, String keyColumn, String databaseId,  
    LanguageDriver lang, String resultSets) {  
  
    if (unresolvedCacheRef) {  
        throw new IncompleteElementException("Cache-ref not yet resolved");  
    }  
  
    id = applyCurrentNamespace(id, false);  
    boolean isSelect = sqlCommandType == SqlCommandType.SELECT;  
  
    // 创建建造器, 设置各种属性  
    MappedStatement.Builder statementBuilder = new MappedStatement  
        .Builder(configuration, id, sqlSource, sqlCommandType)  
        .resource(resource).fetchSize(fetchSize).timeout(timeout)  
        .statementType(statementType).keyGenerator(keyGenerator)  
        .keyProperty(keyProperty).keyColumn(keyColumn)  
        .databaseId(databaseId).lang(lang)  
        .resultOrdered(resultOrdered).resultSets(resultSets)
```

```
.resultMaps(getStatementResultMaps(resultMap, resultType, id))
.flushCacheRequired(valueOrDefault(flushCache, !isSelect))
.resultSetType(resultSetType)
.useCache(valueOrDefault(useCache, isSelect))
.cache(currentCache);

// 获取或创建 ParameterMap
ParameterMap statementParameterMap =
    getStatementParameterMap(parameterMap, parameterType, id);
if (statementParameterMap != null) {
    statementBuilder.parameterMap(statementParameterMap);
}

// 构建 MappedStatement, 没有什么复杂逻辑, 不跟下去了
MappedStatement statement = statementBuilder.build();
// 添加 MappedStatement 到 configuration 的 mappedStatements 集合中
configuration.addMappedStatement(statement);
return statement;
}
```

上面就是 MappedStatement 的构建过程, 没什么特别复杂的地方, 就不多说了。本节分析了映射文件的解析过程, 总的来说, 内容还是比较复杂的, 逻辑比较多。尽管如此, 大家也应把映射文件的解析过程认真分析一遍, 这样会对 MyBatis 有更深入的了解。

3.3 Mapper 接口绑定过程分析

映射文件解析完成后, 并不意味着整个解析过程就结束了。此时还需要通过命名空间绑定 mapper 接口, 这样才能将映射文件中的 SQL 语句和 mapper 接口中的方法绑定在一起, 后续可直接通过调用 mapper 接口方法执行与之对应的 SQL 语句。下面我们来分析一下 mapper 接口的绑定过程。

```
// -☆- XMLMapperBuilder
private void bindMapperForNamespace() {
    // 获取映射文件的命名空间
    String namespace = builderAssistant.getCurrentNamespace();
    if (namespace != null) {
        Class<?> boundType = null;
        try {
            // 根据命名空间解析 mapper 类型
            boundType = Resources.classForName(namespace);
        } catch (ClassNotFoundException e) {}
        if (boundType != null) {
            // 检测当前 mapper 类是否被绑定过
            if (!configuration.hasMapper(boundType)) {
                configuration.addLoadedResource("namespace:" + namespace);
                // 绑定 mapper 类
                configuration.addMapper(boundType);
            }
        }
    }
}

// -☆- Configuration
public <T> void addMapper(Class<T> type) {
    // 通过 MapperRegistry 绑定 mapper 类
    mapperRegistry.addMapper(type);
}

// -☆- MapperRegistry
public <T> void addMapper(Class<T> type) {
    if (type.isInterface()) {
        if (hasMapper(type)) {
            throw new BindingException(".....");
        }
    }
}
```

```
boolean loadCompleted = false;
try {
    // 将 type 和 MapperProxyFactory 进行绑定,
    // MapperProxyFactory 可为 mapper 接口生成代理类
    knownMappers.put(type, new MapperProxyFactory<T>(type));

    // 创建注解解析器。在 MyBatis 中, 有 XML 和 注解两种配置方式可选
    MapperAnnotationBuilder parser =
        new MapperAnnotationBuilder(config, type);
    // 解析注解中的信息
    parser.parse();
    loadCompleted = true;
} finally {
    if (!loadCompleted) {
        knownMappers.remove(type);
    }
}
}
```

以上就是 Mapper 接口的绑定过程。这里简单总结一下：

1. 获取命名空间, 并根据命名空间解析 mapper 类型
2. 将 type 和 MapperProxyFactory 实例存入 knownMappers 中
3. 解析注解中的信息

以上步骤中, 第3步的逻辑较多。如果大家看懂了映射文件的解析过程, 那么注解的解析过程也就不难理解了, 这里就不深入分析了。关于 Mapper 接口的绑定过程就先分析到这。

3.4 处理未完成解析的节点

在解析某些节点的过程中, 如果这些节点引用了其他一些未被解析的配置, 会导致当前节点解析工作无法进行下去。对于这种情况, MyBatis 的做法是抛出 `IncompleteElementException`

异常。外部逻辑会捕捉这个异常，并将节点对应的解析器放入 `incomplet*` 集合中。这个我在分析映射文件解析的过程中进行过相应注释，不知道大家有没有注意到。没注意到也没关系，

```
// -☆- XMLMapperBuilder
public void parse() {
    // 省略部分代码

    // 解析 mapper 节点
    configurationElement(parser.evalNode("/mapper"));

    // 处理未完成解析的节点
    parsePendingResultMaps();
    parsePendingCacheRefs();
    parsePendingStatements();
}
```

待会我会举例说明。下面我们来看一下 MyBatis 是如何处理未完成解析的节点。

如上，`parse` 方法是映射文件的解析入口。在本章的开始，我贴过这个源码。从上面的源码中可以知道有三种节点在解析过程中可能会出现不能完成解析的情况。由于上面三个以 `parsePending` 开头的方法逻辑一致，所以下面我只会分析其中一个方法的源码。简单起见，这里选择分析 `parsePendingCacheRefs` 的源码。下面看一下如何配置映射文件会导致 `<cache-ref>` 节点无法完成解析。

```
<!-- 映射文件 1 -->
<mapper namespace="xyz.coolblog.dao.Mapper1">
    <!-- 引用映射文件 2 中配置的缓存 -->
    <cache-ref namespace="xyz.coolblog.dao.Mapper2"/>
</mapper>

<!-- 映射文件 2 -->
<mapper namespace="xyz.coolblog.dao.Mapper2">
    <cache/>
</mapper>
```

如上, 假设 MyBatis 先解析映射文件 1, 然后再解析映射文件 2。按照这样的解析顺序, 映射文件 1 中的<cache-ref>节点就无法完成解析, 因为它所引用的缓存还未被解析。当映射文件 2 解析完成后, MyBatis 会调用 parsePendingCacheRefs 方法处理在此之前未完成解析的<cache-ref>节点。具体的逻辑如下:

```
private void parsePendingCacheRefs() {
    // 获取 CacheRefResolver 列表
    Collection<CacheRefResolver> incompleteCacheRefs =
        configuration.getIncompleteCacheRefs();
    synchronized (incompleteCacheRefs) {
        Iterator<CacheRefResolver> iter = incompleteCacheRefs.iterator();
        // 通过迭代器遍历列表
        while (iter.hasNext()) {
            try {
                // 尝试解析 <cache-ref> 节点, 若解析失败, 则抛出
                // IncompleteElementException, 此时下面的删除操作不会被执行
                iter.next().resolveCacheRef();
                // 移除 CacheRefResolver 对象。如果代码能执行到此处,
                // 表明已成功解析了 <cache-ref> 节点
                iter.remove();
            } catch (IncompleteElementException e) {
                // 如果再次发生 IncompleteElementException 异常, 表明当前
                // 映射文件中并没有<cache-ref>所引用的缓存。有可能所引用的缓存
                // 在后面的映射文件中, 所以这里不能将解析失败的 CacheRefResolver
                // 从集合中删除
            }
        }
    }
}
```

上面代码不是很长, 逻辑也比较简单, 这里简单总结一下:

1. 获取获取 CacheRefResolver 列表, 并进行遍历
2. 尝试解析<cache-ref>节点, 若解析失败再次抛出异常
3. 若解析成功则列表中移除相关节点

3.5 本章小结

本章对映射文件中的`<cache>`、`<cache-ref>`、`<resultMap>`、`<sql>`、`<select|insert|update|delete>`等节点的解析过程进行了较为详细的分析。同时对 Mapper 接口的绑定过程，以及未完成解析节点处理过程也一并进行了分析。总的来说。映射文件解析过程较为复杂，想完全搞懂并不是特别容易的事。尽管如此，大家在看完本章内容，也应尝试分析一下映射文件的解析逻辑。这样可以加深对 MyBatis 的了解。

第 4 章

SQL 执行流程

前两章分析了配置文件和映射文件的解析过程。经过前面复杂的解析过程后，现在，MyBatis 已经进入了就绪状态，等待使用者发号施令。本章将对 MyBatis 执行 SQL 的过程进行较为详尽的分析。该过程比较复杂，涉及的技术点很多。包括但不限于以下技术点：

1. 为 mapper 接口生成实现类
2. 根据配置信息生成 SQL，并将运行时参数设置到 SQL 中
3. 一二级缓存的实现
4. 插件机制
5. 数据库连接的获取与管理
6. 查询结果的处理，以及延迟加载等

如果大家能掌握上面的技术点，那么对 MyBatis 的原理将会有很深入的理解。限于篇幅问题，本章仅分析以上列表中的第 1 个、第 2 个以及第 6 个技术点，其他技术点将会在后续章节中进行分析。

4.1 SQL 执行入口

在单独使用 MyBatis 进行数据库操作时，我们通常都会先调用 SqlSession 接口的 getMapper 方法为我们的 Mapper 接口生成实现类。然后就可以通过 Mapper 进行数据库操作。比如像下面这样：

```
ArticleMapper articleMapper = session.getMapper(ArticleMapper.class);
Article article = articleMapper.findOne(1);
```

如果大家对 MyBatis 较为了解，会知道 SqlSession 是通过 JDK 动态代理的方式为接口生成代理对象的。在调用接口方法时，相关调用会被代理逻辑拦截。在代理逻辑中可根据方法名及方法归属接口获取到当前方法对应的 SQL 以及其他一些信息，拿到这些信息即可进行数据库操作。

以上是一个简版的 SQL 执行过程，省略了很多细节。下面我们先按照这个简版的流程进行分析，首先来看一下 Mapper 接口的代理对象创建过程。

4.1.1 为 Mapper 接口创建代理对象

本节，我们从 DefaultSqlSession 的 getMapper 方法开始看起，如下：

```
// -☆- DefaultSqlSession
public <T> T getMapper(Class<T> type) {
    return configuration.<T>getMapper(type, this);
}

// -☆- Configuration
public <T> T getMapper(Class<T> type, SqlSession sqlSession) {
    return mapperRegistry.getMapper(type, sqlSession);
}

// -☆- MapperRegistry
```

```

public <T> T getMapper(Class<T> type, SqlSession sqlSession) {
    // 从 knownMappers 中获取与 type 对应的 MapperProxyFactory
    final MapperProxyFactory<T> mapperProxyFactory =
        (MapperProxyFactory<T>) knownMappers.get(type);
    if (mapperProxyFactory == null) {
        throw new BindingException(".....");
    }
    try {
        // 创建代理对象
        return mapperProxyFactory.newInstance(sqlSession);
    } catch (Exception e) {
        throw new BindingException(".....");
    }
}

```

如上，经过连续的调用，Mapper 接口代理对象的创建逻辑初现端倪。如果大家没分析过 MyBatis 配置文件的解析过程，那么可能不知道 knownMappers 集合中的元素是何时存入的，这里简单说明一下。MyBatis 在解析配置文件的<mappers>节点的过程中，会调用 MapperRegistry 的 addMapper 方法将 Class 到 MapperProxyFactory 对象的映射关系存入到 knownMappers。具体的代码就不分析了，大家可以阅读我之前写的文章，或者自行分析相关的代码。

在获取到 MapperProxyFactory 对象后，即可调用工厂方法为 Mapper 接口生成代理对象了。相关逻辑如下：

```

// -☆- MapperProxyFactory
public T newInstance(SqlSession sqlSession) {
    // 创建 MapperProxy 对象，MapperProxy 实现了 InvocationHandler 接口，
    // 代理逻辑封装在此类中
    final MapperProxy<T> mapperProxy =
        new MapperProxy<T>(sqlSession, mapperInterface, methodCache);
    return newInstance(mapperProxy);
}

```

```
protected T newInstance(MapperProxy<T> mapperProxy) {  
    // 通过 JDK 动态代理创建代理对象  
    return (T) Proxy.newProxyInstance(mapperInterface.getClassLoader(),  
        new Class[]{mapperInterface}, mapperProxy);  
}
```

上面的代码首先创建了一个 MapperProxy 对象，该对象实现了 InvocationHandler 接口。然后将对象作为参数传给重载方法，并在重载方法中调用 JDK 动态代理接口为 Mapper 生成代理对象。代理对象已经创建完毕，下面就可以调用接口方法进行数据库操作了。由于接口方法会被代理逻辑拦截，所以下面我们把目光聚焦在代理逻辑上面，看看代理逻辑会做些什么事情。

4.1.2 执行代理逻辑

Mapper 接口方法的代理逻辑首先会对拦截的方法进行一些检测，以决定是否执行后续的数据库操作。对应的代码如下：

```
public Object invoke(Object proxy,  
    Method method, Object[] args) throws Throwable {  
    try {  
        // 如果方法是定义在 Object 类中的，则直接调用  
        if (Object.class.equals(method.getDeclaringClass())) {  
            return method.invoke(this, args);  
        }  
        /*  
        * 下面的代码最早出现在 mybatis-3.4.2 版本中，用于支持 JDK 1.8 中的  
        * 新特性 - 默认方法。这段代码的逻辑就不分析了，有兴趣的同学可以  
        * 去 Github 上看一下相关的讨论（issue #709），链接如下：  
        *  
        * https://github.com/mybatis/mybatis-3/issues/709  
        */  
    } else if (isDefaultMethod(method)) {
```

```

        return invokeDefaultMethod(proxy, method, args);
    }
} catch (Throwable t) {
    throw ExceptionUtil.unwrapThrowable(t);
}

// 从缓存中获取 MapperMethod 对象，若缓存未命中，则创建 MapperMethod 对象
final MapperMethod mapperMethod = cachedMapperMethod(method);

// 调用 execute 方法执行 SQL
return mapperMethod.execute(sqlSession, args);
}

```

如上，代理逻辑会首先检测被拦截的方法是不是定义在 `Object` 中的，比如 `equals`、`hashCode` 方法等。对于这类方法，直接执行即可。除此之外，MyBatis 从 3.4.2 版本开始，对 JDK1.8 接口的默认方法提供了支持，具体就不分析了。完成相关检测后，紧接着从缓存中获取或者创建 `MapperMethod` 对象，然后通过该对象中的 `execute` 方法执行 SQL。在分析 `execute` 方法之前，我们先来看一下 `MapperMethod` 对象的创建过程。`MapperMethod` 的创建过程看似普通，但却包含了一些重要的逻辑，所以不能忽视。

1. 创建 `MapperMethod` 对象

本节来分析一下 `MapperMethod` 的构造方法，看看它的构造方法中都包含了哪些逻辑。如下：

```

public class MapperMethod {

    private final SqlCommand command;
    private final MethodSignature method;

    public MapperMethod(Class<?> mapperInterface,
        Method method, Configuration config) {

```

```
// 创建 SqlCommand 对象, 该对象包含一些和 SQL 相关的信息
this.command = new SqlCommand(config, mapperInterface, method);
// 创建 MethodSignature 对象, 由类名可知, 该对象包含了被拦截方法的一些信息
this.method = new MethodSignature(config, mapperInterface, method);
}
}
```

MapperMethod 构造方法的逻辑很简单, 主要是创建 SqlCommand 和 MethodSignature 对象。这两个对象分别记录了不同的信息, 这些信息在后续的方法调用中都会被用到。下面我们深入到这两个类的构造方法中, 探索它们的初始化逻辑。

- 创建 SqlCommand 对象

前面说了 SqlCommand 中保存了一些和 SQL 相关的信息, 那具体有哪些信息呢? 答案在下面的代码中。

```
public static class SqlCommand {

    private final String name;
    private final SqlCommandType type;

    public SqlCommand(Configuration configuration,
        Class<?> mapperInterface, Method method) {

        final String methodName = method.getName();
        final Class<?> declaringClass = method.getDeclaringClass();
        // 解析 MappedStatement
        MappedStatement ms = resolveMappedStatement(
            mapperInterface, methodName, declaringClass, configuration);

        // 检测当前方法是否有对应的 MappedStatement
        if (ms == null) {
            // 检测当前方法是否有 @Flush 注解
        }
    }
}
```

```

        if (method.getAnnotation(Flush.class) != null) {
            // 设置 name 和 type 遍历
            name = null;
            type = SqlCommandType.FLUSH;
        } else {
            // 若 ms == null 且方法无 @Flush 注解, 此时抛出异常。
            // 这个异常比较常见, 大家应该眼熟吧
            throw new BindingException(".....");
        }
    } else {
        // 设置 name 和 type 变量
        name = ms.getId();
        type = ms.getSqlCommandType();
        if (type == SqlCommandType.UNKNOWN) {
            throw new BindingException(".....");
        }
    }
}
}

```

SqlCommand 的构造方法主要用于初始化它的两个成员变量。代码不是很长, 逻辑也不难理解, 就不多说了。继续往下看。

- 创建 MethodSignature 对象

MethodSignature 即方法签名, 顾名思义, 该类保存了一些和目标方法相关的信息。比如目标方法的返回类型, 目标方法的参数列表信息等。下面, 我们分析一下 MethodSignature 的构造方法。

```

public static class MethodSignature {

    private final boolean returnsMany;
    private final boolean returnsMap;

```



```
private final boolean returnsVoid;
private final boolean returnsCursor;
private final Class<?> returnType;
private final String mapKey;
private final Integer resultHandlerIndex;
private final Integer rowBoundsIndex;
private final ParamNameResolver paramNameResolver;

public MethodSignature(Configuration configuration,
    Class<?> mapperInterface, Method method) {

    // 通过反射解析方法返回类型
    Type resolvedReturnType = TypeParameterResolver
        .resolveReturnType(method, mapperInterface);
    if (resolvedReturnType instanceof Class<?>) {
        this.returnType = (Class<?>) resolvedReturnType;
    } else if (resolvedReturnType instanceof ParameterizedType) {
        this.returnType = (Class<?>) (
            (ParameterizedType) resolvedReturnType).getRawType();
    } else {
        this.returnType = method.getReturnType();
    }

    // 检测返回值类型是否是 void、集合或数组、Cursor、Map 等
    this.returnsVoid = void.class.equals(this.returnType);
    this.returnsMany = configuration.getObjectFactory()
        .isCollection(this.returnType) || this.returnType.isArray();
    this.returnsCursor = Cursor.class.equals(this.returnType);

    // 解析 @MapKey 注解，获取注解内容
    this.mapKey = getMapKey(method);
    this.returnsMap = this.mapKey != null;
```

```

        // 获取 RowBounds 参数在参数列表中的位置，如果参数列表中
        // 包含多个 RowBounds 参数，此方法会抛出异常
        this.rowBoundsIndex = getUniqueParamIndex(method, RowBounds.class);
        // 获取 ResultHandler 参数在参数列表中的位置
        this.resultHandlerIndex =
            getUniqueParamIndex(method, ResultHandler.class);
        // 解析参数列表
        this.paramNameResolver =
            new ParamNameResolver(configuration, method);
    }
}

```

上面的代码用于检测目标方法的返回类型，以及解析目标方法参数列表。其中，检测返回类型的目的是为避免查询方法返回错误的类型。比如我们要求接口方法返回一个对象，结果却返回了对象集合，这会导致类型转换错误。关于返回值类型的解析过程先说到这，下面分析参数列表的解析过程。

```

public class ParamNameResolver {

    private static final String GENERIC_NAME_PREFIX = "param";
    private final SortedMap<Integer, String> names;

    public ParamNameResolver(Configuration config, Method method) {
        // 获取参数类型列表
        final Class<?>[] paramTypes = method.getParameterTypes();
        // 获取参数注解
        final Annotation[][] paramAnnotations =
            method.getParameterAnnotations();
        final SortedMap<Integer, String> map =
            new TreeMap<Integer, String>();
        int paramCount = paramAnnotations.length;
    }
}

```

```
for (int paramIndex = 0; paramIndex < paramCount; paramIndex++) {
    // 检测当前的参数类型是否为 RowBounds 或 ResultHandler
    if (isSpecialParameter(paramTypes[paramIndex])) {
        continue;
    }
    String name = null;
    for (Annotation annotation : paramAnnotations[paramIndex]) {
        if (annotation instanceof Param) {
            hasParamAnnotation = true;
            // 获取 @Param 注解内容
            name = ((Param) annotation).value();
            break;
        }
    }
    // name 为空, 表明未给参数配置 @Param 注解
    if (name == null) {
        // 检测是否设置了 useActualParamName 全局配置
        if (config.isUseActualParamName()) {
            // 通过反射获取参数名称。此种方式要求 JDK 版本为 1.8+,
            // 且要求编译时加入 -parameters 参数, 否则获取到的参数名
            // 仍然是 arg1, arg2, ..., argN
            name = getActualParamName(method, paramIndex);
        }
        if (name == null) {
            /*
             * 使用 map.size() 返回值作为名称, 思考一下为什么不这样写:
             *   name = String.valueOf(paramIndex);
             * 因为如果参数列表中包含 RowBounds 或 ResultHandler, 这两个
             * 参数会被忽略掉, 这样将导致名称不连续。
             *
             * 比如参数列表 (int p1, int p2, RowBounds rb, int p3)
             * - 期望得到名称列表为 ["0", "1", "2"]
             * - 实际得到名称列表为 ["0", "1", "3"]
             */
        }
    }
}
```

```

        name = String.valueOf(map.size());
    }
}
// 存储 paramIndex 到 name 的映射
map.put(paramIndex, name);
}
names = Collections.unmodifiableSortedMap(map);
}
}

```

方法参数列表解析完毕后，可得到参数下标与参数名的映射关系，这些映射关系最终存储在 ParamNameResolver 的 names 成员变量中。这些映射关系将会在后面的代码中被用到，大家留意一下。下面写点代码测试一下 ParamNameResolver 的解析逻辑。如下：

```

public class ParamNameResolverTest {

    @Test
    public void test() throws NoSuchMethodException,
        NoSuchFieldException, IllegalAccessException {
        Configuration config = new Configuration();
        config.setUseActualParamName(false);
        Method method = ArticleMapper.class.getMethod("select",
            Integer.class, String.class, RowBounds.class, Article.class);

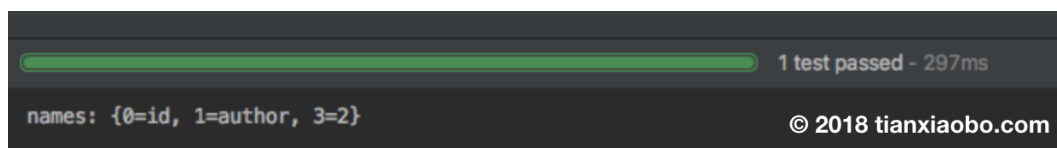
        ParamNameResolver resolver = new ParamNameResolver(config, method);
        Field field = resolver.getClass().getDeclaredField("names");
        field.setAccessible(true);
        // 通过反射获取 ParamNameResolver 私有成员变量 names
        Object names = field.get(resolver);

        System.out.println("names: " + names);
    }
}

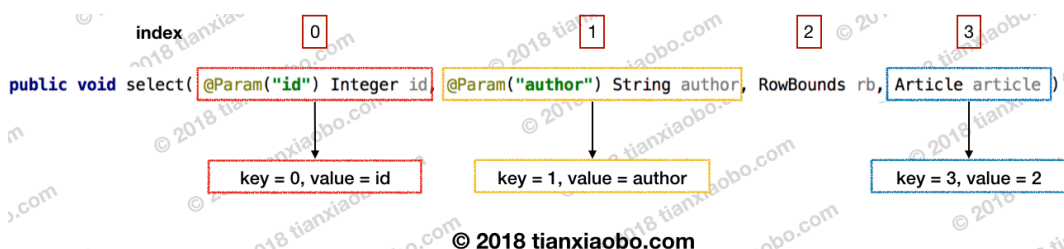
```

```
class ArticleMapper {  
    public void select(@Param("id") Integer id,  
        @Param("author") String author, RowBounds rb, Article article) {}  
}
```

测试结果如下：



参数索引与名称映射图如下：



到此，关于 MapperMethod 的初始化逻辑就分析完了，继续往下分析。

2. 执行 execute 方法

前面已经分析了 MapperMethod 的初始化过程，现在 MapperMethod 创建好了。那么，接下来要做的事情是调用 MapperMethod 的 execute 方法，执行 SQL。代码如下：

```
// -☆- MapperMethod  
public Object execute(SqlSession sqlSession, Object[] args) {  
    Object result;  
    // 根据 SQL 类型执行相应的数据库操作  
    switch (command.getType()) {  
        case INSERT: {  
            // 对用户传入的参数进行转换，下同
```

```
Object param = method.convertArgsToSqlCommandParam(args);
// 执行插入操作, rowCountResult 方法用于处理返回值
result = rowCountResult(
    sqlSession.insert(command.getName(), param));
break;
}
case UPDATE: {
    Object param = method.convertArgsToSqlCommandParam(args);
    // 执行更新操作
    result = rowCountResult(
        sqlSession.update(command.getName(), param));
    break;
}
case DELETE: {
    Object param = method.convertArgsToSqlCommandParam(args);
    // 执行删除操作
    result = rowCountResult(
        sqlSession.delete(command.getName(), param));
    break;
}
case SELECT:
    // 根据目标方法的返回类型进行相应的查询操作
    if (method.returnsVoid() && method.hasResultHandler()) {
        // 如果方法返回值为 void, 但参数列表中包含 ResultHandler, 表明
        // 使用者想通过 ResultHandler 的方式获取查询结果, 而非通过返回值
        // 获取结果
        executeWithResultHandler(sqlSession, args);
        result = null;
    } else if (method.returnsMany()) {
        // 执行查询操作, 并返回多个结果
        result = executeForMany(sqlSession, args);
    } else if (method.returnsMap()) {
        // 执行查询操作, 并将结果封装在 Map 中返回
    }
```

```
        result = executeForMap(sqlSession, args);
    } else if (method.returnsCursor()) {
        // 执行查询操作，并返回一个 Cursor 对象
        result = executeForCursor(sqlSession, args);
    } else {
        Object param = method.convertArgsToSqlCommandParam(args);
        // 执行查询操作，并返回一个结果
        result = sqlSession.selectOne(command.getName(), param);
    }
    break;
case FLUSH:
    // 执行刷新操作
    result = sqlSession.flushStatements();
    break;
default:
    throw new BindingException(".....");
}
// 如果方法的返回值为基本类型，而返回值却为 null，此种情况下应抛出异常
if (result == null && method.getReturnType().isPrimitive()
    && !method.returnsVoid()) {
    throw new BindingException(".....");
}
return result;
}
```

如上，execute 方法主要由一个 switch 语句组成，用于根据 SQL 类型执行相应的数据库操作。该方法的逻辑清晰，不需要太多的分析。不过在上面代码中 convertArgsToSqlCommandParam 方法出现次数比较频繁，这里分析一下：

```
// -☆- MapperMethod
public Object convertArgsToSqlCommandParam(Object[] args) {
    return paramNameResolver.getNamedParams(args);
}
```

```

public Object getNamedParams(Object[] args) {
    final int paramCount = names.size();
    if (args == null || paramCount == 0) {
        return null;
    } else if (!hasParamAnnotation && paramCount == 1) {
        /*
         * 如果方法参数列表无 @Param 注解，且仅有一个非特别参数，则返回该
         * 参数的值。比如如下方法：
         *     List findList(RowBounds rb, String name)
         * names 如下：
         *     names = {1 : "0"}
         * 此种情况下，返回 args[names.firstKey()]，即 args[1] -> name
         */
        return args[names.firstKey()];
    } else {
        final Map<String, Object> param = new ParamMap<Object>();
        int i = 0;
        for (Map.Entry<Integer, String> entry : names.entrySet()) {
            // 添加 <参数名, 参数值> 键值对到 param 中
            param.put(entry.getValue(), args[entry.getKey()]);
            // genericParamName = param + index。比如 param1, param2,... paramN
            final String genericParamName =
                GENERIC_NAME_PREFIX + String.valueOf(i + 1);

            // 检测 names 中是否包含 genericParamName，什么情况下会包含？
            // 答案如下：
            //     使用者显式将参数名称配置为 param1，即 @Param("param1")
            if (!names.containsValue(genericParamName)) {
                // 添加 <param*, value> 到 param 中
                param.put(genericParamName, args[entry.getKey()]);
            }
            i++;
        }
    }
}

```



```
        return param;
    }
}
```

`convertArgsToSqlCommandParam` 是一个空壳方法，该方法最终调用了 `ParamNameResolver` 的 `getNamedParams` 方法。`getNamedParams` 方法的主要逻辑是根据条件返回不同的结果，该方法的代码不是很难理解，我也进行了比较详细的注释，就不多说了。分析完 `convertArgsToSqlCommandParam` 的逻辑，接下来说说 MyBatis 对哪些 SQL 指令提供了支持，如下：

- 查询语句：SELECT
- 更新语句：INSERT/UPDATE/DELETE
- 存储过程：CALL

在上面的列表中，我刻意对 SELECT/INSERT/UPDATE/DELETE 等指令进行了分类，分类依据指令的功能以及 MyBatis 执行这些指令的过程。这里把 SELECT 称为查询语句，INSERT/UPDATE/DELETE 等称为更新语句。下来按照顺序对着两种语句执行过程进行分析，先来分析查询语句的执行过程。

4.2 查询语句的执行过程

查询语句对应的方法比较多，有如下几种：

- `executeWithResultHandler`
- `executeForMany`
- `executeForMap`
- `executeForCursor`

这些方法在内部调用了 `SqlSession` 中的一些 `select*` 方法，比如 `selectList`、`selectMap`、`selectCursor` 等。这些方法的返回值类型是不同的，因此对于每种返回类型，需要有专门的处理方法。以 `selectList` 方法为例，该方法的返回值类型为 `List`。但如果我们的 `Mapper` 或 `Dao` 的接口方法返回值类型为数组，或者 `Set`，直接将 `List` 类型的结果返回给 `Mapper/Dao` 就不合适了。`execute*` 等方法只是对 `select*` 等方法做了一层简单的封装，因此接下来我们应应该把目光放在这些 `select*` 方法上。

4.2.1 selectOne 方法分析

本节选择分析 `selectOne` 方法，而不是其他的方法，大家或许会觉得奇怪。前面提及了 `selectList`、`selectMap`、`selectCursor` 等方法，这里却分析一个未提及的方法。这样做也没什么特别之处，主要原因是 `selectOne` 在内部会调用 `selectList` 方法。这里分析 `selectOne` 方法是为了告知大家，`selectOne` 和 `selectList` 方法是有联系的，同时分析 `selectOne` 方法等同于分析 `selectList` 方法。如果你不信的话，那我们看源码吧，源码面前了无秘密。

```
// -☆- DefaultSqlSession
public <T> T selectOne(String statement, Object parameter) {
    // 调用 selectList 获取结果
    List<T> list = this.<T>selectList(statement, parameter);
    if (list.size() == 1) {
        // 返回结果
        return list.get(0);
    } else if (list.size() > 1) {
        // 如果查询结果大于 1 则抛出异常，这个异常也是很常见的
        throw new TooManyResultsException(".....");
    } else {
        return null;
    }
}
```

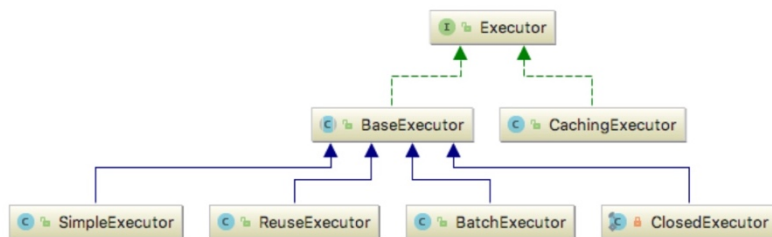
如上，`selectOne` 方法在内部调用 `selectList` 了方法，并取 `selectList` 返回值的第 1 个元素作为自己的返回值。如果 `selectList` 返回的列表元素大于 1，则抛出异常。上面代码比较易懂，就不多说了。下面我们来看看 `selectList` 方法的实现。

```
// -☆- DefaultSqlSession
public <E> List<E> selectList(String statement, Object parameter) {
    // 调用重载方法
    return this.selectList(statement, parameter, RowBounds.DEFAULT);
}
```

```
private final Executor executor;

public <E> List<E> selectList(String statement, Object parameter, RowBounds
rowBounds) {
    try {
        // 获取 MappedStatement
        MappedStatement ms = configuration.getMappedStatement(statement);
        // 调用 Executor 实现类中的 query 方法
        return executor.query(ms, wrapCollection(parameter),
            rowBounds, Executor.NO_RESULT_HANDLER);
    } catch (Exception e) {
        throw ExceptionFactory.wrapException(".....");
    } finally {
        ErrorContext.instance().reset();
    }
}
```

如上，这里要来说说 executor 变量，该变量类型为 Executor。Executor 是一个接口，它的实现类如下：



Executor 有这么多的实现类，大家猜一下 executor 变量对应哪个实现类。要弄清楚这个问题，需要大家到源头去查证。这里提示一下，大家可以跟踪一下 DefaultSqlSessionFactory 的 openSession 方法，很快就能发现 executor 变量创建的踪迹。限于篇幅原因，本文就不分析 openSession 方法的源码了。默认情况下，executor 的类型为 CachingExecutor，该类是一个装饰器类，用于给目标 Executor 增加二级缓存功能。那目标 Executor 是谁呢？默认情况下是 SimpleExecutor。

现在大家搞清楚 executor 变量的身份了，接下来继续分析 selectOne 方法的调用栈。先来看看 CachingExecutor 的 query 方法是怎样实现的。如下：

```
// -☆- CachingExecutor
public <E> List<E> query(MappedStatement ms, Object parameterObject,
    RowBounds rowBounds, ResultHandler resultHandler) throws SQLException {
    // 获取 BoundSql
    BoundSql boundSql = ms.getBoundSql(parameterObject);
    // 创建 CacheKey
    CacheKey key = createCacheKey(ms, parameterObject, rowBounds, boundSql);
    // 调用重载方法
    return query(ms, parameterObject,
        rowBounds, resultHandler, key, boundSql);
}
```

上面的代码用于获取 BoundSql 对象，创建 CacheKey 对象，然后再将这两个对象传给重载方法。BoundSql 的获取过程较为复杂，我将在下一节进行分析。CacheKey 以及接下来即将出现的一二级缓存将会独立成章分析。

上面的方法等代码和 SimpleExecutor 父类 BaseExecutor 中的实现没什么区别，有区别的地方在于这个方法所调用的重载方法。继续往下看。

```
// -☆- CachingExecutor
public <E> List<E> query(MappedStatement ms, Object parameterObject,
    RowBounds rowBounds, ResultHandler resultHandler, CacheKey key,
    BoundSql boundSql) throws SQLException {
    // 从 MappedStatement 中获取缓存
    Cache cache = ms.getCache();
    // 若映射文件中未配置缓存或参照缓存，此时 cache = null
    if (cache != null) {
        flushCacheIfRequired(ms);
        if (ms.isUseCache() && resultHandler == null) {
            ensureNoOutParams(ms, boundSql);
            List<E> list = (List<E>) tcm.getObject(cache, key);
```

```
        if (list == null) {
            // 若缓存未命中，则调用被装饰类的 query 方法
            list = delegate.<E>query(ms, parameterObject,
                                    rowBounds, resultHandler, key, boundSql);
            tcm.putObject(cache, key, list); // issue #578 and #116
        }
        return list;
    }
}

// 调用被装饰类的 query 方法
return delegate.<E>query(
    ms, parameterObject, rowBounds, resultHandler, key, boundSql);
}
```

以上代码涉及到了二级缓存，若二级缓存为空，或未命中，则调用被装饰类的 query 方法。下面来看一下 BaseExecutor 的中签名相同的 query 方法是如何实现的。

```
// -☆- BaseExecutor
public <E> List<E> query(MappedStatement ms, Object parameter,
    RowBounds rowBounds, ResultHandler resultHandler, CacheKey key,
    BoundSql boundSql) throws SQLException {
    if (closed) {
        throw new ExecutorException("Executor was closed.");
    }

    if (queryStack == 0 && ms.isFlushCacheRequired()) {
        clearLocalCache();
    }
    List<E> list;
    try {
        queryStack++;
    }
```

```

// 从一级缓存中获取缓存项
list = resultHandler == null ?
    (List<E>) localCache.getObject(key) : null;
if (list != null) {
    // 存储过程相关处理逻辑，本文不分析存储过程，故该方法不分析了
    handleLocallyCachedOutputParameters(ms, key, parameter, boundSql);
} else {
    // 一级缓存未命中，则从数据库中查询
    list = queryFromDatabase(ms, parameter,
        rowBounds, resultHandler, key, boundSql);
}
} finally {
    queryStack--;
}

if (queryStack == 0) {
    // 从一级缓存中延迟加载嵌套查询结果
    for (DeferredLoad deferredLoad : deferredLoads) {
        deferredLoad.load();
    }
    deferredLoads.clear();
    if (configuration.getLocalCacheScope() == LocalCacheScope.STATEMENT) {
        clearLocalCache();
    }
}
return list;
}

```

上面的方法主要用于从一级缓存中查找查询结果，若缓存未命中，再向数据库进行查询。在上面的代码中，出现了一个新的类 `DeferredLoad`，这个类用于延迟加载。该类的实现并不复杂，但是具体用途让我有点疑惑。这个我目前也未完全搞清楚，就不分析了。接下来，我们来看一下 `queryFromDatabase` 方法的实现。

```
// -☆- BaseExecutor
private <E> List<E> queryFromDatabase(MappedStatement ms, Object parameter,
    RowBounds rowBounds, ResultHandler resultHandler, CacheKey key,
    BoundSql boundSql) throws SQLException {
    List<E> list;
    // 向缓存中存储一个占位符
    localCache.putObject(key, EXECUTION_PLACEHOLDER);
    try {
        // 调用 doQuery 进行查询
        list = doQuery(ms, parameter, rowBounds, resultHandler, boundSql);
    } finally {
        // 移除占位符
        localCache.removeObject(key);
    }
    // 缓存查询结果
    localCache.putObject(key, list);
    if (ms.getStatementType() == StatementType.CALLABLE) {
        localOutputParameterCache.putObject(key, parameter);
    }
    return list;
}
```

上面的代码仍然不是 selectOne 方法调用栈的终点，抛开缓存操作，queryFromDatabase 最终还会调用 doQuery 进行查询。所以下面我们继续进行跟踪。

```
// -☆- SimpleExecutor
public <E> List<E> doQuery(MappedStatement ms, Object parameter,
    RowBounds rowBounds, ResultHandler resultHandler, BoundSql boundSql)
    throws SQLException {
    Statement stmt = null;
    try {
```

```

        Configuration configuration = ms.getConfiguration();
        // 创建 StatementHandler
        StatementHandler handler = configuration.newStatementHandler(
            wrapper, ms, parameter, rowBounds, resultHandler, boundSql);
        // 创建 Statement
        stmt = prepareStatement(handler, ms.getStatementLog());
        // 执行查询操作
        return handler.<E>query(stmt, resultHandler);
    } finally {
        // 关闭 Statement
        closeStatement(stmt);
    }
}

```

doQuery 方法中仍然有不少的逻辑，完全看不到即将要到达终点的趋势，不过这离终点又近了一步。接下来，我们先跳过 StatementHandler 和 Statement 创建过程，这两个对象的创建过程会在后面进行说明。这里，我们以 PreparedStatementHandler 为例，看看它的 query 方法是怎样实现的。如下：

```

// -☆- PreparedStatementHandler
public <E> List<E> query(Statement statement, ResultHandler resultHandler)
    throws SQLException {
    PreparedStatement ps = (PreparedStatement) statement;
    // 执行 SQL
    ps.execute();
    // 处理执行结果
    return resultSetHandler.<E>handleResultSets(ps);
}

```

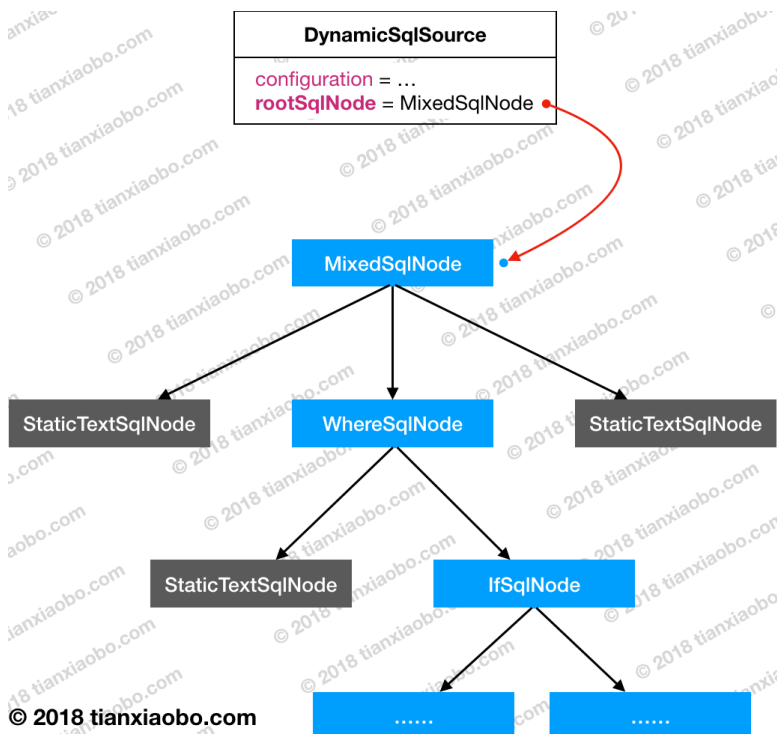
到这里似乎看到了希望，整个调用过程总算要结束了。不过先别高兴的太早，SQL 执行结果的处理过程也很复杂，稍后将会专门拿出一节内容进行分析。

以上就是 selectOne 方法的执行过程，尽管我已经简化了代码分析，但是整个过程看起

来还是很复杂的。查询过程涉及到了很多方法调用，不把这些调用方法搞清楚，很难对 MyBatis 的查询过程有深入的理解。所以在接下来的章节中，我将会对一些重要的调用进行分析。如果大家不满足于泛泛而谈，那么接下来咱们一起进行更深入的探索吧。

4.2.2 获取 BoundSql

在执行 SQL 之前，需要将 SQL 语句完整的解析出来。我们都知道 SQL 是配置在映射文件中的，但由于映射文件中的 SQL 可能会包含占位符#{ }，以及动态 SQL 标签，比如<if>、<where>等。因此，我们并不能直接使用映射文件中配置的 SQL。MyBatis 会将映射文件中的 SQL 解析成一组 SQL 片段。如果某个片段中也包含动态 SQL 相关的标签，那么，MyBatis 会对该片段再次进行分片。最终，一个 SQL 配置将会被解析成一个 SQL 片段树。形如下面的图片：



我们需要对片段树进行解析，以便从每个片段对象中获取相应的内容。然后将这些内容组合起来即可得到一个完成的 SQL 语句，这个完整的 SQL 以及其他的一些信息最终会存储在 BoundSql 对象中。下面我们来看一下 BoundSql 类的成员变量信息，如下：

```
private final String sql;
private final List<ParameterMapping> parameterMappings;
private final Object parameterObject;
private final Map<String, Object> additionalParameters;
private final MetaObject metaParameters;
```

下面用一个表格列举各个成员变量的含义。

变量名	类型	用途
sql	String	一个完整的 SQL 语句，可能会包含问号 ? 占位符
parameterMappings	List	参数映射列表，SQL 中的每个 #{xxx} 占位符都会被解析成相应的 ParameterMapping 对象
parameterObject	Object	运行时参数，即用户传入的参数，比如 Article 对象，或是其他的参数
additionalParameters	Map	附加参数集合，用于存储一些额外的信息，比如 databaseId 等
metaParameters	MetaObject	additionalParameters 的元信息对象

以上对 BoundSql 的成员变量做了简要的说明，部分参数的用途大家现在可能不是很明白。不过不用着急，这些变量在接下来的源码分析过程中会陆续的出现。到时候对着源码多思考，或是写点测试代码调试一下，即可弄懂。

好了，现在准备工作已经做好。接下来，开始分析 BoundSql 的构建过程。我们源码之旅的第一站是 MappedStatement 的 getBoundSql 方法，代码如下：

```
// -☆- MappedStatement
public BoundSql getBoundSql(Object parameterObject) {

    // 调用 sqlSource 的 getBoundSql 获取 BoundSql
    BoundSql boundSql = sqlSource.getBoundSql(parameterObject);
    List<ParameterMapping> parameterMappings =
        boundSql.getParameterMappings();
```

```
if (parameterMappings == null || parameterMappings.isEmpty()) {  
    // 创建新的 BoundSql, 这里的 parameterMap 是 ParameterMap 类型。  
    // 由<ParameterMap> 节点进行配置, 该节点已经废弃, 不推荐使用。  
    // 默认情况下, parameterMap.getParameterMappings() 返回空集合  
    boundSql = new BoundSql(configuration, boundSql.getSql(),  
        parameterMap.getParameterMappings(), parameterObject);  
}  
  
// 省略不重要的逻辑  
  
return boundSql;  
}
```

如上, MappedStatement 的 getBoundSql 在内部调用了 SqlSource 实现类的 getBoundSql 方法。处理此处的调用, 余下的逻辑都不是重要逻辑, 就不啰嗦了。接下来, 我们把目光转移到 SqlSource 实现类的 getBoundSql 方法上。SqlSource 是一个接口, 它有如下几个实现类:

- DynamicSqlSource
- RawSqlSource
- StaticSqlSource
- ProviderSqlSource
- VelocitySqlSource

在如上几个实现类中, 我们应该选择分析哪个实现类的逻辑呢? 首先我们把最后两个排除掉, 不常用。剩下的三个实现类中, 仅前两个实现类会在映射文件解析的过程中被使用。当 SQL 配置中包含\${} (不是#{}) 占位符, 或者包含<if>、<where>等标签时, 会被认为是动态 SQL, 此时使用 DynamicSqlSource 存储 SQL 片段。否则, 使用 RawSqlSource 存储 SQL 配置信息。相比之下 DynamicSqlSource 存储的 SQL 片段类型较多, 解析起来也更为复杂一些。因此下面我将分析 DynamicSqlSource 的 getBoundSql 方法。弄懂这个, RawSqlSource 也不在话下。

```
// -☆- DynamicSqlSource  
public BoundSql getBoundSql(Object parameterObject) {
```

```

// 创建 DynamicContext
DynamicContext context =
    new DynamicContext(configuration, parameterObject);

// 解析 SQL 片段, 并将解析结果存储到 DynamicContext 中
rootSqlNode.apply(context);

SqlSourceBuilder sqlSourceParser = new SqlSourceBuilder(configuration);
Class<?> parameterType = parameterObject == null ?
    Object.class : parameterObject.getClass();

// 构建 StaticSqlSource, 在此过程中将 sql 语句中的占位符 #{ } 替换为问号 ?,
// 并为每个占位符构建相应的 ParameterMapping
SqlSource sqlSource = sqlSourceParser.parse(
    context.getSql(), parameterType, context.getBindings());

// 调用 StaticSqlSource 的 getBoundSql 获取 BoundSql
BoundSql boundSql = sqlSource.getBoundSql(parameterObject);

// 将 DynamicContext 的 ContextMap 中的内容拷贝到 BoundSql 中
for(Map.Entry<String, Object> entry : context.getBindings().entrySet()){
    boundSql.setAdditionalParameter(entry.getKey(), entry.getValue());
}
return boundSql;
}

```

如上, DynamicSqlSource 的 getBoundSql 方法的代码看起来不多, 但是逻辑却并不简单。该方法由数个步骤组成, 这里总结一下:

1. 创建 DynamicContext
2. 解析 SQL 片段, 并将解析结果存储到 DynamicContext 中
3. 解析 SQL 语句, 并构建 StaticSqlSource
4. 调用 StaticSqlSource 的 getBoundSql 获取 BoundSql
5. 将 DynamicContext 的 ContextMap 中的内容拷贝到 BoundSql 中

如上 5 个步骤中，第 5 步为常规操作，就不多说了，其他步骤将会在接下来章节中一一进行分析。按照顺序，我们先来分析 DynamicContext 的实现。

1. DynamicContext

DynamicContext 是 SQL 语句构建的上下文，每个 SQL 片段解析完成后，都会将解析结果存入 DynamicContext 中。待所有的 SQL 片段解析完毕后，一条完整的 SQL 语句就会出现在 DynamicContext 对象中。下面我们来看一下 DynamicContext 类的定义。

```
public class DynamicContext {

    public static final String PARAMETER_OBJECT_KEY = "_parameter";
    public static final String DATABASE_ID_KEY = "_databaseId";

    private final ContextMap bindings;
    private final StringBuilder sqlBuilder = new StringBuilder();

    public DynamicContext(
        Configuration configuration, Object parameterObject) {
        // 创建 ContextMap
        if (parameterObject != null && !(parameterObject instanceof Map)) {
            MetaObject metaObject =
                configuration.newMetaObject(parameterObject);
            bindings = new ContextMap(metaObject);
        } else {
            bindings = new ContextMap(null);
        }

        // 存放运行时参数 parameterObject 以及 databaseId
        bindings.put(PARAMETER_OBJECT_KEY, parameterObject);
        bindings.put(DATABASE_ID_KEY, configuration.getDatabaseId());
    }
}
```

上面只贴了 DynamicContext 类的部分代码。其中 sqlBuilder 变量用于存放 SQL 片段的解析结果，bindings 则用于存储一些额外的信息，比如运行时参数和 databaseId 等。bindings 类型为 ContextMap，ContextMap 定义在 DynamicContext 中，是一个静态内部类。该类继承自 HashMap，并覆写了 get 方法。它的代码如下：

```
static class ContextMap extends HashMap<String, Object> {
    private MetaObject parameterMetaObject;

    public ContextMap(MetaObject parameterMetaObject) {
        this.parameterMetaObject = parameterMetaObject;
    }

    @Override
    public Object get(Object key) {
        String strKey = (String) key;
        // 检查是否包含 strKey，若包含则直接返回
        if (super.containsKey(strKey)) {
            return super.get(strKey);
        }
        if (parameterMetaObject != null) {
            // 从运行时参数中查找结果
            return parameterMetaObject.getValue(strKey);
        }

        return null;
    }
}
```

DynamicContext 对外提供了两个接口，用于操作 sqlBuilder。分别如下：

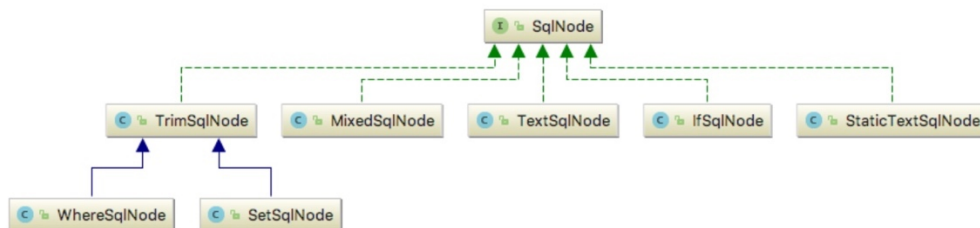
```
public void appendSql(String sql) {
    sqlBuilder.append(sql);
    sqlBuilder.append(" ");
}
```

```
public String getSql() {  
    return sqlBuilder.toString().trim();  
}
```

以上就是对 DynamicContext 的简单介绍，DynamicContext 的源码不难理解，这里就不多说了。继续往下分析。

2. 解析 SQL 片段

对于一个包含了`${}`占位符，或`<if>`、`<where>`等标签的 SQL，在解析的过程中，会被分解成多个片段。每个片段都有对应的类型，每种类型的片段都有不同的解析逻辑。在源码中，片段这个概念等价于 sql 节点，即 `SqlNode`。`SqlNode` 是一个接口，它有众多的实现类。其继承体系如下：



上图只画出了部分的实现类，还有一小部分没画出来，不过这并不影响接下来的分析。在众多实现类中，`StaticTextSqlNode` 用于存储静态文本，`TextSqlNode` 用于存储带有`${}`占位符的文本，`IfSqlNode` 则用于存储`<if>`节点的内容。`MixedSqlNode` 内部维护了一个 `SqlNode` 集合，用于存储各种各样的 `SqlNode`。接下来，我将会对 `MixedSqlNode`、`StaticTextSqlNode`、`TextSqlNode`、`IfSqlNode`、`WhereSqlNode` 以及 `TrimSqlNode` 等进行分析，其他的实现类请大家自行分析。

```
public class MixedSqlNode implements SqlNode {  
    private final List<SqlNode> contents;  
  
    public MixedSqlNode(List<SqlNode> contents) {  
        this.contents = contents;  
    }  
}
```

```

@Override
public boolean apply(DynamicContext context) {
    // 遍历 SqlNode 集合
    for (SqlNode sqlNode : contents) {
        // 调用 sqlNode 对象本身的 apply 方法解析 sql
        sqlNode.apply(context);
    }
    return true;
}
}

```

MixedSqlNode 可以看做是 SqlNode 实现类对象的容器，凡是实现了 SqlNode 接口的类都可以存储到 MixedSqlNode 中，包括它自己。MixedSqlNode 解析方法 apply 逻辑比较简单，即遍历 SqlNode 集合，并调用其他 SqlNode 实现类对象的 apply 方法解析 sql。那下面我们来看看其他 SqlNode 实现类的 apply 方法是怎样实现的。

```

public class StaticTextSqlNode implements SqlNode {
    private final String text;

    public StaticTextSqlNode(String text) {
        this.text = text;
    }

    @Override
    public boolean apply(DynamicContext context) {
        context.appendSql(text);
        return true;
    }
}

```

StaticTextSqlNode 用于存储静态文本，所以它不需要什么解析逻辑，直接将其存储的 SQL 片段添加到 DynamicContext 中即可。StaticTextSqlNode 的实现比较简单，看起来很轻松。下面分析一下 TextSqlNode。


```
public class TextSqlNode implements SqlNode {

    private final String text;
    private final Pattern injectionFilter;

    @Override
    public boolean apply(DynamicContext context) {
        // 创建 ${} 占位符解析器
        GenericTokenParser parser = createParser(
            new BindingTokenParser(context, injectionFilter));
        // 解析 ${} 占位符，并将解析结果添加到 DynamicContext 中
        context.appendSql(parser.parse(text));
        return true;
    }

    private GenericTokenParser createParser(TokenHandler handler) {
        // 创建占位符解析器，GenericTokenParser 是一个通用解析器，
        // 并非只能解析 ${} 占位符
        return new GenericTokenParser("${", "}", handler);
    }

    private static class BindingTokenParser implements TokenHandler {

        private DynamicContext context;
        private Pattern injectionFilter;

        public BindingTokenParser(
            DynamicContext context, Pattern injectionFilter) {
            this.context = context;
            this.injectionFilter = injectionFilter;
        }

        @Override
```

```

public String handleToken(String content) {
    Object parameter = context.getBindings().get("_parameter");
    if (parameter == null) {
        context.getBindings().put("value", null);
    } else if (SimpleTypeRegistry.isSimpleType(parameter.getClass())) {
        context.getBindings().put("value", parameter);
    }
    // 通过 ONGL 从用户传入的参数中获取结果
    Object value = OgnlCache
        .getValue(content, context.getBindings());
    String srtValue = (value == null ? "" : String.valueOf(value));
    // 通过正则表达式检测 srtValue 有效性
    checkInjection(srtValue);
    return srtValue;
}
}
}

```

如上，GenericTokenParser 是一个通用的标记解析器，用于解析形如\${xxx}、#{xxx}等标记。GenericTokenParser 负责将标记中的内容抽取出来，并将标记内容交给相应的TokenHandler 去处理。BindingTokenParser 负责解析标记内容，并将解析结果返回给GenericTokenParser，用于替换\${xxx}标记。举个例子说明一下吧，如下。

我们有这样一个 SQL 语句，用于从 article 表中查询某个作者所写的文章。如下：

```
SELECT * FROM article WHERE author = '${author}'
```

假设我们传入的 author 值为 tianxiaobo，那么该 SQL 最终会被解析成如下的结果：

```
SELECT * FROM article WHERE author = 'tianxiaobo'
```

一般情况下，使用\${author}接受参数都没什么问题。但是怕就怕在有人不怀好意，构建了一些恶意的参数。当用这些恶意的参数替换\${author}时就会出现灾难性问题——SQL 注

入。比如我们构建这样一个参数 `author=tianxiaobo';DELETE FROM article;#`，然后我们把这个参数传给 `TextSqlNode` 进行解析。得到的结果如下

```
SELECT * FROM article WHERE author = 'tianxiaobo'; DELETE FROM article;#'
```

看到没，由于传入的参数没有经过转义，最终导致了一条 SQL 被恶意参数拼接成了两条 SQL。更要命的是，第二天 SQL 会把 `article` 表的数据清空，这个后果就很严重了（从删库到跑路）。这就是为什么我们不应该在 SQL 语句中是用 `${}` 占位符，风险太大。

分析完 `TextSqlNode` 的逻辑，接下来，分析 `IfSqlNode` 的实现。

```
public class IfSqlNode implements SqlNode {

    private final ExpressionEvaluator evaluator;
    private final String test;
    private final SqlNode contents;

    public IfSqlNode(SqlNode contents, String test) {
        this.test = test;
        this.contents = contents;
        this.evaluator = new ExpressionEvaluator();
    }

    @Override
    public boolean apply(DynamicContext context) {
        // 通过 ONGL 评估 test 表达式的结果
        if (evaluator.evaluateBoolean(test, context.getBindings())) {
            // 若 test 表达式中的条件成立，则调用其他节点的 apply 方法进行解析
            contents.apply(context);
            return true;
        }
        return false;
    }
}
```

IfSqlNode 对应的是<iftest='xxx'>节点，<if>节点是日常开发中使用频次比较高的一个节点。它的具体用法我想大家都很熟悉了，这里就不多啰嗦。IfSqlNode 的 apply 方法逻辑并不复杂，首先是通过 ONGL 检测 test 表达式是否为 true，如果为 true，则调用其他节点的 apply 方法继续进行解析。需要注意的是<if>节点中也可嵌套其他的动态节点，并非只有纯文本。因此 contents 变量遍历指向的是 MixedSqlNode，而非 StaticTextSqlNode。

关于 IfSqlNode 就说到这，接下来分析 WhereSqlNode 的实现。

```
public class WhereSqlNode extends TrimSqlNode {
    /** 前缀列表 */
    private static List<String> prefixList = Arrays.asList(
        "AND ", "OR ", "AND\n", "OR\n", "AND\r", "OR\r", "AND\t", "OR\t");

    public WhereSqlNode(Configuration configuration, SqlNode contents) {
        // 调用父类的构造方法
        super(configuration, contents, "WHERE", prefixList, null, null);
    }
}
```

在 MyBatis 中，WhereSqlNode 和 SetSqlNode 都是基于 TrimSqlNode 实现的，所以上面的代码看起来很简单。WhereSqlNode 对应于<where>节点，关于该节点的用法以及它的应用场景，大家请自行查阅资料。我在分析源码的过程中，默认大家已经知道了该节点的用途和应用场景。

接下来，我们把目光聚焦在 TrimSqlNode 的实现上。

```
public class TrimSqlNode implements SqlNode {

    private final SqlNode contents;
    private final String prefix;
    private final String suffix;
    private final List<String> prefixesToOverride;
    private final List<String> suffixesToOverride;
    private final Configuration configuration;
```

```
@Override
public boolean apply(DynamicContext context) {
    // 创建具有过滤功能的 DynamicContext
    FilteredDynamicContext filteredDynamicContext =
        new FilteredDynamicContext(context);
    // 解析节点内容
    boolean result = contents.apply(filteredDynamicContext);
    // 过滤掉前缀和后缀
    filteredDynamicContext.applyAll();
    return result;
}
}
```

如上，apply 方法首选调用了其他 SqlNode 的 apply 方法解析节点内容，这步操作完成后，FilteredDynamicContext 中会得到一条 SQL 片段字符串。接下来需要做的事情是过滤字符串前缀和后缀，并添加相应的前缀和后缀。这个事情由 FilteredDynamicContext 负责，FilteredDynamicContext 是 TrimSqlNode 的私有内部类。我们去看一下它的代码。

```
private class FilteredDynamicContext extends DynamicContext {

    private DynamicContext delegate;
    /** 构造方法会将下面两个布尔值置为 false */
    private boolean prefixApplied;
    private boolean suffixApplied;
    private StringBuilder sqlBuffer;

    public void applyAll() {
        sqlBuffer = new StringBuilder(sqlBuffer.toString().trim());
        String trimmedUppercaseSql =
            sqlBuffer.toString().toUpperCase(Locale.ENGLISH);
        if (trimmedUppercaseSql.length() > 0) {
            // 引用前缀和后缀，也就是对 sql 进行过滤操作，移除掉前缀或后缀
            applyPrefix(sqlBuffer, trimmedUppercaseSql);
        }
    }
}
```

```

        applySuffix(sqlBuffer, trimmedUppercaseSql);
    }
    // 将当前对象的 sqlBuffer 内容添加到代理类中
    delegate.appendSql(sqlBuffer.toString());
}

private void applyPrefix(StringBuilder sql, String trimmedUppercaseSql){
    if (!prefixApplied) {
        // 设置 prefixApplied 为 true, 以下逻辑仅会被执行一次
        prefixApplied = true;
        if (prefixesToOverride != null) {
            for (String toRemove : prefixesToOverride) {
                // 检测当前 sql 字符串是否包含前缀, 比如 'AND ', 'AND\t'等
                if (trimmedUppercaseSql.startsWith(toRemove)) {
                    // 移除前缀
                    sql.delete(0, toRemove.trim().length());
                    break;
                }
            }
        }
    }

    // 插入前缀, 比如 WHERE
    if (prefix != null) {
        sql.insert(0, " ");
        sql.insert(0, prefix);
    }
}

// 该方法逻辑与 applyPrefix 大同小异, 大家自行分析
private void applySuffix(
    StringBuilder sql, String trimmedUppercaseSql){
}
}

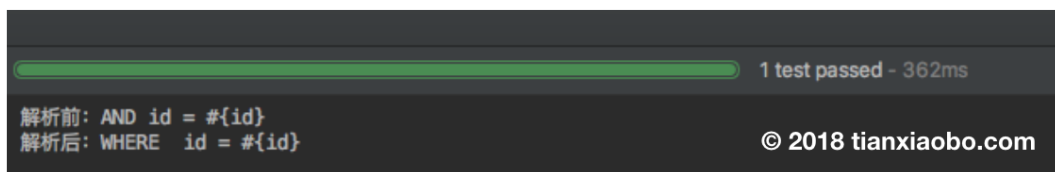
```

在上面的代码中，我们重点关注 `applyAll` 和 `applyPrefix` 方法，其他的方法大家自行分析。`applyAll` 方法的逻辑比较简单，首先从 `sqlBuffer` 中获取 SQL 字符串。然后调用 `applyPrefix` 和 `applySuffix` 进行过滤操作。最后将过滤后的 SQL 字符串添加到被装饰的类中。`applyPrefix` 方法会首先检测 SQL 字符串是不是以 "AND", "OR", 或 "AND\n", "OR\n" 等前缀开头，若是则将前缀从 `sqlBuffer` 中移除。然后将前缀插入到 `sqlBuffer` 的首部，整个逻辑就结束了。下面写点代码简单验证一下，如下：

```
public class SqlNodeTest {

    @Test
    public void testWhereSqlNode() throws IOException {
        String sqlFragment = "AND id = #{id}";
        MixedSqlNode msn = new MixedSqlNode(
            Arrays.asList(new StaticTextSqlNode(sqlFragment)));
        WhereSqlNode wsn = new WhereSqlNode(new Configuration(), msn);
        DynamicContext dc = new DynamicContext(
            new Configuration(), new ParamMap<>());
        wsn.apply(dc);
        System.out.println("解析前: " + sqlFragment);
        System.out.println("解析后: " + dc.getSql());
    }
}
```

测试结果如下：



```
解析前: AND id = #{id}
解析后: WHERE id = #{id}
```

1 test passed - 362ms

© 2018 tianxiaobo.com

3. 解析#{ }占位符

经过前面的解析，我们已经能从 `DynamicContext` 获取到完整的 SQL 语句了。但这并不意味着解析过程就结束了，因为当前的 SQL 语句中还有一种占位符没有处理，即 `#{ }`。与 `${ }`

占位符的处理方式不同，MyBatis 并不会直接将#{ }占位符替换为相应的参数值。#{ }占位符的解析逻辑这里先不多说，等相应的源码分析完了，答案就明了了。

#{ }占位符的解析逻辑是包含在 SqlSourceBuilder 的 parse 方法中，该方法最终会将解析后的 SQL 以及其他的一些数据封装到 StaticSqlSource 中。下面，一起来看一下 SqlSourceBuilder 的 parse 方法。

```
// -☆- SqlSourceBuilder
public SqlSource parse(String originalSql, Class<?> parameterType,
    Map<String, Object> additionalParameters) {
    // 创建 #{ } 占位符处理器
    ParameterMappingTokenHandler handler = new
    ParameterMappingTokenHandler(
        configuration, parameterType, additionalParameters);
    // 创建 #{ } 占位符解析器
    GenericTokenParser parser = new GenericTokenParser("#{", "}", handler);
    // 解析 #{ } 占位符，并返回解析结果
    String sql = parser.parse(originalSql);
    // 封装解析结果到 StaticSqlSource 中，并返回
    return new StaticSqlSource(
        configuration, sql, handler.getParameterMappings());
}
```

如上，GenericTokenParser 的用途上一节已经介绍过了，就不多说了。接下来，我们重点关注#{ }占位符处理器 ParameterMappingTokenHandler 的逻辑。

```
public String handleToken(String content) {
    // 获取 content 的对应的 ParameterMapping
    parameterMappings.add(buildParameterMapping(content));
    // 返回 ?
    return "?";
}
```


ParameterMappingTokenHandler 的 handleToken 方法看起来比较简单，但实际上并非如此。GenericTokenParser 负责将#{ } 占位符中的内容抽取出来，并将抽取出的内容传给 handleToken 方法。handleToken 方法负责将传入的参数解析成对应的 ParameterMapping 对象，这步操作由 buildParameterMapping 方法完成。下面我们看一下 buildParameterMapping 的源码。

```
private ParameterMapping buildParameterMapping(String content) {  
    /*  
     * 将 #{xxx} 占位符中的内容解析成 Map。大家可能很好奇一个普通的字符串是怎么  
     * 解析成 Map 的，举例说明一下。如下：  
     *  
     *   #{age,javaType=int,jdbcType=NUMERIC,typeHandler=MyTypeHandler}  
     *  
     * 上面占位符中的内容最终会被解析成如下的结果：  
     *  
     * {  
     *   "property": "age",  
     *   "typeHandler": "MyTypeHandler",  
     *   "jdbcType": "NUMERIC",  
     *   "javaType": "int"  
     * }  
     *  
     * parseParameterMapping 内部依赖 ParameterExpression 对字符串进行解析，  
     * ParameterExpression 的逻辑不是很复杂，这里就不分析了。  
     */  
    Map<String, String> propertiesMap = parseParameterMapping(content);  
    String property = propertiesMap.get("property");  
    Class<?> propertyType;  
    // metaParameters 为 DynamicContext 成员变量 bindings 的元信息对象  
    if (metaParameters.hasGetter(property)) {  
        propertyType = metaParameters.getGetterType(property);  
    }  
    /*  
     * parameterType 是运行时参数的类型。如果用户传入的是单个参数，比如 Article
```

```

* 对象, 此时 parameterType 为 Article.class。如果用户传入的多个参数, 比如
* [id = 1, author = "coolblog"], MyBatis 会使用 ParamMap 封装这些参数,
* 此时 parameterType 为 ParamMap.class。如果 parameterType 有相应的
* TypeHandler, 这里则把 parameterType 设为 propertyType
*/
} else if (typeHandlerRegistry.hasTypeHandler(parameterType)) {
    propertyType = parameterType;
} else if (JdbcType.CURSOR.name().equals(propertiesMap.get("jdbcType"))){
    propertyType = java.sql.ResultSet.class;
} else if (property == null || Map.class.isAssignableFrom(parameterType)){
    // 如果 property 为空, 或 parameterType 是 Map 类型,
    // 则将 propertyType 设为 Object.class
    propertyType = Object.class;
} else {
    // 代码逻辑走到此分支中, 表明 parameterType 是一个自定义的类,
    // 比如 Article, 此时为该创建元信息对象
    MetaClass metaClass = MetaClass.forClass(
        parameterType, configuration.getReflectorFactory());
    // 检测参数对象有没有与 property 想对应的 getter 方法
    if (metaClass.hasGetter(property)) {
        // 获取成员变量的类型
        propertyType = metaClass.getGetterType(property);
    } else {
        propertyType = Object.class;
    }
}

// ----- 分割线 -----
ParameterMapping.Builder builder = new ParameterMapping.Builder(
    configuration, property, propertyType);

// 将 propertyType 赋值给 javaType
Class<?> javaType = propertyType;
String typeHandlerAlias = null;

```

```
// 遍历 propertiesMap
for (Map.Entry<String, String> entry : propertiesMap.entrySet()) {
    String name = entry.getKey();
    String value = entry.getValue();
    if ("javaType".equals(name)) {
        // 如果用户明确配置了 javaType, 则以用户的配置为准
        javaType = resolveClass(value);
        builder.javaType(javaType);
    } else if ("jdbcType".equals(name)) {
        // 解析 jdbcType
        builder.jdbcType(resolveJdbcType(value));
    } else if ("mode".equals(name)) {...}
    else if ("numericScale".equals(name)) {...}
    else if ("resultMap".equals(name)) {...}
    else if ("typeHandler".equals(name)) {
        typeHandlerAlias = value;
    }
    else if ("jdbcTypeName".equals(name)) {...}
    else if ("property".equals(name)) {...}
    else if ("expression".equals(name)) {
        throw new BuilderException(".....");
    } else {
        throw new BuilderException(".....");
    }
}

if (typeHandlerAlias != null) {
    // 解析 TypeHandler
    builder.typeHandler(resolveTypeHandler(javaType, typeHandlerAlias));
}

// 构建 ParameterMapping 对象
return builder.build();
}
```

如上，buildParameterMapping 代码很多，逻辑看起来很复杂。但是它做的事情却不是很多，只有 3 件事情。如下：

1. 解析 content
2. 解析 propertyType，对应分割线之上的代码
3. 构建 ParameterMapping 对象，对应分割线之下的代码

buildParameterMapping 代码比较多，不太好理解，下面写个示例演示一下。如下：

```
public class SqlSourceBuilderTest {

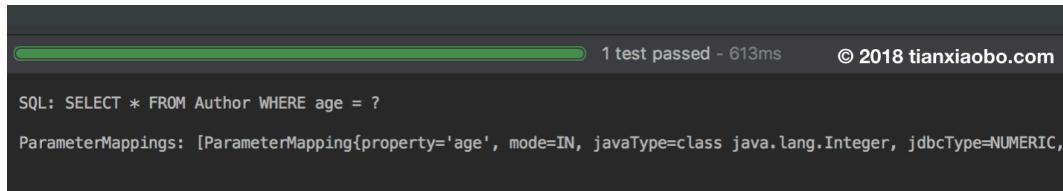
    @Test
    public void test() {
        // 带有复杂 #{ } 占位符的参数，接下来会解析这个占位符
        String sql = "SELECT * FROM Author WHERE age ="
            + "#{age,javaType=int,jdbcType=NUMERIC}";
        SqlSourceBuilder sqlSourceBuilder =
            new SqlSourceBuilder(new Configuration());
        SqlSource sqlSource = sqlSourceBuilder.parse(
            sql, Author.class, new HashMap<>());
        BoundSql boundSql = sqlSource.getBoundSql(new Author());

        System.out.println(String.format("SQL: %s\n", boundSql.getSql()));
        System.out.println(String.format(
            "ParameterMappings: %s", boundSql.getParameterMappings()));
    }
}

public class Author {
    private Integer id;
    private String name;
    private Integer age;

    // 省略 getter/setter
}
```

测试结果如下：



```
1 test passed - 613ms © 2018 tianxiaobo.com
SQL: SELECT * FROM Author WHERE age = ?
ParameterMappings: [ParameterMapping{property='age', mode=IN, javaType=class java.lang.Integer, jdbcType=NUMERIC,
```

正如测试结果所示，SQL 中的#{age,...} 占位符被替换成了问号?。#{age,...} 也被解析成了一个 ParameterMapping 对象。

本节的最后，我们再来看一下 StaticSqlSource 的创建过程。如下：

```
public class StaticSqlSource implements SqlSource {
    private final String sql;
    private final List<ParameterMapping> parameterMappings;
    private final Configuration configuration;

    public StaticSqlSource(Configuration configuration, String sql) {
        this(configuration, sql, null);
    }

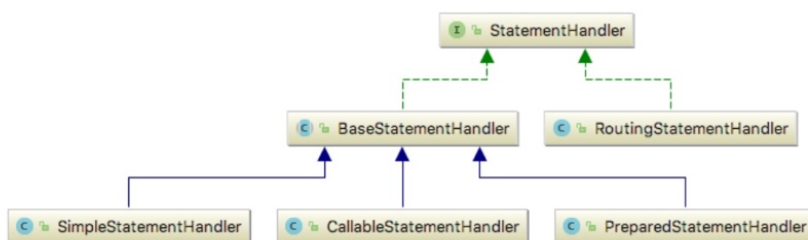
    public StaticSqlSource(Configuration configuration, String sql,
        List<ParameterMapping> parameterMappings) {
        this.sql = sql;
        this.parameterMappings = parameterMappings;
        this.configuration = configuration;
    }

    @Override
    public BoundSql getBoundSql(Object parameterObject) {
        // 创建 BoundSql 对象
        return new BoundSql(
            configuration, sql, parameterMappings, parameterObject);
    }
}
```

上面代码没有什么太复杂的地方，从上面代码中可以看出 BoundSql 的创建过程也很简单。正因为前面经历了这么复杂的解析逻辑，BoundSql 的创建过程才会如此简单。到此，关于 BoundSql 构建的过程就分析完了。

4.2.3 创建 StatementHandler

在 MyBatis 的源码中，StatementHandler 是一个非常核心接口。之所以说它核心，是因为从代码分层的角度来说，StatementHandler 是 MyBatis 源码的边界，再往下层就是 JDBC 层面的接口了。StatementHandler 需要和 JDBC 层面的接口打交道，它要做的事情有很多。在执行 SQL 之前，StatementHandler 需要创建合适的 Statement 对象，然后填充参数值到 Statement 对象中，最后通过 Statement 对象执行 SQL。这还不算完，待 SQL 执行完毕，还要去处理查询结果等。这些过程看似简单，但实现起来却很复杂。好在，这些过程对应的逻辑并不需要我们亲自实现。好了，其他的就不多说了。下面我们来看一下 StatementHandler 的继承体系。



上图中，最下层的三种 StatementHandler 实现类与三种不同的 Statement 进行交互，这个不难看出来。但 RoutingStatementHandler 则是一个奇怪的存在，因为 JDBC 中并不存在 RoutingStatement。那它有什么用呢？接下来，我们到代码中寻找答案。

```
// -☆- Configuration
public StatementHandler newStatementHandler(Executor executor,
    MappedStatement mappedStatement, Object parameterObject,
    RowBounds rowBounds, ResultHandler resultHandler, BoundSql boundSql) {
    // 创建具有路由功能的 StatementHandler
    StatementHandler statementHandler = new RoutingStatementHandler(
        executor, mappedStatement, parameterObject, rowBounds,
        resultHandler, boundSql);
}
```

```
// 应用插件到 StatementHandler 上
statementHandler =
    (StatementHandler) interceptorChain.pluginAll(statementHandler);
return statementHandler;
}
```

如上，newStatementHandler 方法在创建 StatementHandler 之后，还会应用插件到 StatementHandler 上。关于 MyBatis 的插件机制，后面独立成章进行讲解，这里就不分析了。下面分析 RoutingStatementHandler 的代码。

```
public class RoutingStatementHandler implements StatementHandler {

    private final StatementHandler delegate;

    public RoutingStatementHandler(Executor executor, MappedStatement ms,
        Object parameter, RowBounds rowBounds, ResultHandler resultHandler,
        BoundSql boundSql) {

        // 根据 StatementType 创建不同的 StatementHandler
        switch (ms.getStatementType()) {
            case STATEMENT:
                delegate = new SimpleStatementHandler(executor, ms,
                    parameter, rowBounds, resultHandler, boundSql);
                break;
            case PREPARED:
                delegate = new PreparedStatementHandler(executor, ms,
                    parameter, rowBounds, resultHandler, boundSql);
                break;
            case CALLABLE:
                delegate = new CallableStatementHandler(executor,
                    ms, parameter, rowBounds, resultHandler, boundSql);
                break;
        }
    }
}
```

```

        default:
            throw new ExecutorException(".....");
    }
}

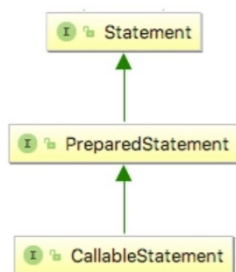
// 其他方法逻辑均由别的 StatementHandler 代理完成, 就不贴代码了
}

```

RoutingStatementHandler 的构造方法会根据 MappedStatement 中的 statementType 变量创建不同的 StatementHandler 实现类。默认情况下, statementType 值为 PREPARED。关于 StatementHandler 创建的过程就先分析到这, StatementHandler 创建完成了, 后续要做到事情是创建 Statement, 以及将运行时参数和 Statement 进行绑定。

4.2.4 设置运行时参数到 SQL 中

JDBC 提供了三种 Statement 接口, 分别是 Statement、PreparedStatement 和 CallableStatement。他们的关系如下:



上面三个接口的层级分明, 其中 Statement 接口提供了执行 SQL, 获取执行结果等基本功能。PreparedStatement 在此基础上, 对 IN 类型的参数提供了支持。使得我们可以使用运行时参数替换 SQL 中的问号?占位符, 而不用手动拼接 SQL。CallableStatement 则是在 PreparedStatement 基础上, 对 OUT 类型的参数提供了支持, 该种类型的参数用于保存存储过程输出的结果。本节将分析 PreparedStatement 的创建, 以及设置运行时参数到 SQL 中的过程。其他两种 Statement 的处理过程, 大家请自行分析。Statement 的创建入口是在 SimpleExecutor 的 prepareStatement 方法中, 下面从这个方法开始进行分析。


```
// -☆- SimpleExecutor
private Statement prepareStatement(StatementHandler handler, Log
statementLog) throws SQLException {
    Statement stmt;
    // 获取数据库连接
    Connection connection = getConnection(statementLog);
    // 创建 Statement,
    stmt = handler.prepare(connection, transaction.getTimeout());
    // 为 Statement 设置 IN 参数
    handler.parameterize(stmt);
    return stmt;
}
```

上面代码的逻辑比较简单，总共包含三个步骤。如下：

1. 获取数据库连接
2. 创建 Statement
3. 为 Statement 设置 IN 参数

上面三个步骤看起来并不难实现，实际上如果大家愿意写的话，也能写出来。不过 MyBatis 对这三个步骤进行了一些拓展，实现上也相对复杂一些。以获取数据库连接为例，MyBatis 并未没有在 getConnection 方法中直接调用 JDBC DriverManager 的 getConnection 方法获取获取连接，而是通过数据源获取连接。MyBatis 提供了两种基于 JDBC 接口的数据源，分别为 PooledDataSource 和 UnpooledDataSource。创建或获取数据库连接的操作最终是由这两个数据源执行。本节不会分析以上两种数据源的源码，相关分析会在下一章中展开。

接下来，我将分析 PreparedStatement 的创建，以及 IN 参数设置的过程。按照顺序，先来分析 PreparedStatement 的创建过程。如下：

```
// -☆- PreparedStatementHandler
public Statement prepare(Connection connection, Integer transactionTimeout)
throws SQLException {
    Statement statement = null;
    try {
        // 创建 Statement
    }
}
```

```

        statement = instantiateStatement(connection);
        // 设置超时和 FetchSize
        setStatementTimeout(statement, transactionTimeout);
        setFetchSize(statement);
        return statement;
    } catch (SQLException e) {
        closeStatement(statement);
        throw e;
    } catch (Exception e) {
        closeStatement(statement);
        throw new ExecutorException(".....");
    }
}

protected Statement instantiateStatement(Connection connection)
    throws SQLException {
    String sql = boundSql.getSql();
    // 根据条件调用不同的 prepareStatement 方法创建 PreparedStatement
    if (mappedStatement.getKeyGenerator() instanceof Jdbc3KeyGenerator) {
        String[] keyColumnNames = mappedStatement.getKeyColumns();
        if (keyColumnNames == null) {
            return connection.prepareStatement(
                sql, PreparedStatement.RETURN_GENERATED_KEYS);
        } else {
            return connection.prepareStatement(sql, keyColumnNames);
        }
    } else if (mappedStatement.getResultSetType() != null) {
        return connection.prepareStatement(sql,
            mappedStatement.getResultSetType().getValue(),
            ResultSet.CONCUR_READ_ONLY);
    } else {
        return connection.prepareStatement(sql);
    }
}
}

```

PreparedStatement 的创建过程没什么复杂的地方，就不多说了。下面分析运行时参数是如何被设置到 SQL 中的过程。

```
// -☆- PreparedStatementHandler
public void parameterize(Statement statement) throws SQLException {
    // 通过参数处理器 ParameterHandler 设置运行时参数到 PreparedStatement 中
    parameterHandler.setParameters((PreparedStatement) statement);
}

public class DefaultParameterHandler implements ParameterHandler {
    private final TypeHandlerRegistry typeHandlerRegistry;
    private final MappedStatement mappedStatement;
    private final Object parameterObject;
    private final BoundSql boundSql;
    private final Configuration configuration;

    public void setParameters(PreparedStatement ps) {
        // 从 BoundSql 中获取 ParameterMapping 列表, 每个 ParameterMapping
        // 与原始 SQL 中的 #{xxx} 占位符一一对应
        List<ParameterMapping> parameterMappings =
            boundSql.getParameterMappings();
        if (parameterMappings != null) {
            for (int i = 0; i < parameterMappings.size(); i++) {
                ParameterMapping parameterMapping=parameterMappings.get(i);
                // 检测参数类型, 排除掉 mode 为 OUT 类型的 parameterMapping
                if (parameterMapping.getMode() != ParameterMode.OUT) {
                    Object value;
                    // 获取属性名
                    String propertyName = parameterMapping.getProperty();
                    // 检测 BoundSql 的 additionalParameters 是否包含 propertyName
                    if (boundSql.hasAdditionalParameter(propertyName)) {
                        value=boundSql.getAdditionalParameter(propertyName);
                    } else if (parameterObject == null) {
                        value = null;
                    }
                }
            }
        }
    }
}
```


如上代码，分割线以上的大段代码用于获取#{xxx}占位符属性所对应的运行时参数。分割线以下的代码则是获取#{xxx}占位符属性对应的 TypeHandler，并在最后通过 TypeHandler 将运行时参数值设置到 PreparedStatement 中。

4.2.5 #{ } 占位符的解析与参数的设置过程梳理

前面两节的内容比较多，本节将对前两节的部分内容进行梳理，以便大家能够更好理解这两节内容之间的联系。假设我们有这样一条 SQL 语句：

```
SELECT * FROM author WHERE name = #{name} AND age = #{age}
```

这个 SQL 语句中包含两个#{ } 占位符，在运行时这两个占位符会被解析成两个 ParameterMapping 对象。如下：

```
ParameterMapping{property='name', mode=IN,  
    javaType=class java.lang.String, jdbcType=null, ...}
```

和

```
ParameterMapping{property='age', mode=IN,  
    javaType=class java.lang.Integer, jdbcType=null, ...}
```

#{xxx} 占位符解析完毕后，得到的 SQL 如下：

```
SELECT * FROM Author WHERE name = ? AND age = ?
```

这里假设下面这个方法与上面的 SQL 对应：

```
Author findByNameAndAge(@Param("name")String name, @Param("age")Integer  
age)
```

该方法的参数列表会被 ParamNameResolver 解析成一个 map，如下：

```
{  
    0: "name",  
    1: "age"  
}
```

假设该方法在运行时有如下的调用：

```
findByNameAndAge("tianxiaobo", 20)
```

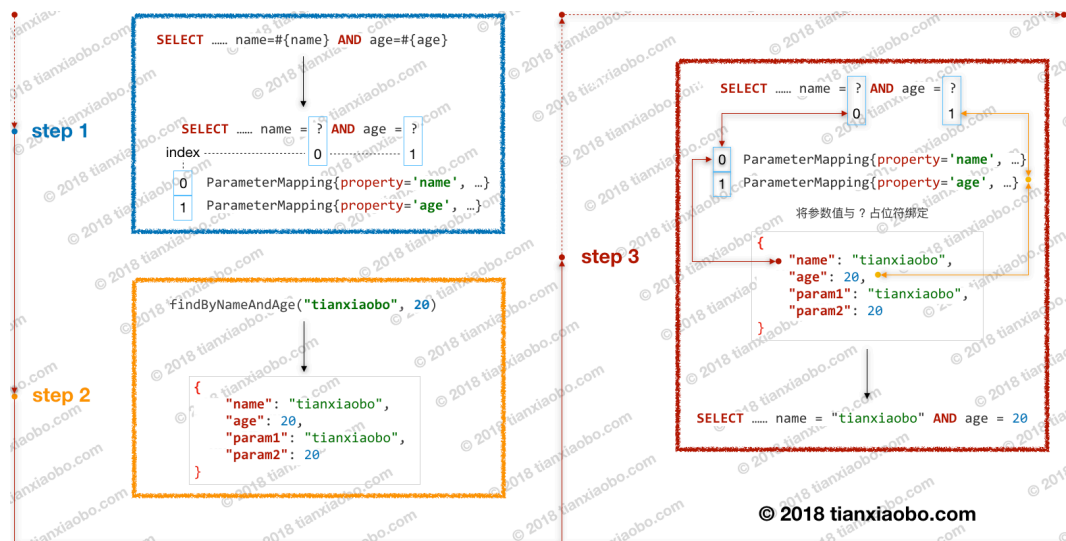
此时，需要再次借助 ParamNameResolver 的力量。这次我们将参数名和运行时的参数值绑定起来，得到如下的映射关系。

```
{  
    "name": "tianxiaobo",  
    "age": 20,  
    "param1": "tianxiaobo",  
    "param2": 20  
}
```

下一步，我们要将运行时参数设置到 SQL 中。由于原 SQL 经过解析后，占位符信息已经被擦除掉了，我们无法直接将运行时参数 SQL 中。不过好在，这些占位符信息被记录在了 ParameterMapping 中了，MyBatis 会将 ParameterMapping 会按照#{}占位符的解析顺序存入到 List 中。这样我们通过 ParameterMapping 在列表中的位置确定它与 SQL 中的哪一个占位符相关联。同时通过 ParameterMapping 中的 property 字段，我们可以到“参数名与参数值”映射表中查找具体的参数值。这样，我们就可以将参数值准确的设置到 SQL 中了，此时 SQL 如下：

```
SELECT * FROM Author WHERE name = "tianxiaobo" AND age = 20
```

整个流程如下图所示。



当运行时参数被设置到 SQL 中后，下一步要做的事情是执行 SQL，然后处理 SQL 执行结果。对于更新操作，数据库一般返回一个 int 行数值，表示受影响行数，这个处理起来比较简单。但对于查询操作，返回的结果类型多变，处理方式也很复杂。接下来，我们就来看看 MyBatis 是如何处理查询结果的。

4.2.6 处理查询结果

MyBatis 可以将查询结果，即结果集 `ResultSet` 自动映射成实体类对象。这样使用者就无需再手动操作结果集，并将数据填充到实体类对象中。这可大大降低开发的工作量，提高工作效率。在 MyBatis 中，结果集的处理工作由结果集处理器 `ResultSetHandler` 执行。`ResultSetHandler` 是一个接口，它只有一个实现类 `DefaultResultSetHandler`。结果集的处理入口方法是 `handleResultSets`，下面来看一下该方法的实现。

```
public List<Object> handleResultSets(Statement stmt) throws SQLException {

    final List<Object> multipleResults = new ArrayList<Object>();

    int resultSetCount = 0;
    // 获取第一个结果集
```

```

ResultSetWrapper rsw = getFirstResultSet(stmt);

List<ResultMap> resultMaps = mappedStatement.getResultMaps();
int resultMapCount = resultMaps.size();
validateResultMapsCount(rsw, resultMapCount);

while (rsw != null && resultMapCount > resultSetCount) {
    ResultMap resultMap = resultMaps.get(resultSetCount);
    // 处理结果集
    handleResultSet(rsw, resultMap, multipleResults, null);
    // 获取下一个结果集
    rsw = getNextResultSet(stmt);
    cleanUpAfterHandlingResultSet();
    resultSetCount++;
}

// 以下逻辑均与多结果集有关，就不分析了，代码省略
String[] resultSets = mappedStatement.getResultSets();
if (resultSets != null) {...}

return collapseSingleResultList(multipleResults);
}

private ResultSetWrapper getFirstResultSet(Statement stmt)
throws SQLException {
    // 获取结果集
    ResultSet rs = stmt.getResultSet();
    while (rs == null) {
        /*
         * 移动 ResultSet 指针到下一个上，有些数据库驱动可能需要使用者
         * 先调用 getMoreResults 方法，然后才能调用 getResultSet 方法
         * 获取到第一个 ResultSet
         */
    }
}

```



```
        if (stmt.getMoreResults()) {
            rs = stmt.getResultSet();
        } else {
            if (stmt.getUpdateCount() == -1) {
                break;
            }
        }
    }
}
/*
 * 这里并不直接返回 ResultSet，而是将其封装到 ResultSetWrapper 中。
 * ResultSetWrapper 中包含了 ResultSet 一些元信息，比如列名称、
 * 每列对应的 JdbcType、以及每列对应的 Java 类名（class name，譬如
 * java.lang.String）等。
 */
return rs != null ? new ResultSetWrapper(rs, configuration) : null;
}
```

如上，该方法首先从 Statement 中获取第一个结果集，然后调用 `handleResultSet` 方法对该结果集进行处理。一般情况下，如果我们不调用存储过程，不会涉及到多结果集的问题。由于存储过程并不是很常用，所以关于多结果集的处理逻辑我就不分析了。下面，我们把目光聚焦在单结果集的处理逻辑上。

```
private void handleResultSet(ResultSetWrapper rsw, ResultMap resultMap,
    List<Object> multipleResults, ResultMapping parentMapping)
    throws SQLException {
    try {
        if (parentMapping != null) {
            // 多结果集相关逻辑，不分析了
            handleRowValues(rsw, resultMap,
                null, RowBounds.DEFAULT, parentMapping);
        } else {
            /*
```

```

    * 检测 resultHandler 是否为空。ResultHandler 是一个接口，使用者可
    * 实现该接口，这样我们可以通过 ResultHandler 自定义接收查询结果的
    * 动作。比如我们可将结果存储到 List、Map 亦或是 Set，甚至丢弃，
    * 这完全取决于大家的实现逻辑。
    */
    if (resultHandler == null) {
        // 创建默认的结果处理器
        DefaultResultHandler defaultResultHandler =
            new DefaultResultHandler(objectFactory);
        // 处理结果集的行数据
        handleRowValues(rsw, resultMap,
            defaultResultHandler, rowBounds, null);
        multipleResults.add(defaultResultHandler.getResultList());
    } else {
        // 处理结果集的行数据
        handleRowValues(rsw, resultMap, resultHandler, rowBounds, null);
    }
}
} finally {
    closeResultSet(rsw.getResultSet());
}
}
}

```

在上面代码中，出镜率最高的 handleRowValues 方法，该方法用于处理结果集中的数据。下面来看一下这个方法的逻辑。

```

public void handleRowValues(ResultSetWrapper rsw, ResultMap resultMap,
    ResultHandler<?> resultHandler, RowBounds rowBounds,
    ResultMapping parentMapping) throws SQLException {
    if (resultMap.hasNestedResultMaps()) {
        ensureNoRowBounds();
        checkResultHandler();
        // 处理嵌套映射，关于嵌套映射本文就不分析了
    }
}

```

```
        handleRowValuesForNestedResultMap(rsw,
            resultMap, resultHandler, rowBounds, parentMapping);
    } else {
        // 处理简单映射
        handleRowValuesForSimpleResultMap(rsw,
            resultMap, resultHandler, rowBounds, parentMapping);
    }
}
```

handleRowValues 方法中针对两种映射方式进行了处理。一种是嵌套映射，另一种是简单映射。本文所说的嵌套查询是指<ResultMap>中嵌套了一个<ResultMap>，关于此种映射的处理方式本节就不进行分析了。下面我将详细分析简单映射的处理逻辑，如下：

```
private void handleRowValuesForSimpleResultMap(ResultSetWrapper rsw,
    ResultMap resultMap, ResultHandler<?> resultHandler, RowBounds rowBounds,
    ResultMapping parentMapping) throws SQLException {
    DefaultResultContext<Object> resultContext =
        new DefaultResultContext<Object>();
    // 根据 RowBounds 定位到指定行记录
    skipRows(rsw.getResultSet(), rowBounds);
    // 检测是否还有更多行的数据需要处理
    while (shouldProcessMoreRows(resultContext, rowBounds) &&
        rsw.getResultSet().next()) {
        // 获取经过鉴别器处理后的 ResultMap
        ResultMap discriminatedResultMap = resolveDiscriminatedResultMap(
            rsw.getResultSet(), resultMap, null);
        // 从 resultSet 中获取结果
        Object rowValue = getRowValue(rsw, discriminatedResultMap);
        // 存储结果
        storeObject(resultHandler, resultContext,
            rowValue, parentMapping, rsw.getResultSet());
    }
}
```

上面方法的逻辑较多，这里简单总结一下。如下：

1. 根据 RowBounds 定位到指定行记录
2. 循环处理多行数据
3. 使用鉴别器处理 ResultMap
4. 映射 ResultSet，得到映射结果 rowValue
5. 存储结果

在如上几个步骤中，鉴别器相关的逻辑就不分析了，不是很常用。第 2 步的检测逻辑比较简单，也忽略了。下面分析第一个步骤对应的代码逻辑。如下：

```
private void skipRows(ResultSet rs, RowBounds rowBounds)
    throws SQLException {
    // 检测 rs 的类型，不同的类型行数据定位方式是不同的
    if (rs.getType() != ResultSet.TYPE_FORWARD_ONLY) {
        if (rowBounds.getOffset() != RowBounds.NO_ROW_OFFSET) {
            // 直接定位到 rowBounds.getOffset() 位置处
            rs.absolute(rowBounds.getOffset());
        }
    } else {
        for (int i = 0; i < rowBounds.getOffset(); i++) {
            /*
             * 通过多次调用 rs.next() 方法实现行数据定位。
             * 当 Offset 数值很大时，这种效率很低下
             */
            rs.next();
        }
    }
}
```

MyBatis 默认提供了 RowBounds 用于分页，从上面的代码中可以看出，这并非是一个高效的分页方式。除了使用 RowBounds，还可以使用一些第三方分页插件进行分页。关于第三方的分页插件，大家请自行查阅资料，这里就不展开说明了。下面分析一下 ResultSet 的映射过程，如下：

```
private Object getRowValue(ResultSetWrapper rsw, ResultMap resultMap)
    throws SQLException {
    final ResultLoaderMap lazyLoader = new ResultLoaderMap();
    // 创建实体类对象, 比如 Article 对象
    Object rowValue = createResultObject(rsw, resultMap, lazyLoader, null);
    if (rowValue != null &&
        !hasTypeHandlerForResultObject(rsw, resultMap.getType())) {
        final MetaObject metaObject = configuration.newMetaObject(rowValue);
        boolean foundValues = this.useConstructorMappings();

        // 检测是否应该自动映射结果集
        if (shouldApplyAutomaticMappings(resultMap, false)) {
            // 进行自动映射
            foundValues = applyAutomaticMappings(
                rsw, resultMap, metaObject, null) || foundValues;
        }
        // 根据 <resultMap> 节点中配置的映射关系进行映射
        foundValues = applyPropertyMappings(
            rsw, resultMap, metaObject, lazyLoader, null) || foundValues;
        foundValues = lazyLoader.size() > 0 || foundValues;
        rowValue = foundValues || configuration.isReturnInstanceForEmptyRow()
            ? rowValue : null;
    }
    return rowValue;
}
```

上面的方法中的重要逻辑已经注释出来了, 这里再简单总结一下。如下:

1. 创建实体类对象
2. 检测结果集是否需要自动映射, 若需要则进行自动映射
3. 按<resultMap>中配置的映射关系进行映射

这三处代码的逻辑比较复杂, 接下来按顺序进行分节说明。首先分析实体类的创建过程。

1. 创建实体类对象

在我们的印象里，创建实体类对象是一个很简单过程。直接通过 new 关键字，或通过反射即可完成。大家可能会想，把这么简单过程也拿出来说说，怕是有凑字数的嫌疑。实则不然，MyBatis 的维护者写了不少逻辑，以保证能成功创建实体类对象。如果实在无法创建，则抛出异常。下面我们来看一下 MyBatis 创建实体类对象的过程。

```
// -☆- DefaultResultSetHandler
private Object createResultObject(ResultSetWrapper rsw,
    ResultMap resultMap, ResultLoaderMap lazyLoader, String columnPrefix)
    throws SQLException {

    this.useConstructorMappings = false;
    final List<Class<?>> constructorArgTypes = new ArrayList<Class<?>>();
    final List<Object> constructorArgs = new ArrayList<Object>();

    // 调用重载方法创建实体类对象
    Object resultObject = createResultObject(rsw,
        resultMap, constructorArgTypes, constructorArgs, columnPrefix);
    // 检测实体类是否有相应的类型处理器
    if (resultObject != null &&
        !hasTypeHandlerForResultObject(rsw, resultMap.getType())) {
        final List<ResultMapping> propertyMappings =
            resultMap.getPropertyResultMappings();
        for (ResultMapping propertyMapping : propertyMappings) {
            // 如果开启了延迟加载，则为 resultObject 生成代理类
            if (propertyMapping.getNestedQueryId() != null &&
                propertyMapping.isLazy()) {
                // 创建代理类，默认使用 Javassist 框架生成代理类。由于实体类通常
                // 不会实现接口，所以不能使用 JDK 动态代理 API 为实体类生成代理。
                resultObject = configuration.getProxyFactory()
                    .createProxy(resultObject, lazyLoader, configuration,
                        objectFactory, constructorArgTypes, constructorArgs);
            }
        }
    }
}
```

```
                break;
            }
        }
    }
    this.useConstructorMappings =
        resultObject != null && !constructorArgTypes.isEmpty();
    return resultObject;
}
```

创建实体类对象的逻辑被封装在了 `createResultObject` 的重载方法中，关于该方法，待会再分析。创建好实体类对后，还需要对<resultMap>中配置的映射信息进行检测。若发现有关联查询，且关联查询结果的加载方式为延迟加载，此时需为实体类生成代理类。举个例子说明一下，假设有如下两个实体类：

```
/** 作者类 */
public class Author {
    private Integer id;
    private String name;
    private Integer age;
    private Integer sex;
}

/** 文章类 */
public class Article {
    private Integer id;
    private String title;
    // 一对一关系
    private Author author;
    private String content;
}
```

如上，Article 对象中的数据由一条 SQL 从 article 表中查询。Article 类有一个 author 字段，该字段的数据由另一条 SQL 从 author 表中查出。我们在将 article 表的查询结果填充到 Article 类对象中时，并不希望 MyBaits 立即执行另一条 SQL 查询 author 字段对应的数据。而是期望在我们调用 article.getAuthor()方法时，MyBaits 再执行另一条 SQL 从 author 表中查询出所需的数据。若如此，我们需要改造 getAuthor 方法，以保证调用该方法时可让 MyBaits 执行相关的 SQL。关于延迟加载后面将会进行详细的分析，这里先说这么多。下面分析 createResultObject 重载方法的逻辑，如下：

```
private Object createResultObject(ResultSetWrapper rsw, ResultMap
resultMap, List<Class<?>> constructorArgTypes, List<Object>
constructorArgs, String columnPrefix) throws SQLException {

    final Class<?> resultType = resultMap.getType();
    final MetaClass metaType =
        MetaClass.forClass(resultType, reflectorFactory);
    // 获取 <constructor> 节点对应的 ResultMapping
    final List<ResultMapping> constructorMappings =
        resultMap.getConstructorResultMappings();

    // 检测是否有与返回值类型相对应的 TypeHandler，若有则直接从
    // 通过 TypeHandler 从结果集中提取数据，并生成返回值对象
    if (hasTypeHandlerForResultObject(rsw, resultType)) {
        // 通过 TypeHandler 获取提取，并生成返回值对象
        return createPrimitiveResultObject(rsw, resultMap, columnPrefix);
    } else if (!constructorMappings.isEmpty()) {
        // 通过 <constructor> 节点配置的映射信息从 ResultSet 中提取数据，
        // 然后将这些数据传给指定构造方法，即可创建实体类对象
        return createParameterizedResultObject(rsw, resultType,
            constructorMappings, constructorArgTypes,
            constructorArgs, columnPrefix);
    } else if (resultType.isInterface() || metaType.hasDefaultConstructor()){
        // 通过 ObjectFactory 调用目标类的默认构造方法创建实例
        return objectFactory.create(resultType);
    } else if (shouldApplyAutomaticMappings(resultMap, false)) {
```



```
// 通过自动映射查找合适的构造方法创建实例
return createByConstructorSignature(rsw, resultType,
    constructorArgTypes, constructorArgs, columnPrefix);
}
throw new ExecutorException(".....");
}
```

createResultObject 方法中包含了 4 种创建实体类对象的方式。一般情况下，若无特殊要求，MyBatis 会通过 ObjectFactory 调用默认构造方法创建实体类对象。ObjectFactory 是一个接口，大家可以实现这个接口，以按照自己的逻辑控制对象的创建过程。至此，实体类对象创建好了，接下来要做的事情是将结果集中的数据映射到实体类对象中。

2. 结果集映射

在 MyBatis 中，结果集自动映射有三种等级。这三种等级官方文档上有所说明，这里直接引用一下。如下：

- NONE - 禁用自动映射。仅设置手动映射属性
- PARTIAL - 将自动映射结果除了那些有内部定义内嵌结果映射的(joins)
- FULL - 自动映射所有

除了以上三种等级，我们还可以显示配置<resultMap>节点的 autoMapping 属性，以启用或者禁用指定 ResultMap 的自动映射设定。下面，来看一下自动映射相关的逻辑。

```
private boolean shouldApplyAutomaticMappings(
    ResultMap resultMap, boolean isNested) {
    // 检测 <resultMap> 是否配置了 autoMapping 属性
    if (resultMap.getAutoMapping() != null) {
        // 返回 autoMapping 属性
        return resultMap.getAutoMapping();
    } else {
        if (isNested) {
            // 对于嵌套 resultMap，仅当全局的映射行为为 FULL 时，才进行自动映射
            return AutoMappingBehavior.FULL ==
                configuration.getAutoMappingBehavior();
        }
    }
}
```

```

    } else {
        // 对于普通的 resultMap，只要全局的映射行为不为 NONE，即可进行自动映射
        return AutoMappingBehavior.NONE !=
            configuration.getAutoMappingBehavior();
    }
}
}
}

```

shouldApplyAutomaticMappings 方法用于检测是否应为当前结果集应用自动映射。检测结果取决于<resultMap>节点的 autoMapping 属性，以及全局自动映射行为。上面代码的逻辑不难理解，就不多说了。下面来分析 MyBatis 是如何进行自动映射的。

```

private boolean applyAutomaticMappings(ResultSetWrapper rsw, ResultMap
    resultMap, MetaObject metaObject, String columnPrefix)
    throws SQLException {

    // 获取 UnMappedColumnAutoMapping 列表
    List<UnMappedColumnAutoMapping> autoMapping = createAutomaticMappings(
        rsw, resultMap, metaObject, columnPrefix);
    boolean foundValues = false;

    if (!autoMapping.isEmpty()) {
        for (UnMappedColumnAutoMapping mapping : autoMapping) {
            // 通过 TypeHandler 从结果集中获取指定列的数据
            final Object value = mapping.typeHandler
                .getResult(rsw.getResultSet(), mapping.column);
            if (value != null) {
                foundValues = true;
            }
            if (value != null || (configuration.isCallSettersOnNulls() &&
                !mapping.primitive)) {
                // 通过元信息对象设置 value 到实体类对象的指定字段上
                metaObject.setValue(mapping.property, value);
            }
        }
    }
}

```

```
        }  
    }  
}  
return foundValues;  
}
```

applyAutomaticMappings 方法的代码不多，逻辑也不是很复杂。首先是获取 UnMappedColumnAutoMapping 集合，然后遍历该集合，并通过 TypeHandler 从结果集中获取数据，最后再将获取到的数据设置到实体类对象中。虽然逻辑上看起来没什么复杂的东西，但如果不清楚 UnMappedColumnAutoMapping 的用途，是无法理解上面代码的逻辑的。所以这里简单介绍一下 UnMappedColumnAutoMapping 的用途。UnMappedColumnAutoMapping 用于记录未配置在<resultMap>节点中的映射关系。该类定义在 DefaultResultSetHandler 内部，它的代码如下：

```
private static class UnMappedColumnAutoMapping {  
  
    private final String column;  
    private final String property;  
    private final TypeHandler<?> typeHandler;  
    private final boolean primitive;  
  
    public UnMappedColumnAutoMapping(String column, String property,  
        TypeHandler<?> typeHandler, boolean primitive) {  
        this.column = column;  
        this.property = property;  
        this.typeHandler = typeHandler;  
        this.primitive = primitive;  
    }  
}
```

以上就是 UnMappedColumnAutoMapping 类的所有代码，没什么逻辑，仅用于记录映射关系。下面看一下获取 UnMappedColumnAutoMapping 集合的过程。

```
// -☆- DefaultResultSetHandler
private List<UnMappedColumnAutoMapping> createAutomaticMappings(
    ResultSetWrapper rsw, ResultMap resultMap, MetaObject metaObject,
    String columnPrefix) throws SQLException {
    final String mapKey = resultMap.getId() + ":" + columnPrefix;
    // 从缓存中获取 UnMappedColumnAutoMapping 列表
    List<UnMappedColumnAutoMapping> autoMapping =
        autoMappingsCache.get(mapKey);
    // 缓存未命中
    if (autoMapping == null) {
        autoMapping = new ArrayList<UnMappedColumnAutoMapping>();
        // 从 ResultSetWrapper 中获取未配置在 <resultMap> 中的列名
        final List<String> unmappedColumnNames =
            rsw.getUnmappedColumnNames(resultMap, columnPrefix);
        for (String columnName : unmappedColumnNames) {
            String propertyName = columnName;
            if (columnPrefix != null && !columnPrefix.isEmpty()) {
                if (columnName.toUpperCase(Locale.ENGLISH)
                    .startsWith(columnPrefix)) {
                    // 获取不包含列名前缀的属性名
                    propertyName =
                        columnName.substring(columnPrefix.length());
                } else {
                    continue;
                }
            }
            // 将下划线形式的列名转成驼峰式, 比如 AUTHOR_NAME -> authorName
            final String property = metaObject.findProperty(
                propertyName, configuration.isMapUnderscoreToCamelCase());
            if (property != null && metaObject.hasSetter(property)) {
                // 检测当前属性是否存在于 resultMap 中
                if (resultMap.getMappedProperties().contains(property)) {
                    continue;
                }
            }
        }
    }
}
```

```
// 获取属性对于的类型
final Class<?> propertyType =
    metaObject.getSetterType(property);
if (typeHandlerRegistry.hasTypeHandler(
    propertyType, rsw.getJdbcType(columnName))) {
    // 获取类型处理器
    final TypeHandler<?> typeHandler =
        rsw.getTypeHandler(propertyType, columnName);
    // 封装上面获取到的信息到 UnMappedColumnAutoMapping 对象中
    autoMapping.add(new UnMappedColumnAutoMapping(
        columnName, property, typeHandler,
        propertyType.isPrimitive()));
} else {
    configuration.getAutoMappingUnknownColumnBehavior()
        .doAction(mappedStatement,
            columnName, property, propertyType);
}
} else {
    // 若 property 为空，或实体类中无 property 属性，此时无法完成
    // 列名与实体类属性建立映射关系。针对这种情况，有三种处理方式，
    // 1. 什么都不做
    // 2. 仅打印日志
    // 3. 抛出异常
    // 默认情况下，是什么都不做
    configuration.getAutoMappingUnknownColumnBehavior()
        .doAction(mappedStatement, columnName,
            (property != null) ? property : propertyName, null);
}
}
// 写入缓存
autoMappingsCache.put(mapKey, autoMapping);
}
return autoMapping;
}
```

上面的代码有点多，不过不用太担心，耐心看一下，还是可以看懂的。下面总结一下这个方法的逻辑。

1. 从 `ResultSetWrapper` 中获取未配置在 `<resultMap>` 中的列名
2. 遍历上一步获取到的列名列表
3. 若列名包含列名前缀，则移除列名前缀，得到属性名
4. 将下划线形式的列名转成驼峰式
5. 获取属性类型
6. 获取类型处理器
7. 创建 `UnMappedColumnAutoMapping` 实例

以上步骤中，除了第一步，其他都是常规操作，无需过多说明。下面来分析第一个步骤的逻辑，如下：

```
// ☆- ResultSetWrapper
public List<String> getUnmappedColumnNames(ResultMap resultMap,
    String columnPrefix) throws SQLException {

    List<String> unmappedColumnNames = unmappedColumnNamesMap.get(
        getMapKey(resultMap, columnPrefix));
    if (unmappedColumnNames == null) {
        // 加载已映射与未映射列名
        loadMappedAndUnmappedColumnNames(resultMap, columnPrefix);
        // 获取未映射列名
        unmappedColumnNames = unmappedColumnNamesMap.get(
            getMapKey(resultMap, columnPrefix));
    }
    return unmappedColumnNames;
}

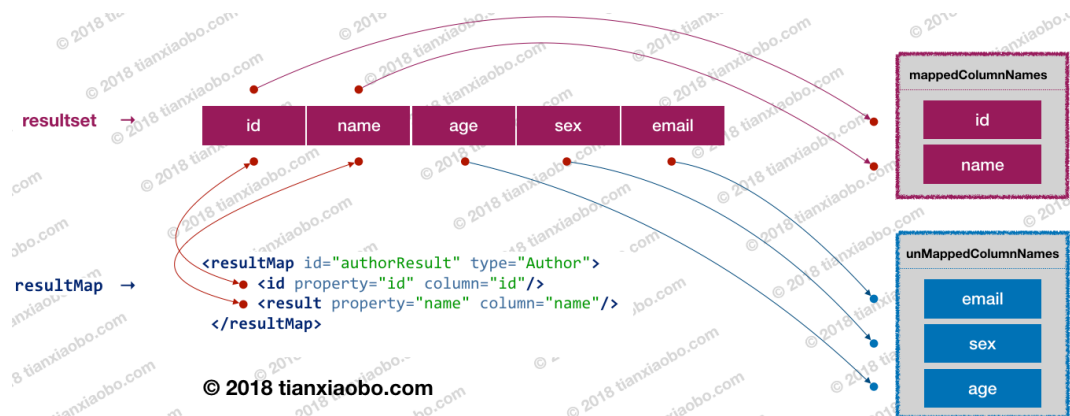
private void loadMappedAndUnmappedColumnNames(ResultMap resultMap,
    String columnPrefix) throws SQLException {
    List<String> mappedColumnNames = new ArrayList<String>();
    List<String> unmappedColumnNames = new ArrayList<String>();
}
```

```
final String upperColumnPrefix = columnPrefix == null ?
    null : columnPrefix.toUpperCase(Locale.ENGLISH);
// 为 <resultMap> 中的列名拼接前缀
final Set<String> mappedColumns = prependPrefixes(
    resultMap.getMappedColumns(), upperColumnPrefix);

// 遍历 columnNames, columnNames 是 ResultSetWrapper 的成员变量,
// 保存了当前结果集中的所有列名
for (String columnName : columnNames) {
    final String upperColumnName =
        columnName.toUpperCase(Locale.ENGLISH);
    // 检测已映射列名集合中是否包含当前列名
    if (mappedColumns.contains(upperColumnName)) {
        mappedColumnNames.add(upperColumnName);
    } else {
        // 将列名存入 unmappedColumnNames 中
        unmappedColumnNames.add(columnName);
    }
}

// 缓存列名集合
mappedColumnNamesMap.put(
    getMapKey(resultMap, columnPrefix), mappedColumnNames);
unMappedColumnNamesMap.put(
    getMapKey(resultMap, columnPrefix), unmappedColumnNames);
}
```

如上，已映射列名与未映射列名的分拣逻辑并不复杂，这里简述一下相关逻辑。首先是从当前数据集中获取列名集合，然后获取<resultMap>中配置的列名集合。之后遍历数据集中的列名集合，并判断列名是否被配置在了<resultMap>节点中。若配置了，则表明该列名已有映射关系，此时该列名存入 mappedColumnNames 中。若未配置，则表明列名未与实体类的某个字段形成映射关系，此时该列名存入 unmappedColumnNames 中。这样，列名的分拣工作就完成了。分拣过程示意图如下：



如上图所示,实体类 `Author` 的 `id` 和 `name` 字段与列名 `id` 和 `name` 被配置在了 `<resultMap>` 中,它们之间形成了映射关系。列名 `age`、`sex` 和 `email` 未配置在 `<resultMap>` 中,因此未与 `Author` 中的字段形成映射,所以他们最终都被放入了 `unMappedColumnNames` 集合中。弄懂了未映射列名获取的过程,自动映射的代码逻辑就不难懂了。好了,关于自动映射的分析就先到这,接下来分析一下 MyBatis 是如何将结果集中的数据填充到已映射的实体类字段中的。

```
// -☆- DefaultResultSetHandler
private boolean applyPropertyMappings(ResultSetWrapper rsw, ResultMap
resultMap, MetaObject metaObject, ResultLoaderMap lazyLoader, String
columnPrefix) throws SQLException {

    // 获取已映射的列名
    final List<String> mappedColumnNames =
        rsw.getMappedColumnNames(resultMap, columnPrefix);
    boolean foundValues = false;
    // 获取 ResultMapping
    final List<ResultMapping> propertyMappings =
        resultMap.getPropertyResultMappings();
    for (ResultMapping propertyMapping : propertyMappings) {
        // 拼接列名前缀,得到完整列名
        String column = prependPrefix(
            propertyMapping.getColumn(), columnPrefix);
```



```
if (propertyMapping.getNestedResultMapId() != null) {
    column = null;
}
/*
 * 下面的 if 分支由三个或条件组合而成，三个条件的含义如下：
 * 条件一：检测 column 是否为 {prop1=col1, prop2=col2} 形式，该
 *          种形式的 column 一般用于关联查询
 * 条件二：检测当前列名是否被包含在已映射的列名集合中，
 *          若包含则可进行数据集映射操作
 * 条件三：多结果集相关，暂不分析
 */
if (propertyMapping.isCompositeResult()
    || (column != null && mappedColumnNames.contains(
        column.toUpperCase(Locale.ENGLISH)))
    || propertyMapping.getResultSet() != null) {

    // 从结果集中获取指定列的数据
    Object value = getPropertyMappingValue(rsw.getResultSet(),
        metaObject, propertyMapping, lazyLoader, columnPrefix);

    final String property = propertyMapping.getProperty();
    if (property == null) {
        continue;

        // 若获取到的值为 DEFERED，则延迟加载该值
    } else if (value == DEFERED) {
        foundValues = true;
        continue;
    }
    if (value != null) {
        foundValues = true;
    }
    if (value != null || (configuration.isCallSettersOnNulls() &&
        !metaObject.getSetterType(property).isPrimitive())) {
```

```

        // 将获取到的值设置到实体类对象中
        metaObject.setValue(property, value);
    }
}
}
return foundValues;
}

private Object getPropertyMappingValue(ResultSet rs, MetaObject
    metaResultObject, ResultMapping propertyMapping, ResultLoaderMap
    lazyLoader, String columnPrefix) throws SQLException {

    if (propertyMapping.getNestedQueryId() != null) {
        // 获取关联查询结果，下一节分析
        return getNestedQueryMappingValue(rs, metaResultObject,
propertyMapping, lazyLoader, columnPrefix);
    } else if (propertyMapping.getResultSet() != null) {
        addPendingChildRelation(rs, metaResultObject, propertyMapping);
        return DEFERED;
    } else {
        final TypeHandler<?> typeHandler = propertyMapping.getTypeHandler();
        // 拼接前缀
        final String column = prependPrefix(propertyMapping.getColumn(),
columnPrefix);
        // 从 ResultSet 中获取指定列的值
        return typeHandler.getResult(rs, column);
    }
}
}

```

applyPropertyMappings 方法首先从 ResultSetWrapper 中获取已映射列名集合 mappedColumnNames，从 ResultMap 获取映射对象 ResultMapping 集合。然后遍历 ResultMapping 集合，在此过程中调用 getPropertyMappingValue 获取指定指定列的数据，最后将获取到的数据设置到实体类对象中。到此，基本的结果集映射过程就分析完了。

3. 关联查询与延迟加载

我们在学习 MyBatis 框架时, 会经常碰到一对一, 一对多的使用场景。对于这样的场景, 通常我们可以用一条 SQL 进行多表查询完成任务。当然我们也可以使用关联查询, 将一条 SQL 拆成两条去完成查询任务。MyBatis 提供了两个标签用于支持一对一和一对多的使用场景, 分别是<association>和<collection>。下面我来演示一下如何使用<association>完成一对一的关联查询。先来看看实体类的定义:

```
/** 作者类 */
public class Author {
    private Integer id;
    private String name;
    private Integer age;
    private Integer sex;
    private String email;

    // 省略 getter/setter
}

/** 文章类 */
public class Article {
    private Integer id;
    private String title;
    // 一对一关系
    private Author author;
    private String content;
    private Date createTime;

    // 省略 getter/setter
}
```

相关表记录如下:

id	author_id	title	type	content	create_time
1	1	MyBatis 源码分析系列文章导读	8	MyBatis 源码分析系列文章导读	2018-07-15 15:30:09
2	2	HashMap 源码详细分析(JDK1.8)	1	HashMap 源码详细分析(JDK1.8)	2018-01-18 15:29:13
3	1	Java CAS 原理分析	1	Java CAS 原理分析	2018-05-15 15:28:33
4	1	Spring IOC 容器源码分析 - 获取单例 bean	4	Spring IOC 容器源码分析 - 获取单例 bean	2018-06-01 00:00:00
5	1	Spring IOC 容器源码分析 - 循环依赖的解决办法	4	Spring IOC 容器源码分析 - 循环依赖的解决办法	2018-06-08 00:00:00
6	2	Spring AOP 源码分析系列文章导读	4	Spring AOP 源码分析系列文章导读	2018-06-17 00:00:00
7	2	Spring AOP 源码分析 - 创建代理对象	4	Spring AOP 源码分析 - 创建代理对象	2018-06-20 00:00:00
8	1	Spring MVC 原理探秘 - 一个请求的旅行过程	4	Spring MVC 原理探秘 - 一个请求的旅行过程	2018-06-29 00:00:00
9	2	Spring MVC 原理探秘 - 容器的创建过程	4	Spring MVC 原理探秘 - 容器的创建过程	2018-06-30 00:00:00
10	2	Spring IOC 容器源码分析系列文章导读	4	Spring IOC 容器源码分析系列文章导读	2018-05-30 00:00:00

article

id	name	age	sex	email
1	coolblog.xyz	28	0	coolblog.xyz@outlook.com
2	nullllun	29	1	coolblog.xyz@outlook.com

author

接下来看一下 Mapper 接口与映射文件的定义。

```
public interface ArticleDao {
    Article findOne(@Param("id") int id);
    Author findAuthor(@Param("id") int authorId);
}
```

```
<mapper namespace="xyz.coolblog.chapter4.dao.ArticleDao">
    <resultMap id="articleResult" type="Article">
        <result property="createTime" column="create_time"/>
        <association property="author" column="author_id"
            javaType="Author" select="findAuthor"/>
    </resultMap>

    <select id="findOne" resultMap="articleResult">
        SELECT
            id, author_id, title, content, create_time
        FROM
            article
```

```
        WHERE
            id = #{id}
    </select>

    <select id="findAuthor" resultType="Author">
        SELECT
            id, name, age, sex, email
        FROM
            author
        WHERE
            id = #{id}
    </select>
</mapper>
```

好了，必要的准备工作做完了，下面可以写测试代码了。如下：

```
public class OneToOneTest {

    private SqlSessionFactory sqlSessionFactory;

    @Before
    public void prepare() throws IOException {
        String resource = "chapter4/mybatis-config.xml";
        InputStream inputStream = Resources.getResourceAsStream(resource);
        sqlSessionFactory = new
            SqlSessionFactoryBuilder().build(inputStream);
        inputStream.close();
    }

    @Test
    public void testOne2One() {
        SqlSession session = sqlSessionFactory.openSession();
        try {
```

```

        ArticleDao articleDao = session.getMapper(ArticleDao.class);
        Article article = articleDao.findOne(1);

        Author author = article.getAuthor();
        article.setAuthor(null);

        System.out.println("\narticles info:");
        System.out.println(article);

        System.out.println("\nauthor info:");
        System.out.println(author);
    } finally {
        session.close();
    }
}
}
}

```

测试结果如下：

```

1 test passed - 2s 30ms © 2018 tianxiaobo.com
DEBUG [main] - ==> Preparing: SELECT id, author_id, title, content, create_time FROM article WHERE id = ?
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, author_id, title, content, create_time SQL-1
TRACE [main] - <== Row: 1, 1, MyBatis 源码分析系列文章导读, <<BLOB>>, 2018-07-15 15:30:09
DEBUG [main] - <== Total: 1
DEBUG [main] - ==> Preparing: SELECT id, name, age, sex, email FROM author WHERE id = ? SQL-2
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, name, age, sex, email
TRACE [main] - <== Row: 1, coolblog.xyz, 28, 0, coolblog.xyz@outlook.com
DEBUG [main] - <== Total: 1

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', type=null, author=Author{id=1, name='coolblog.xyz', age=28, sex=0}

author info:
Author{id=1, name='coolblog.xyz', age=28, sex=0, email='coolblog.xyz@outlook.com'}

```

如上,从上面的输出结果中可以看出,我们在调用 ArticleDao 的 findOne 方法时,MyBatis 执行了两条 SQL,完成了一对一的查询需求。理解了上面的例子后,下面就可以深入到源码中,看看 MyBatis 是如何实现关联查询的。接下来从 getNestedQueryMappingValue 方法开始分析,如下:

```

private Object getNestedQueryMappingValue(ResultSet rs,
    MetaObject metaResultObject, ResultMapping propertyMapping,
    ResultLoaderMap lazyLoader, String columnPrefix) throws SQLException {
    // 获取关联查询 id, id = 命名空间 + <association> 的 select 属性值
    final String nestedQueryId = propertyMapping.getNestedQueryId();
    final String property = propertyMapping.getProperty();
    // 根据 nestedQueryId 获取 MappedStatement
    final MappedStatement nestedQuery =
        configuration.getMappedStatement(nestedQueryId);
    final Class<?> nestedQueryParameterType =
        nestedQuery.getParameterMap().getType();
    /*
     * 生成关联查询语句参数对象, 参数类型可能是一些包装类, Map 或是自定义的实体类,
     * 具体类型取决于配置信息。以上面的例子为基础, 下面分析不同配置对
     * 参数类型的影响:
     * 1. <association column="author_id">
     *     column 属性值仅包含列信息, 参数类型为 author_id 列对应的类型,
     *     这里为 Integer
     * 2. <association column="{id=author_id, name=title}">
     *     column 属性值包含了属性名与列名的复合信息, MyBatis 会根据列名从
     *     ResultSet 中获取列数据, 并将列数据设置到实体类对象的指定属性中, 比如:
     *     Author{id=1, name="MyBatis 源码分析系列文章导读", age=null, ...}
     *     或是以键值对 <属性, 列数据> 的形式, 将两者存入 Map 中。比如:
     *     {"id": 1, "name": "MyBatis 源码分析系列文章导读"}
     *
     * 至于参数类型到底为实体类还是 Map, 取决于关联查询语句的配置信息。比如:
     *     <select id="findAuthor"> -> 参数类型为 Map
     *     <select id="findAuthor" parameterType="Author">
     *         -> 参数类型为实体类
     */
    final Object nestedQueryParameterObject=prepareParameterForNestedQuery(
        rs, propertyMapping, nestedQueryParameterType, columnPrefix);
    Object value = null;

```

```
if (nestedQueryParameterObject != null) {
    // 获取 BoundSql
    final BoundSql nestedBoundSql =
        nestedQuery.getBoundSql(nestedQueryParameterObject);
    final CacheKey key = executor.createCacheKey(nestedQuery,
        nestedQueryParameterObject, RowBounds.DEFAULT, nestedBoundSql);
    final Class<?> targetType = propertyMapping.getJavaType();
    // 检查一级缓存是否保存了关联查询结果
    if (executor.isCached(nestedQuery, key)) {
        // 从一级缓存中获取关联查询的结果, 并通过 metaResultObject
        // 将结果设置到相应的实体类对象中
        executor.deferLoad(nestedQuery,
            metaResultObject, property, key, targetType);
        value = DEFERED;
    } else {
        // 创建结果加载器
        final ResultLoader resultLoader = new ResultLoader(
            configuration, executor, nestedQuery,
            nestedQueryParameterObject, targetType, key, nestedBoundSql);
        // 检测当前属性是否需要延迟加载
        if (propertyMapping.isLazy()) {
            // 添加延迟加载相关的对象到 loaderMap 集合中
            lazyLoader.addLoader(
                property, metaResultObject, resultLoader);
            value = DEFERED;
        } else {
            // 直接执行关联查询
            value = resultLoader.loadResult();
        }
    }
}
return value;
}
```


上面对关联查询进行了比较多的注释，导致该方法看起来有点复杂。当然，真实的逻辑确实有点复杂，因为它还调用了其他的很多方法。下面先来总结一下该方法的逻辑：

1. 根据 `nestedQueryId` 获取 `MappedStatement`
2. 生成参数对象
3. 获取 `BoundSql`
4. 检测一级缓存中是否有关联查询的结果，若有，则将结果设置到实体类对象中
5. 若一级缓存未命中，则创建结果加载器 `ResultLoader`
6. 检测当前属性是否需要延迟加载，若需要，则添加延迟加载相关的对象到 `loaderMap` 集合中
7. 如不需要延迟加载，则直接通过结果加载器加载结果

如上，`getNestedQueryMappingValue` 方法中逻辑多是都是和延迟加载有关。除了延迟加载，以上流程中针对一级缓存的检查是十分有必要的，若缓存命中，可直接取用结果，无需再在执行关联查询 SQL。若缓存未命中，接下来就要按部就班执行延迟加载相关逻辑，接下来，分析一下 MyBatis 延迟加载是如何实现的。首先我们来看一下添加延迟加载相关对象到 `loaderMap` 集合中的逻辑，如下：

```
// -☆- ResultLoaderMap
public void addLoader(String property, MetaObject metaResultObject,
ResultLoader resultLoader) {
    // 将属性名转为大写
    String upperFirst = getUppercaseFirstProperty(property);
    if (!upperFirst.equalsIgnoreCase(property) &&
        loaderMap.containsKey(upperFirst)) {
        throw new ExecutorException(".....");
    }
    // 创建 LoadPair，并将 <大写属性名, LoadPair 对象> 键值对添加到 loaderMap 中
    loaderMap.put(upperFirst,
        new LoadPair(property, metaResultObject, resultLoader));
}
```

`addLoader` 方法的参数最终都传给了 `LoadPair`，该类的 `load` 方法会在内部调用 `ResultLoader` 的 `loadResult` 方法进行关联查询，并通过 `metaResultObject` 将查询结果设置到实体类对象中。那 `LoadPair` 的 `load` 方法由谁调用呢？答案是实体类的代理对象。下面我们修

改一下上面示例中的部分代码，演示一下延迟加载。首先，我们需要在 MyBatis 配置文件的 <settings> 节点中加入或覆盖如下配置：

```
<!-- 开启延迟加载 -->
<setting name="lazyLoadingEnabled" value="true"/>
<!-- 关闭积极的加载策略 -->
<setting name="aggressiveLazyLoading" value="false"/>
<!-- 延迟加载的触发方法 -->
<setting name="lazyLoadTriggerMethods" value="equals,hashCode"/>
```

上面三个配置 MyBatis 官方文档中有较为详细的介绍，大家可以参考官方文档，这里就不详细介绍了。下面修改一下测试类的代码：

```
public class OneToOneTest {

    private SqlSessionFactory sqlSessionFactory;

    @Before
    public void prepare() throws IOException {...}

    @Test
    public void testOne2One2() {
        SqlSession session = sqlSessionFactory.openSession();
        try {
            ArticleDao articleDao = session.getMapper(ArticleDao.class);
            Article article = articleDao.findOne(1);

            System.out.println("\narticles info:");
            System.out.println(article);

            System.out.println("\n延迟加载 author 字段: ");
            // 通过 getter 方法触发延迟加载
            Author author = article.getAuthor();
        }
    }
}
```

```

        System.out.println("\narticles info:");
        System.out.println(article);
        System.out.println("\nauthor info:");
        System.out.println(author);
    } finally {
        session.close();
    }
}
}

```

测试结果如下：

```

1 test passed - 1s 493ms © 2018 tianxiaobo.com
DEBUG [main] - ==> Preparing: SELECT id, author_id, title, content, create_time FROM article WHERE id = ?
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, author_id, title, content, create_time
TRACE [main] - <== Row: 1, 1, MyBatis 源码分析系列文章导读, <<BLOB>>, 2018-07-15 15:30:09
DEBUG [main] - <== Total: 1

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', type=null, author=null, content='MyBatis 源码分析系列文章导读', creat

延迟加载 author 字段:
DEBUG [main] - ==> Preparing: SELECT id, name, age, sex, email FROM author WHERE id = ?
DEBUG [main] - ==> Parameters: 1(Integer)
TRACE [main] - <== Columns: id, name, age, sex, email
TRACE [main] - <== Row: 1, coolblog.xyz, 28, 0, coolblog.xyz@outlook.com
DEBUG [main] - <== Total: 1

articles info:
Article{id=1, title='MyBatis 源码分析系列文章导读', type=null, author=Author{id=1, name='coolblog.xyz', age=28, sex=0, email='coolblog.xyz@outlook.com'}, content='MyBatis 源码分析系列文章导读', creat

author info:
Author{id=1, name='coolblog.xyz', age=28, sex=0, email='coolblog.xyz@outlook.com'}

```

从上面结果中可以看出，我们在未调用 `getAuthor` 方法时，`Article` 对象中的 `author` 字段为 `null`。调用该方法后，再次输出 `Article` 对象，发现其 `author` 字段有值了，表明 `author` 字段的延迟加载逻辑被触发了。既然调用 `getAuthor` 可以触发延迟加载，那么该方法一定被做过手脚了，不然该方法应该返回 `null` 才是。实际情况确实如此，MyBatis 会为需要延迟加载的类生成代理类，代理逻辑会拦截实体类的方法调用。默认情况下，MyBatis 会使用 `Javassist` 为实体类生成代理，代理逻辑封装在 `JavassistProxyFactory` 类中，下面一起看一下。

```

// -☆- JavassistProxyFactory
public Object invoke(Object enhanced, Method method, Method methodProxy,
    Object[] args) throws Throwable {

```

```

final String methodName = method.getName();
try {
    synchronized (lazyLoader) {
        if (WRITE_REPLACE_METHOD.equals(methodName)) {
            // 针对 writeReplace 方法的处理逻辑，与延迟加载无关，不分析了
        } else {
            if (lazyLoader.size() > 0 &&
                !FINALIZE_METHOD.equals(methodName)) {
                // 如果 aggressive 为 true，或触发方法（比如 equals，
                // hashCode 等）被调用，则加载所有的所有延迟加载的数据
                if (aggressive ||
                    lazyLoadTriggerMethods.contains(methodName)) {
                    lazyLoader.loadAll();
                } else if (PropertyNamer.isSetter(methodName)) {
                    final String property =
                        PropertyNamer.methodToProperty(methodName);
                    // 如果使用者显示调用了 setter 方法，则将相应的
                    // 延迟加载类从 loaderMap 中移除
                    lazyLoader.remove(property);
                } // 检测使用者是否调用 getter 方法
            } else if (PropertyNamer.isGetter(methodName)) {
                final String property =
                    PropertyNamer.methodToProperty(methodName);
                // 检测该属性是否有相应的 LoadPair 对象
                if (lazyLoader.hasLoader(property)) {
                    // 执行延迟加载逻辑
                    lazyLoader.load(property);
                }
            }
        }
    }
}
}
// 调用被代理类的方法

```

```
        return methodProxy.invoke(enhanced, args);
    } catch (Throwable t) {
        throw ExceptionUtil.unwrapThrowable(t);
    }
}
```

如上，代理方法首先会检查 `aggressive` 是否为 `true`，如果不满足，再去检查 `lazyLoadTriggerMethods` 是否包含当前方法名。这里两个条件只要一个为 `true`，当前实体类中所有需要延迟加载。`aggressive` 和 `lazyLoadTriggerMethods` 两个变量的值取决于下面的配置。

```
<setting name="aggressiveLazyLoading" value="false"/>
<setting name="lazyLoadTriggerMethods" value="equals,hashCode"/>
```

回到上面的代码中。如果执行线程未进入第一个条件分支，那么紧接着，代理逻辑会检查使用者是不是调用了实体类的 `setter` 方法。如果调用了，就将该属性对应的 `LoadPair` 从 `loaderMap` 中移除。为什么要这么做呢？答案是：使用者既然手动调用 `setter` 方法，说明使用者想自定义某个属性的值。此时，延迟加载逻辑不应该再修改该属性的值，所以这里从 `loaderMap` 中移除属性对应的 `LoadPair`。最后如果使用者调用的是某个属性的 `getter` 方法，且该属性配置了延迟加载，此时延迟加载逻辑就会被触发。那接下来，我们来看看延迟加载逻辑是怎样实现的。

```
// -☆- ResultLoaderMap
public boolean load(String property) throws SQLException {
    // 从 loaderMap 中移除 property 所对应的 LoadPair
    LoadPair pair = loaderMap.remove(property.toUpperCase(Locale.ENGLISH));
    if (pair != null) {
        // 加载结果
        pair.load();
        return true;
    }
    return false;
}
```

```

// -☆- LoadPair
public void load() throws SQLException {
    if (this.metaResultObject == null) {
        throw new IllegalArgumentException("metaResultObject is null");
    }
    if (this.resultLoader == null) {
        throw new IllegalArgumentException("resultLoader is null");
    }

    // 调用重载方法
    this.load(null);
}

public void load(final Object userObject) throws SQLException {
    // 若 metaResultObject 和 resultLoader 为 null，则创建相关对象。
    // 在当前调用情况下，两者均不为 null，条件不成立。篇幅原因，下面代码不分析了
    if (this.metaResultObject == null || this.resultLoader == null) {...}

    // 线程安全检测
    if (this.serializationCheck == null) {
        final ResultLoader old = this.resultLoader;
        // 重新创建新的 ResultLoader 和 ClosedExecutor，
        // ClosedExecutor 是非线程安全的
        this.resultLoader = new ResultLoader(old.configuration,
            new ClosedExecutor(), old.mappedStatement, old.parameterObject,
            old.targetType, old.cacheKey, old.boundSql);
    }

    // 调用 ResultLoader 的 loadResult 方法加载结果，
    // 并通过 metaResultObject 设置结果到实体类对象中
    this.metaResultObject.setValue(property, this.resultLoader.loadResult());
}

```

上面的代码比较多，但是没什么特别的逻辑，我们重点关注最后一行有效代码就行了。下面看一下 ResultLoader 的 loadResult 方法逻辑是怎样的。

```
public Object loadResult() throws SQLException {
    // 执行关联查询
    List<Object> list = selectList();
    // 抽取结果
    resultObject = resultExtractor.extractObjectFromList(list, targetType);
    return resultObject;
}

private <E> List<E> selectList() throws SQLException {
    Executor localExecutor = executor;
    if (Thread.currentThread().getId() != this.creatorThreadId ||
        localExecutor.isClosed()) {
        localExecutor = newExecutor();
    }
    try {
        // 通过 Executor 就行查询，这个之前已经分析过了
        return localExecutor.<E>query(mappedStatement, parameterObject,
            RowBounds.DEFAULT, Executor.NO_RESULT_HANDLER, cacheKey, boundSql);
    } finally {
        if (localExecutor != executor) {
            localExecutor.close(false);
        }
    }
}
```

如上，我们在 ResultLoader 中终于看到了执行关联查询的代码，即 selectList 方法中的逻辑。该方法在内部通过 Executor 进行查询。至于查询结果的抽取过程，并不是本节所关心的点，因此大家自行分析吧。到此，关于关联查询与延迟加载就分析完了。

4. 存储映射结果

存储映射结果是“查询结果”处理流程中的最后一环，实际上也是查询语句执行过程的最后一环。本节内容分析完，整个查询过程就分析完了，那接下来让我们带着喜悦的心情来分析映射结果存储逻辑。

```
private void storeObject(ResultHandler<?> resultHandler,
    DefaultResultContext<Object> resultContext, Object rowValue, ResultMapping
    parentMapping, ResultSet rs) throws SQLException {
    if (parentMapping != null) {
        // 多结果集相关，不分析了
        linkToParents(rs, parentMapping, rowValue);
    } else {
        // 存储结果
        callResultHandler(resultHandler, resultContext, rowValue);
    }
}

private void callResultHandler(ResultHandler<?> resultHandler,
    DefaultResultContext<Object> resultContext, Object rowValue) {
    // 设置结果到 resultContext 中
    resultContext.nextResultObject(rowValue);
    // 从 resultContext 获取结果，并存储到 resultHandler 中
    ((ResultHandler<Object>) resultHandler).handleResult(resultContext);
}
```

上面方法显示将 rowValue 设置到 ResultContext 中，然后再将 ResultContext 对象作为参数传给 ResultHandler 的 handleResult 方法。下面我们分别看一下 ResultContext 和 ResultHandler 的实现类。如下：

```
public class DefaultResultContext<T> implements ResultContext<T> {
    private T resultObject;
    private int resultCount;
```



```
    /** 状态字段 */
    private boolean stopped;

    // 省略部分代码

    @Override
    public boolean isStopped() {
        return stopped;
    }

    public void nextResultObject(T resultObject) {
        resultCount++;
        this.resultObject = resultObject;
    }

    @Override
    public void stop() {
        this.stopped = true;
    }
}
```

DefaultResultContext 中包含了一个状态字段，表明结果上下文的状态。在处理多行数据时，MyBatis 会检查该字段的值，已决定是否需要后续的处理。该类的逻辑比较简单，不多说了。下面再来看一下 DefaultResultHandler 的源码。

```
public class DefaultResultHandler implements ResultHandler<Object> {

    private final List<Object> list;

    public DefaultResultHandler() {
        list = new ArrayList<Object>();
    }
}
```

```
@Override
public void handleResult(ResultContext<? extends Object> context) {
    // 添加结果到 list 中
    list.add(context.getResultObject());
}

public List<Object> getResultList() {
    return list;
}
}
```

如上，DefaultResultHandler 默认使用 List 存储结果。除此之外，如果 Mapper（或 Dao）接口方法返回值为 Map 类型，此时则需要另一种 ResultHandler 实现类处理结果，即 DefaultMapResultHandler。关于 DefaultMapResultHandler 的源码大家自行分析吧啊，本节就不展开了。

4.3 更新语句的执行过程

上一节较为完整的分析了查询语句的执行过程。尽管有些地方一笔带过了，但多数细节都分析到了。如果大家搞懂了查询语句的执行过程，那么理解更新语句的执行过程也将不在话下。执行更新语句所需处理的情况较之查询语句要简单不少，两者最大的区别更新语句的执行结果类型单一，处理逻辑要简单不少。除此之外，两者在缓存的处理上也有比较大的区别。更新过程会立即刷新缓存，而查询过程则不会。至于其他的不同点，这里就不一一列举了。下面开始分析更新语句的执行过程。

4.3.1 更新语句执行过程全貌

首先，我们还是从 MapperMethod 的 execute 方法开始看起。

```
// -☆- MapperMethod
public Object execute(SqlSession sqlSession, Object[] args) {
```

```
Object result;
switch (command.getType()) {
    case INSERT: { // 执行插入语句
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(
            sqlSession.insert(command.getName(), param));
        break;
    }
    case UPDATE: { // 执行更新语句
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(
            sqlSession.update(command.getName(), param));
        break;
    }
    case DELETE: { // 执行删除语句
        Object param = method.convertArgsToSqlCommandParam(args);
        result = rowCountResult(
            sqlSession.delete(command.getName(), param));
        break;
    }
    case SELECT:
        // ...
        break;
    case FLUSH:
        // ...
        break;
    default:
        throw new BindingException(".....");
}
if (result == null && method.getReturnType().isPrimitive() &&
    !method.returnsVoid()) {...}
return result;
}
```

如上，插入、更新以及删除操作最终都调用了 SqlSession 接口中的方法。这三个方法返回值均是受影响行数，是一个整型值。rowCountResult 方法负责处理这个整型值，该方法的逻辑暂时先不分析，放在最后分析。下面分析 SqlSession 的实现类 DefaultSqlSession 的代码。

```
// -☆- DefaultSqlSession
public int insert(String statement, Object parameter) {
    return update(statement, parameter);
}

public int delete(String statement, Object parameter) {
    return update(statement, parameter);
}

public int update(String statement, Object parameter) {
    try {
        dirty = true;
        // 获取 MappedStatement
        MappedStatement ms = configuration.getMappedStatement(statement);
        // 调用 Executor 的 update 方法
        return executor.update(ms, wrapCollection(parameter));
    } catch (Exception e) {
        throw ExceptionFactory.wrapException(".....");
    } finally {
        ErrorContext.instance().reset();
    }
}
```

如上，insert 和 delete 方法最终都调用了同一个 update 方法，这就是为什么我把他们归为一类的原因。既然它们最终调用的都是同一个方法，那么 MyBatis 为什么还要在 SqlSession 中提供这么多方法呢，难道只提供 update 方法不行么？答案是：只提供一个 update 方法从实现上来说完全可行，但是从接口的语义化的角度来说，这样做并不好。一般情况下，使用者觉得 update 接口方法应该仅负责执行 UPDATE 语句，如果它还兼职执行其他的 SQL 语句，会让使用者产生疑惑。对于对外的接口，接口功能越单一，语义越清晰越好。在日常开

发中，我们在为客户端提供接口时，也应该这样做。比如我之前写过一个文章评论的开关接口，我写的接口如下：

```
Result openComment();  
Result closeComment();
```

上面接口语义比较清晰，同时没有参数，后端不用校验参数，客户端同学也不用思考传什么值。如果我像下面这样定义接口：

```
Result updateCommentStatus(Integer status);    // 0 - 关闭, 1 - 开启
```

首先这个方法没有上面两个方法语义清晰，其次需要传入一个整型状态值，客户端需要注意传值，后端也要进行校验。好了，关于接口语义化就先说这么多。扯多了，回归正题，下面分析 Executor 的 update 方法。如下：

```
// -☆- CachingExecutor  
public int update(MappedStatement ms, Object parameterObject)  
    throws SQLException {  
    // 刷新二级缓存  
    flushCacheIfRequired(ms);  
    return delegate.update(ms, parameterObject);  
}  
  
// -☆- BaseExecutor  
public int update(MappedStatement ms, Object parameter)  
    throws SQLException {  
    if (closed) {  
        throw new ExecutorException("Executor was closed.");  
    }  
    // 刷新一级缓存  
    clearLocalCache();  
    return doUpdate(ms, parameter);  
}
```

Executor 实现类中的方法在进行下一步操作之前,都会先刷新各自的缓存。默认情况下,insert、update 和 delete 操作都会清空一二级缓存。清空缓存的逻辑不复杂,大家自行分析。下面分析 doUpdate 方法,该方法是一个抽象方法,因此我们到 BaseExecutor 的子类 SimpleExecutor 中看看该方法是如何实现的。

```
// -☆- SimpleExecutor
public int doUpdate(MappedStatement ms, Object parameter)
    throws SQLException {
    Statement stmt = null;
    try {
        Configuration configuration = ms.getConfiguration();
        // 创建 StatementHandler
        StatementHandler handler = configuration.newStatementHandler(
            this, ms, parameter, RowBounds.DEFAULT, null, null);
        // 创建 Statement
        stmt = prepareStatement(handler, ms.getStatementLog());
        // 调用 StatementHandler 的 update 方法
        return handler.update(stmt);
    } finally {
        closeStatement(stmt);
    }
}
```

StatementHandler 和 Statement 的创建过程前面已经分析过,这里就不重复分析了。下面分析 PreparedStatementHandler 的 update 方法。

```
// -☆- PreparedStatementHandler
public int update(Statement statement) throws SQLException {
    PreparedStatement ps = (PreparedStatement) statement;
    // 执行 SQL
    ps.execute();
    // 返回受影响行数
    int rows = ps.getUpdateCount();
}
```

```
// 获取用户传入的参数值，参数值类型可能是普通的实体类，也可能是 Map
Object parameterObject = boundSql.getParameterObject();

KeyGenerator keyGenerator = mappedStatement.getKeyGenerator();
// 获取自增主键的值，并将值填入到参数对象中
keyGenerator.processAfter(executor, mappedStatement, ps, parameterObject);
return rows;
}
```

PreparedStatementHandler 的 update 方法的逻辑比较清晰明了了，更新语句的 SQL 会在该方法中被执行。执行结果为受影响行数，对于 insert 语句，有时候我们还想获取自增主键的值，因此我们需要进行一些额外的操作。这些额外操作的逻辑封装在 KeyGenerator 的实现类中，下面我们一起看一下 KeyGenerator 的实现逻辑。

4.3.2 KeyGenerator

KeyGenerator 是一个接口，目前它有三个实现类，分别如下：

- Jdbc3KeyGenerator
- SelectKeyGenerator
- NoKeyGenerator

Jdbc3KeyGenerator 用于获取插入数据后的自增主键数值。某些数据库不支持自增主键，需要手动填写主键字段，此时需要借助 SelectKeyGenerator 获取主键值。至于 NoKeyGenerator，这是一个空实现，没什么可说的。本节将分析 Jdbc3KeyGenerator 的源码，至于 SelectKeyGenerator，大家可自行分析。

```
// -☆- Jdbc3KeyGenerator
public void processBefore(Executor executor, MappedStatement ms,
    Statement stmt, Object parameter) {
    // 空方法
}

public void processAfter(Executor executor, MappedStatement ms,
    Statement stmt, Object parameter) {
```

```

        processBatch(ms, stmt, getParameters(parameter));
    }

    public void processBatch(MappedStatement ms, Statement stmt,
        Collection<Object> parameters) {
        ResultSet rs = null;
        try {
            rs = stmt.getGeneratedKeys();
            final Configuration configuration = ms.getConfiguration();
            final TypeHandlerRegistry typeHandlerRegistry =
                configuration.getTypeHandlerRegistry();
            // 获取主键字段
            final String[] keyProperties = ms.getKeyProperties();
            // 获取结果集 ResultSet 的元数据
            final ResultSetMetaData rsmd = rs.getMetaData();
            TypeHandler<?>[] typeHandlers = null;
            // ResultSet 中数据的列数要大于等于主键的数量
            if (keyProperties != null &&
                rsmd.getColumnCount() >= keyProperties.length) {
                // 遍历 parameters
                for (Object parameter : parameters) {
                    // 对于批量插入, ResultSet 会返回多行数据
                    if (!rs.next()) {
                        break;
                    }
                    final MetaObject metaParam =
                        configuration.newMetaObject(parameter);
                    if (typeHandlers == null) {
                        // 为每个主键属性获取 TypeHandler
                        typeHandlers = getTypeHandlers(typeHandlerRegistry,
                            metaParam, keyProperties, rsmd);
                    }
                }
            }
        }
    }

```



```
        // 填充结果到运行时参数中
        populateKeys(rs, metaParam, keyProperties, typeHandlers);
    }
}
} catch (Exception e) {
    throw new ExecutorException(...);
} finally {...}
}

private Collection<Object> getParameters(Object parameter) {
    Collection<Object> parameters = null;
    if (parameter instanceof Collection) {
        parameters = (Collection) parameter;
    } else if (parameter instanceof Map) {
        Map parameterMap = (Map) parameter;
        // 如果 parameter 是 Map 类型, 则从其中提取指定 key 对应的值。
        // 至于 Map 中为什么会出现 collection/list/array 等键。大家
        // 可以参考 DefaultSqlSession 的 wrapCollection 方法
        if (parameterMap.containsKey("collection")) {
            parameters = (Collection) parameterMap.get("collection");
        } else if (parameterMap.containsKey("list")) {
            parameters = (List) parameterMap.get("list");
        } else if (parameterMap.containsKey("array")) {
            parameters = Arrays.asList((Object[])parameterMap.get("array"));
        }
    }
    if (parameters == null) {
        parameters = new ArrayList<Object>();
        // 将普通的对象添加到 parameters 中
        parameters.add(parameter);
    }
    return parameters;
}
```

Jdbc3KeyGenerator 的 processBefore 方法是一个空方法,processAfter 则是一个空壳方法,只有一行代码。Jdbc3KeyGenerator 的重点在 processBatch 方法中,由于存在批量插入的情况,所以该方法的名字类包含 batch 单词,表示可处理批量插入的结果集。processBatch 方法的逻辑并不是很复杂,主要流程如下:

1. 获取主键数组(keyProperties)
2. 获取 ResultSet 元数据
3. 遍历参数列表,为每个主键属性获取 TypeHandler
4. 从 ResultSet 中获取主键数据,并填充到参数中

在上面流程中,第 1~3 步骤都是常规操作,第 4 个步骤需要分析一下。如下:

```
private void populateKeys(ResultSet rs, MetaObject metaParam,
    String[] keyProperties, TypeHandler<?>[] typeHandlers)
    throws SQLException {
    // 遍历 keyProperties
    for (int i = 0; i < keyProperties.length; i++) {
        // 获取主键属性
        String property = keyProperties[i];
        TypeHandler<?> th = typeHandlers[i];
        if (th != null) {
            // 从 ResultSet 中获取某列的值
            Object value = th.getResult(rs, i + 1);
            // 设置结果值到运行时参数中
            metaParam.setValue(property, value);
        }
    }
}
```

populateKeys 方法首先是遍历主键数组,然后通过 TypeHandler 从 ResultSet 中获取自增主键的值,最后再通过元信息对象将自增主键的值设置到参数中。

以上就是 Jdbc3KeyGenerator 的原理分析,这里写个示例演示一下。本次演示所用到的实体类如下:

```
public class Author {  
    private Integer id;  
    private String name;  
    private Integer age;  
    private Integer sex;  
    private String email;  
}
```

Mapper 接口和映射文件内容如下:

```
public interface AuthorDao {  
    int insertMany(List<Author> authors);  
}
```

```
<insert id="insertMany" keyProperty="id" useGeneratedKeys="true">  
    INSERT INTO  
        author (`name`, `age`, `sex`, `email`)  
    VALUES  
        <foreach item="author" index="index" collection="list" separator=",">  
            ({author.name}, {author.age}, {author.sex}, {author.email})  
        </foreach>  
</insert>
```

测试代码如下:

```
public class InsertManyTest {  
  
    private SqlSessionFactory sqlSessionFactory;  
  
    @Before  
    public void prepare() throws IOException {  
        String resource = "chapter4/mybatis-config.xml";  
        InputStream inputStream = Resources.getResourceAsStream(resource);  
    }  
}
```

```
sqlSessionFactory = new
    SqlSessionFactoryBuilder().build(inputStream);
inputStream.close();
}

@Test
public void testInsertMany() {
    SqlSession session = sqlSessionFactory.openSession();
    try {
        List<Author> authors = new ArrayList<>();
        // 添加多个 Author 对象到 authors 中
        authors.add(new Author(
            "tianxiaobo-1", 20, 0, "coolblog.xyz@outlook.com"));
        authors.add(new Author(
            "tianxiaobo-2", 18, 0, "coolblog.xyz@outlook.com"));

        System.out.println("\nBefore Insert: ");
        authors.forEach(author -> System.out.println(" " + author));

        AuthorDao authorDao = session.getMapper(AuthorDao.class);
        authorDao.insertMany(authors);
        session.commit();

        System.out.println("\nAfter Insert: ");
        authors.forEach(author -> System.out.println(" " + author));
    } finally {
        session.close();
    }
}
```

上面的测试代码创建了一个 Author 集合，并向集合中插入了两个 Author 对象。然后将集合中的元素批量插入到 author 表中，得到如下结果：

```

1 test passed - 1s 462ms © 2018 tianxiaobo.com

Before Insert:
Author{id=null, name='tianxiaobo-1', age=20, sex=0, email='coolblog.xyz@outlook.com'}
Author{id=null, name='tianxiaobo-2', age=18, sex=0, email='coolblog.xyz@outlook.com'}

DEBUG [main] - ==> Preparing: INSERT INTO author (`name`, `age`, `sex`, `email`) VALUES (?, ?, ?, ?) , (?, ?, ?, ?)
DEBUG [main] - ==> Parameters: tianxiaobo-1(String), 20(Integer), 0(Integer), coolblog.xyz@outlook.com(String), tianxiaobo-2(String), 18(Integer), 0(Integer), coolblog.xyz@outlook.com(String)
DEBUG [main] - <== Updates: 2

After Insert:
Author{id=3, name='tianxiaobo-1', age=20, sex=0, email='coolblog.xyz@outlook.com'}
Author{id=4, name='tianxiaobo-2', age=18, sex=0, email='coolblog.xyz@outlook.com'}

```

如上图，执行插入语句前，列表中元素的 id 字段都为 null。插入数据后，列表元素中的 id 字段均被赋值了。好了，到此，关于 Jdbc3KeyGenerator 的原理与使用就分析完了。

4.3.3 处理更新结果

更新语句的执行结果是一个整型值，表示本次更新所影响的行数。由于返回值类型简单，因此处理逻辑也很简单。下面我们简单看一下，放松放松。

```

// -☆- MapperMethod
private Object rowCountResult(int rowCount) {
    final Object result;
    // 这里的 method 类型为 MethodSignature，即方法签名
    if (method.returnsVoid()) {
        // 方法返回类型为 void，则不用返回结果，这里将结果置空
        result = null;
    } else if (Integer.class.equals(method.getReturnType()) ||
               Integer.TYPE.equals(method.getReturnType())) {
        // 方法返回类型为 Integer 或 int，直接赋值返回即可
        result = rowCount;
    } else if (Long.class.equals(method.getReturnType()) ||
               Long.TYPE.equals(method.getReturnType())) {
        // 如果返回值类型为 Long 或者 long，这里强转一下即可
        result = (long) rowCount;
    } else if (Boolean.class.equals(method.getReturnType()) ||
               Boolean.TYPE.equals(method.getReturnType())) {

```

```

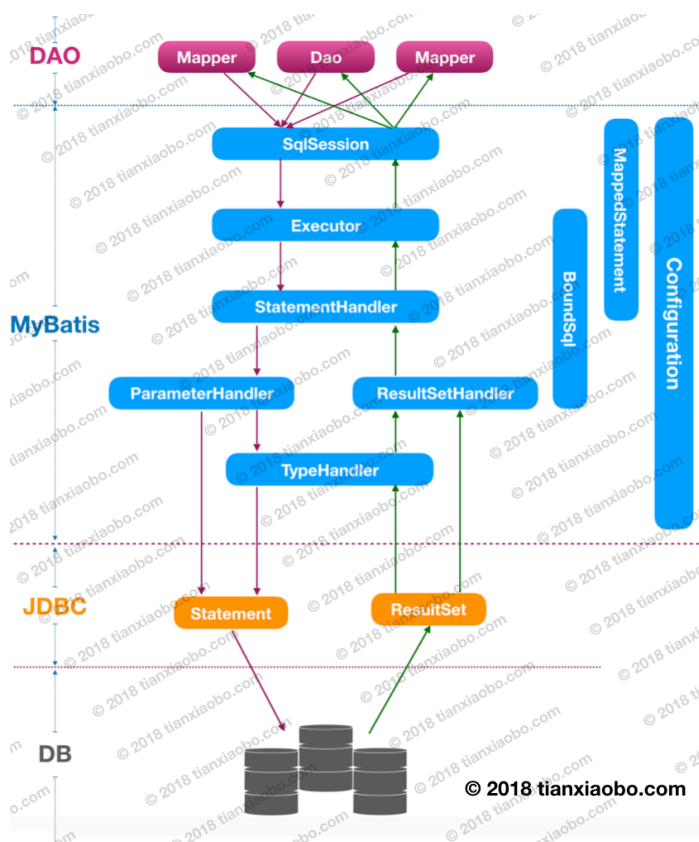
// 方法返回类型为布尔类型，若 rowCount > 0，则返回 true，否则返回 false
result = rowCount > 0;
} else {...}
return result;
}

```

上面就是 MyBatis 处理更新语句执行结果的逻辑。足够简单，就不多说了。

4.4 SQL 执行过程总结

经过前面前面的分析，相信大家对 MyBatis 执行 SQL 的过程都有比较深入的理解。本章的最后，用一张图 MyBatis 的执行过程进行一个总结。如下：



在 MyBatis 中，SQL 执行过程的实现代码是有层次的，每层都有相应的功能。比如，SqlSession 是对外接口的接口，因此它提供了各种语义清晰的方法，供使用者调用。Executor 层做的事情较多，比如一二级缓存功能就是嵌入在该层内的。StatementHandler 层主要是与 JDBC 层面的接口打交道。至于 ParameterHandler 和 ResultSetHandler，一个负责向 SQL 中设置运行时参数，另一个负责处理 SQL 执行结果，它们俩可以看做是 StatementHandler 辅助类。最后看一下右边横跨数层的类，Configuration 是一个全局配置类，很多地方都依赖它。MappedStatement 对应 SQL 配置，包含了 SQL 配置的相关信息。BoundSql 中包含了已完成解析的 SQL 语句，以及运行时参数等。到此，关于 SQL 的执行过程就分析完了。

4.5 本章小结

本章较为详细地分析了两种 SQL 语句的执行过程。整个执行过程比较复杂，牵涉的技术点比较多，包含但不限于 SQL 语句的解析过程、#{ } 占位符解析过程、运行时参数绑定逻辑，以及结果集处理逻辑等。对于这些技术点，如果大家若能掌握，那么对 MyBatis 的原理会有很深的理解。需要说明的是，尽管本章篇幅已经很长了，但仍有技术点未涉及到，比如存储过程的执行细节，SelectKeyGenerator 的原理等。对于这部分技术点，大家可自行分析。

第 5 章

内置数据源

本篇文章将向大家介绍 MyBatis 内置数据源的实现逻辑。MyBatis 支持三种数据源配置，分别为 UNPOOLED、POOLED 和 JNDI。并提供了两种数据源实现，分别是 UnpooledDataSource 和 PooledDataSource。在这三种数据源配置中，UNPOOLED 和 POOLED 是我们最常用的两种配置，这两种数据源也是本章要重点分析的对象。至于 JNDI，MyBatis 提供这种数据源的目的是为了让其能够运行在 EJB 或应用服务器等容器中，这一点官方文档中有所说明。由于 JNDI 数据源在日常开发中使用甚少，因此，本篇文章不打算分析 JNDI 数据源相关源码。

5.1 内置数据源初始化过程

在详细分析 UnpooledDataSource 和 PooledDataSource 两种数据源实现之前，我们先来了解一下数据源的配置与初始化过程。先来看一下数据源配置方法，如下：


```
<dataSource type="UNPOOLED|POOLED">
  <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
  <property name="url" value="jdbc:mysql..."/>
  <property name="username" value="root"/>
  <property name="password" value="1234"/>
</dataSource>
```

数据源的配置是内嵌在<environment>节点中的，MyBatis 在解析<environment>节点时，会一并解析数据源的配置。MyBatis 会根据具体的配置信息，为不同的数据源创建相应工厂类，通过工厂类即可创建数据源实例。关于数据源配置的解析以及数据源工厂类的创建过程，本书第二章分析过，这里就不赘述了。下面我们来看一下数据源工厂类的实现逻辑。

```
public class UnpooledDataSourceFactory implements DataSourceFactory {
    private static final String DRIVER_PROPERTY_PREFIX = "driver.";
    private static final int DRIVER_PROPERTY_PREFIX_LENGTH =
        DRIVER_PROPERTY_PREFIX.length();

    protected DataSource dataSource;

    public UnpooledDataSourceFactory() {
        // 创建 UnpooledDataSource 对象
        this.dataSource = new UnpooledDataSource();
    }

    @Override
    public void setProperties(Properties properties) {
        Properties driverProperties = new Properties();
        // 为 dataSource 创建元信息对象
        MetaObject metaDataSource = SystemMetaObject.forObject(dataSource);
        // 遍历 properties 键列表，properties 由配置文件解析器传入
        for (Object key : properties.keySet()) {
            String propertyName = (String) key;
```

```

// 检测 propertyName 是否以 "driver." 开头
if (propertyName.startsWith(DRIVER_PROPERTY_PREFIX)) {
    String value = properties.getProperty(propertyName);
    // 存储配置信息到 driverProperties 中
    driverProperties.setProperty(propertyName
        .substring(DRIVER_PROPERTY_PREFIX_LENGTH), value);
} else if (metaDataSource.hasSetter(propertyName)) {
    String value = (String) properties.get(propertyName);
    // 按需转换 value 类型
    Object convertedValue = convertValue(
        metaDataSource, propertyName, value);
    // 设置转换后的值到 UnpooledDataSourceFactory 指定属性中
    metaDataSource.setValue(propertyName, convertedValue);
} else {
    throw new DataSourceException(".....");
}
}

if (driverProperties.size() > 0) {
    // 设置 driverProperties 到 UnpooledDataSourceFactory 的
    // driverProperties 属性中
    metaDataSource.setValue("driverProperties", driverProperties);
}
}

private Object convertValue(
    MetaObject metaDataSource, String propertyName, String value) {
    Object convertedValue = value;
    // 获取属性对应的 setter 方法的参数类型
    Class<?> targetType = metaDataSource.getSetterType(propertyName);
    // 按照 setter 方法的参数类型进行类型转换
    if (targetType == Integer.class || targetType == int.class) {
        convertedValue = Integer.valueOf(value);
    } else if (targetType == Long.class || targetType == long.class) {

```

```
        convertedValue = Long.valueOf(value);
    } else if(targetType == Boolean.class || targetType == boolean.class){
        convertedValue = Boolean.valueOf(value);
    }
    return convertedValue;
}

@Override
public DataSource getDataSource() {
    return dataSource;
}
}
```

在 `UnpooledDataSourceFactory` 的源码中，除了 `setProperties` 方法稍复杂一点，其他的都比较简单。下面看看 `PooledDataSourceFactory` 的源码。

```
public class PooledDataSourceFactory extends UnpooledDataSourceFactory {

    public PooledDataSourceFactory() {
        // 创建 PooledDataSource
        this.dataSource = new PooledDataSource();
    }
}
```

以上就是 `PooledDataSource` 类的所有源码，`PooledDataSourceFactory` 继承自 `UnpooledDataSourceFactory`，复用了父类的逻辑，因此它的实现很简单。关于两种数据源的创建过程就先分析到这，接下来，我们去探究一下两种数据源是怎样实现的。

5.2 UnpooledDataSource

`UnpooledDataSource`，从名称上即可知道，该种数据源不具有池化特性。该种数据源每

次会返回一个新的数据库连接，而非复用旧的连接。由于 `UnpooledDataSource` 无需提供连接池功能，因此它的实现非常简单。核心的方法有三个，分别如下：

1. `initializeDriver` - 初始化数据库驱动
2. `doGetConnection` - 获取数据连接
3. `configureConnection` - 配置数据库连接

下面将按照顺序分节对相关方法进行分析，由于 `configureConnection` 方法比较简单，因此我把它和 `doGetConnection` 放在一节中进行分析。

5.2.1 初始化数据库驱动

回顾我们一开始学习使用 JDBC 访问数据库时的情景，在执行 SQL 之前，通常都是先获取数据库连接。一般步骤都是加载数据库驱动，然后通过 `DriverManager` 获取数据库连接。`UnpooledDataSource` 也是使用 JDBC 访问数据库的，因此它获取数据库连接的过程也大致如此，只不过会稍有不同。下面我们一起看一下。

```
// -☆- UnpooledDataSource
private synchronized void initializeDriver() throws SQLException {
    // 检测缓存中是否包含了与 driver 对应的驱动实例
    if (!registeredDrivers.containsKey(driver)) {
        Class<?> driverType;
        try {
            // 加载驱动类型
            if (driverClassLoader != null) {
                // 使用 driverClassLoader 加载驱动
                driverType = Class.forName(driver, true, driverClassLoader);
            } else {
                // 通过其他 ClassLoader 加载驱动
                driverType = Resources.classForName(driver);
            }

            // 通过反射创建驱动实例
            Driver driverInstance = (Driver) driverType.newInstance();
```

```
        // 注册驱动，注意这里是将 Driver 代理类 DriverProxy 对象注册到
        // 而非 Driver 对象本身。DriverProxy 中并没有什么特别的逻辑，就不分析。
        DriverManager.registerDriver(new DriverProxy(driverInstance));
        // 缓存驱动类名和实例
        registeredDrivers.put(driver, driverInstance);
    } catch (Exception e) {
        throw new SQLException(".....");
    }
}
```

如上，initializeDriver 方法主要包含三步操作，分别如下：

1. 加载驱动
2. 通过反射创建驱动实例
3. 注册驱动实例

这三步都是都是常规操作，比较容易理解。上面代码中出现了缓存相关的逻辑，这个是为了避免重复注册驱动。因为 initializeDriver 方法并不是在 UnpooledDataSource 初始化时被调用的，而是在获取数据库连接时被调用的。因此这里需要做个检测，避免每次获取数据库连接时都重新注册驱动。这个是一个比较小的点，大家看代码时注意一下即可。下面看一下获取数据库连接的逻辑。

5.2.2 获取数据库连接

在使用 JDBC 时，我们都是通过 DriverManager 的接口方法获取数据库连接。本节所要分析的源码也不例外，一起看一下吧。

```
// -☆- UnpooledDataSource
public Connection getConnection() throws SQLException {
    return doGetConnection(username, password);
}
```

```
private Connection doGetConnection(String username, String password)
    throws SQLException {
    Properties props = new Properties();
    if (driverProperties != null) {
        props.putAll(driverProperties);
    }
    if (username != null) {
        // 存储 user 配置
        props.setProperty("user", username);
    }
    if (password != null) {
        // 存储 password 配置
        props.setProperty("password", password);
    }
    // 调用重载方法
    return doGetConnection(props);
}

private Connection doGetConnection(Properties properties)
    throws SQLException {
    // 初始化驱动
    initializeDriver();
    // 获取连接
    Connection connection = DriverManager.getConnection(url, properties);
    // 配置连接，包括自动提交以及事务等级
    configureConnection(connection);
    return connection;
}

private void configureConnection(Connection conn) throws SQLException {
    if (autoCommit != null && autoCommit != conn.getAutoCommit()) {
        // 设置自动提交
        conn.setAutoCommit(autoCommit);
    }
}
```

```
    }  
    if (defaultTransactionIsolationLevel != null) {  
        // 设置事务隔离级别  
        conn.setTransactionIsolation(defaultTransactionIsolationLevel);  
    }  
}
```

如上，上面方法将一些配置信息放入到 Properties 对象中，然后将数据库连接和 Properties 对象传给 DriverManager 的 getConnection 方法即可获取到数据库连接。

5.3 PooledDataSource

PooledDataSource 内部实现了连接池功能，用于复用数据库连接。因此，从效率上来说，PooledDataSource 要高于 UnpooledDataSource。PooledDataSource 需要借助一些辅助类帮助它完成连接池的功能，所以接下来，我们先来认识一下相关的辅助类。

5.3.1 辅助类介绍

PooledDataSource 需要借助两个辅助类帮其完成功能，这两个辅助类分别是 PoolState 和 PooledConnection。PoolState 用于记录连接池运行时的状态，比如连接获取次数，无效连接数量等。同时 PoolState 内部定义了两个 PooledConnection 集合，用于存储空闲连接和活跃连接。PooledConnection 内部定义了一个 Connection 类型的变量，用于指向真实的数据库连接。以及一个 Connection 的代理类，用于对部分方法调用进行拦截。至于为什么要拦截，随后将进行分析。除此之外，PooledConnection 内部也定义了一些字段，用于记录数据库连接的一些运行时状态。接下来，我们来看一下 PooledConnection 的定义。

```
class PooledConnection implements InvocationHandler {  
    private static final String CLOSE = "close";  
    private static final Class<?>[] IFACES =  
        new Class<?>[]{Connection.class};  
    private final int hashCode;
```

```
private final PooledDataSource dataSource;
// 真实的数据库连接
private final Connection realConnection;
// 数据库连接代理
private final Connection proxyConnection;
// 从连接池中取出连接时的时间戳
private long checkoutTimestamp;
// 数据库连接创建时间
private long createdTimestamp;
// 数据库连接最后使用时间
private long lastUsedTimestamp;
// connectionTypeCode = (url + username + password).hashCode()
private int connectionTypeCode;
// 表示连接是否有效
private boolean valid;

public PooledConnection(
    Connection connection, PooledDataSource dataSource) {
    this.hashCode = connection.hashCode();
    this.realConnection = connection;
    this.dataSource = dataSource;
    this.createdTimestamp = System.currentTimeMillis();
    this.lastUsedTimestamp = System.currentTimeMillis();
    this.valid = true;
    // 创建 Connection 的代理类对象
    this.proxyConnection = (Connection) Proxy.newProxyInstance(
        Connection.class.getClassLoader(), IFACES, this);
}

@Override
public Object invoke(Object proxy, Method method, Object[] args)
    throws Throwable {...}
}
```


下面再来看看 PoolState 的定义。

```
public class PoolState {

    protected PooledDataSource dataSource;

    // 空闲连接列表
    protected final List<PooledConnection> idleConnections =
        new ArrayList<PooledConnection>();
    // 活跃连接列表
    protected final List<PooledConnection> activeConnections =
        new ArrayList<PooledConnection>();
    // 从连接池中获取连接的次数
    protected long requestCount = 0;
    // 请求连接总耗时（单位：毫秒）
    protected long accumulatedRequestTime = 0;
    // 连接执行时间总耗时
    protected long accumulatedCheckoutTime = 0;
    // 执行时间超时的连接数
    protected long claimedOverdueConnectionCount = 0;
    // 超时时间累加值
    protected long accumulatedCheckoutTimeOfOverdueConnections = 0;
    // 等待时间累加值
    protected long accumulatedWaitTime = 0;
    // 等待次数
    protected long hadToWaitCount = 0;
    // 无效连接数
    protected long badConnectionCount = 0;
}
```

上面对 PooledConnection 和 PoolState 的定义进行了一些注释，这两个类中有很多字段用来记录运行时状态。但在这些字段并非核心，因此大家知道每个字段的用途就行了。

5.3.2 获取连接

前面已经说过，PooledDataSource 会将用过的连接进行回收，以便可以复用连接。因此从 PooledDataSource 获取连接时，如果空闲链接列表里有连接时，可直接取用。那如果没有空闲连接怎么办呢？此时有两种解决办法，要么创建新连接，要么等待其他连接完成任务。具体怎么做，需视情况而定。下面我们深入到源码中一探究竟。

```
public Connection getConnection() throws SQLException {
    // 返回 Connection 的代理对象
    return popConnection(dataSource.getUsername(),
        dataSource.getPassword()).getProxyConnection();
}

private PooledConnection popConnection(String username, String password)
    throws SQLException {
    boolean countedWait = false;
    PooledConnection conn = null;
    long t = System.currentTimeMillis();
    int localBadConnectionCount = 0;

    while (conn == null) {
        synchronized (state) {
            // 检测空闲连接集合（idleConnections）是否为空
            if (!state.idleConnections.isEmpty()) {
                // idleConnections 不为空，表示有空闲连接可以使用
                conn = state.idleConnections.remove(0);
            } else {
                // 暂无空闲连接可用，但如果活跃连接数还未超出限制
                // （poolMaximumActiveConnections），则可创建新的连接
                if (state.activeConnections.size() <
                    poolMaximumActiveConnections) {
                    // 创建新连接
                }
            }
        }
    }
}
```

```
        conn = new PooledConnection(
            dataSource.getConnection(), this);
    } else {    // 连接池已满，不能创建新连接
        // 取出运行时间最长的连接
        PooledConnection oldestActiveConnection =
            state.activeConnections.get(0);
        // 获取运行时长
        long longestCheckoutTime =
            oldestActiveConnection.getCheckoutTime();
        // 检测运行时长是否超出限制，即超时
        if (longestCheckoutTime > poolMaximumCheckoutTime) {
            // 累加超时相关的统计字段
            state.claimedOverdueConnectionCount++;
            state.accumulatedCheckoutTimeOfOverdueConnections +=
                longestCheckoutTime;
            state.accumulatedCheckoutTime += longestCheckoutTime;

            // 从活跃连接集合中移除超时连接
            state.activeConnections
                .remove(oldestActiveConnection);
            // 若连接未设置自动提交，此处进行回滚操作
            if (!oldestActiveConnection
                .getRealConnection().getAutoCommit()){
                try {
                    oldestActiveConnection
                        .getRealConnection().rollback();
                } catch (SQLException e) {...}
            }
            // 创建一个新的 PooledConnection，注意，此处复用
            // oldestActiveConnection 的 realConnection 变量
            conn = new PooledConnection(
                oldestActiveConnection.getRealConnection(), this);
        }
    }
}
```

```

        // 复用 oldestActiveConnection 的一些信息，注意
        // PooledConnection 中的 createdTimestamp 用于记录
        // Connection 的创建时间，而非 PooledConnection
        // 的创建时间。所以这里要复用原连接的时间信息。
        conn.setCreatedTimestamp(
            oldestActiveConnection.getCreatedTimestamp());
        conn.setLastUsedTimestamp(
            oldestActiveConnection.getLastUsedTimestamp());
        // 设置连接为无效状态
        oldestActiveConnection.invalidate();
    } else {    // 运行时间最长的连接并未超时
        try {
            if (!countedWait) {
                state.hadToWaitCount++;
                countedWait = true;
            }
            long wt = System.currentTimeMillis();
            // 当前线程进入等待状态
            state.wait(poolTimeToWait);
            state.accumulatedWaitTime +=
                System.currentTimeMillis() - wt;
        } catch (InterruptedException e) {
            break;
        }
    }
}

if (conn != null) {
    // 检测连接是否有效，isValid 方法除了会检测 valid 是否为 true，
    // 还会通过 PooledConnection 的 pingConnection 方法执行 SQL 语句，
    // 检测连接是否可用。pingConnection 方法的逻辑不复杂，大家自行分析。

    if (conn.isValid()) {

```

```
        if (!conn.getRealConnection().getAutoCommit()) {
            // 进行回滚操作
            conn.getRealConnection().rollback();
        }
        conn.setConnectionTypeCode(assembleConnectionTypeCode(
            dataSource.getUrl(), username, password));
        // 设置统计字段
        conn.setCheckoutTimestamp(System.currentTimeMillis());
        conn.setLastUsedTimestamp(System.currentTimeMillis());
        state.activeConnections.add(conn);
        state.requestCount++;
        state.accumulatedRequestTime +=
            System.currentTimeMillis() - t;
    } else {
        // 连接无效，此时累加无效连接相关的统计字段
        state.badConnectionCount++;
        localBadConnectionCount++;
        conn = null;
        if (localBadConnectionCount > (poolMaximumIdleConnections
            + poolMaximumLocalBadConnectionTolerance)) {
            throw new SQLException(...);
        }
    }
}

}

}

if (conn == null) {
    throw new SQLException(...);
}

return conn;
}
```

上面代码冗长，过程比较复杂，下面把代码逻辑梳理一下。从连接池中获取连接首先会遇到两种情况：

1. 连接池中有空闲连接
2. 连接池中无空闲连接

对于第一种情况，处理措施就很简单了，把连接取出返回即可。对于第二种情况，则要进行细分，会有如下的情况。

1. 活跃连接数没有超出最大活跃连接数
2. 活跃连接数超出最大活跃连接数

对于上面两种情况，第一种情况比较好处理，直接创建新的连接即可。至于第二种情况，需要再次进行细分。

1. 活跃连接的运行时间超出限制，即超时了
2. 活跃连接未超时

对于第一种情况，我们直接将超时连接强行中断，并进行回滚，然后复用部分字段重新创建 `PooledConnection` 即可。对于第二种情况，目前没有更好的处理方式了，只能等待了。下面用一段伪代码演示各种情况及相应的处理措施，如下：

```
if (连接池中有空闲连接) {  
    1. 将连接从空闲连接集合中移除  
} else {  
    if (活跃连接数未超出限制) {  
        1. 创建新连接  
    } else {  
        1. 从活跃连接集合中取出第一个元素  
        2. 获取连接运行时长  
  
        if (连接超时) {  
            1. 将连接从活跃集合中移除  
            2. 复用原连接的成员变量，并创建新的 PooledConnection 对象  
        } else {  
            1. 线程进入等待状态  
            2. 线程被唤醒后，重新执行以上逻辑  
        }  
    }  
}
```

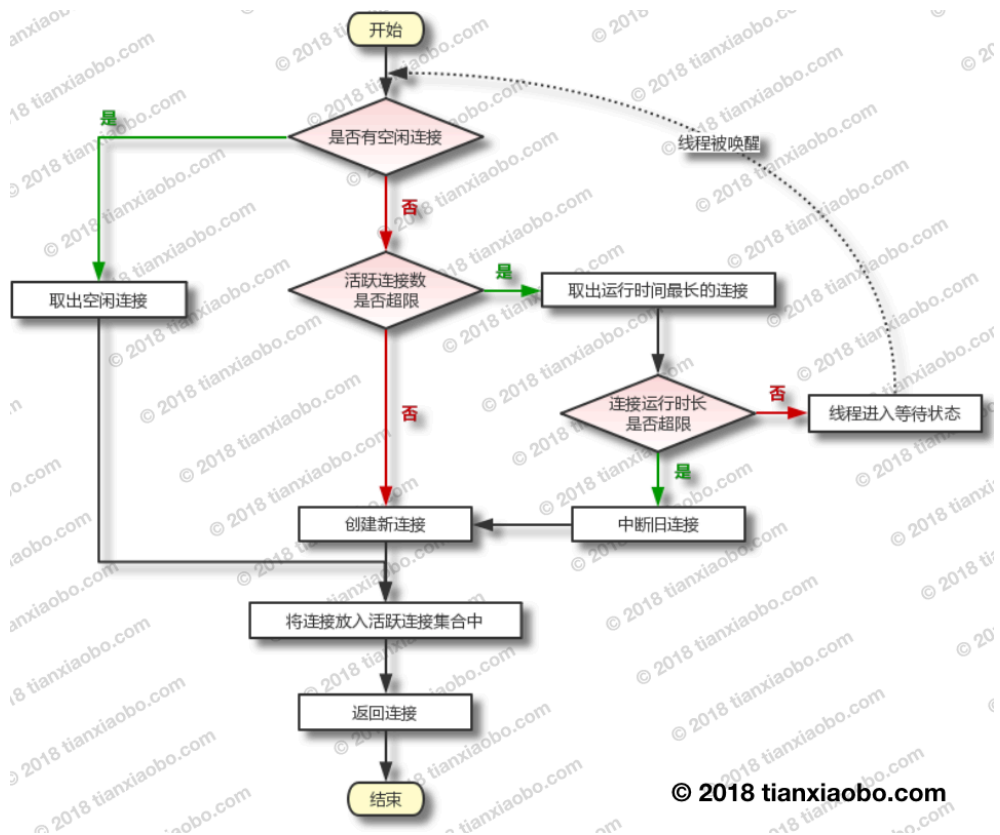
```

    }
}

```

1. 将连接添加到活跃连接集合中
2. 返回连接

最后用一个流程图大致描绘 popConnection 的逻辑，如下：



5.3.3 回收连接

相比获取连接，回收连接的逻辑要简单的多。回收连接成功与否只取决于空闲连接集合的状态，所需处理情况很少，因此比较简单。下面看一下相关的代码。

```
protected void pushConnection(PooledConnection conn) throws SQLException {
    synchronized (state) {
        // 从活跃连接池中移除连接
        state.activeConnections.remove(conn);
        if (conn.isValid()) {
            // 空闲连接集合未满足
            if (state.idleConnections.size() < poolMaximumIdleConnections
                && conn.getConnectionTypeCode() == expectedConnectionTypeCode) {
                state.accumulatedCheckoutTime += conn.getCheckoutTime();
                // 回滚未提交的事务
                if (!conn.getRealConnection().getAutoCommit()) {
                    conn.getRealConnection().rollback();
                }
                // 创建新的 PooledConnection
                PooledConnection newConn = new PooledConnection(
                    conn.getRealConnection(), this);
                state.idleConnections.add(newConn);
                // 复用时间信息
                newConn.setCreatedTimestamp(conn.getCreatedTimestamp());
                newConn.setLastUsedTimestamp(conn.getLastUsedTimestamp());

                // 将原连接置为无效状态
                conn.invalidate();
                // 通知等待的线程
                state.notifyAll();
            } else { // 空闲连接集合已满
                state.accumulatedCheckoutTime += conn.getCheckoutTime();
                // 回滚未提交的事务
                if (!conn.getRealConnection().getAutoCommit()) {
                    conn.getRealConnection().rollback();
                }
            }
            // 关闭数据库连接
            conn.getRealConnection().close();
        }
    }
}
```



```
        conn.invalidate();
    }
    } else {
        state.badConnectionCount++;
    }
}
}
```

上面代码首先将连接从活跃连接集合中移除,然后再根据空闲集合是否有空闲空间进行后续处理。如果空闲集合未满,此时复用原连接的字段信息创建新的连接,并将其放入空闲集合中即可。若空闲集合已满,此时无需回收连接,直接关闭即可。`pushConnection` 方法的逻辑并不复杂,就不多说了。

我们知道获取连接的方法 `popConnection` 是由 `getConnection` 方法调用的,那回收连接的方法 `pushConnection` 是由谁调用的呢? 答案是 `PooledConnection` 中的代理逻辑。相关代码如下:

```
// -☆- PooledConnection
public Object invoke(Object proxy, Method method, Object[] args)
    throws Throwable {
    String methodName = method.getName();
    // 检测 close 方法是否被调用,若被调用则拦截之
    if (CLOSE.hashCode() == methodName.hashCode() &&
        CLOSE.equals(methodName)){
        // 将回收连接中,而不是直接将连接关闭
        dataSource.pushConnection(this);
        return null;
    } else {
        try {
            if (!Object.class.equals(method.getDeclaringClass())) {
                checkConnection();
            }
        }
    }
}
```

```
        // 调用真实连接的目标方法
        return method.invoke(realConnection, args);
    } catch (Throwable t) {
        throw ExceptionUtil.unwrapThrowable(t);
    }
}
}
```

在上一节中，`getConnection` 方法返回的是 `Connection` 代理对象，不知道大家有没有注意到。代理对象中的方法被调用时，会被上面的代理逻辑所拦截。如果代理对象的 `close` 方法被调用，MyBatis 并不会直接调用真实连接的 `close` 方法关闭连接，而是调用 `pushConnection` 方法回收连接。同时会唤醒处于睡眠中的线程，使其恢复运行，整个过程并不复杂。

5.4 本章小结

本篇文章对 MyBatis 两种内置数据源进行了较为详细的分析，总的来说，这两种数据源的源码都不是很理解。大家在阅读源码的过程中，首先应搞懂源码的主要逻辑，然后再去分析一些边边角角的逻辑。不要一开始就陷入各种细节中，容易迷失方向。

第 6 章

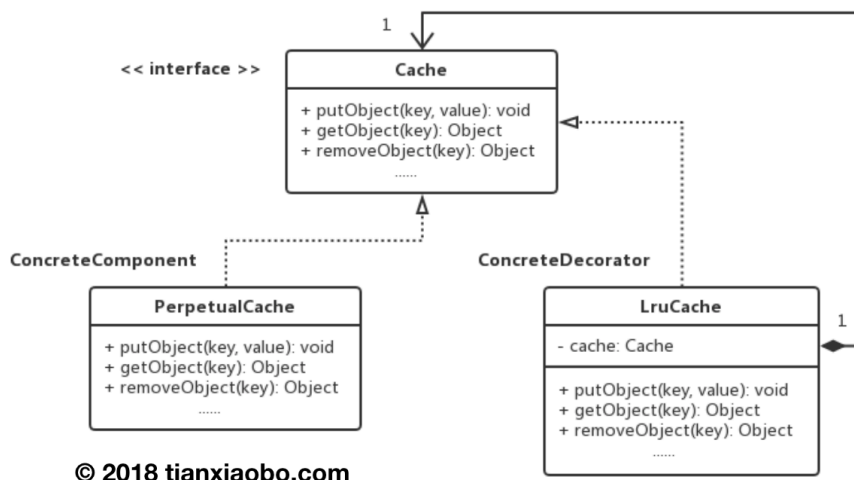
缓存机制

在 Web 应用中，缓存是必不可少的组件。通常我们都会用 Redis 或 memcached 等缓存中间件，拦截大量奔向数据库的请求，以减轻数据库压力。作为一个重要的组件，MyBatis 自然也在内部提供了相应的支持。通过在框架层面增加缓存功能，可减轻数据库的压力，同时又可以提升查询速度，可谓一举两得。MyBatis 缓存结构由一级缓存和二级缓存构成，这两级缓存均是使用 Cache 接口的实现类。因此本章将首先会向大家介绍 Cache 几种实现类的源码，然后再分析一级和二级缓存的实现。

6.1 缓存类介绍

在 MyBatis 中，Cache 是缓存接口，定义了一些基本的缓存操作，所有缓存类都应该实现该接口。MyBatis 内部提供了丰富的缓存实现类，比如具有基本缓存功能的 PerpetualCache，具有 LRU 策略的缓存 LruCache，以及可保证线程安全的缓存 SynchronizedCache 和具备阻塞功能的缓存 BlockingCache 等。除此之外，还有很多缓存实现类，这里就不一一列举了。需要特别说明的是，MyBatis 在实现缓存模块的过程中，使用

了装饰模式。在以上几种缓存实现类中，PerpetualCache 相当于装饰模式中的 ConcreteComponent。LruCache、SynchronizedCache 和 BlockingCache 等相当于装饰模式中的 ConcreteDecorator。它们的关系如下：



以上对 Cache 接口的实现类进行了简单的介绍，接下来，我们一起深入到源码中，看看这些缓存类的实现。

6.1.1 PerpetualCache

PerpetualCache 是一个具有基本功能的缓存类，内部使用了 HashMap 实现缓存功能。它的源码如下：

```

public class PerpetualCache implements Cache {

    private final String id;

    private Map<Object, Object> cache = new HashMap<Object, Object>();

    public PerpetualCache(String id) {
        this.id = id;
    }
}

```

```
@Override
public String getId() {
    return id;
}

@Override
public int getSize() {
    return cache.size();
}

@Override
public void putObject(Object key, Object value) {
    // 存储键值对到 HashMap
    cache.put(key, value);
}

@Override
public Object getObject(Object key) {
    // 查找缓存项
    return cache.get(key);
}

@Override
public Object removeObject(Object key) {
    // 移除缓存项
    return cache.remove(key);
}

@Override
public void clear() {
    cache.clear();
}
}
```

上面是 PerpetualCache 的全部代码,很简单。接下来,我们通过装饰类对该类进行装饰,使其功能变的丰富起来。

6.1.2 LruCache

LruCache,顾名思义,是一种具有 LRU 策略的缓存实现类。除此之外,MyBatis 还提供了具有 FIFO 策略的缓存 FifoCache。不过并未提供 LFU 缓存,如果大家有兴趣,可以自行拓展。接下来,我们来看一下 LruCache 的实现。

```
public class LruCache implements Cache {
    private final Cache delegate;
    private Map<Object, Object> keyMap;
    private Object eldestKey;

    public LruCache(Cache delegate) {
        this.delegate = delegate;
        setSize(1024);
    }

    public int getSize() {
        return delegate.getSize();
    }

    public void setSize(final int size) {
        // 初始化 keyMap, 注意, keyMap 的类型继承自 LinkedHashMap,
        // 并覆盖了 removeEldestEntry 方法
        keyMap = new LinkedHashMap<Object, Object>(size, .75F, true) {
            // 覆盖 LinkedHashMap 的 removeEldestEntry 方法
            @Override
            protected boolean removeEldestEntry(
                Map.Entry<Object, Object> eldest) {
                boolean tooBig = size() > size;
                if (tooBig) {
                    // 获取将要被移除缓存项的键值

```

```
        eldestKey = eldest.getKey();
    }
    return tooBig;
}
};
}

@Override
public void putObject(Object key, Object value) {
    // 存储缓存项
    delegate.putObject(key, value);
    cycleKeyList(key);
}

@Override
public Object getObject(Object key) {
    // 刷新 key 在 keyMap 中的位置
    keyMap.get(key);
    // 从被装饰类中获取相应缓存项
    return delegate.getObject(key);
}

@Override
public Object removeObject(Object key) {
    // 从被装饰类中移除相应的缓存项
    return delegate.removeObject(key);
}

@Override
public void clear() {
    delegate.clear();
    keyMap.clear();
}
```

```
private void cycleKeyList(Object key) {  
    // 存储 key 到 keyMap 中  
    keyMap.put(key, key);  
    if (eldestKey != null) {  
        // 从被装饰类中移除相应的缓存项  
        delegate.removeObject(eldestKey);  
        eldestKey = null;  
    }  
}  
}
```

LruCache 的 keyMap 属性是实现 LRU 策略的关键，该属性类型继承自 LinkedHashMap，并覆盖了 removeEldestEntry 方法。LinkedHashMap 可保持键值对的插入顺序，当插入一个新的键值对时，LinkedHashMap 内部的 tail 节点会指向最新插入的节点。head 节点则指向第一个被插入的键值对，也就是最久未被访问的那个键值对。默认情况下，LinkedHashMap 仅维护键值对的插入顺序。若要基于 LinkedHashMap 实现 LRU 缓存，还需通过构造方法将 LinkedHashMap 的 accessOrder 属性设为 true，此时 LinkedHashMap 会维护键值对的访问顺序。比如，上面代码中 getObject 方法中执行了这样一句代码 keyMap.get(key)，目的是刷新 key 对应的键值对在 LinkedHashMap 的位置。LinkedHashMap 会将 key 对应的键值对移动到链表的尾部，尾部节点表示最久刚被访问过或者插入的节点。除了需将 accessOrder 设为 true，还需覆盖 removeEldestEntry 方法。LinkedHashMap 在插入新的键值对时会调用该方法，以决定是否在插入新的键值对后，移除老的键值对。在上面的代码中，当被装饰类的容量超出了 keyMap 的所规定的容量（由构造方法传入）后，keyMap 会移除最长时间未被访问的键，并将该键保存到 eldestKey 中，然后由 cycleKeyList 方法将 eldestKey 传给被装饰类的 removeObject 方法，移除相应的缓存项目。

上面介绍了 LinkedHashMap 是如何实现 LRU 特性的，这个是理解 LruCache 的源码的关键所在，所以大家务必搞懂。如果大家想深入了解 LinkedHashMap 的源码，也可参考我之前写的文章 [LinkedHashMap 源码详细分析](#)。好了，关于 LruCache 就先分析这么多了。

6.1.3 BlockingCache

BlockingCache 实现了阻塞特性，该特性是基于 Java 重入锁实现的。同一时刻下，BlockingCache 仅允许一个线程访问指定 key 的缓存项，其他线程将会被阻塞住。下面我们来看一下 BlockingCache 的源码。

```
public class BlockingCache implements Cache {

    private long timeout;
    private final Cache delegate;
    private final ConcurrentHashMap<Object, ReentrantLock> locks;

    public BlockingCache(Cache delegate) {
        this.delegate = delegate;
        this.locks = new ConcurrentHashMap<Object, ReentrantLock>();
    }

    @Override
    public void putObject(Object key, Object value) {
        try {
            // 存储缓存项
            delegate.putObject(key, value);
        } finally {
            // 释放锁
            releaseLock(key);
        }
    }

    @Override
    public Object getObject(Object key) {
        // 请求锁
        acquireLock(key);
        Object value = delegate.getObject(key);
    }
}
```

```
// 若缓存命中，则释放锁。需要注意的是，未命中则不释放锁
if (value != null) {
    // 释放锁
    releaseLock(key);
}
return value;
}

@Override
public Object removeObject(Object key) {
    // 释放锁
    releaseLock(key);
    return null;
}

private ReentrantLock getLockForKey(Object key) {
    ReentrantLock lock = new ReentrantLock();
    // 存储 <key, Lock> 键值对到 locks 中
    ReentrantLock previous = locks.putIfAbsent(key, lock);
    return previous == null ? lock : previous;
}

private void acquireLock(Object key) {
    Lock lock = getLockForKey(key);
    if (timeout > 0) {
        try {
            // 尝试加锁
            boolean acquired = lock.tryLock(
                timeout, TimeUnit.MILLISECONDS);
            if (!acquired) {
                throw new CacheException("...");
            }
        } catch (InterruptedException e) {
            throw new CacheException("...");
        }
    }
}
```

```
        }
    } else {
        // 加锁
        lock.lock();
    }
}

private void releaseLock(Object key) {
    // 获取与当前 key 对应的锁
    ReentrantLock lock = locks.get(key);
    if (lock.isHeldByCurrentThread()) {
        // 释放锁
        lock.unlock();
    }
}
}
```

如上, 在查询缓存时, `getObject` 方法会先获取与 `key` 对应的锁, 并加锁。若缓存命中, `getObject` 方法会释放锁, 否则将一直锁定。`getObject` 方法若返回 `null`, 表示缓存未命中。此时 `MyBatis` 会向数据库发起查询请求, 并调用 `putObject` 方法存储查询结果。此时, `putObject` 方法会将指定 `key` 对应的锁进行解锁, 这样被阻塞的线程即可恢复运行。

上面的描述有点啰嗦, 倒是 `BlockingCache` 类的注释说到比较简单明了。这里引用一下:

It sets a lock over a cache key when the element is not found in cache. This way, other threads will wait until this element is filled instead of hitting the database.

这段话的意思是, 当指定 `key` 对应元素不存在于缓存中时, `BlockingCache` 会根据 `lock` 进行加锁。此时, 其他线程将会进入等待状态, 直到与 `key` 对应的元素被填充到缓存中。而不是让所有线程都去访问数据库。

在上面代码中, `removeObject` 方法的逻辑很奇怪, 仅调用了 `releaseLock` 方法释放锁, 却没有调用被装饰类的 `removeObject` 方法移除指定缓存项。这样做是为什么呢? 大家可以先思考, 答案将在分析二级缓存的相关逻辑时揭晓。

6.2 CacheKey

在 MyBatis 中，引入缓存的目的是为提高查询效率，降低数据库压力。既然 MyBatis 引入了缓存，那么大家思考过缓存中的 key 和 value 的值分别是什么吗？大家可能很容易能回答出 value 的内容，不就是 SQL 的查询结果吗。那 key 是什么呢？是字符串，还是其他什么对象？如果是字符串的话，那么大家首先能想到的是用 SQL 语句作为 key。但这是不对的，比如：

```
SELECT * FROM author where id > ?
```

id > 1 和 id > 10 查出来的结果可能是不同的，所以我们不能简单的使用 SQL 语句作为 key。从这里可以看出来，运行时参数将会影响查询结果，因此我们的 key 应该涵盖运行时参数。除此之外呢，如果进行分页查询，查询结果也会不同，因此 key 也应该涵盖分页参数。综上，我们不能使用简单的 SQL 语句作为 key。应该考虑使用一种复合对象，能涵盖可影响查询结果的因子。在 MyBatis 中，这种复合对象就是 CacheKey。下面来看一下它的定义。

```
public class CacheKey implements Cloneable, Serializable {

    private static final int DEFAULT_MULTIPLYER = 37;
    private static final int DEFAULT_HASHCODE = 17;

    // 乘子，默认为 37
    private final int multiplier;
    // CacheKey 的 hashCode，综合了各种影响因子
    private int hashCode;
    // 校验和
    private long checksum;
    // 影响因子个数
    private int count;
    // 影响因子集合
    private List<Object> updateList;
```

```
public CacheKey() {  
    this.hashCode = DEFAULT_HASHCODE;  
    this.multiplier = DEFAULT_MULTIPLYER;  
    this.count = 0;  
    this.updateList = new ArrayList<Object>();  
}  
}
```

如上，除了 `multiplier` 是恒定不变的，其他变量将在更新操作中被修改。下面看一下更新操作的代码。

```
/** 每当执行更新操作时，表示有新的影响因子参与计算 */  
public void update(Object object) {  
    int baseHashCode = object == null ? 1 : ArrayUtil.hashCode(object);  
    // 自增 count  
    count++;  
    // 计算校验和  
    checksum += baseHashCode;  
    // 更新 baseHashCode  
    baseHashCode *= count;  
  
    // 计算 hashCode  
    hashCode = multiplier * hashCode + baseHashCode;  
  
    // 保存影响因子  
    updateList.add(object);  
}
```

当不断有新的影响因子参与计算时，`hashCode` 和 `checksum` 将会变得愈发复杂和随机。这样可降低冲突率，使 `CacheKey` 可在缓存中更均匀的分布。`CacheKey` 最终要作为键存入 `HashMap`，因此它需要覆盖 `equals` 和 `hashCode` 方法。下面我们来看一下这两个方法的实现。

```
public boolean equals(Object object) {  
    // 检测是否为同一个对象  
    if (this == object) {  
        return true;  
    }  
    // 检测 object 是否为 CacheKey  
    if (!(object instanceof CacheKey)) {  
        return false;  
    }  
    final CacheKey cacheKey = (CacheKey) object;  
    // 检测 hashCode 是否相等  
    if (hashCode != cacheKey.hashCode) {  
        return false;  
    }  
    // 检测校验和是否相同  
    if (checksum != cacheKey.checksum) {  
        return false;  
    }  
    // 检测 count 是否相同  
    if (count != cacheKey.count) {  
        return false;  
    }  
  
    // 如果上面的检测都通过了，下面分别对每个影响因子进行比较  
    for (int i = 0; i < updateList.size(); i++) {  
        Object thisObject = updateList.get(i);  
        Object thatObject = cacheKey.updateList.get(i);  
        if (!ArrayUtil.equals(thisObject, thatObject)) {  
            return false;  
        }  
    }  
    return true;  
}
```

```
public int hashCode() {  
    // 返回 hashCode 变量  
    return hashCode;  
}
```

`equals` 方法的检测逻辑比较严格, 对 `CacheKey` 中多个成员变量进行了检测, 已保证两者相等。`hashCode` 方法比较简单, 返回 `hashCode` 变量即可。关于 `CacheKey` 就先分析到这, `CacheKey` 在一二级缓存中会被用到, 接下来还会看到它的身影。

6.3 一级缓存

在进行数据库查询之前, MyBatis 首先会检查以及缓存中是否有相应的记录, 若有的话直接返回即可。一级缓存是数据库的最后一道防护, 若一级缓存未命中, 查询请求将落到数据库上。一级缓存是在 `BaseExecutor` 被初始化的, 下面我们来看一下相关的初始化逻辑:

```
public abstract class BaseExecutor implements Executor {  
    protected PerpetualCache localCache;  
    // 省略其他字段  
  
    protected BaseExecutor(Configuration configuration,  
        Transaction transaction) {  
        this.localCache = new PerpetualCache("LocalCache");  
        // 省略其他字段初始化方法  
    }  
}
```

如上, 一级缓存的类型为 `PerpetualCache`, 没有被其他缓存类装饰过。一级缓存所存储从查询结果会在 MyBatis 执行更新操作 (`INSERT/UPDATE/DELETE`), 以及提交和回滚事务时被清空。下面我们来看一下访问一级缓存的逻辑。

```

public <E> List<E> query(MappedStatement ms, Object parameter,
    RowBounds rowBounds, ResultHandler resultHandler) throws SQLException {
    BoundSql boundSql = ms.getBoundSql(parameter);
    // 创建 CacheKey
    CacheKey key = createCacheKey(ms, parameter, rowBounds, boundSql);
    return query(ms, parameter, rowBounds, resultHandler, key, boundSql);
}

public <E> List<E> query(MappedStatement ms, Object parameter,
    RowBounds rowBounds, ResultHandler resultHandler, CacheKey key,
    BoundSql boundSql) throws SQLException {

    List<E> list;
    try {
        queryStack++;
        // 查询一级缓存
        list = resultHandler == null ?
            (List<E>) localCache.getObject(key) : null;
        if (list != null) {
            // 存储过程相关逻辑, 忽略
            handleLocallyCachedOutputParameters(ms, key, parameter, boundSql);
        } else {
            // 缓存未命中, 则从数据库中查询
            list = queryFromDatabase(ms, parameter,
                rowBounds, resultHandler, key, boundSql);
        }
    } finally {
        queryStack--;
    }

    return list;
}

```


如上,在访问一级缓存之前,MyBatis 首先会调用 `createCacheKey` 方法创建 `CacheKey`。下面我们来看一下 `createCacheKey` 方法的逻辑:

```
public CacheKey createCacheKey(MappedStatement ms, Object parameterObject,
    RowBounds rowBounds, BoundSql boundSql) {
    if (closed) {
        throw new ExecutorException("Executor was closed.");
    }
    // 创建 CacheKey 对象
    CacheKey cacheKey = new CacheKey();
    // 将 MappedStatement 的 id 作为影响因子进行计算
    cacheKey.update(ms.getId());
    // RowBounds 用于分页查询, 下面将它的两个字段作为影响因子进行计算
    cacheKey.update(rowBounds.getOffset());
    cacheKey.update(rowBounds.getLimit());
    // 获取 sql 语句, 并进行计算
    cacheKey.update(boundSql.getSql());
    List<ParameterMapping> parameterMappings =
        boundSql.getParameterMappings();
    TypeHandlerRegistry typeHandlerRegistry =
        ms.getConfiguration().getTypeHandlerRegistry();
    for (ParameterMapping parameterMapping : parameterMappings) {
        if (parameterMapping.getMode() != ParameterMode.OUT) {
            Object value;
            // 当前大段代码用于获取 SQL 中的占位符 #{xxx} 对应的运行时参数,
            // 前文有类似分析, 这里忽略了
            String propertyName = parameterMapping.getProperty();
            if (boundSql.hasAdditionalParameter(propertyName)) {
                value = boundSql.getAdditionalParameter(propertyName);
            } else if (parameterObject == null) {
                value = null;
            } else if (typeHandlerRegistry.hasTypeHandler(
                parameterObject.getClass())) {
                value = parameterObject;
```

```

        } else {
            MetaObject metaObject =
                configuration.newMetaObject(parameterObject);
            value = metaObject.getValue(propertyName);
        }

        // 让运行时参数参与计算
        cacheKey.update(value);
    }
}

if (configuration.getEnvironment() != null) {
    // 获取 Environment id 遍历, 并让其参与计算
    cacheKey.update(configuration.getEnvironment().getId());
}

return cacheKey;
}

```

在计算 CacheKey 的过程中, 有很多影响因子参与了计算。比如 MappedStatement 的 id 字段, SQL 语句, 分页参数, 运行时变量, Environment 的 id 字段等。通过让这些影响因子参与计算, 可以很好的区分不同查询请求。所以, 我们可以简单的把 CacheKey 看做是一个查询请求的 id。有了 CacheKey, 我们就可以使用它读写缓存了。在上面代码中, 若一级缓存为命中, BaseExecutor 会调用 queryFromDatabase 查询数据库, 并将查询结果写入缓存中。下面看一下 queryFromDatabase 的逻辑。

```

private <E> List<E> queryFromDatabase(MappedStatement ms, Object parameter,
RowBounds rowBounds, ResultHandler resultHandler, CacheKey key, BoundSql
boundSql) throws SQLException {
    List<E> list;
    // 向缓存中存储一个占位符
    localCache.putObject(key, EXECUTION_PLACEHOLDER);
    try {
        // 查询数据库
        list = doQuery(ms, parameter, rowBounds, resultHandler, boundSql);
    }
}

```

```
    } finally {  
        // 移除占位符  
        localCache.removeObject(key);  
    }  
    // 存储查询结果  
    localCache.putObject(key, list);  
  
    // 存储过程相关逻辑, 忽略  
    if (ms.getStatementType() == StatementType.CALLABLE) {  
        localOutputParameterCache.putObject(key, parameter);  
    }  
    return list;  
}
```

到此, 关于一级缓存相关的逻辑就差不多分析完了。一级缓存的逻辑比较简单, 大家可以简单过一遍。

6.4 二级缓存

二级缓存构建在一级缓存之上, 在收到查询请求时, MyBatis 首先会查询二级缓存。若二级缓存未命中, 再去查询一级缓存。与一级缓存不同, 二级缓存和具体的命名空间绑定, 一级缓存则是和 `SqlSession` 绑定。在按照 MyBatis 规范使用 `SqlSession` 的情况下, 一级缓存不存在并发问题。二级缓存则不然, 二级缓存可在多个命名空间间共享。这种情况下, 会存在并发问题, 因此需要针对性的去处理。除了并发问题, 二级缓存还存在事务问题, 相关问题将在接下来进行分析。下面先来看一下访问二级缓存的逻辑。

```
// -☆- CachingExecutor  
public <E> List<E> query(MappedStatement ms, Object parameterObject,  
    RowBounds rowBounds, ResultHandler resultHandler) throws SQLException {  
    BoundSql boundSql = ms.getBoundSql(parameterObject);  
    // 创建 CacheKey
```

```

        CacheKey key = createCacheKey(ms, parameterObject, rowBounds, boundSql);
        return query(ms, parameterObject, rowBounds, resultHandler, key, boundSql);
    }

    public <E> List<E> query(MappedStatement ms, Object parameterObject,
        RowBounds rowBounds, ResultHandler resultHandler, CacheKey key,
        BoundSql boundSql) throws SQLException {
        // 从 MappedStatement 中获取 Cache, 注意这里的 Cache
        // 并非是在 CachingExecutor 中创建的
        Cache cache = ms.getCache();
        // 如果配置文件中没有配置 <cache>, 则 cache 为空
        if (cache != null) {
            flushCacheIfRequired(ms);
            if (ms.isUseCache() && resultHandler == null) {
                ensureNoOutParams(ms, boundSql);
                // 访问二级缓存
                List<E> list = (List<E>) tcm.getObject(cache, key);
                // 缓存未命中
                if (list == null) {
                    // 向一级缓存或者数据库进行查询
                    list = delegate.<E>query(ms, parameterObject,
                        rowBounds, resultHandler, key, boundSql);
                    // 缓存查询结果
                    tcm.putObject(cache, key, list);
                }
                return list;
            }
        }

        return delegate.<E>query(
            ms, parameterObject, rowBounds, resultHandler, key, boundSql);
    }
}

```

如上，注意二级缓存是从 MappedStatement 中获取的，而非由 CachingExecutor 创建。由于 MappedStatement 存在于全局配置中，可以被多个 CachingExecutor 获取到，这样就会出现线程安全问题。除此之外，若不加以控制，多个事务共用一个缓存实例，会导致脏读问题。线程安全问题可以通过 SynchronizedCache 装饰类解决，该装饰类会在 Cache 实例构造期间被添加上。至于脏读问题，需要借助其他类来处理，也就是上面代码中 tcm 变量对应的类型。下面分析一下。

```
/** 事务缓存管理器 */
public class TransactionalCacheManager {

    // Cache 与 TransactionalCache 的映射关系表
    private final Map<Cache, TransactionalCache> transactionalCaches =
        new HashMap<Cache, TransactionalCache>();

    public void clear(Cache cache) {
        // 获取 TransactionalCache 对象，并调用该对象的 clear 方法，下同
        getTransactionalCache(cache).clear();
    }

    public Object getObject(Cache cache, CacheKey key) {
        return getTransactionalCache(cache).getObject(key);
    }

    public void putObject(Cache cache, CacheKey key, Object value) {
        getTransactionalCache(cache).putObject(key, value);
    }

    public void commit() {
        for (TransactionalCache txCache : transactionalCaches.values()) {
            txCache.commit();
        }
    }

    public void rollback() {
```

```

        for (TransactionalCache txCache : transactionalCaches.values()) {
            txCache.rollback();
        }
    }

    private TransactionalCache getTransactionalCache(Cache cache) {
        // 从映射表中获取 TransactionalCache
        TransactionalCache txCache = transactionalCaches.get(cache);
        if (txCache == null) {
            // TransactionalCache 也是一种装饰类，为 Cache 增加事务功能
            txCache = new TransactionalCache(cache);
            transactionalCaches.put(cache, txCache);
        }
        return txCache;
    }
}

```

TransactionalCacheManager 内部维护了 Cache 实例与 TransactionalCache 实例间的映射关系，该类也仅负责维护两者的映射关系，真正做事的还是 TransactionalCache。TransactionalCache 是一种缓存装饰器，可以为 Cache 实例增加事务功能。我在之前提到的脏读问题正是由该类进行处理的。下面分析一下该类的逻辑。

```

public class TransactionalCache implements Cache {

    private final Cache delegate;
    private boolean clearOnCommit;
    // 在事务被提交前，所有从数据库中查询的结果将缓存在此集合中
    private final Map<Object, Object> entriesToAddOnCommit;
    // 在事务被提交前，当缓存未命中时，CacheKey 将会被存储在此集合中
    private final Set<Object> entriesMissedInCache;

    @Override
    public Object getObject(Object key) {

```

```
// 查询 delegate 所代表的缓存
Object object = delegate.getObject(key);
if (object == null) {
    // 缓存未命中, 则将 key 存入到 entriesMissedInCache 中
    entriesMissedInCache.add(key);
}
if (clearOnCommit) {
    return null;
} else {
    return object;
}
}

@Override
public void putObject(Object key, Object object) {
    // 将键值对存入到 entriesToAddOnCommit 中, 而非 delegate 缓存中
    entriesToAddOnCommit.put(key, object);
}

@Override
public Object removeObject(Object key) {
    return null;
}

@Override
public void clear() {
    clearOnCommit = true;
    // 清空 entriesToAddOnCommit, 但不清空 delegate 缓存
    entriesToAddOnCommit.clear();
}

public void commit() {
    // 根据 clearOnCommit 的值决定是否清空 delegate
    if (clearOnCommit) {
```

```
        delegate.clear();
    }

    // 刷新未缓存的结果到 delegate 缓存中
    flushPendingEntries();
    // 重置 entriesToAddOnCommit 和 entriesMissedInCache
    reset();
}

public void rollback() {
    unlockMissedEntries();
    reset();
}

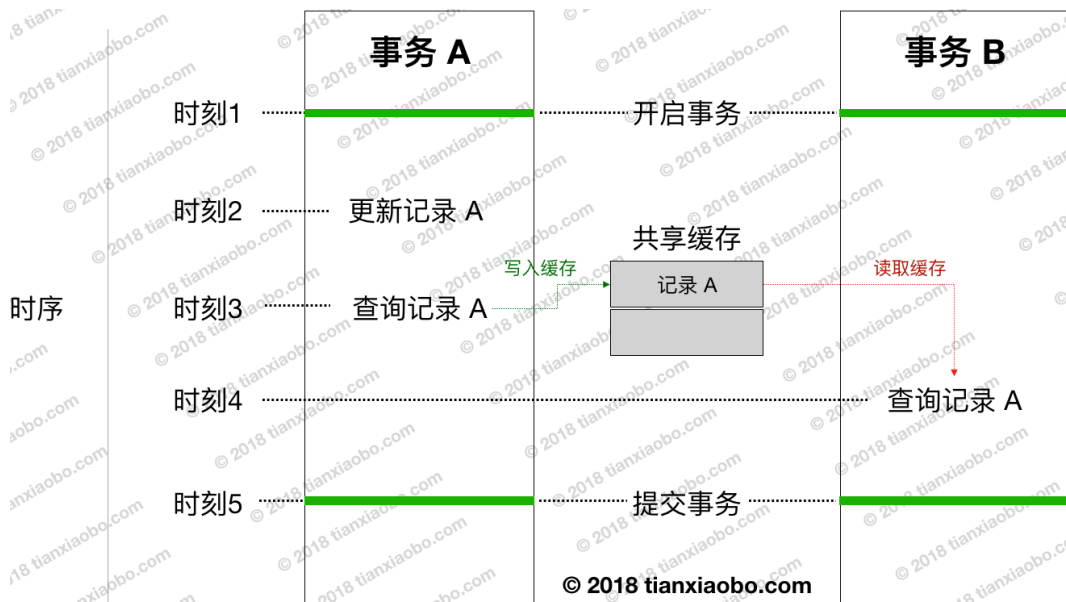
private void reset() {
    clearOnCommit = false;
    // 清空集合
    entriesToAddOnCommit.clear();
    entriesMissedInCache.clear();
}

private void flushPendingEntries() {
    for (Map.Entry<Object, Object> entry :
        entriesToAddOnCommit.entrySet()){
        // 将 entriesToAddOnCommit 中的内容转存到 delegate 中
        delegate.putObject(entry.getKey(), entry.getValue());
    }
    for (Object entry : entriesMissedInCache) {
        if (!entriesToAddOnCommit.containsKey(entry)) {
            // 存入空值
            delegate.putObject(entry, null);
        }
    }
}
```

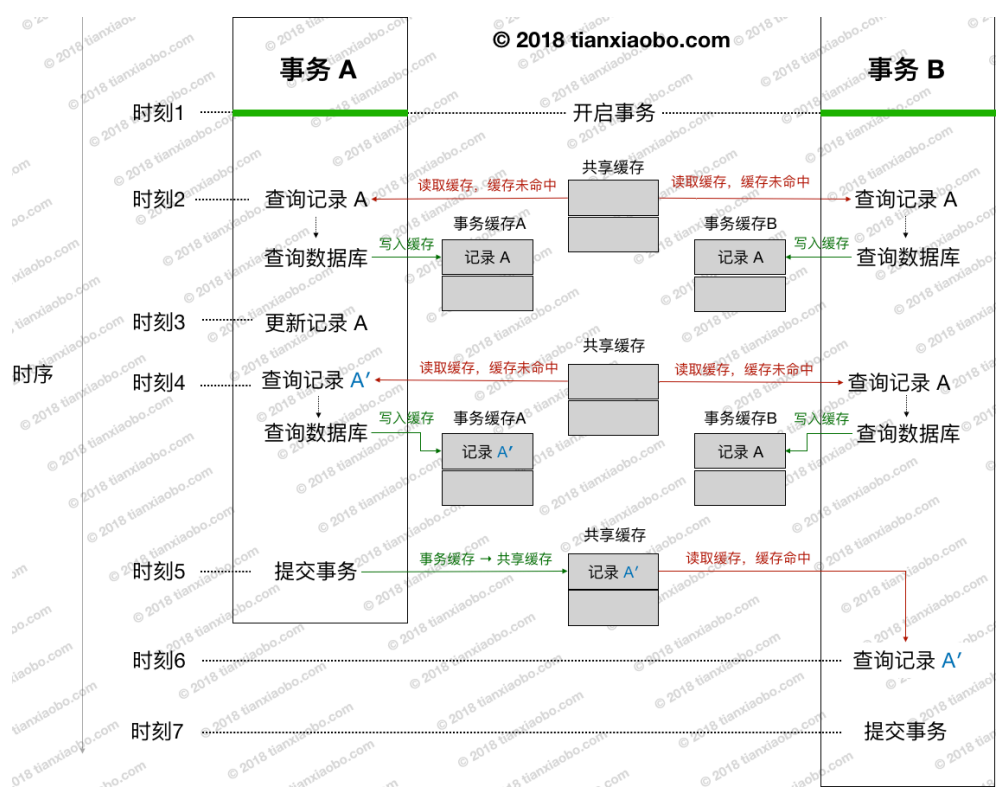


```
private void unlockMissedEntries() {  
    for (Object entry : entriesMissedInCache) {  
        try {  
            // 调用 removeObject 进行解锁  
            delegate.removeObject(entry);  
        } catch (Exception e) {  
            log.warn("...");  
        }  
    }  
}
```

在 TransactionalCache 的代码中，我们要重点关注 entriesToAddOnCommit 集合，TransactionalCache 中的很多方法都会与这个集合打交道。该集合用于存储从查询的结果，那为什么要将结果保存在该集合中，而非 delegate 所表示的缓存中呢？主要是因为直接存到 delegate 会导致脏数据问题。下面通过一张图演示一下脏数据问题发生的过程，假设两个线程开启两个不同的事务，它们的执行过程如下：



如上图，时刻 2，事务 A 对记录 A 进行了更新。时刻 3，事务 A 从数据库查询记录 A，并将记录 A 写入缓存中。时刻 4，事务 B 查询记录 A，由于缓存中存在记录 A，事务 B 直接从缓存中取数据。这个时候，脏数据问题就发生了。事务 B 在事务 A 未提交情况下，读取到了事务 A 所修改的记录。为了解决这个问题，我们可以为每个事务引入一个独立的缓存。查询数据时，仍从 delegate 缓存(以下统称为共享缓存)中查询。若缓存未命中，则查询数据库。存储查询结果时，并不直接存储查询结果到共享缓存中，而是先存储到事务缓存中，也就是 entriesToAddOnCommit 集合。当事务提交时，再将事务缓存中的缓存项转存到共享缓存中。这样，事务 B 只能在事务 A 提交后，才能读取到事务 A 所做的修改，解决了脏读问题。整个过程大致如下：



如上，时刻 2，事务 A 和 B 同时查询记录 A。此时共享缓存中还没有数据，所以两个事务均会向数据库发起查询请求，并将查询结果存储到各自的事务缓存中。时刻 3，事务 A 更新记录 A，这里把更新后的记录 A 记为 A'。时刻 4，两个事务再次进行查询。此时，事务 A 读取到的记录为修改后的值，而事务 B 读取到的记录仍为原值。时刻 5，事务 A 被提交，并将事务缓存 A 中的内容转存到共享缓存中。时刻 6，事务 B 再次查询记录 A，

由于共享缓存中有相应的数据,所以直接取缓存数据即可。因此得到记录 A',而非记录 A。但由于事务 A 已经提交,所以事务 B 读取到的记录 A' 并非是脏数据。MyBatis 引入事务缓存解决了脏读问题,事务间只能读取到其他事务提交后的内容,这相当于事务隔离级别中的“读已提交 (Read Committed)”。但需要注意的是,MyBatis 缓存事务机制只能解决脏读问题,并不能解决“不可重复读”问题。再回到上图,事务 B 在被提交前进行了三次查询。前两次查询得到的结果为记录 A,最后一次查询得到的结果为 A'。最有一次的查询结果与前两次不同,这就会导致“不可重复读”的问题。MyBatis 的缓存事务机制最高只支持“读已提交”,并不能解决“不可重复读”问题。即使数据库使用了更高的隔离级别解决了这个问题,但因 MyBatis 缓存事务机制级别较低。此时仍然会导致“不可重复读”问题的发生,这个在日常开发中需要注意一下。下面写点测试代码验证 MyBatis 所导致的“不可重复读”问题,首先看一下实体类:

```
public class Student {  
    private Integer id;  
    private String name;  
    private Integer age;  
}
```

对应的数据表如下:

```
      student  
+----+-----+-----+  
| id | name      | age |  
+----+-----+-----+  
|  1 | coolblog | 20  |  
+----+-----+-----+
```

Dao 接口与映射文件定义如下:

```
public interface StudentDao {  
    Student findOne(@Param("id") Integer id);  
    int update(@Param("id") Integer id, @Param("name") String name);  
}
```

```
<mapper namespace="xyz.coolblog.dao.StudentDao">

    <!-- 注意要在映射文件中配置缓存 -->
    <cache/>

    <select id="findOne" resultType="xyz.coolblog.model.Student">
        SELECT
            `id`, `name`, `age`
        FROM
            student
        WHERE
            id = #{id}
    </select>

    <update id="update">
        UPDATE
            student
        SET
            `name` = #{name}
        WHERE
            id = #{id}
    </update>
</mapper>
```

测试代码如下：

```
public class TransactionalCacheTest {
    private SqlSessionFactory sqlSessionFactory;
    private CountDownLatch countDownLatch = new CountDownLatch(1);

    @Before
    public void prepare() throws IOException {
        String resource = "mybatis-transactional-cache-config.xml";
```

```
        InputStream inputStream = Resources.getResourceAsStream(resource);
        sqlSessionFactory=new SqlSessionFactoryBuilder().build(inputStream);
        inputStream.close();
    }

    @Test
    public void testTransactional() throws Exception {
        ExecutorService es = Executors.newFixedThreadPool(2);
        // 开启两个线程
        Future<String> fa = es.submit(this::transactionalA);
        Future<String> fb = es.submit(this::transactionalB);

        countDownLatch.countDown();
        es.awaitTermination(6, TimeUnit.SECONDS);

        System.out.println(fa.get());
        System.out.println("\n ----- 分割线 ----- \n");
        System.out.println(fb.get());
    }

    private String transactionalA() throws Exception {
        SqlSession sqlSession = sqlSessionFactory.openSession();
        StudentDao studentDao = sqlSession.getMapper(StudentDao.class);
        countDownLatch.await();

        StringBuilder sb = new StringBuilder();
        sb.append("时刻 1: 开启事务 A\n");
        sb.append("时刻 2: 查询记录 A\n");

        Student s1 = studentDao.findOne(1);
        sb.append(s1).append("\n");

        sb.append("时刻 3: 更新记录 A\n");
    }
}
```

```

        studentDao.update(1, "tianxiaobo");
        sb.append("时刻 4: 查询记录 A'\n");
        Student s2 = studentDao.findOne(1);
        sb.append(s2).append("\n");

        // 此处睡眠 1 秒, 让事务 B 在事务 A 提交前, 完成时刻 4 的查询请求
        Thread.sleep(1000);

        sb.append("时刻 5: 提交事务 A");
        sqlSession.commit();

        return sb.toString();
    }

    private String transactionalB() throws Exception {
        SqlSession sqlSession = sqlSessionFactory.openSession();
        StudentDao studentDao = sqlSession.getMapper(StudentDao.class);
        countDownLatch.await();

        StringBuilder sb = new StringBuilder();
        sb.append("时刻 1: 开启事务 B\n");
        sb.append("时刻 2: 查询数据 A\n");
        Student s1 = studentDao.findOne(1);
        sb.append(s1).append("\n");
        sb.append("时刻 3: ----- \n");
        sb.append("时刻 4: 查询数据 A\n");
        Student s2 = studentDao.findOne(1);
        sb.append(s2).append("\n");

        // 此处睡眠 3 秒, 等待事务 A 提交
        Thread.sleep(3000);

        sb.append("时刻 5: ----- \n");
    }

```

```

        sb.append("时刻 6: 查询数据 A'\n");
        Student s3 = studentDao.findOne(1);
        sb.append(s3).append("\n");

        sb.append("时刻 7: 提交事务 B");
        sqlSession.commit();
        return sb.toString();
    }
}

```

最后对输出结果进行简单的美化，如下：



如上图，事务 B 在时刻 2 和时刻 4 读取到的记录与数据库中的记录一致，表示可重复读。但当事务 A 提交后，事务 B 在时刻 6 读取到的数据则是事务 A 修改的内容，这个时候就出现了“不可重复读”问题。以上测试是基于 MySQL 数据可读，MySQL 默认事务级别为“可重复读”。

```

3. mysql -uroot -p (mysql)
mysql> select @@tx_isolation;
+-----+
| @@tx_isolation |
+-----+
| REPEATABLE-READ |
+-----+
1 row in set, 1 warning (0.00 sec)

```

下面在本地开启两个 MySQL 客户端，模拟上面的执行流程。最终结果如下：

事务 A

时刻1

时刻2

时刻3

时刻4

时刻5

时刻6

时刻7

事务 B

时刻1

时刻2

时刻3

时刻4

时刻5

时刻6

时刻7

从测试结果可以看出，不可重复读问题并未发生，事务 B 三次查询结果均相同。到此，关于 MyBatis 二级缓存所引发的问题就分析完了。

接下来，我们再来看一下 entriesMissedInCache 集合，这个集合是用于存储未命中缓存的查询请求所对应的 CacheKey。单独分析与 entriesMissedInCache 相关的逻辑没什么意义，要搞清 entriesMissedInCache 的实际用途，需要把它和 BlockingCache 的逻辑结合起来进行分析。在 BlockingCache，同一时刻仅允许一个线程通过 getObject 方法查询指定 key 对应的缓存项。如果缓存未命中，getObject 方法不会释放锁，导致其他线程被阻塞住。其他线程要想恢复运行，必须进行解锁，解锁逻辑由 BlockingCache 的 putObject 和 removeObject 方法执行。其中 putObject 会在 TransactionalCache 的 flushPendingEntries 方法中被调用，removeObject 方法则由 TransactionalCache 的 unlockMissedEntries 方法调用。flushPendingEntries 和 unlockMissedEntries 最终都会遍历 entriesMissedInCache 集合，并将集合元素传给 BlockingCache 的相关方法。这样可以解开指定 key 对应的锁，让阻塞线程恢复运行。

最后特别说明一下，本节内容参考了《MyBatis 技术内幕》一书中关于缓存的一些分析，这里向这本书的作者表示感谢。如果大家不是很能看懂上面的内容，也可参考这本书的部分章节。

6.5 本章小结

本章对一些缓存类、CacheKey，以及一二级缓存的实现进行了深入的分析。其中，缓存类和 CacheKey 的实现比较简单。在一二级缓存中，一级缓存的实现相对简单，二级缓存则要复杂一些。二级缓存涉及到线程安全，事务等问题。线程安全问题比较好解决，由并发所导致的事务问题处理起来则要麻烦一些。MyBatis 二级缓存的事务级别比较低，这一点需要注意一下。

第 7 章

插件机制

一般情况下，开源框架都会提供插件或其他形式的拓展点，供开发者自行拓展。这样的好处是显而易见的，一是增加了框架的灵活性。二是开发者可以结合实际需求，对框架进行拓展，使其能够更好的工作。以 MyBatis 为例，我们可基于 MyBatis 插件机制实现分页、分表，监控等功能。由于插件和业务无关，业务也无法感知插件的存在。因此可以无感植入插件，在无形中增强功能。

开发 MyBatis 插件需要对 MyBatis 比较深了解才行，一般来说最好能够掌握 MyBatis 的源码，门槛相对较高。本篇文章在分析完 MyBatis 插件机制后，会手写一个简单的分页插件，帮助大家更好的掌握 MyBatis 插件的编写。

7.1 插件机制原理

我们在编写插件时，除了需要让插件类实现 `Interceptor` 接口外，还需要通过注解标注该插件的拦截点。所谓拦截点指的是插件所能拦截的方法，MyBatis 所允许拦截的方法如下：

1. Executor: update, query, flushStatements, commit, rollback, getTransaction, close, isClosed
2. ParameterHandler: getParameterObject, setParameters
3. ResultSetHandler: handleResultSets, handleOutputParameters
4. StatementHandler: prepare, parameterize, batch, update, query

如果我们想要拦截 Executor 的 query 方法，那么可以这样定义插件。

```
@Intercepts({
    @Signature(
        type = Executor.class,
        method = "query",
        args = {MappedStatement.class, Object.class, RowBounds.class,
        ResultHandler.class}
    )
})
public class ExamplePlugin implements Interceptor {
    // 省略逻辑
}
```

除此之外，我们还需将插件配置到相关文件中。这样 MyBatis 在启动时可以加载插件，并保存插件实例到相关对象（InterceptorChain，拦截器链）中。待准备工作做完后，MyBatis 处于就绪状态。我们在执行 SQL 时，需要先通过 DefaultSqlSessionFactory 创建 SqlSession。Executor 实例会在创建 SqlSession 的过程中被创建，Executor 实例创建完毕后，MyBatis 会通过 JDK 动态代理为实例生成代理类。这样，插件逻辑即可在 Executor 相关方法被调用前执行。以上就是 MyBatis 插件机制的基本原理。接下来，我们来看一下原理背后对应的源码是怎样的。

7.1.1 植入插件逻辑

本节，我将以 Executor 为例，分析 MyBatis 是如何为 Executor 实例植入插件逻辑的。Executor 实例是在开启 SqlSession 时被创建的，因此，下面我们从源头进行分析。先来看一下 SqlSession 开启的过程。

```
// -☆- DefaultSqlSessionFactory
public SqlSession openSession() {
    return openSessionFromDataSource(
        configuration.getDefaultExecutorType(), null, false);
}

private SqlSession openSessionFromDataSource(ExecutorType execType,
    TransactionIsolationLevel level, boolean autoCommit) {
    Transaction tx = null;
    try {
        // 省略部分逻辑

        // 创建 Executor
        final Executor executor = configuration.newExecutor(tx, execType);
        return new DefaultSqlSession(configuration, executor, autoCommit);
    }
    catch (Exception e) {...}
    finally {...}
}
```

Executor 的创建过程封装在 Configuration 中，我们跟进去看看。

```
// -☆- Configuration
public Executor newExecutor(Transaction transaction,
    ExecutorType executorType) {
    executorType = executorType == null ?
        defaultExecutorType : executorType;
    executorType = executorType == null ?
        ExecutorType.SIMPLE : executorType;
    Executor executor;

    // 根据 executorType 创建相应的 Executor 实例
    if (ExecutorType.BATCH == executorType) {...}
```

```
else if (ExecutorType.REUSE == executorType) {...}
else {
    executor = new SimpleExecutor(this, transaction);
}
if (cacheEnabled) {
    executor = new CachingExecutor(executor);
}

// 植入插件
executor = (Executor) interceptorChain.pluginAll(executor);
return executor;
}
```

如上,newExecutor 方法在创建好 Executor 实例后,紧接着通过拦截器链 interceptorChain 为 Executor 实例植入代理逻辑。那下面我们看一下 InterceptorChain 的代码是怎样的。

```
public class InterceptorChain {
    private final List<Interceptor> interceptors =
        new ArrayList<Interceptor>();

    public Object pluginAll(Object target) {
        // 遍历拦截器集合
        for (Interceptor interceptor : interceptors) {
            // 调用拦截器的 plugin 方法植入相应的插件逻辑
            target = interceptor.plugin(target);
        }
        return target;
    }

    /** 添加插件实例到 interceptors 集合中 */
    public void addInterceptor(Interceptor interceptor) {
        interceptors.add(interceptor);
    }

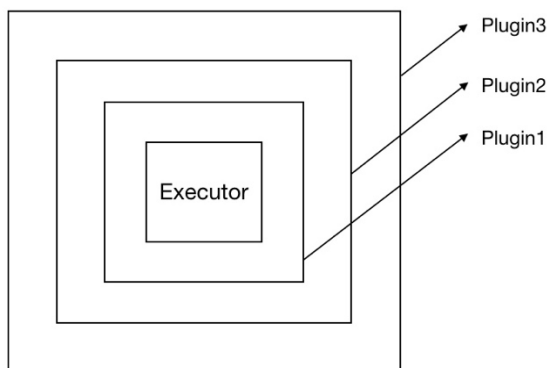
    /** 获取插件列表 */
}
```

```

public List<Interceptor> getInterceptors() {
    return Collections.unmodifiableList(interceptors);
}
}

```

以上是 InterceptorChain 的全部代码，比较简单。它的 pluginAll 方法会调用具体插件的 plugin 方法植入相应的插件逻辑。如果有多个插件，则会多次调用 plugin 方法，最终生成一个层层嵌套的代理类。形如下面：



当 Executor 的某个方法被调用的时候，插件逻辑会先行执行。执行顺序由外而内，比如上图的执行顺序为 plugin3 → plugin2 → Plugin1 → Executor。

plugin 方法是由具体的插件类实现，不过该方法代码一般比较固定，所以下面找个示例分析一下。

```

// -☆- ExamplePlugin
public Object plugin(Object target) {
    return Plugin.wrap(target, this);
}

// -☆- Plugin
public static Object wrap(Object target, Interceptor interceptor) {
    // 获取插件类 @Signature 注解内容，并生成相应的映射结构。形如下面：

```

```
// {  
//     Executor.class : [query, update, commit],  
//     ParameterHandler.class : [getParameterObject, setParameters]  
// }  
Map<Class<?>, Set<Method>> signatureMap = getSignatureMap(interceptor);  
Class<?> type = target.getClass();  
// 获取目标类实现的接口  
Class<?>[] interfaces = getAllInterfaces(type, signatureMap);  
if (interfaces.length > 0) {  
    // 通过 JDK 动态代理为目标类生成代理类  
    return Proxy.newProxyInstance(  
        type.getClassLoader(),  
        interfaces,  
        new Plugin(target, interceptor, signatureMap));  
}  
return target;  
}
```

如上, plugin 方法在内部调用了 Plugin 类的 wrap 方法, 用于为目标对象生成代理。Plugin 类实现了 InvocationHandler 接口, 因此它可以作为参数传给 Proxy 的 newProxyInstance 方法。

到这里, 关于插件植入的逻辑就分析完了。接下来, 我们来看看插件逻辑是怎样执行的。

7.1.2 执行插件逻辑

Plugin 实现了 InvocationHandler 接口, 因此它的 invoke 方法会拦截所有的方法调用。invoke 方法会对所拦截的方法进行检测, 以决定是否执行插件逻辑。该方法的逻辑如下:

```
// -☆- Plugin  
public Object invoke(Object proxy, Method method, Object[] args)  
    throws Throwable {  
    try {
```

```
// 获取被拦截方法列表, 比如: signatureMap.get(Executor.class),
// 可能返回 [query, update, commit]
Set<Method> methods = signatureMap.get(method.getDeclaringClass());
// 检测方法列表是否包含被拦截的方法
if (methods != null && methods.contains(method)) {
    // 执行插件逻辑
    return interceptor.intercept(
        new Invocation(target, method, args));
}
// 执行被拦截的方法
return method.invoke(target, args);
} catch (Exception e) {
    throw ExceptionUtil.unwrapThrowable(e);
}
}
```

invoke 方法的代码比较少, 逻辑不难理解。首先, invoke 方法会检测被拦截方法是否配置在插件的 @Signature 注解中, 若是, 则执行插件逻辑, 否则执行被拦截方法。插件逻辑封装在 intercept 中, 该方法的参数类型为 Invocation。Invocation 主要用于存储目标类, 方法以及方法参数列表。下面简单看一下该类的定义。

```
public class Invocation {

    private final Object target;
    private final Method method;
    private final Object[] args;

    public Invocation(Object target, Method method, Object[] args) {
        this.target = target;
        this.method = method;
        this.args = args;
    }
}
```



```
public Object proceed()  
    throws InvocationTargetException, IllegalAccessException {  
    // 调用被拦截的方法  
    return method.invoke(target, args);  
}  
}
```

关于插件的执行逻辑就分析到这，整个过程不难理解，大家简单看看即可。

7.2 实现一个分页插件

为了更好的向大家介绍 MyBatis 的插件机制，本节将实现一个 MySQL 数据库分页插件。相关代码如下：

```
@Intercepts({  
    @Signature(  
        type = Executor.class,    // 目标类  
        method = "query",        // 目标方法  
        args = {MappedStatement.class,  
                Object.class, RowBounds.class, ResultHandler.class}  
    )  
})  
public class MySqlPagingPlugin implements Interceptor {  
  
    private static final Integer MAPPED_STATEMENT_INDEX = 0;  
    private static final Integer PARAMETER_INDEX = 1;  
    private static final Integer ROW_BOUNDS_INDEX = 2;  
  
    @Override  
    public Object intercept(Invocation invocation) throws Throwable {  
        Object[] args = invocation.getArgs();  

```

```

RowBounds rb = (RowBounds) args[ROW_BOUNDS_INDEX];
// 无需分页
if (rb == RowBounds.DEFAULT) {
    return invocation.proceed();
}

// 将原 RowBounds 参数设为 RowBounds.DEFAULT，关闭 MyBatis 内置的分页机制
args[ROW_BOUNDS_INDEX] = RowBounds.DEFAULT;

MappedStatement ms = (MappedStatement) args[MAPPED_STATEMENT_INDEX];
BoundSql boundSql = ms.getBoundSql(args[PARAMETER_INDEX]);

// 获取 SQL 语句，拼接 limit 语句
String sql = boundSql.getSql();
String limit = String.format(
    "LIMIT %d,%d", rb.getOffset(), rb.getLimit());
sql = sql + " " + limit;

// 创建一个 StaticSqlSource，并将拼接好的 sql 传入
SqlSource sqlSource = new StaticSqlSource(
    ms.getConfiguration(), sql, boundSql.getParameterMappings());

// 通过反射获取并设置 MappedStatement 的 sqlSource 字段
Field field = MappedStatement.class.getDeclaredField("sqlSource");
field.setAccessible(true);
field.set(ms, sqlSource);

// 执行被拦截方法
return invocation.proceed();
}

@Override
public Object plugin(Object target) {

```

```
        return Plugin.wrap(target, this);
    }

    @Override
    public void setProperties(Properties properties) {
    }
}
```

上面的分页插件通过 RowBounds 参数获取分页信息，并生成相应的 limit 语句。之后拼接 sql，并使用该 sql 作为参数创建 StaticSqlSource。最后通过反射替换 MappedStatement 对象中的 sqlSource 字段。下面，写点测试代码验证一下插件是否可以正常运行。先来看一下 Dao 接口与映射文件的定义：

```
public interface StudentDao {
    List<Student> findByPaging(@Param("id") Integer id, RowBounds rb);
}
```

```
<mapper namespace="xyz.coolblog.chapter7.dao.StudentDao">
    <select id="findByPaging"
        resultType="xyz.coolblog.chapter7.model.Student">
        SELECT
            `id`, `name`, `age`
        FROM
            student
        WHERE
            id > #{id}
    </select>
</mapper>
```

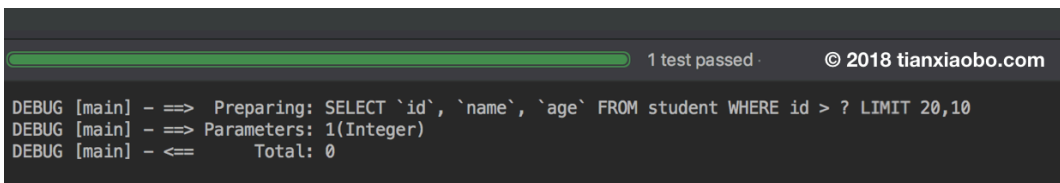
测试代码如下：

```
public class PluginTest {
    private SqlSessionFactory sqlSessionFactory;

    @Before
    public void prepare() throws IOException {
        String resource = "chapter7/mybatis-config.xml";
        InputStream inputStream = Resources.getResourceAsStream(resource);
        sqlSessionFactory=new SqlSessionFactoryBuilder().build(inputStream);
        inputStream.close();
    }

    @Test
    public void testPlugin() {
        SqlSession session = sqlSessionFactory.openSession();
        try {
            StudentDao studentDao = session.getMapper(StudentDao.class);
            studentDao.findByPaging(1, new RowBounds(20, 10));
        } finally {
            session.close();
        }
    }
}
```

上面代码运行之后，会打印如下日志。



The screenshot shows a JUnit test runner interface. At the top, a green progress bar indicates '1 test passed'. To the right of the progress bar, the text '© 2018 tianxiaobo.com' is displayed. Below the progress bar, the debug output is shown in a monospaced font. The output consists of three lines: 'DEBUG [main] - ==> Preparing: SELECT `id`, `name`, `age` FROM student WHERE id > ? LIMIT 20,10', 'DEBUG [main] - ==> Parameters: 1(Integer)', and 'DEBUG [main] - <== Total: 0'.

```
1 test passed · © 2018 tianxiaobo.com
DEBUG [main] - ==> Preparing: SELECT `id`, `name`, `age` FROM student WHERE id > ? LIMIT 20,10
DEBUG [main] - ==> Parameters: 1(Integer)
DEBUG [main] - <== Total: 0
```

在上面的输出中，SQL 语句中包含了 LIMIT 字样，这说明插件生效了。

7.3 本章小结

到此，关于 MyBatis 插件机制就分析完了。总体来说，MyBatis 插件机制比较简单。但实现一个插件却较为复杂，需要对 MyBatis 比较了解才行。因此，若想写出高效的插件，还需深入学习源码才行。

附录

MyBatis 源码分析系列文章列表

更新时间	标题
2018-07-16	MyBatis 源码分析系列文章导读
2018-07-20	MyBatis 源码分析 - 配置文件解析过程
2018-07-30	MyBatis 源码分析 - 映射文件解析过程
2018-08-17	MyBatis 源码分析 - SQL 的执行过程
2018-08-19	MyBatis 源码分析 - 内置数据源
2018-08-25	MyBatis 源码分析 - 缓存原理
2018-08-26	MyBatis 源码分析 - 插件机制